$author-display-name

08/14/2026

Versal Devices Integrated 100G Multirate Ethernet MAC Subsystem Product Guide (PG314)

$author-display-name

Versal Devices Integrated 100G Multirate Ethernet MAC Subsystem Product Guide (PG314)

Introduction

The AMD Versal™ Adaptive SoC Integrated 100G Multirate Ethernet MAC (MRMAC) is a high-performance, low latency, adaptable Ethernet integrated hard IP, targeting numerous customer networking applications. The block can be configured for up to four ports with independent MAC and PHY functions at the IEEE Standard MAC Rates from 10GE to 100GE, and an overall maximum bandwidth of 100GE. The IP supports various FECs and IEEE 1588 Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems (IEEE 1588) hardware timestamping.

Features

- Supports 1x100GE, 2x50GE, 1x40GE, 4x25GE, 4x10GE, and mixed modes
- User-side AXI4-Stream interface at ~390.625 MHz AXI4-Stream clock
- 80-bit interface to the serial transceiver for 1 × 100GE, 2 x 50GE, and 4 × 25GE mode. 32-bit interface to the serial transceiver for 1 x 40GE and 4 x 10GE
- IEEE 1588 Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems (IEEE 1588) one-step and two-step hardware timestamping at ingress and egress at full bit width
- Pause frame processing including priority-based flow control
- Optional built-in RS-FEC/ Firecode FEC blocks for 1 × 100GE, 2x50GE, 1x40GE, 4x25GE, and 4x10GE modes

IP Facts

Subsystem IP Facts Table	
Subsystem Specifics	
Supported Device Family (1)	AMD Versal™ architecture
Supported User Interfaces	AXI4-Stream, AXI4-Lite
Resources	Performance and Resource Utilization
Provided with Subsystem	
Design Files	Encrypted RTL
Example Design	Verilog
Test Bench	Verilog
Constraints File	Xilinx Design Constraints (XDC)
Simulation Model	Verilog
Supported S/W Driver	Linux Kernel. The drivers for 10G and 25GE are available here.
Tested Design Flows (2)	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see the Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973).
Synthesis	Synopsys or Vivado synthesis
Support	

Subsystem IP Facts Table	
Release Notes and Known Issues	Master Answer Record: 75817
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

1. For a complete list of supported devices, see the Vivado IP catalog.
2. For the supported versions of the tools, see the Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973).

Overview

Navigating Content by Design Process

AMD documentation is organized around a set of standard design processes to help you find relevant content for your current development task. This document covers the following design processes:

- **Hardware, IP, and Platform Development**
  Creating the PL IP blocks for the hardware platform, creating PL kernels, subsystem functional simulation, and evaluating the AMD Vivado timing, resource, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
  - Port Descriptions
  - Register Space
  - Clocking
  - Resets
  - Customizing and Generating the Subsystem
  - Example Design

Subsystem Overview

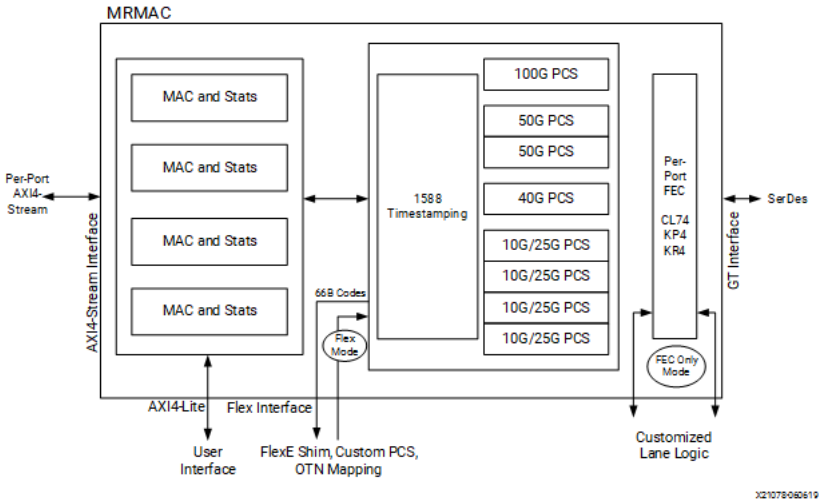
This product guide describes the function and operation of the AMD Versal™ devices integrated 100G Multirate Ethernet MAC subsystem, including how to design, customize, and implement the MRMAC.

The MRMAC subsystem handles all protocol related functions of an Ethernet MAC, PCS, and FEC, including handshaking, synchronizing, and error checking. An AXI4-Stream interface is provided for packet data, and an AXI4-Lite interface is provided for statistics and management.

The subsystem can provide up to four independent Ethernet ports and is designed to be flexible for use in many different applications. To reduce latency, the datapath does not perform any buffering other than the pipelining required to perform the required operations. Received data is passed directly to the user interface in a cut-through manner, allowing the flexibility to implement any required buffering scheme. Similarly, the transmit path consists of minimal pipeline and buffering to provide reliable cut-through operation.

The MRMAC subsystem can be configured to include forward error correction (FEC).

Figure: MRMAC High-Level Block Diagram



The following lists the supported rates, FEC types, and configurations:

Table: Supported Configuration Combinations

Data Rates	Data Path Functions	Integrated PCS Options
1 × 100GE [1]	MAC+PCS, PCS Only	No FEC, IEEE 802.3 CL91 RS(528,514) “KR4” FEC and CL91 RS(544,514) “KP4” FEC
2 × 50GE [1]		No FEC, IEEE 802.3 CL134 RS(544,514) “KP4” FEC, 25/50GE Consortium RSFEC/CL74 FEC
1 × 40GE [1]		No FEC, IEEE 802.3 CL74 FEC
4 × 25GE [1]		No FEC, IEEE 802.3 CL108 and 25/50GE Consortium RS(528,514) FEC, CL74 FEC
4 × 10GE [1]		No FEC, IEEE 802.3 CL74 FEC
4 × FC32 [1]	FEC Only	N/A
1 x 106.2GE (Overclocked 100GE) [1, 2]	Same as 100GE	Same as 100GE
1 x 112.84G FEC Only	FEC Only	IEEE 802.3 CL91 RS(528,514) "KR4" FEC and CL91 RS(544,514) "KP4" FEC
2 x 56.42G FEC Only [1]	FEC Only	IEEE 802.3 CL134 RS(544,514) “KP4” FEC, 50GE Consortium "KR4" RSFEC
1 x 64GFC [2]	FEC Only (requires additional programmable logic for transcoding and other functions)	CL91 RS(544,514) "KP4" FEC

Data Rates	Data Path Functions	Integrated PCS Options
2 x 64GFC [2]	FEC Only (requires additional programmable logic for transcoding and other functions)	CL91 RS(544,514) "KP4" FEC

1. Only these configurations are available in v2.0.
2. Supported in faster devices only.
3. MRMAC is limited to 40G speed for AMD Versal™ AI Edge Series devices. Therefore, the allowed data rates are 1x40G, 4x25G, and 4x10G.

Features Summary

The subsequent sections show the MRMAC subsystem features.

Protocol Support

The following features are supported:

- IEEE 1588 1-step and 2-step timestamping (including transparent clocking)
- Custom preamble insertion, received preamble extraction
- IEEE 802.1Q Express and Preemption traffic support through IEEE 802.3 Clause 99
- Programmable inter packet gap (IPG)
- Link status and alignment monitoring and status reporting
- 64B/66B decoding and encoding as defined in IEEE 802.3
- Scrambling and descrambling using x58 + x39 + 1 polynomial
- IPG insertion and deletion
- Optional frame check sequence (FCS) calculation and addition in the transmit direction
- FCS checking and optional FCS removal in the receive direction
- Class-based and Global Flow Control
- Pause packet processing and transmission
- Bit interleaved parity (BIP8) with hooks for customer-specific implementations

Interfaces

- User-side AXI4-Stream interface
- Network-side Flex Port interfacing supporting:
  - OTN interface for ITU defined PHY mapping points
  - Flex Ethernet (FlexE) interface to PHYs
  - PCS support mode for all rates (without the rate adaptation)
  - FEC Only (RS encode/decode only; transcoding bypassed)
  - 100G FlexO (lane deskew and RS encode/decode)
  - 32GFC (RS encode/decode and transcoding)
  - 64GFC (using FEC-only and programmable logic)
  -
- AXI4-Lite interface to access the control and statistics registers
- Transceiver interfaces for the following transceivers:
  - Up to 4 x GTYs
  - Up to 4 x GTYPs
  - Up to 4 x GTMs

Statistics

- Detailed statistics gathering (broadside bus and AXI4-Lite implementation)
  - Total bytes
  - Total packets
  - Good bytes
  - Good packets
  - Unicast packets
  - Multicast packets
  - Broadcast packets
  - Pause packets
  - Virtual local area network (VLAN) tagged packets
  - 64B/66B code violations
  - Bad preambles
  - Bad FCS
  - FEC code words
  - FEC correctable and uncorrectable code words
  - FEC symbol errors
  - Packet histogram for packets sized between:
    - 64
    - 65-127
    - 128-255
    - 256-511
    - 512-1023
    - 1024-1518
    - 1519-1522
    - 1523-1548
    - 1549-2047
    - 2048-4095
    - 4096-8191
    - 8192-9215

Additional Features

- Low data path latency
- Overclocking 100GE (4 × 28.21G) on -2LP and above
- Embedded Statistics Counters
- Status monitoring of decoding functions
- Real time switching of ports to support different optics configurations
- PCS Lane marker framing and deframing including swapping of each PCS Lane
- Dynamic and static deskew functions

Applications

The MRMAC subsystem is a highly flexible, IEEE-compliant network interface for applications that require a very high bit rate, such as:

- Ethernet switches
- IP routers
- Data center switches
- Communications equipment

The capability to interconnect devices at 10, 25, 40, 50, and 100 Gb/s Ethernet rates becomes especially relevant for next-generation data center networks where:

- To keep up with increasing CPU and storage bandwidth, rack or blade servers must support aggregate throughputs faster than 10 Gb/s (single lane) or 20 Gb/s (dual lane) from their Network Interface Card or LAN-on-Motherboard (LOM) networking ports.
- Given the increased bandwidth to endpoints, uplinks from Top-of-Rack (TOR) or Blade switches need to transition from 40 Gb/s (four lanes) to 100 Gb/s (four lanes) while ideally maintaining the same per-lane breakout capability.
- Due to the expected adoption of 100GBASE-CR4/KR4/SR4/LR4, SerDes and cabling technologies are already being developed and deployed to support 25 Gb/s per physical lane, twinax cable, or fiber.

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the End User License.

For more information and to generate a no charge license key, visit the AMD Versal™ Devices Integrated Multirate Ethernet MAC Subsystem product web page.

The licensing requirements for the core features are outlined in following table.

Table: Licensing Requirements

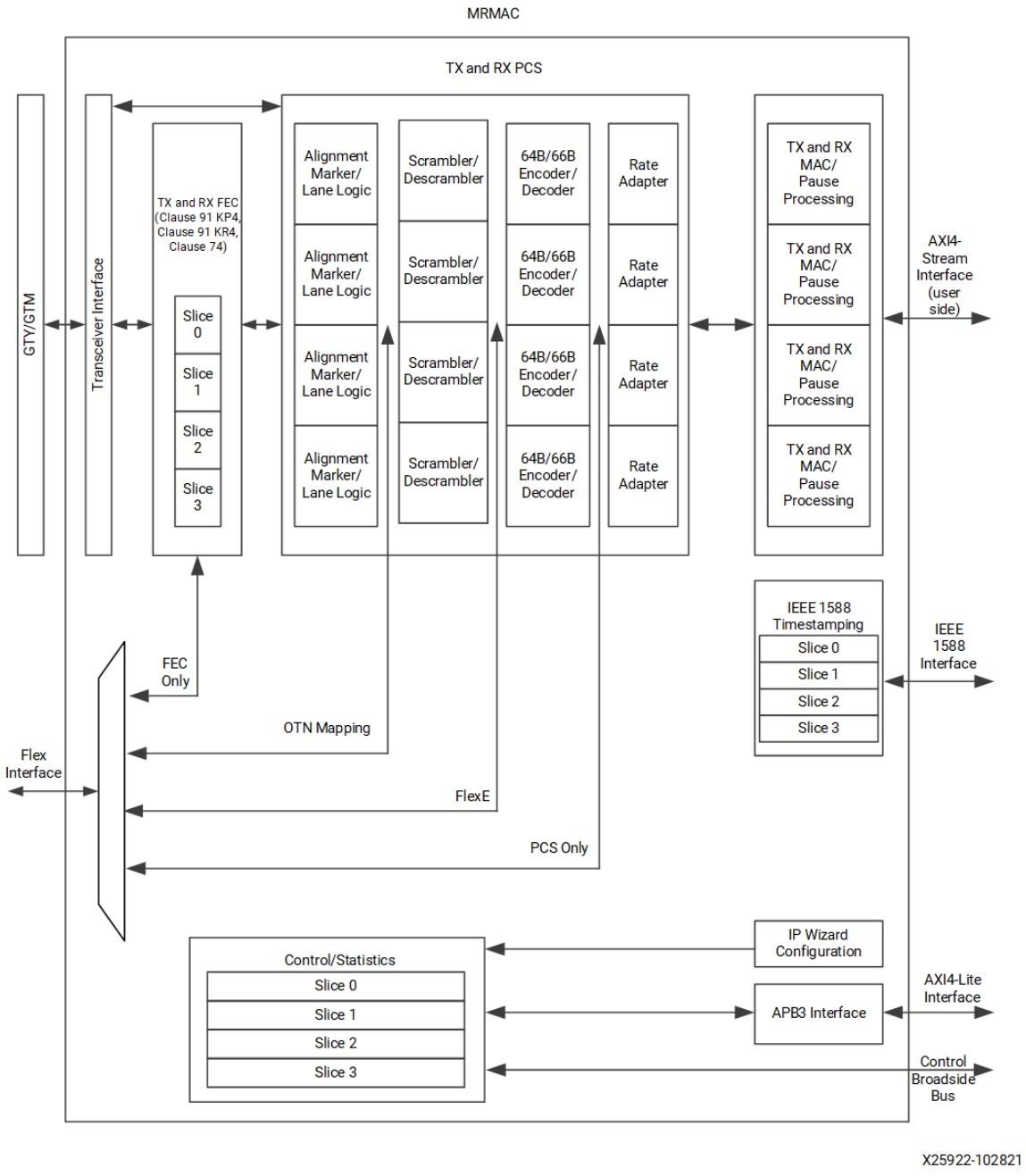
Subsystem Product Name	License Key Feature	Part Number
AMD Versal Devices Integrated 100G Multirate Ethernet MAC Subsystem	mrmac	N/A

Information about other AMD LogiCORE™ IP modules is available at the Intellectual Property page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your local sales representative.

Product Specification

The MRMAC can operate as four independent MAC and PHYs or its resources can be combined for higher-bandwidth operation. Note the sliced nature of the design. The functional block diagram of the subsystem is shown in the following figure. For a list of all FECs, see the following related information.

Figure: MRMAC Detailed Block Diagram



Related Information

Port FEC Mode

Standards

This subsystem adheres to the following standards:

- IEEE 802.3-2018 standard for 10GE, 25GE, 40GE, 50GE, and 100GE Ethernet MAC rates
- IEEE 802.3cd-2018 relating to 50GE and 100GE
- IEEE 802.3-2022 Clause 99. MAC Merge sublayer

The IP also supports the 25GE Ethernet Consortium Schedule 3 specification for 25GE and 50GE.

Performance and Resource Utilization

For full details about performance and resource use, see the Performance and Resource Use web page.

Operating Modes

This section describes the various operating modes of the different subsystems within the MRMAC.

Port Data Rate

The MRMAC architecture is composed of four independent Ethernet ports, each capable of 10/25GE data rate. The port resources can be dynamically combined to produce higher IEEE Ethernet rates, up to an overall bandwidth of 100GE. The ports can be statically configured through the IP wizard or dynamically configured during runtime through the AXI4-Lite interface.

Note: Dynamically reconfiguring the client rates of the MRMAC has clocking implications. Ensure that you understand the details surrounding clocking (see Clocking), and if you intend to take advantage of this capability, review the Use Cases in the Clocking Use Cases.

A port data rate is configured using the `ctl_data_rate_<N>` field (N = 0...3) of each MODE_REG of the port. The following table shows the values. All values not listed are reserved.

Table: Port Data Rate

Port	Configurable Modes (ctl_data_rate_<N>[2:0])				
	10GE	25GE	40GE	50GE	100GE
0	000	001	010	011	100
1	000	001	N/A	N/A	N/A
2	000	001	N/A	011	N/A
3	000	001	N/A	N/A	N/A

1. N/A denotes that the data rate is not supported by that port. All values not listed in the table are reserved.

You can configure Port 0 to operate at 10GE, 25GE, 40GE, 50GE, or 100GE data rates. However, when operating at 50GE data rate, port 0 consumes the data path resources of port 1, making it unavailable for independent operation. When port 0 is configured for 40GE or 100GE data rate operation, all internal data path resources are consumed. Consequently, ports 1, 2, and 3 are unavailable for independent operation.

Similarly, when port 2 is configured to operate at 50GE data rate, it consumes the data path of port 3, making port 3 unavailable for independent operation. Modes can be mixed-and-matched. For example, the user logic could configure ports 0 and 1 as 10GE and 25GE respectively, and then configure port 2 as a single 50GE client.

After configuration completes, a change in the MODE_REG register of any port requires a reset to be issued to that port.

Related Information

Clocking
Clocking Use Cases

Port AXI4-Stream Modes

The AXI4-Stream interface is dynamically re-configurable during runtime to support the selected mode of Ethernet operation. The MRMAC AXI4-Stream interface can be configured as four independent interfaces or ganged together as a wider bus to support the higher data rates. The number of 64-bit data word lanes can be configured to tailor the interface towards low latency performance or low frequency clock rates.

The following table lists the available configuration options. The desired mode is selected using the control field `ctl_axis_cfg_<N>[2:0]` (N = 0... 3) of each MODE_REG register of the port. The following table explains the meaning of the various options (segmented versus non-segmented bus structure and low latency versus independent clock modes).

Table: MRMAC AXI4-Stream Modes

Selected Data Rate	Selected AXI Clock Mode	AXI Clock Frequency (MHz)	AXI4-Stream Data Width (Bits)	AXI4-Stream Segment Mode	MODE Configuration (ctl_axis_cfg_<N>)
100GE	Low Latency	Same as tx_core_clk/ rx_core_clk (for example, 644.531)	256	Non-Segmented	3'b000
	Independent	390.625	384	Non-Segmented	3'b101
	Independent	322.265	384	Segmented	3'b111
40GE/50GE	Low Latency	Same as tx_core_clk/ rx_core_clk (for example, 644.531)	128	Non-Segmented	3'b000
	Independent	322.265	256	Non-Segmented	3'b101
25GE	Low Latency	Same as tx_core_clk/ rx_serdes_clk (for example, 644.531)	64	Non-Segmented	3'b000
	Independent	390.625	64	Non-Segmented	3'b001
	Independent	322.265	128	Non-Segmented	3'b101
10GE	Low Latency	tx_core_clk, rx_serdes_clk (for example, 644.531)	32	Non-Segmented	3'b000
	Independent	322.265	32	Non-Segmented	3'b001

1. All other values of `ctl_axis_cfg_<N>` field not listed in the table are reserved.

AXI4-Stream Segment Modes

If the MRMAC is operating at 100GE data rate, you can further configure the AXI4-Stream interface in either Non-Segmented or Segmented mode. For all other data rates, the AXI4-Stream only operates in non-segmented mode.

Configuration of 100GE port to operate in segmented versus non-segmented AXI4-Stream mode is through the MODE_REG register `ctl_axi_cfg_N` field, as listed in the previous table.

Non-Segmented Mode

In non-segmented mode, the per-port `rx_axis_tvalid`/`tx_axis_tvalid` signals indicate a transaction is present on the AXI bus. When an AXI4-Stream bus cycle contains the end of a packet (EOP) data, only a single packet can be transferred at a time. The next packet start must wait until the following cycle. Full AXI4-Stream port description and waveforms are provided in the User Interfaces section.

Related Information

User Interfaces

Segmented Mode

Segmented mode is suitable when a single high-bandwidth client requires efficient lowest-operating frequency access to the AXI4-Stream interface. The segmented nature of the bus allows the packing of a packet start on the segment-word immediately following the end of the previous packet, improving interface efficiency. In the MRMAC implementation, segmented mode is intended for (and limited to) 100GE operation. Full AXI4-Stream port description and waveforms are provided in the User Interfaces section.

Related Information

User Interfaces

Port AXI4-Stream Clocking Modes

To allow flexibility in the potential dynamic rate switching of ports, the MRMAC has two AXI4-Stream bus clock inputs to the MRMAC per direction (TX and RX), `tx_axi_clk[2,0]` and `rx_axi_clk[2,0]`. The `tx_axi_clk[0]` and `rx_axi_clk[0]` pins are used to clock ports 0 and 1. The `tx_axi_clk[2]` and `rx_axi_clk[2]` are used to clock ports 2 and 3.

Table: AXI4-Stream Clock Active for Each Data Rate

Port	rx_axi_clk and tx_axi_clk Usage for Configured Data Rates				
	10GE	25GE	40GE	50GE	100GE
0	rx_axi_clk[0], tx_axi_clk[0]	rx_axi_clk[0], tx_axi_clk[0]	rx_axi_clk[0], tx_axi_clk[0]	rx_axi_clk[0], tx_axi_clk[0]	rx_axi_clk[0], tx_axi_clk[0]
1	rx_axi_clk[0], tx_axi_clk[0]	rx_axi_clk[0], tx_axi_clk[0]	N/A	N/A	N/A
2	rx_axi_clk[2], tx_axi_clk[2]	rx_axi_clk[2], tx_axi_clk[2]	N/A	rx_axi_clk[2], tx_axi_clk[2]	N/A
3	rx_axi_clk[2], tx_axi_clk[2]	rx_axi_clk[2], tx_axi_clk[2]	N/A	N/A	N/A

The MRMAC uses the `rx_axi_clk` and `tx_axi_clk` to output some latency critical status signals, thus the IP block requires that the `rx_axi_clk` and `tx_axi_clk` inputs be driven by valid clocks regardless of the AXI4-Stream operating mode. The section for the independent AXI clocking mode describes the acceptable `rx_axi_clk` and `tx_axi_clk`.

In Flexible interface mode, it is necessary to provide `tx_axi_clk` and `rx_axi_clk` clocks to enable operation of certain high-performance device statistics and alarms, such as real-time FEC alarms.

The AXI4-Stream user interface can be configured to operate in one of two clocking modes, depending on system latency and performance requirements. As reflected in Table 1, the desired clock mode is selected using the `ctl_axis_cfg_<N>` field of the port MODE_REG register.

✎ **Important:** All ports of the MRMAC must be operated in the same AXI4-Stream clocking mode. A change in the clocking of the AXI4-Stream interface requires a reset of the full MRMAC IP.

Independent AXI Clock Mode

In independent (asynchronous) clock mode, the AXI4-Stream interface is run off an independent clock which can be asynchronous to the other MRMAC clocks allowing maximum flexibility. For available AXI clocks, see Table 1.

Note: It is acceptable to provide the same clock to both transmit and receive clock ports.

The port's `tx_axi_clk` and `rx_axi_clk` clocks must be a frequency of at least 50% (or greater) of the `tx_core_clk` and `rx_core_clk` (or `rx_serdes_clk` if the port is operating in 10/25GE data rate). It is acceptable if the `tx_axi_clk` is driven by a source that is /2 of the `tx_core_clk`, and the `rx_axi_clk` is connected to a source of /2 of the `rx_core_clk` (or /2 `rx_serdes_clk` if the port is operating in 10/25GE data rate).

Related Information

Port AXI4-Stream Clocking Modes

Low Latency Mode

In Low Latency mode, the AXI4-Stream interface is clocked internally by the same high-speed clock connected to `tx_core_clk` and `rx_core_clk` pins (or `tx_core_clk` and `rx_serdes_clk` pins when a port is operating at 10/25GE data rate). By leveraging a single clock throughout the datapath, this mode reduces latency by bypassing retiming stages and domain crossing buffering.

Even in the Low Latency mode, it is necessary to provide `tx_axi_clk` and `rx_axi_clk` clocks to enable operation of certain high-performance device statistics and alarms, such as real-time FEC alarms and flow control signaling.

Port FEC Mode

Forward error correction (FEC) is available on the PHYs of the MRMAC port. FEC can be enabled or disabled on a per-port basis.

The FEC_CONFIGURATION_REG needs to be configured appropriately for the FEC to work. For example, the Clause 49 flag should be set to 1 for modes which require Clause 49 transcoding (25GE, 50GE, Fibre Channel) and set to 0 for modes which require Clause 82 transcoding (100GE). CTL_TX_FEC_FOUR_LANE_PMD needs to set to 1 for 100CAUI-4 with RSFEC (it should be set to 0 100GAUI-2 and other modes). For additional details, refer to the Register description sheet of MRMAC Register document.

The various FEC operating modes are selected through the `ctl_fec_mode` field of the FEC_CONFIGURATION_REG configuration register. A change in the FEC_CONFIGURATION_REG of the port requires a reset being driven to that port after configuration is complete.

For a 50G configuration, the alignment marker spacing changes depending on whether RSFEC is enabled. The 50G alignment marker spacing should be set to 0x3FFF when RSFEC is disabled and 0x4FFF when RSFEC is enabled, via ctl_tx_vl_length_minus1_50ge_0/2 and ctl_rx_vl_length_minus1_50ge_0/2.

Table: FEC Operating Modes

MRMAC Port	Operating Rate	ctl_fec_mode_<N>[3:0]	Description

MRMAC Port	Operating Rate	ctl_fec_mode_<N>[3:0]	Description
0	All rates	'b0000	FEC Disabled
	100GE	'b1000	IEEE 802.3 RS(528,514) FEC
		'b1010	IEEE P802.3cd/D3.5 CL91 RS(544,514) FEC
		'b1011	ITU-T FlexEO RS(544,514) FEC
	50GE	'b0100	25G/50G Ethernet Consortium RS(528,517) FEC
		'b0101	IEEE 802.3 CL134 RS(544,514) FEC
		'b1111	IEEE 802.3 CL74 FEC
	40GE	'b1101	IEEE 802.3 CL74 FEC
	25GE	'b0010	25G/50G Ethernet Consortium RS(528,514) FEC
		'b0011	IEEE 802.3 CL108 RS(528,514) FEC
		'b1110	IEEE 802.3 CL74 FEC
	10GE	'b1100	IEEE 802.3 CL74 FEC
	32GFEC	'b0001	32GFEC (Valid only for Flex Interface)
1	All rates	'b0000	FEC Disabled
	25GE	'b0010	25G/50G Ethernet Consortium RS(528,514) FEC
		'b0011	IEEE 802.3 CL108 RS(528,514) FEC
		'b1110	IEEE 802.3 CL74 FEC
	10GE	'b1100	IEEE 802.3 CL74 FEC
	32GFEC	'b0001	32GFEC (Valid only for Flex Interface)

MRMAC Port	Operating Rate	ctl_fec_mode_<N>[3:0]	Description
2	All rates	'b0000	FEC Disabled
	50GE	'b0100	25G/50G Ethernet Consortium RS(528,514) FEC
		'b0101	IEEE 802.3 CL134 RS(544,514) FEC
		'b1111	IEEE CL74 25/50GE Ethernet Consortium
	25GE	'b0010	25G/50G Ethernet Consortium RS(528,514) FEC
		'b0011	IEEE 802.3 CL108 RS(528,514) FEC
		'b1110	IEEE 802.3 CL74 FEC
	10GE	'b1100	IEEE 802.3 CL74 FEC
	32GFEC	'b0001	32GFEC (Valid only for Flex Interface)
3	All rates	'b0000	FEC Disabled
	25GE	'b0010	25G/50G Ethernet Consortium RS(528,514) FEC
		'b0011	IEEE 802.3 CL108 RS(528,514) FEC
		'b1110	IEEE 802.3 CL74 FEC
	10GE	'b1100	IEEE 802.3 CL74 FEC
	32GFEC	'b0001	32GFEC (Valid only for Flex Interface)

GT Quad Transceiver (SerDes) Modes

The MRMAC is capable of interfacing with the AMD Versal™ GTY transceivers through the programmable logic region. Each GT Quad lane has a dedicated transceiver interface on the MRMAC IP subsystem.

The MRMAC has PCS/PMA lane logic functions including gearbox, scrambling, lane alignment, and lane deskew. For this reason, the transceivers connected to the MRMAC must be configured to operate in RAW mode. The default transceiver lane interface configuration is 16b RAW interface for 10.3125 Gb/s lane logic, 40b RAW interface for 25.78/26.56 Gb/s lane logic, and 80b RAW interface for 53.125 Gb/s lane logic.

For multi-lane PMD lane data rates (such as 100GE or 40GE), the MRMAC GT Quad interface can support per-lane GT Quad recovered clocks in the RX direction. This option reduces transceiver latency and improves 1588 timestamping accuracy at the cost of requiring per-lane FPGA clocking resources.

For designs desiring to reduce the clocking resources consumed by the MRMAC link, the IP also supports having the transceivers configured with per-lane deskew buffering enabled, allowing all RX PMD lanes to share a common master lane clock.

The following table shows a list of all the MRMAC supported interface options for GTY transceivers including the associated line rates, GT datapath width, clock frequency, and the corresponding `ctl_serdes_width` field setting. The `ctl_serdes_width` setting is found in the MODE_REG register for each port.

Table: GT Quad Operating Modes

MRMAC Operating Mode	GT Lane Line Rate (Gb/s)	GT Interface Data Width (bits)	GT Interface Clock (MHz)	ctl_serdes_width [2:0]	GT Mode NRZ/PAM4
10GE Narrow	10.3125	16	644.5313	b000	NRZ
10GE Wide	10.3125	32	322.2656	b100	NRZ
25GE Narrow	25.78125	40	644.5313	b010	NRZ
25GE Wide	25.78125	80	322.2656	b110	NRZ
40GE XLAUI-4 Narrow	10.3125	16	644.5313	b000	NRZ
40GE XLAUI-4 Wide	10.3125	32	322.2656	b100	NRZ
50GE 50GAUI-2 (KP4 FEC) Narrow	26.5625	40	664.0625	b010	NRZ
50GE 50GAUI-2 (KP4 FEC) Wide	26.5625	80	332.0312	b110	NRZ
50GE LAUI-2 Consortium Narrow	25.78125	40	644.5313	b010	NRZ
50GE LAUI-2 Consortium Wide	25.78125	80	322.2656	b110	NRZ
50GE 50LAUI-1 Narrow	51.5625	80	644.531	b011	PAM4
50GE 50GAUI-1 (KP4 FEC) Narrow	53.125	80	664.0625	b011	PAM4
50GE 50GAUI-1 Wide	53.125	160	664.0625	b110	PAM4
100GE CAUI-4 Narrow	25.78125	40	644.5313	b010	NRZ
100GE CAUI-4 Wide	25.78125	80	322.2656	b110	NRZ
100GE CAUI-4 (Overclocking) Wide	28.21	80	352.625	b110	NRZ
100GE 100GAUI-4 (KP4 FEC) Narrow	26.5625	40	664.0625	b010	NRZ
100GE 100GAUI-4 (KP4 FEC) Wide	26.5625	80	332.0312	b110	NRZ
100GE 100GAUI-2 (KP4 FEC) Narrow	53.125	80	664.0625	b011	PAM4

MRMAC Operating Mode	GT Lane Line Rate (Gb/s)	GT Interface Data Width (bits)	GT Interface Clock (MHz)	ctl_serdes_width [2:0]	GT Mode NRZ/PAM4
100GE 100GAUI-2 Wide	53.125	160	332.03125	b110	PAM4
100GE 100GAUI-2 (Overclocking)	56.42	160	352.625	b110	PAM4
100GE CAUI-2 Narrow	51.5625	80	644.531	b011	PAM4
100GE 100GAUI-1 Narrow	106.25	160	664.0625	b010	PAM4
100GE 100GAUI-1 Wide	106.25	160	332.03125	b110	PAM4
FC32 Single Lane	28.05	80	350.625	b110	NRZ

1. All the MRMAC 160-bit and 320-bit presets are operating in GAUI-4/CAUI-4 (80-bit) mode and using an external interleaving logic, for the required operation (160/320 bit). Therefore, the ctl_serdes_width setting value for these configs is the same as the CAUI-4/GAUI-4 value.
2. 10G/40G narrow (16-bit) mode cannot be used with GTM because GTM only supports a minimum data width of 32 bits.
3. -1 speed grade does not support narrow mode, GAUI, and over clocking.

Related Information

Clocking Use Cases
Transceiver (SerDes) Interface

GT Quad Transceiver Clocking Modes

The GT Quad Operating Modes table lists the supported MRMAC GT Quad interface clock frequency for each operating mode. All GT Quad lane interfaces for a port operating in a multi-lane operating modes have the same configuration. The `ctl_serdes_width[2]` bit selects the GT Quad interface to operate in narrow interface width or wider x2 GT Quad interface width.

The following table shows a list of MRMAC-supported interface options for FEC-only mode, including the associated data rates, datapath width, and clock frequency.

Table: Transceiver FEC-Only Operating Modes

Operating Mode	RS (528,514)	RS (544,514)	FEC-Only Rate (Gb/s)	FEC-Only Interface Data Width (Bits)	FEC-Only Interface Clock (MHz)	FEC-Only Logic Internal Clock (MHz)
25.78125 FEC-only	✓	-	25.78125	80	322.2656	644.5313
51.5625G FEC-only	✓	-	51.5625	160	322.2656	644.5313
53.125G FEC-only	-	✓	53.125	160	332.0312	664.0625
56.42G FEC-only	-	✓	56.42	160	352.625	705.25
103.125 FEC-only	✓	-	103.125	320	322.2656	644.5313

Operating Mode	RS (528,514)	RS (544,514)	FEC-Only Rate (Gb/s)	FEC-Only Interface Data Width (Bits)	FEC-Only Interface Clock (MHz)	FEC-Only Logic Internal Clock (MHz)
106.125G FEC-only	-	✓	106.25	320	332.0312	664.0625
112.84G FEC-only	-	✓	112.84	320	352.625	705.25

When configured in x2 width mode, the GT Quad interface from the lane is clocked by `rx_alt_serdes_clk[n]` and `tx_alt_serdes_clk[n]`. The `rx_serdes_clk[n]` connects to a GT Quad transceiver generated clock whose frequency is x2 `rx_alt_serdes_clk[n]`. The `tx_core_clk[m]` is driven by the GT Quad generated x2 `tx_alt_serdes_clk[n]` clock (where n is the lane number and m is the port number).

A full pin description of the GT Quad interface is provided in the later Transceiver Interface section and the GT interface clocking examples are in the Clocking Use Cases section.

Related Information

Clocking Use Cases
Transceiver (SerDes) Interface

Flex Interface Modes

The MRMAC features a flexible interface (Flex I/F) which permits the user logic to bypass the MAC logic and directly access the internal hardened resources of the MRMAC. Each port's Flex I/F can be configured to access a tap point to and from the internal MRMAC transmit/receive PCS datapath. In this approach, the Flex I/F of the port can be configured to any of the following:

- BASE-R PCS n × 66b word interface (PCS/BASE-R mode)
- FlexE PHY n × 66b word interface (FlexE mode)
- Transparent PHY n × 66b interface (OTN mapping mode)
- Direct access to the FEC resources (FEC only mode)

The Flex I/F can be configured into between one to four independent ports, supporting 4 × 10/25GE, 2 × 50GE, 1 × 40GE, or 1 × 100GE operation.

For each port, the `ctl_data_rate` field of the MODE_REG sets the operating data rate for the Flex I/F. For the TX direction, the `ctl_tx_flexif_input_enable` enables the Flex I/F input (disabling the AXI4-Stream input interface) and `ctl_tx_flexif_select` sets the operating mode, both fields are in the CONFIGURATION_TX_REG register. For the RX direction, the `ctl_rx_flexif_select` field of the CONFIGURATION_RX_REG register sets the RX Flex I/F mode.

Flex Interface Modes of Operation

The following table describes the available Flex I/F modes of operation. Note the dependence of device speed grades on the available modes.

Table: Flex Interface Modes of Operation

MRMAC Configuration	CL74	RS(528,514)	RS(544,514)	GT Line Rate (Gb/s)	Number of GT Lanes	GT Interface Data Width (Bits)	GT Data Rate (MHz)	Flex I/F Clock (Minimum) (MHz)	Supported Flex I/F Modes			
									PCS/BASE-R Mode	FlexE	OTN	FEC Only Mode
10GE	✓	-	-	10.3125	1	16	644.5313	332.2656	✓	✓	✓	-

MRMAC Configuration	CL74	RS(528,514)	RS(544,514)	GT Line Rate (Gb/s)	Number of GT Lanes	GT Interface Data Width (Bits)	GT Data Rate (MHz)	Flex I/F Clock (Minimum) (MHz)	Supported Flex I/F Modes			
									PCS/BASE-R Mode	FlexE	OTN	FEC Only Mode
10GE	✓	-	-	10.3125	1	32	322.2656	322.2656	✓	✓	✓	✓
25GE	✓	-	-	25.78125	1	40	644.5313	322.2656	✓	✓	✓	-
25GE	-	✓	-	25.78125	1	40	644.5313	322.2656	✓	✓	✓	-
25GE	✓	✓	-	25.78125	1	80	322.2656	322.2656	✓	✓	✓	✓
40GE XLAUI-4	-	-	-	10.3125	4	16	644.5313	322.2656	✓	✓	✓	-
40GE XLAUI-4	-	-	-	10.3125	4	32	322.2656	322.2656	✓	✓	✓	-
50GE LAUI-2	-	✓	-	25.78125	2	40	644.5313	322.2656	✓	✓	✓	-
50GE LAUI-2	-	✓	-	25.78125	2	80	322.2656	322.2656	✓	✓	✓	✓
50GE 50GAUI-2	-	-	✓	26.5625	2	40	664.0625	332.03125	✓	✓	✓	✓
50GE	-	-	✓	53.125	1	80	664.0625	332.03125	✓	✓	✓	✓
100GE CAUI-4	-	✓	-	25.78125	4	40	644.5313	322.2656	✓	✓	✓	-
100GE CAUI-4	-	✓	-	25.78125	4	80	322.2656	322.2656	✓	✓	✓	✓
100GE 100GAUI-4	-	-	✓	26.5625	4	40	644.0625	332.03125	✓	✓	✓	-

MRMAC Configuration	CL74	RS(528,514)	RS(544,514)	GT Line Rate (Gb/s)	Number of GT Lanes	GT Interface Data Width (Bits)	GT Data Rate (MHz)	Flex I/F Clock (Minimum) (MHz)	Supported Flex I/F Modes			
									PCS/BASE-R Mode	FlexE	OTN	FEC Only Mode
100GE overclocking: 109.42GE – No FEC 109.42GE – RS(528,514) FEC 106.20GE – RS(544,514) FEC	–	✓	✓	28.21	4	80	352.625	352.625	–	–	–	–
100GE	–	–	✓	53.125	2	160	332.03125	332.03125	–	–	–	✓
100GE Overclocking	–	–	✓	56.42	2	160	352.625	352.625	–	–	–	✓
FC32	–	–	–	28.05	1	80	350.625	350.625	–	–	–	✓

Flex Interface Receive Operating Modes

The following table lists the configuration options for the RX path (from SerDes interface to the Flex I/F). The configuration is set in the `ctl_rx_flexif_select_N` field of the CONFIGURATION_RX_REG register of each port. All other values not listed in the table are reserved and invalid.

Table: Flex Interface Receive Operating Modes

ctl_rx_flexif_select_N[3:0]	Operating Mode	Application Description
b0000	Disable descrambler function and disable AM removal function.	100GE/40GE OTN mapping point.
b0001	Enable descrambler function and enable AMs removed.	FlexE PHY mode with no EBLOCK replacement.
b0011	Enable descrambler function, enable AMs removed, and enable 66-bit sync header error replacement by EBLOCKs.	FlexE PHY mode with EBLOCK replacement.

ctl_rx_flexif_select_N[3:0]	Operating Mode	Application Description
b0111	Enable descrambler function, enable AMs removed, enable 66-bit sync header errors replacement by EBLOCKs, and enable PCS receive state machine.	PCS mode with 66-bit BASER interface.
b1111	Enable descrambler function, enable AMs removed, enable 66-bit sync header errors replacement by EBLOCKs, enable PCS receive state machine, and enable IEEE 802.3 CL49 to CL82 66-bit block conversion. The CL49 to CL82 66-bit block conversion is not included/verified in the current MRMAC.	Client PCS interface to a FlexE system.
b0010	FEC Only mode. Disable scrambling, disable AM removal, and direct connection to FEC decode block.	FC32, FlexO, and custom FEC use case.

1. In FEC Only mode, the FEC Mode register is used to select the application-specific FEC encoding/decoding.

Flex Interface Transmit Operating Modes

The following table lists the configuration options for the TX path (from Flex I/F to SerDes interface). The configuration is set in the `ctl_tx_flexif_select_N` field of the CONFIGURATION_TX_REG register of each port. All other values not listed in the table are reserved and invalid.

Table: Flex Interface Transmit Operating Modes

ctl_tx_flexif_select_N[2:0]	Operating Mode	Application Description
b000	Disable scrambler function and disable AM insertion function.	100GE/40GE OTN mapping point.
b001	Enable scrambler function and enable AMs insertion.	FlexE PHY mode.
b011	Enable scrambler function, enable AM insertion, and enable PCS TX state machine.	PCS mode with 66-bit BASER interface.
b111	Enable scrambler function, enable AM insertion, enable PCS TX state machine, and enable IEEE 802.3 CL49 to CL82 66-bit block conversion. The CL82 to CL49 66-bit block conversion is not included/verified in the current MRMAC.	Client PCS interface to a FlexE system.
b010	FEC Only Mode	FC32, FlexO, or direct FEC mode.

ctl_tx_flexif_select_N[2:0]	Operating Mode	Application Description

1. In FEC Only mode, the FEC Mode register is used to select the application-specific FEC encoding/decoding.
2. MRMAC Dynamic Switching between FEC-only mode and full MRMAC mode. In FEC-only mode, FEC registers are accessible, but not all of the MRMAC registers are fully accessible. When doing a dynamic switch from FEC-only to full mode, you should be careful to change the FEC mode first. Specifically, ctl_tx_fec_transcode_bypass should be set to 0 before making other non-FEC configuration changes. When doing a dynamic switch from full mode to FEC-only mode, ctl_tx_fec_transcode_bypass should be set to '1' last.

Port Descriptions

The following sections describe the MRMAC subsystem interfaces.

AXI4-Stream User Interface

The primary datapath interface to the MRMAC is a high performance AXI4-Stream interface. Transmit and receive frame data presents to and collects from this interface.

The following sections describe the AXI4-Stream signals which the signals to use for each supported data rate. The same signals are used for segmented and non-segmented modes, but their interpretation by user logic and interface logic can differ. Also, different groups of signals are used by different data rates. All unused input signals (including those inputs associated with unused or disabled modes) must be driven to 0.

The MRMAC can be dynamically reconfigured during runtime to support a user-selected mode of operation (exchanging data rate for a port). In this case, the user logic must be careful to properly drive the correct signals depending on the mode of operation.

✎ Note: All ports of the MRMAC must operate in the same AXI4-Stream clocking mode (Independent or Low Latency Clocking modes).

Non-Segmented Mode

✎ Note: In the following tables, <N> = port number 0 to 3 and <M> is the word number 0 to 7.

Table: Non-Segmented AXI4-Stream Interface Signal Descriptions – TX Direction

Port Name	I/O	Description
tx_axis_tvalid_<N>	I	Asserted by user logic to indicate that the current bus cycle is valid.
tx_axis_tready_<N>	O	Driven by the MRMAC to indicate that the interface is able to accept the data currently present on the bus. If the tready signal deasserts during a transfer, the user logic must hold the current data on the bus until tready is asserted again.
tx_axis_tlast_<N>	I	Indicates that the current bus cycle contains the last data transfer for the packet. This is the only time a partially-filled data word is permitted.
tx_axis_tdata<M>[63:0]	I	Packet data, in Little-Endian order.

Port Name	I/O	Description
tx_axis_tkeep_user_<M>[7:0]	I	tkeep[7:0]. A per-byte data valid indication for last word. Valid when tlast is 1. tkeep indicates whether the data found in the corresponding tdata lane is valid. The mapping is as follows: • tkeep[0] = tdata[7:0] is valid • tkeep[1] = tdata[15:0] is valid • ... • tkeep[7] = tdata[63:56] is valid Data bytes always fill the tdata signal contiguously from LSB to MSB with no gaps in between. Therefore, the legal patterns of the tkeep signal are: • 0000_0001 (1 byte valid) • 0000_0011 (2 bytes valid) • 0000_0111 (3 bytes valid) • ... • 1111_1111 (all bytes valid)
tx_axis_tkeep_user_<M>[8]	I	ERR. Error indication for the packet. Valid when corresponding tlast is 1.
tx_axis_tkeep_user_<M>[9]	I	Preempt. Indicates that the packet in progress is to be preempted. Valid when tlast is 1. For more details on frame preemption support, see the related information below.
tx_axis_tkeep_user_<M>[10]	I	Resume. Indicates that the packet being sent on the AXI4-Stream interface is a resumption of a previously preempted packet. This signal is to be asserted at the beginning of new packet transfer. For more details on frame preemption support, see the related information below.
tx_preamblein_<N>[55:0]	I	This signal is used to convey preamble + SFD values when custom preamble mode is enabled. It is also used when Frame Preemption is enabled. The tx_preamblein signal must be provided during the first transfer of data. When enabled, six preamble bytes followed by one SFD byte must be provided to populate the packet's 64B/66B Start code.

Port Name	I/O	Description
tx_ptp_1588op_in_<N>[1:0]	I	Select the desired PTP operation. • 2'b00 – No operation: no timestamp is taken and the frame is not modified. • 2'b01 – 1-step: a timestamp should be taken and inserted into the frame. The timestamp is also returned to the client using the additional ports of 2-step operation. • 2'b10 – 2-step: a timestamp should be taken and returned to the client using the additional ports of 2-step operation. The frame itself is not modified. • 2'b11 – Reserved.
tx_ptp_cf_offset_<N>[15:0]	I	This specifies the offset (in octets) to the PTP Correction field. Offset 0 corresponds to the first byte of the Ethernet Destination Address. If UDP checksum adjustment is required (tx_ptp_upd_chksum_in is set), the first byte of the checksum is assumed to be located at tx_ptp_cf_offset – 10 octets. Note, this value must be even, and for timestamping applications not following IEEE 1588 protocol but wanting to use the 1-step timestamp insertion feature, any user specified correction field offset must be a minimum of 18 bytes from the end of the packet.
tx_ptp_upd_chksum_in_<N>	I	Update UDP checksum: • 1'b0: PTP frame does not contain a UDP checksum • 1'b1: PTP frame does contain a UDP checksum which the core is required to recalculate. The location of the checksum is ascertained as described in the tx_ptp_cf_offset field.
tx_ptp_tag_field_in_<N>[15:0]	I	Identifying tag to use for egress PTP packet. This should be valid during the SOP cycle on the AXI4-Stream interface.
tx_ptp_tstamp_tag_out_<N>[15:0]	O	Egress Tag. Valid when tx_ptp_tstamp_valid_out<N> is set.
tx_ptp_tstamp_valid_out_<N>	O	Indicates that Egress tag/timestamp is valid.

Port Name	I/O	Description
tx_ptp_rsfec_offset_out_<N>[6:0]	O	Egress RSFEC codeword offset of start of packet. This is only valid when RSFEC is enabled.
tx_ptp_tstamp_out_<N>[54:0]	O	Egress Timestamp. Valid when tx_ptp_tstamp_valid_out_<N> is set.

Table: Non-Segmented AXI4-Stream Interface Signal Descriptions - RX Direction

Port Name	I/O	Description
rx_axis_tvalid_<N>	O	Indicates that the data and flags on the AXI4-Stream bus output are valid. The user logic must accept the data transfer. There is no backpressure mechanism.
rx_axis_tlast_<N>	O	Indicates that the current bus cycle output contains the last data transfer for the packet.
rx_axis_tdata<M>[63:0]	O	Contains the data for the received packet in Little-Endian order.
rx_axis_tkeep_user_<M>[7:0]	O	tkeep[7:0]. A per-byte data valid indication for last word. Valid when tlast is 1.
rx_axis_tkeep_user_<M>[8]	O	Err. Error indication for the received packet. Valid when tlast is 1. A value of 1 indicates that an error was detected in the received frame and it should be discarded.
rx_axis_tkeep_user_<M>[9]	O	Preempt. Indicates that the packet being output on the bus was preempted. The preambleout field (below) reflects the Preemption header, which allows the user logic to handle reassembly. Valid when tlast is 1. For more details on frame preemption support, see the related information below.
rx_preambleout_<N>[55:0]	O	This signal is used to convey extracted preamble + SFD values when the custom preamble mode is enabled. It is also used when Frame Preemption is enabled. The rx_preambleout signal is provided during the first transfer of data. When enabled, the six preamble bytes followed by one SFD bytes carried by a packet's 64B/66B Start code are provided.
rx_ptp_rsfec_offset_out_<N>[6:0]	O	Ingress RSFEC codeword offset of start of packet. This is only valid when RSFEC is enabled.

Port Name	I/O	Description
rx_ptp_tstamp_out_<N>[54:0]	O	Ingress timestamp. ✎ **Note:** This signal is also shared with the Flex I/F. When used in Flex I/F mode (ctl_pcs_rx_ts_en_<N>==1), the timestamp is valid for the first word of the data.

Related Information

Frame Preemption Support

10G Signaling

10G operation necessitates a 32-bit data bus. So, half of the `axis_tdata<N>` signal is unused (Bits[63:32]). Additionally, only even-numbered `tdata` signals are used. Up to four 10G clients can be active.

Table: 10G Non-Segmented Signaling for 32 Bits

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[31:0]	rx_axis_tdata0[31:0]
		tkeep	rx_axis_tkeep_user_0[3:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[31:0]	tx_axis_tdata0[31:0]
		tkeep	tx_axis_tkeep_user_0[3:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Client	Direction	Function	Signaling
1	RX	valid	rx_axis_tvalid_1
		last	rx_axis_tlast_1
		data[31:0]	rx_axis_tdata2[31:0]
		tkeep	rx_axis_tkeep_user_2[3:0]
		err	rx_axis_tkeep_user_2[8]
		preempt	rx_axis_tkeep_user_2[9]
		resume	rx_axis_tkeep_user_2[10]
		preamble_out	rx_preambleout_1[55:0]
	TX	ready	tx_axis_tready_1
		valid	tx_axis_tvalid_1
		last	tx_axis_tlast_1
		data[31:0]	tx_axis_tdata2[31:0]
		tkeep	tx_axis_tkeep_user_2[3:0]
		err	tx_axis_tkeep_user_2[8]
		preempt	tx_axis_tkeep_user_2[9]
		resume	tx_axis_tkeep_user_2[10]
		preamble_in	tx_axis_preamblein_1[55:0]

Client	Direction	Function	Signaling
2	RX	valid	rx_axis_tvalid_2
		last	rx_axis_tlast_2
		data[31:0]	rx_axis_tdata4[31:0]
		tkeep	rx_axis_tkeep_user_4[3:0]
		err	rx_axis_tkeep_user_4[8]
		preempt	rx_axis_tkeep_user_4[9]
		resume	rx_axis_tkeep_user_4[10]
		preamble_out	rx_preambleout_2[55:0]
	TX	ready	tx_axis_tready_2
		valid	tx_axis_tvalid_2
		last	tx_axis_tlast_2
		data[31:0]	tx_axis_tdata4[31:0]
		tkeep	tx_axis_tkeep_user_4[3:0]
		err	tx_axis_tkeep_user_4[8]
		preempt	tx_axis_tkeep_user_4[9]
		resume	tx_axis_tkeep_user_4[10]
		preamble_in	tx_axis_preamblein_2[55:0]

Client	Direction	Function	Signaling
3	RX	valid	rx_axis_tvalid_3
		last	rx_axis_tlast_3
		data[31:0]	rx_axis_tdata6[31:0]
		tkeep	rx_axis_tkeep_user_6[3:0]
		err	rx_axis_tkeep_user_6[8]
		preempt	rx_axis_tkeep_user_6[9]
		resume	rx_axis_tkeep_user_6[10]
		preamble_out	rx_preambleout_3[55:0]
	TX	ready	tx_axis_tready_3
		valid	tx_axis_tvalid_3
		last	tx_axis_tlast_3
		data[31:0]	tx_axis_tdata6[31:0]
		tkeep	tx_axis_tkeep_user_6[3:0]
		err	tx_axis_tkeep_user_6[8]
		preempt	tx_axis_tkeep_user_6[9]
		resume	tx_axis_tkeep_user_6[10]
		preamble_in	tx_axis_preamblein_3[55:0]

25G Signaling

25G operation can use either a 64/128-bit data bus depending on the clock mode that is in use. Up to four 25G clients can be active.

Table: 25G Non-Segmented Signaling for 64 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		tkeep	rx_axis_tkeep_user_0[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		tkeep	tx_axis_tkeep_user_0[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Client	Direction	Function	Signaling
1	RX	valid	rx_axis_tvalid_1
		last	rx_axis_tlast_1
		data[63:0]	rx_axis_tdata2[63:0]
		tkeep	rx_axis_tkeep_user_2[7:0]
		err	rx_axis_tkeep_user_2[8]
		preempt	rx_axis_tkeep_user_2[9]
		resume	rx_axis_tkeep_user_2[10]
		preamble_out	rx_preambleout_1[55:0]
	TX	ready	tx_axis_tready_1
		valid	tx_axis_tvalid_1
		last	tx_axis_tlast_1
		data[63:0]	tx_axis_tdata2[63:0]
		tkeep	tx_axis_tkeep_user_2[7:0]
		err	tx_axis_tkeep_user_2[8]
		preempt	tx_axis_tkeep_user_2[9]
		resume	tx_axis_tkeep_user_2[10]
		preamble_in	tx_axis_preamblein_1[55:0]

Client	Direction	Function	Signaling
2	RX	valid	rx_axis_tvalid_2
		last	rx_axis_tlast_2
		data[63:0]	rx_axis_tdata4[63:0]
		tkeep	rx_axis_tkeep_user_4[7:0]
		err	rx_axis_tkeep_user_4[8]
		preempt	rx_axis_tkeep_user_4[9]
		resume	rx_axis_tkeep_user_4[10]
		preamble_out	rx_preambleout_2[55:0]
	TX	ready	tx_axis_tready_2
		valid	tx_axis_tvalid_2
		last	tx_axis_tlast_2
		data[63:0]	tx_axis_tdata4[63:0]
		tkeep	tx_axis_tkeep_user_4[7:0]
		err	tx_axis_tkeep_user_4[8]
		preempt	tx_axis_tkeep_user_4[9]
		resume	tx_axis_tkeep_user_4[10]
		preamble_in	tx_axis_preamblein_2[55:0]

Client	Direction	Function	Signaling
3	RX	valid	rx_axis_tvalid_3
		last	rx_axis_tlast_3
		data[63:0]	rx_axis_tdata6[63:0]
		tkeep	rx_axis_tkeep_user_6[7:0]
		err	rx_axis_tkeep_user_6[8]
		preempt	rx_axis_tkeep_user_6[9]
		resume	rx_axis_tkeep_user_6[10]
		preamble_out	rx_preambleout_3[55:0]
	TX	ready	tx_axis_tvalid_3
		valid	tx_axis_tlast_3
		last	tx_axis_tready_3
		data[63:0]	tx_axis_tdata6[63:0]
		tkeep	tx_axis_tkeep_user_6[7:0]
		err	tx_axis_tkeep_user_6[8]
		preempt	tx_axis_tkeep_user_6[9]
		resume	tx_axis_tkeep_user_6[10]
		preamble_in	tx_axis_preamblein_3[55:0]

Table: 25G Non-Segmented Signaling for 128 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		data[127:64]	rx_axis_tdata1[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_1[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		data[127:64]	tx_axis_tdata1[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_1[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Client	Direction	Function	Signaling
1	RX	valid	rx_axis_tvalid_1
		last	rx_axis_tlast_1
		data[63:0]	rx_axis_tdata2[63:0]
		data[127:64]	rx_axis_tdata3[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_2[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_3[7:0]
		err	rx_axis_tkeep_user_2[8]
		preempt	rx_axis_tkeep_user_2[9]
		resume	rx_axis_tkeep_user_2[10]
		preamble_out	rx_preambleout_1[55:0]
	TX	ready	tx_axis_tready_1
		valid	tx_axis_tvalid_1
		last	tx_axis_tlast_1
		data[63:0]	tx_axis_tdata2[63:0]
		data[127:64]	tx_axis_tdata3[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_2[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_3[7:0]
		err	tx_axis_tkeep_user_2[8]
		preempt	tx_axis_tkeep_user_2[9]
		resume	tx_axis_tkeep_user_2[10]
		preamble_in	tx_axis_preamblein_1[55:0]

Client	Direction	Function	Signaling
2	RX	valid	rx_axis_tvalid_2
		last	rx_axis_tlast_2
		data[63:0]	rx_axis_tdata4[63:0]
		data[127:64]	rx_axis_tdata5[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_4[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_5[7:0]
		err	rx_axis_tkeep_user_4[8]
		preempt	rx_axis_tkeep_user_4[9]
		resume	rx_axis_tkeep_user_4[10]
		preamble_out	rx_preambleout_2[55:0]
	TX	ready	tx_axis_tready_2
		valid	tx_axis_tvalid_2
		last	tx_axis_tlast_2
		data[63:0]	tx_axis_tdata4[63:0]
		data[127:64]	tx_axis_tdata5[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_4[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_5[7:0]
		err	tx_axis_tkeep_user_4[8]
		preempt	tx_axis_tkeep_user_4[9]
		resume	tx_axis_tkeep_user_4[10]
		preamble_in	tx_axis_preamblein_2[55:0]

Client	Direction	Function	Signaling
3	RX	valid	rx_axis_tvalid_3
		last	rx_axis_tlast_3
		data[63:0]	rx_axis_tdata6[63:0]
		data[127:64]	rx_axis_tdata7[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_6[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_7[7:0]
		err	rx_axis_tkeep_user_6[8]
		preempt	rx_axis_tkeep_user_6[9]
		resume	rx_axis_tkeep_user_6[10]
		preamble_out	rx_preambleout_3[55:0]
	TX	ready	tx_axis_tvalid_3
		valid	tx_axis_tlast_3
		last	tx_axis_tlast_3
		data[63:0]	tx_axis_tdata6[63:0]
		data[127:64]	tx_axis_tdata7[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_6[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_7[7:0]
		err	tx_axis_tkeep_user_6[8]
		preempt	tx_axis_tkeep_user_6[9]
		resume	tx_axis_tkeep_user_6[10]
		preamble_in	tx_axis_preamblein_3[55:0]

40G Signaling

40G operation is only supported on Port 0. The bus can operate in 128/256-bit modes.

Table: 40G Non-Segmented Signaling for 128 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		data[127:64]	rx_axis_tdata1[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_1[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		data[127:64]	tx_axis_tdata1[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_1[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Table: 40G Non-Segmented Signaling for 256 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		data[127:64]	rx_axis_tdata1[63:0]
		data[191:128]	rx_axis_tdata2[63:0]
		data[255:192]	rx_axis_tdata3[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	rx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	rx_axis_tkeep_user_3[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		data[127:64]	tx_axis_tdata1[63:0]
		data[191:128]	tx_axis_tdata2[63:0]
		data[255:192]	tx_axis_tdata3[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	tx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	tx_axis_tkeep_user_3[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

50G Signaling

50G operation requires either a 128/256-bit interface. Up to two 50G clients can be active.

Table: 50G Non-Segmented Signaling for 128 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		data[127:64]	rx_axis_tdata1[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_1[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		data[127:64]	tx_axis_tdata1[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_1[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Client	Direction	Function	Signaling
2	RX	valid	rx_axis_tvalid_2
		last	rx_axis_tlast_2
		data[63:0]	rx_axis_tdata4[63:0]
		data[127:64]	rx_axis_tdata5[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_4[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_5[7:0]
		err	rx_axis_tkeep_user_4[8]
		preempt	rx_axis_tkeep_user_4[9]
		resume	rx_axis_tkeep_user_4[10]
		preamble_out	rx_preambleout_2[55:0]
	TX	ready	tx_axis_tready_2
		valid	tx_axis_tvalid_2
		last	tx_axis_tlast_2
		data[63:0]	tx_axis_tdata4[63:0]
		data[127:64]	tx_axis_tdata5[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_4[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_5[7:0]
		err	tx_axis_tkeep_user_4[8]
		preempt	tx_axis_tkeep_user_4[9]
		resume	tx_axis_tkeep_user_4[10]
		preamble_in	tx_axis_preamblein_2[55:0]

Table: 50G Non-Segmented Signaling for 256 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		data[127:64]	rx_axis_tdata1[63:0]
		data[191:128]	rx_axis_tdata2[63:0]
		data[255:192]	rx_axis_tdata3[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	rx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	rx_axis_tkeep_user_3[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		data[127:64]	tx_axis_tdata1[63:0]
		data[191:128]	tx_axis_tdata2[63:0]
		data[255:192]	tx_axis_tdata3[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	tx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	tx_axis_tkeep_user_3[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Client	Direction	Function	Signaling
2	RX	valid	rx_axis_tvalid_2
		last	rx_axis_tlast_2
		data[63:0]	rx_axis_tdata4[63:0]
		data[127:64]	rx_axis_tdata5[63:0]
		data[191:128]	rx_axis_tdata6[63:0]
		data[255:192]	rx_axis_tdata7[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_4[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_5[7:0]
		tkeep[23:16]	rx_axis_tkeep_user_6[7:0]
		tkeep[31:24]	rx_axis_tkeep_user_7[7:0]
		err	rx_axis_tkeep_user_4[8]
		preempt	rx_axis_tkeep_user_4[9]
		resume	rx_axis_tkeep_user_4[10]
		preamble_out	rx_preambleout_2[55:0]
	TX	ready	tx_axis_tready_2
		valid	tx_axis_tvalid_2
		last	tx_axis_tlast_2
		data[63:0]	tx_axis_tdata4[63:0]
		data[127:64]	tx_axis_tdata5[63:0]
		data[191:128]	tx_axis_tdata6[63:0]
		data[255:192]	tx_axis_tdata7[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_4[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_5[7:0]
		tkeep[23:16]	tx_axis_tkeep_user_6[7:0]
		tkeep[31:24]	tx_axis_tkeep_user_7[7:0]
		err	tx_axis_tkeep_user_4[8]
		preempt	tx_axis_tkeep_user_4[9]
		resume	tx_axis_tkeep_user_4[10]
		preamble_in	tx_axis_preamblein_2[55:0]

100G Signaling

In 100G mode, the bus can operate either in 256/384-bit modes. Only one 100G client can be active, as Client 0.

Table: 100G Non-Segmented Signaling for 256 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		data[127:64]	rx_axis_tdata1[63:0]
		data[191:128]	rx_axis_tdata2[63:0]
		data[255:192]	rx_axis_tdata3[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	rx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	rx_axis_tkeep_user_3[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		data[127:64]	tx_axis_tdata1[63:0]
		data[191:128]	tx_axis_tdata2[63:0]
		data[255:192]	tx_axis_tdata3[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	tx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	tx_axis_tkeep_user_3[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Table: 100G Non-Segmented Signaling for 384 Bits

Client	Direction	Function	Signaling

Client	Direction	Function	Signaling
0	RX	valid	rx_axis_tvalid_0
		last	rx_axis_tlast_0
		data[63:0]	rx_axis_tdata0[63:0]
		data[127:64]	rx_axis_tdata1[63:0]
		data[191:128]	rx_axis_tdata2[63:0]
		data[255:192]	rx_axis_tdata3[63:0]
		data[319:256]	rx_axis_tdata4[63:0]
		data[383:320]	rx_axis_tdata5[63:0]
		tkeep[7:0]	rx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	rx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	rx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	rx_axis_tkeep_user_3[7:0]
		tkeep[39:32]	rx_axis_tkeep_user_4[7:0]
		tkeep[47:40]	rx_axis_tkeep_user_5[7:0]
		err	rx_axis_tkeep_user_0[8]
		preempt	rx_axis_tkeep_user_0[9]
		resume	rx_axis_tkeep_user_0[10]
		preamble_out	rx_preambleout_0[55:0]

Client	Direction	Function	Signaling
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		last	tx_axis_tlast_0
		data[63:0]	tx_axis_tdata0[63:0]
		data[127:64]	tx_axis_tdata1[63:0]
		data[191:128]	tx_axis_tdata2[63:0]
		data[255:192]	tx_axis_tdata3[63:0]
		data[319:256]	tx_axis_tdata4[63:0]
		data[383:320]	tx_axis_tdata5[63:0]
		tkeep[7:0]	tx_axis_tkeep_user_0[7:0]
		tkeep[15:8]	tx_axis_tkeep_user_1[7:0]
		tkeep[23:16]	tx_axis_tkeep_user_2[7:0]
		tkeep[31:24]	tx_axis_tkeep_user_3[7:0]
		tkeep[39:32]	tx_axis_tkeep_user_4[7:0]
		tkeep[47:40]	tx_axis_tkeep_user_5[7:0]
		err	tx_axis_tkeep_user_0[8]
		preempt	tx_axis_tkeep_user_0[9]
		resume	tx_axis_tkeep_user_0[10]
		preamble_in	tx_axis_preamblein_0[55:0]

Segmented Mode

Segmented 100G mode uses a 384-bit datapath. This mode is limited to a single Ethernet client (Client 0) and consequently <N> = 0 for the various `tx_axis_*_<N>` signals. Higher numbered signals such as `tx_axis_tvalid_(1-3)`, `tx_axis_tready_(1-3)`, etc., are unused and should be driven to 0.

✎ **Note:** `tx_axis_tlast` and `rx_axis_tlast` signals are not used in the segmented mode.

Table: Segmented AXI4-Stream Interface Signal Descriptions – TX Direction

Port Name	I/O	Description
tx_axis_tvalid_0	I	Asserted by user logic to indicate that the current bus cycle is valid.
tx_axis_tready_0	O	Driven by the MRMAC to indicate that the interface is able to accept the data currently present on the bus. If the tready signal deasserts during a transfer the user logic must hold the current data on the bus until tready is asserted again.
tx_axis_tdata<M>[63:0]	I	Segmented packet data, in Little-Endian order.

Port Name	I/O	Description
tx_axis_tkeep_user_<M>[0]	I	ENA. Setting this input to 1 indicates that the associated tx_axis_tdata(0-5) signal contains valid data, and that the other flags on the associated tx_axis_tkeep_user_* signals are valid. When set to 0, all the corresponding tkeep_user flags are ignored by the interface.
tx_axis_tkeep_user_<M>[1]	I	SOP. Setting this input to 1 indicates that the associated tx_axis_tdata(0-5) segment contains the beginning of a new packet. Only valid when ENA is 1.
tx_axis_tkeep_user_<M>[2]	I	EOP. Setting this input to 1 indicates that the associated tx_axis_tdata(0-5) segment contains the last byte of a packet transfer. Only valid when ENA is 1.
tx_axis_tkeep_user_<M>[3]	I	ERR. Setting this input to 1 indicates the packet in transfer is to be aborted with an error. The packet is marked poisoned with an invalid FCS, as is typical in cut-through architectures. This flag is only valid in a segment where both EOP and ENA are set to 1.
tx_axis_tkeep_user_<M>[6:4]	I	MTY[2:0]. Empty indicates the number of empty (unused) bytes in the final (EOP) segment. Valid only in the segment where EOP and ENA are asserted to 1. 3'd0 - 8 bytes valid 3'd1 - 7 bytes valid ... 3'd7 - 1 byte valid
tx_axis_tkeep_user_<M>[7]	I	Preempt. When set to 1, it indicates that the packet in progress is to be preempted. When preempting a packet the interface requires that both EOP and ENA be set to 1 as well. For more details on frame preemption support, see the related information below.
tx_axis_tkeep_user_<M>[8]	I	Resume. Set to 1 when the currently transmitting packet is a resumption of a previously preempted packet. This signal is to be asserted at the beginning of new packet transfer. It is only valid in a segment where the corresponding SOP and ENA are asserted.

Port Name	I/O	Description
tx_preamblein_0[55:0]	I	This signal is used to convey preamble when custom preamble mode is enabled. It is also used when Frame Preemption is enabled. The tx_preamblein_0 signal must be provided during the first transfer of data. When enabled, seven preamble bytes must be provided. The MRMAC always sets the first byte to 0x55.
tx_ptp_1588op_in_0[1:0]	I	Select the desired PTP operation. • 2'b00 – No operation: no timestamp is taken and the frame is not modified. • 2'b01 – 1-step: a timestamp should be taken and inserted into the frame. The timestamp is also returned to the client using the additional ports of 2-step operation. • 2'b10 – 2-step: a timestamp should be taken and returned to the client using the additional ports of 2-step operation. The frame itself is not modified. • 2'b11 – Reserved.
tx_ptp_cf_offset_0[15:0]	I	This specifies the offset (in octets) to the PTP Correction field. Offset 0 corresponds to the first byte of the Ethernet Destination Address. If UDP checksum adjustment is required (tx_ptp_upd_chksum_in is set), the first byte of the checksum is assumed to be located at tx_ptp_cf_offset – 10 octets. Note, this value must be even, and for timestamping applications not following IEEE 1588 protocol but wanting to use the 1-step timestamp insertion feature, any user specified correction field offset must be a minimum of 18 bytes from the end of the packet.
tx_ptp_upd_chksum_in_0	I	Update UDP checksum: • 1'b0: PTP frame does not contain a UDP checksum • 1'b1: PTP frame does contain a UDP checksum which the core is required to recalculate. The location of the checksum is ascertained as described in the tx_ptp_cf_offset field.

Port Name	I/O	Description
tx_ptp_tag_field_in_0[15:0]	I	Identifying tag to use for egress PTP packet. This should be valid during the SOP cycle on the AXI4-Stream interface.
tx_ptp_tag_out_0[15:0]	O	Egress Tag. Valid when tx_ptp_tstamp_valid_out<N> is set.
tx_ptp_tstamp_valid_out_0	O	Indicates that Egress tag/timestamp is valid.
tx_ptp_rsfec_offset_out_0[6:0]	O	Egress RSFEC codeword offset of start of packet. This is only valid when RSFEC is enabled.
tx_ptp_tstamp_out_0[54:0]	O	Egress Timestamp. Valid when tx_ptp_tstamp_valid_out_<N> is set.

Table: Segmented AXI4-Stream Interface Signal Descriptions – RX Direction

Port Name	I/O	Description
rx_axis_tvalid_0	O	Indicates that the data and flags on the AXI4-Stream bus are valid.
rx_axis_tdata<M>[63:0]	O	Contains the data for the received packet in Little-Endian order.
rx_axis_tkeep_user_<M>[0]	O	ENA. When this output is 1, it indicates that the associated rx_axis_tdata(0-5) signal contains valid data, and that the other flags on the associated rx_axis_tkeep_user_(0-5) signal are valid. When set to 0, all the corresponding tkeep_user flags are ignored by the interface.
rx_axis_tkeep_user_<M>[1]	O	SOP. When this output is 1, it indicates that the associated rx_axis_tdata(0-5) segment contains the beginning of a new packet. Only valid when ENA is 1.
rx_axis_tkeep_user_<M>[2]	O	EOP. When this output is 1, it indicates that the associated rx_axis_tdata(0-5) segment contains the last byte of a packet transfer. Only valid when ENA is 1.
rx_axis_tkeep_user_<M>[3]	O	ERR. When this output is 1, it indicates the packet in transfer was found to contain an error. This flag is only valid in a segment where both EOP and ENA are set to 1.

Port Name	I/O	Description
rx_axis_tkeep_user_<M>[6:4]	O	MTY[2:0]. Empty indicates the number of empty (unused) bytes in the final (EOP) segment. Valid only in the segment where EOP and ENA are asserted to 1. 3'd0 – 8 bytes valid 3'd1 – 7 bytes valid ... 3'd7 – 1 byte valid
rx_axis_tkeep_user_<M>[7]	O	Preempt. When set to 1, it indicates that the packet in progress was preempted. For more details on frame preemption support, see the related information below.
rx_axis_tkeep_user_<M>[8]	O	Resume. When set to 1, it indicates that the currently active packet is a resumption of a previously preempted packet. This signal is to be asserted at the beginning of new packet transfer. It is only valid in a segment where the corresponding SOP and ENA are asserted. For more details on frame preemption support, see the related information below.
rx_preambleout_0[55:0]	O	This signal is used to convey extracted preamble + SFD values when the custom preamble mode is enabled. It is also used when Frame Preemption is enabled. The rx_preambleout signal is provided during the first transfer of data. When enabled, the six preamble bytes followed by one SFD bytes carried by a packet's 64B/66B Start code are provided..
rx_ptp_rsfec_offset_out_0[6:0]	O	Ingress RSFEC codeword offset of start of packet. This is only valid when RSFEC is enabled.
rx_ptp_tstamp_out_0[54:0]	O	Ingress timestamp. ✎ **Note:** This signal is also shared with the Flex I/F. When used in Flex I/F mode (ctl_pcs_rx_ts_en_<N>==1), the timestamp is valid for the first word of the data.

Related Information

Frame Preemption Support

The following table describes the connections that must be made for single-client 100G in the segmented mode. In segmented mode, the AXI4-Stream bus is divided up into six segments with each segment consisting of ena, sop, eop, err, mty, preempt, and resume flags, along with the corresponding 64-bit data segment.

Table: 100G Segmented Signaling for 384 Bits

Client	Direction	Function	Signaling
Segment M (M = 0 ... 5)	RX	valid	rx_axis_tvalid_0
		preamble_out	rx_preambleout_0[55:0]
		segM_data[63:0]	rx_axis_tdata_M[63:0]
		segM_ena	rx_axis_tkeep_user_M[0]
		segM_sop	rx_axis_tkeep_user_M[1]
		segM_eop	rx_axis_tkeep_user_M[2]
		segM_err	rx_axis_tkeep_user_M[3]
		segM_mty	rx_axis_tkeep_user_M[6:4]
		segM_preempt	rx_axis_tkeep_user_M[7]
		segM_resume	rx_axis_tkeep_user_M[8]
	TX	ready	tx_axis_tready_0
		valid	tx_axis_tvalid_0
		preamblein	tx_preamblein_0[55:0]
		segM_data[63:0]	tx_axis_tdata_M[63:0]
		segM_ena	tx_axis_tkeep_user_M[0]
		segM_sop	tx_axis_tkeep_user_M[1]
		segM_eop	tx_axis_tkeep_user_M[2]
		segM_err	tx_axis_tkeep_user_M[3]
		segM_mty	tx_axis_tkeep_user_M[6:4]
		segM_preempt	tx_axis_tkeep_user_M[7]
		segM_resume	tx_axis_tkeep_user_M[8]

Flexible Interface

This section describes the flexible interface (Flex I/F) signals. The following tables define the signal description section, the direction, and meaning of the various signals. The subsequent sections describe how the signals are hooked up for the various client rates.

Table: Flexible Interface Signal Descriptions – TX Direction

Port Name	I/O	Description

Port Name	I/O	Description
tx_flexif_clk[3:0]	I	Clock for the corresponding slice TX Flex Interface. The clock frequency must be the 50% of the corresponding tx_core_clk frequency for that slice.
tx_flexif_data(0-7)[65:0]	I	66-bit data retrieved from the datapath.
tx_flex_ena_(0-3)	I	Enable signal, indicating that the corresponding tx_flexif_data word is valid.
tx_flex_stall_(0-3)	O	Backpressure signal (for the corresponding TX Flex Interface slice) from the MRMAC TX PHY to the user logic. User logic must provide a corresponding gap in the data after exactly one clock cycle.
tx_flexe_almarker(0-7)	I	Indicates that the corresponding tx_flexif_data(0-7) contains an alignment maker and that the scrambler (if active) must not scramble the datapath. AMs must be left unscrambled.
stat_tx_flexif_err_(0-3)	O	Indicates that an |E| codeword was detected on the corresponding slice of the TX Flex Interface.
fec_tx_din_valid_(0-3)	I	TX FEC Data Input Valid
fec_tx_din_start_(0-3)	I	TX FEC Data Input Start
fec_tx_din_is_am_(0-3)	I	TX FEC Data Input AM

Table: Flexible Interface Signal Descriptions - RX Direction

Port Name	I/O	Description
rx_flex_data(0-7)[65:0]	O	66-bit data from the RX PHY.
rx_flex_ena_(0-3)	O	Indicates that the corresponding rx_flex_data(0-7) is valid. The data words corresponding to rx_flex_ena_(0-3) depend on the configured mode. For more details on signaling, see the following related information.
rx_flex_lane0	O	Indication that the rx_flex_data0 bus contains PCS Lane 0's data.
rx_flex_almarker(0-7)	O	Indication that the data on the corresponding rx_flex_data(0-7) bus contains an Alignment Marker. For modes that do not contain alignment markers, this indicates the first cycle after the alignment marker location.

Port Name	I/O	Description
rx_flex_bip8(0-7)	O	Indicates that the corresponding rx_flex_data(0-7) contains a BIP8 field.
fec_rx_dout_valid_(0-3)	O	RX FEC Data Output Valid
fec_rx_dout_start_(0-3)	O	RX FEC Data Output Start
fec_rx_dout_flags_(0-3)	O	RX FEC Data Output Flags
fec_rx_dout_is_am_(0-3)	O	RX FEC Data Output AM
stat_rx_local_fault_(0-3)	O	Indicates PCS logic (on the corresponding slice of the RX Flex Interface) is in local fault state.
stat_rx_internal_local_fault_(0-3)	O	Indicates PCS logic (on the corresponding slice of the RX Flex Interface) is in local fault due to internal issue such as reset or loss of alignment.
stat_rx_test_pattern_mismatch_(0-3)	O	Indicates PCS logic (on the corresponding slice of the RX Flex Interface) received test pattern mismatch when configured in PCS test_pattern mode.
stat_rx_valid_ctrl_code	O	Indicates that PCS decoder has seen a valid control code. Allows you to monitor for valid Ethernet datastream.
stat_rx_pcs_bad_code	O	Indicates the PCS decoder received a malformed 66b code word, or improper code word, and transitioned to the E state. This statistic is enabled only when the ctl_rx_flexif_select mode is configured to enable the PCS receive state-machine or EBLOCK replacement, specifically when ctl_rx_flexif_select is set to 0x3, 0x7, or 0xF.
stat_rx_hi_ber	O	Indicates that the PCS is in the High BER state as identified by IEEE 802.3 CL82 (100/50/40GE) or CL49 (10/25GE).
stat_rx_status	O	Indicates that the Port is out of fault and not in the HI BER state.
stat_rx_flexif_err	O	Indicates that a Flex I/F Error has occurred and the data and flags on the RX Flex I/F is in error. The port's RX datapath needs to be reset.

Each port of the Flex I/F resets along with the overall port logic with the assertion of the appropriate `tx_core_reset[3:0]` and `rx_core_reset[3:0]` pins. The TX direction reset by the toggling of the `tx_core_reset` and RX logic by the toggling of

that port's `rx_core_reset` pin. Asserting reset on a port's `rx_core_reset` pin alters both the PCS and the Client Monitoring functions of the Flex I/F.

Related Information

Flex I/F Active Bus Ports by Configuration

Flex I/F Active Bus Ports by Configuration

The following tables list the active data widths and signaling for the various rates.

Flex I/F Active Bus Ports for 10G/25G Operation

Table: Flex I/F Active Bus Ports for 10G/25G

Port	Data Rate	Flex I/F Active Signals		Data Rate	Flex I/F Active Signals	
		TX	RX		TX	RX
0	10GE	tx_flex_ena_0 [1] fec_tx_din_valid_0 [2] fec_tx_din_start_0 [2] fec_tx_din_is_am_0 [2] tx_flex_stall_0 tx_flex_data0	rx_flex_ena_0 [1] fec_rx_dout_valid_0 [2] fec_rx_dout_start_0 [2] fec_rx_dout_flags_0 [2] fec_rx_dout_is_am_0 [2] rx_flex_data0	25GE	tx_flex_ena_0 [1] fec_tx_din_valid_0 [2] fec_tx_din_start_0 [2] fec_tx_din_is_am_0 [2] tx_flex_stall_0 tx_flex_data0 tx_flex_data1	rx_flex_ena_0 [1] fec_rx_dout_valid_0 [2] fec_rx_dout_start_0 [2] fec_rx_dout_flags_0 [2] fec_rx_dout_is_am_0 [2] rx_flex_data0 rx_flex_data1
1	10GE	tx_flex_ena_1 [1] fec_tx_din_valid_1 [2] fec_tx_din_start_1 [2] fec_tx_din_is_am_1 [2] tx_flex_stall_1 tx_flex_data2	rx_flex_ena_1 [1] fec_rx_dout_valid_1 [2] fec_rx_dout_start_1 [2] fec_rx_dout_flags_1 [2] fec_rx_dout_is_am_1 [2] rx_flex_data2	25GE	tx_flex_ena_1 [1] fec_tx_din_valid_1 [2] fec_tx_din_start_1 [2] fec_tx_din_is_am_1 [2] tx_flex_stall_1 tx_flex_data2 tx_flex_data3	rx_flex_ena_1 [1] fec_rx_dout_valid_1 [2] fec_rx_dout_start_1 [2] fec_rx_dout_flags_1 [2] fec_rx_dout_is_am_1 [2] rx_flex_data2 rx_flex_data3

Port	Data Rate	Flex I/F Active Signals		Data Rate	Flex I/F Active Signals	
		TX	RX		TX	RX
2	10GE	tx_flex_ena_2 [1] fec_tx_din_valid_2 [2] fec_tx_din_start_2 [2] fec_tx_din_is_am_2 [2] tx_flex_stall_2 tx_flex_data4	rx_flex_ena_2 [1] fec_rx_dout_valid_2 [2] fec_rx_dout_start_2 [2] fec_rx_dout_flags_2 [2] fec_rx_dout_is_am_2 [2] rx_flex_data4	25GE	tx_flex_ena_2 [1] fec_tx_din_valid_2 [2] fec_tx_din_start_2 [2] fec_tx_din_is_am_2 [2] tx_flex_stall_2 tx_flex_data4 tx_flex_data5	rx_flex_ena_2 [1] fec_rx_dout_valid_2 [2] fec_rx_dout_start_2 [2] fec_rx_dout_flags_2 [2] fec_rx_dout_is_am_2 [2] rx_flex_data4 rx_flex_data5
3	10GE	tx_flex_ena_3 [1] fec_tx_din_valid_3 [2] fec_tx_din_start_3 [2] fec_tx_din_is_am_3 [2] tx_flex_stall_3 tx_flex_data6	rx_flex_ena_3 [1] fec_rx_dout_valid_3 [2] fec_rx_dout_start_3 [2] fec_rx_dout_flags_3 [2] fec_rx_dout_is_am_3 [2] rx_flex_data6	25GE	tx_flex_ena_3 [1] fec_tx_din_valid_3 [2] fec_tx_din_start_3 [2] fec_tx_din_is_am_3 [2] tx_flex_stall_3 tx_flex_data6 tx_flex_data7	rx_flex_ena_3 [1] fec_rx_dout_valid_3 [2] fec_rx_dout_start_3 [2] fec_rx_dout_flags_3 [2] fec_rx_dout_is_am_3 [2] rx_flex_data6 rx_flex_data7

1. Non-FEC only modes, all 66 data bits used.
2. For FEC only modes, [39:0] data bits used.

Flex I/F Active Bus Ports for 40G/50G Operation

Table: Flex I/F Active Bus Ports for 40G/50G

Port	Data Rate	Flex I/F Active Signals		Data Rate	Flex I/F Active Signals	
		TX	RX		TX	RX

Port	Data Rate	Flex I/F Active Signals		Data Rate	Flex I/F Active Signals	
		TX	RX		TX	RX
0	40GE	tx_flex_ena_0 [1] fec_tx_din_valid_0 [2] fec_tx_din_start_0 [2] fec_tx_din_is_am_0 [2] tx_flex_stall_0 tx_flex_data0 tx_flex_data1 tx_flex_data2 tx_flex_data3 tx_flex_almarker0 tx_flex_almarker1	rx_flex_ena_0 [1] fec_rx_dout_valid_0 [2] fec_rx_dout_start_0 [2] fec_rx_dout_flags_0 [2] fec_rx_dout_is_am_0 [2] rx_flex_bip80 rx_flex_bip81 rx_flex_almarker0 rx_flex_almarker1 rx_flex_data0 rx_flex_data1 rx_flex_data2 rx_flex_data3	50GE	tx_flex_ena_0 [1] fec_tx_din_valid_0 [2] fec_tx_din_start_0 [2] fec_tx_din_is_am_0 [2] tx_flex_stall_0 tx_flex_data0 tx_flex_data1 tx_flex_data2 tx_flex_data3 tx_flex_almarker0 tx_flex_almarker1 tx_flex_almarker2 tx_flex_almarker3	rx_flex_ena_0 [1] fec_rx_dout_valid_0 [2] fec_rx_dout_start_0 [2] fec_rx_dout_flags_0 [2] fec_rx_dout_is_am_0 [2] rx_flex_bip80 rx_flex_bip81 rx_flex_bip82 rx_flex_bip83 rx_flex_almarker0 rx_flex_almarker1 rx_flex_almarker2 rx_flex_almarker3 rx_flex_data0 rx_flex_data1 rx_flex_data2 rx_flex_data3

Port	Data Rate	Flex I/F Active Signals		Data Rate	Flex I/F Active Signals	
		TX	RX		TX	RX
2	N/A			50GE	tx_flex_ena_2 [1] fec_rx_dout_valid_2 [2] fec_rx_dout_start_2 [2] fec_rx_dout_flags_2 [2] fec_rx_dout_is_am_2 [2] tx_flex_stall_2 tx_flex_data4 tx_flex_data5 tx_flex_data6 tx_flex_data7 tx_flex_almarker4 tx_flex_almarker5 tx_flex_almarker6 tx_flex_almarker7	rx_flex_ena_2 [1] fec_rx_dout_valid_2 [2] fec_rx_dout_start_2 [2] fec_rx_dout_flags_2 [2] fec_rx_dout_is_am_2 [2] rx_flex_almarker4 rx_flex_almarker5 rx_flex_almarker6 rx_flex_almarker7 rx_flex_bip84 rx_flex_bip85 rx_flex_bip86 rx_flex_bip87 rx_flex_data4 rx_flex_data5 rx_flex_data6 rx_flex_data7

1. Non-FEC only modes, all 66 data bits used.
2. For FEC only modes, [39:0] data bits used.

Flex I/F Active Bus Ports for 100G Operation

Table: Flex I/F Active Bus Ports for 100G

Port	Data Rate	Flex I/F Active Signals		Data Rate	Flex I/F Active Signals	
		TX	RX		TX	RX

Port	Data Rate	Flex I/F Active Signals		Data Rate	Flex I/F Active Signals	
		TX	RX		TX	RX
0	100GE	tx_flex_ena_0 [1] fec_tx_din_valid_0 [2] fec_tx_din_start_0 [2] fec_tx_din_is_am_0 [2] tx_flex_stall_0 tx_flex_data0 tx_flex_data1 tx_flex_data2 tx_flex_data3 tx_flex_data4 tx_flex_almarker0 tx_flex_almarker1 tx_flex_almarker2 tx_flex_almarker3 tx_flex_almarker4	rx_flex_ena_0 [1] fec_rx_dout_valid_0 [2] fec_rx_dout_start_0 [2] fec_rx_dout_flags_0 [2] fec_rx_dout_is_am_0 [2] rx_flex_lane0 rx_flex_bip80 rx_flex_bip81 rx_flex_bip82 rx_flex_bip83 rx_flex_bip84 rx_flex_almarker0 rx_flex_almarker1 rx_flex_almarker2 rx_flex_almarker3 rx_flex_almarker4 rx_flex_data0 rx_flex_data1 rx_flex_data2 rx_flex_data3 rx_flex_data4		N/A	

1. Non-FEC only modes, all 66 data bits are used.
2. For FEC only modes, [39:0] data bits are used.

Transceiver (SerDes) Interface

The clock and data connections between the MRMAC transceiver interface and the GT transceivers varies depending on the selected GT technology and the configured operating mode. This section describes the connectivity between the MRMAC and the GT Transceivers.

Table: Transceiver Interface Signal Descriptions

Port Name	I/O	Clock	Description
rx_serdes_clk[3:0]	I	N/A	Per-lane receive GT clock.
rx_alt_serdes_clk[3:0]	I	N/A	Per-lane receive GT clock (x2 width mode).
tx_core_clk[3:0]	I	N/A	Per-Port transmit GT clock. The same clock is used for all TX GT lanes for a given Port0-3. This clock is also used in the port core logic.
tx_alt_serdes_clk[3:0]	I	N/A	Per-lane transmit GT clock (x2 width mode).
rx_serdes_reset[3:0]	I	rx_serdes_clk[3:0]	Per-lane RX lane-logic reset.
tx_serdes_reset[3:0]	I	rx_serdes_clk[3:0]	Per-lane RX lane-logic reset.
rx_serdes_data(0-3)[79:0]	I	rx_serdes_clk[3:0], rx_alt_serdes_clk[3:0]	Receive GT data. This data is clocked by the associated rx_serdes_clk or the rx_alt_serdes_clk depending on operating mode.
tx_serdes_data(0-3)[79:0]	O	tx_core_clk[3:0], tx_alt_serdes_clk[3:0]	Transmit GT data. This data is clocked by the associated tx_core_clk or the tx_alt_serdes_clk depending on operating mode.

Transceiver Signaling

To minimize consumption of routing resources, the various clock and data signals take on different functions depending on the configured MRMAC operating mode. The following table details the meaning of the various clock and data signals for the given MRMAC configuration settings. For FEC only modes, data/clocks are shared with the transceiver interface.

Table: Transceiver Signaling

MRMAC Configuration	Active MRMAC Port	Number of Active GT Lanes	Active GT Clocks	Active GT Interface Data Bus

MRMAC Configuration	Active MRMAC Port	Number of Active GT Lanes	Active GT Clocks	Active GT Interface Data Bus
10/25GE Narrow SerDes	0	1	rx_serdes_clk[0]	rx_serdes_data0[39:0]
			tx_core_clk[0]	tx_serdes_data0[39:0]
	1	1	rx_serdes_clk[1]	rx_serdes_data1[39:0]
			tx_core_clk[1]	tx_serdes_data1[39:0]
	2	1	rx_serdes_clk[2]	rx_serdes_data2[39:0]
			tx_core_clk[2]	tx_serdes_data2[39:0]
	3	1	rx_serdes_clk[3]	rx_serdes_data3[39:0]
			tx_core_clk[3]	tx_serdes_data3[39:0]

MRMAC Configuration	Active MRMAC Port	Number of Active GT Lanes	Active GT Clocks	Active GT Interface Data Bus
10/25GE Wide SerDes	0	1	rx_serdes_clk[0] rx_alt_serdes_clk[0]	rx_serdes_data0[79:0] fec_rx_din_start_0
			tx_core_clk[0] tx_alt_serdes_clk[0]	tx_serdes_data0[79:0] fec_tx_dout_start_0
	1	1	rx_serdes_clk[1] rx_alt_serdes_clk[1]	rx_serdes_data1[79:0] fec_rx_din_start_1
			tx_core_clk[1] tx_alt_serdes_clk[1]	tx_serdes_data1[79:0] fec_tx_dout_start_1
	2	1	rx_serdes_clk[2] rx_alt_serdes_clk[2]	rx_serdes_data2[79:0] fec_rx_din_start_2
			tx_core_clk[2] tx_alt_serdes_clk[2]	tx_serdes_data2[79:0] fec_tx_dout_start_2
	3	1	rx_serdes_clk[3] rx_alt_serdes_clk[3]	rx_serdes_data3[79:0] fec_rx_din_start_3
			tx_core_clk[3] tx_alt_serdes_clk[3]	tx_serdes_data3[79:0] fec_tx_dout_start_3

MRMAC Configuration	Active MRMAC Port	Number of Active GT Lanes	Active GT Clocks	Active GT Interface Data Bus
40GE Narrow SerDes	0	4	rx_serdes_clk[0] rx_serdes_clk[1] rx_serdes_clk[2] rx_serdes_clk[3]	rx_serdes_data0[15:0] rx_serdes_data1[15:0] rx_serdes_data2[15:0] rx_serdes_data3[15:0]
			tx_core_clk[0]	tx_serdes_data0[15:0] tx_serdes_data1[15:0] tx_serdes_data2[15:0] tx_serdes_data3[15:0]
40GE Wide SerDes	0	4	rx_serdes_clk[0] rx_serdes_clk[1] rx_serdes_clk[2] rx_serdes_clk[3] rx_alt_serdes_clk[0] rx_alt_serdes_clk[1] rx_alt_serdes_clk[2] rx_alt_serdes_clk[3]	rx_serdes_data0[32:0] rx_serdes_data1[32:0] rx_serdes_data2[32:0] rx_serdes_data3[32:0]
			tx_core_clk[0] tx_alt_serdes_clk[0] tx_alt_serdes_clk[1] tx_alt_serdes_clk[2] tx_alt_serdes_clk[3]	tx_serdes_data0[32:0] tx_serdes_data1[32:0] tx_serdes_data2[32:0] tx_serdes_data3[32:0]

MRMAC Configuration	Active MRMAC Port	Number of Active GT Lanes	Active GT Clocks	Active GT Interface Data Bus
50GE Narrow SerDes	0	1	rx_serdes_clk[0]	rx_serdes_data0[39:0]
			tx_core_clk[0]	tx_serdes_data0[39:0]
		2	rx_serdes_clk[1]	rx_serdes_data1[39:0]
			N/A	tx_serdes_data1[39:0]
	2	1	rx_serdes_clk[2]	rx_serdes_data2[39:0]
			tx_core_clk[2]	tx_serdes_data2[39:0]
		2	rx_serdes_clk[3]	rx_serdes_data3[39:0]
			N/A	tx_serdes_data3[39:0]
50GE Wide SerDes	0	2	rx_serdes_clk[0] rx_alt_serdes_clk[0]	rx_serdes_data0[79:0] fec_rx_din_start_0
			tx_core_clk[0] tx_alt_serdes_clk[0]	tx_serdes_data0[79:0] fec_tx_dout_start_0
	2	2	rx_serdes_clk[2] rx_alt_serdes_clk[2]	rx_serdes_data2[79:0] fec_rx_din_start_2
			tx_core_clk[2] tx_alt_serdes_clk[2]	tx_serdes_data2[79:0] fec_tx_dout_start_2

MRMAC Configuration	Active MRMAC Port	Number of Active GT Lanes	Active GT Clocks	Active GT Interface Data Bus
100GE Narrow SerDes	0	4	rx_serdes_clk[0] rx_serdes_clk[1] rx_serdes_clk[2] rx_serdes_clk[3]	rx_serdes_data0[39:0] rx_serdes_data1[39:0] rx_serdes_data2[39:0] rx_serdes_data3[39:0]
			tx_core_clk[0]	tx_serdes_data0[39:0] tx_serdes_data1[39:0] tx_serdes_data2[39:0] tx_serdes_data3[39:0]
100GE Wide SerDes	0	4	rx_serdes_clk[0] rx_serdes_clk[1] rx_serdes_clk[2] rx_serdes_clk[3] rx_alt_serdes_clk[0] rx_alt_serdes_clk[1] rx_alt_serdes_clk[2]	rx_serdes_data0[79:0] rx_serdes_data1[79:0] rx_serdes_data2[79:0] rx_serdes_data3[79:0]
			tx_core_clk[0] tx_alt_serdes_clk[0]	tx_serdes_data0[79:0] tx_serdes_data1[79:0] tx_serdes_data2[79:0] tx_serdes_data3[79:0]

1. The `fec_tx_dout_start` and `fec_rx_din_start` are only used in the FEC only modes and clocked by `alt_serdes_clk`.

The design must satisfy several rules when connecting the MRMAC to the transceivers. See the Transceiver Selection Rules section for details.

AXI4-Lite Register Interface

The MRMAC contains a soft logic 32-bit AXI4-Lite interface block to allow access to the integrated IP's APB3 interface. Through the AXI4-Lite interface, you can access the internal configuration, status, and statistics registers.

For more details on the AXI4-Lite interface, see the AXI to APB Bridge LogiCORE IP Product Guide (PG073).

Table: AXI4-Lite Interface Signal Descriptions

Port Name	I/O	Description
s_axi_aclk	I	This clock is used for both the AXI4-Lite bridge Soft Logic and MRMAC APB3 port clock. ✎ **Note:** The s_axi_aclk must be present for the MRMAC to function. If this clock is interrupted, the MRMAC enters an error state.
s_axi_areset	I	Active-High reset for the AXI4-Lite port. Asserting this reset alters the Port control logic and stops any in-flight writes/reads. It does not reset the internal configuration registers with exceptions listed below.
s_axi_*	I/O	See the AXI to APB Bridge LogiCORE IP Product Guide (PG073) and the Vivado Design Suite: AXI Reference Guide (UG1037).
s_axi_pslverr	O	MRMAC AXI4-Lite slave error indication. This signal is Low during successful operation. When High, the signal indicates one of two issues has occurred: 1. An AXI bus protocol error has occurred. 2. There was a read/write attempt timeout failure during the transaction to the Port registers. This type of failure indicates that the MRMAC block could be in reset, or similar unrecoverable failure. The AXI4-Lite port and the MRMAC must be reset following such a failure. ✎ **Note:** Writes or reads to non-existent/invalid register locations, nor a port being held in reset does not trigger a slave error indication. A set of debug signals stat_rsvd_out[179:178] are provided for diagnostics.

Port Name	I/O	Description
pm_tick[3:0]	I	Performance monitoring statistics tick signal.
pm_rdy[3:0]	O	Performance monitoring statistics counters ready flag. These per-port flags indicate that the internal statistics engine has completed a user-directed update cycle and are ready for access.

A complete description of the register map and the individual registers can be found in the Register Space section.

When the statistics engine is configured in extended mode by setting the `ctl_counter_extend_<N>` field of the per-port MODE_REG register to a 1, then the Statistics counter extension port is used to provide an increment for a set of external soft logic counters. The `ext_count_addr` bus cycles through all counters. When the internal 48-bit counter pointed to by the `ext_count_addr` address overflows, the increment signal `ext_count_inc` pulses to a 1.

Table: Statistics Counter Extension Signal Descriptions

Port Name	I/O	Description
ext_count_addr[8:0]	O	Indicates the address of the counter to increment with the value on the ext_count_inc bus.
ext_count_flags[3:0]	O	Indicates that a port has been issued a tick event at this point in the register rotation.
ext_count_inc	O	Increment value for the counter addressed by ext_count_addr bus.

Related Information

Statistics Monitoring
Register Space

AXI4-Lite Clock and Reset

The AXI4-Lite interface has its own clock and reset. The AXI4-Lite clock (`s_axi_aclk`) is independent of the remainder of the MRMAC clocks and can be any frequency to a maximum of 300 MHz.

✎ **Important:** The AXI4-Lite clock must be present and stable for the MRMAC to operate. An interruption in the AXI4-Lite clock is likely to result in an unrecoverable internal MRMAC error. When the AXI4-Lite clock has returned and is stable, this requires a full reset-sequence to bring the MRMAC back to a stable condition.

Asserting the AXI4-Lite `s_axi_areset` pin results in:

- A reset of the AXI4-Lite port and MRMAC APB3 control logic, stopping any in-flight writes/reads.
- A reset of the current accumulated statistics and status registers for all slices. If only the AXI4-Lite port reset is asserted, then the internal statistics and status engines immediately begins monitoring the MRMAC operation and restarts the accumulations of the statistics counters.
- A reset of all ports in User Scratch registers: USER_REG_0-3 to `0x0`.
- A reset of all ports in Reset registers: RESET_REG_0-3 to `0x0`.

An AXI4-Lite reset does not reset the internal configuration registers.

In addition to the `s_axi_aclk`, the statistics and status registers require a port's core clock (`rx_core_clk[N]` and `tx_core_clk[N]`) be active and stable to operate. Unless the entire MRMAC is held in reset, all AXI4-Lite write/read transactions complete without asserting `s_axi_pslverr`. If `s_axi_pslv_err` is asserted, an event, such as an unstable AXI4-Lite or core clock during a transaction has occurred, and the MRMAC must be reset. If an RX statistics register read is required while the `rx_serdes_clk[N]` might be de-stabilized (that is, due to a GT RX reset), you can opt to clock the `rx_core_clk[N]` off of the `tx_core_clk[N]` domain.

If the AXI4-Lite port attempts a read or write access to the memory space of a disabled port (for example, ports with their clock tied to 0/1 or which are held in reset) or to a non-existent register address, the transaction completes and one of the debug signals `stat_rsvd_out[179:178]` toggle for one `s_axi_aclk` cycle. In these cases, the value on the AXI4-Lite port's output is invalid.

MRMAC Control Ports

In addition to the AXI4-Lite configuration registers, the MRMAC IP requires some control inputs illustrated in the following table. All control ports are inputs.

Table: MRMAC Control Port Descriptions

Port Name	Clock Domain	Description
ctl_rx_fec_fc32_ra_mode_0	s_axi_aclk	RX_RESET needed after change to CTL_RX_FEC_FC32_RA_MODE_*.
ctl_rx_fec_fc32_ra_mode_1		
ctl_rx_fec_fc32_ra_mode_2		
ctl_rx_fec_fc32_ra_mode_3		
ctl_rx_pause_ack_0[8:0]	rx_axi_clk[0]	RX Pause Processing Acknowledge Each bit of ctl_rx_pause_ack_N[8:0] is associated with a pause priority (Bit[8] is for global pause signaling).
ctl_rx_pause_ack_1[8:0]		
ctl_rx_pause_ack_2[8:0]	rx_axi_clk[1]	
ctl_rx_pause_ack_3[8:0]		
ctl_rx_pause_enable_0[8:0]	rx_axi_clk[0]	RX Pause Packet Enable Each bit of ctl_rx_pause_enable_N[8:0] is associated with a pause priority (Bit[8] is for global pause signaling).
ctl_rx_pause_enable_1[8:0]		
ctl_rx_pause_enable_2[8:0]	rx_axi_clk[1]	
ctl_rx_pause_enable_3[8:0]		
ctl_rx_ptp_st_adjust_0[31:0]	rx_ts_clk[0]	Specifies System Timer Adjust
ctl_rx_ptp_st_adjust_1[31:0]	rx_ts_clk[1]	
ctl_rx_ptp_st_adjust_2[31:0]	rx_ts_clk[2]	
ctl_rx_ptp_st_adjust_3[31:0]	rx_ts_clk[3]	
ctl_rx_ptp_st_adjust_type_0[1:0]	rx_ts_clk[0]	Specifies the desired System Timer Adjustment Type
ctl_rx_ptp_st_adjust_type_1[1:0]	rx_ts_clk[1]	
ctl_rx_ptp_st_adjust_type_2[1:0]	rx_ts_clk[2]	
ctl_rx_ptp_st_adjust_type_3[1:0]	rx_ts_clk[3]	

Port Name	Clock Domain	Description
ctl_rx_ptp_st_adjust_vld_0	rx_ts_clk[0]	Specifies the System Timer Adjust is valid.
ctl_rx_ptp_st_adjust_vld_1	rx_ts_clk[1]	
ctl_rx_ptp_st_adjust_vld_2	rx_ts_clk[2]	
ctl_rx_ptp_st_adjust_vld_3	rx_ts_clk[3]	
ctl_rx_ptp_st_overwrite_0	rx_ts_clk[0]	System Timer Overwrite
ctl_rx_ptp_st_overwrite_1	rx_ts_clk[1]	
ctl_rx_ptp_st_overwrite_2	rx_ts_clk[2]	
ctl_rx_ptp_st_overwrite_3	rx_ts_clk[3]	
ctl_rx_ptp_st_sync_0	rx_ts_clk[0]	System Timer Sync
ctl_rx_ptp_st_sync_1	rx_ts_clk[1]	
ctl_rx_ptp_st_sync_2	rx_ts_clk[2]	
ctl_rx_ptp_st_sync_3	rx_ts_clk[3]	
ctl_rx_ptp_systemtimer_0[54:0]	rx_ts_clk[0]	System timer input in units of 2^{-8} nanoseconds (unsigned). This field maps to Bits[62:8] of the correction field format in IEEE 1588-2008 (with the bottom 8 bits set to 0).
ctl_rx_ptp_systemtimer_1[54:0]	rx_ts_clk[1]	
ctl_rx_ptp_systemtimer_2[54:0]	rx_ts_clk[2]	
ctl_rx_ptp_systemtimer_3[54:0]	rx_ts_clk[3]	
ctl_tx_lane0_vlm_bip7_override_01	tx_axi_clk[0]	Indicates that the bip7 byte value is overridden.
ctl_tx_lane0_vlm_bip7_override_23	tx_axi_clk[1]	
ctl_tx_lane0_vlm_bip7_override_value_01[7:0]	tx_axi_clk[0]	Indicates that the bip7 byte value has a override value.
ctl_tx_lane0_vlm_bip7_override_value_23[7:0]	tx_axi_clk[1]	
ctl_tx_pause_req_0[8:0]	tx_axi_clk[0]	TX Pause Request Each bit of ctl_tx_pause_req_N[8:0] is associated with a pause priority (Bit[8] is for global pause signaling).
ctl_tx_pause_req_1[8:0]	tx_axi_clk[0]	
ctl_tx_pause_req_2[8:0]	tx_axi_clk[1]	
ctl_tx_pause_req_3[8:0]	tx_axi_clk[1]	
ctl_tx_ptp_st_adjust_0[31:0]	tx_ts_clk[0]	Specifies System Timer Adjust
ctl_tx_ptp_st_adjust_1[31:0]	tx_ts_clk[1]	
ctl_tx_ptp_st_adjust_2[31:0]	tx_ts_clk[2]	
ctl_tx_ptp_st_adjust_3[31:0]	tx_ts_clk[3]	
ctl_tx_ptp_st_adjust_type_0[1:0]	tx_ts_clk[0]	Specifies the desired System Timer Adjustment Type
ctl_tx_ptp_st_adjust_type_1[1:0]	tx_ts_clk[1]	
ctl_tx_ptp_st_adjust_type_2[1:0]	tx_ts_clk[2]	
ctl_tx_ptp_st_adjust_type_3[1:0]	tx_ts_clk[3]	

Port Name	Clock Domain	Description
ctl_tx_ptp_st_adjust_vld_0	tx_ts_clk[0]	Specifies the System Timer Adjust is valid.
ctl_tx_ptp_st_adjust_vld_1	tx_ts_clk[1]	
ctl_tx_ptp_st_adjust_vld_2	tx_ts_clk[2]	
ctl_tx_ptp_st_adjust_vld_3	tx_ts_clk[3]	
ctl_tx_ptp_st_overwrite_0	tx_ts_clk[0]	System Timer Overwrite
ctl_tx_ptp_st_overwrite_1	tx_ts_clk[1]	
ctl_tx_ptp_st_overwrite_2	tx_ts_clk[2]	
ctl_tx_ptp_st_overwrite_3	tx_ts_clk[3]	
ctl_tx_ptp_st_sync_0	tx_ts_clk[0]	System Timer Sync
ctl_tx_ptp_st_sync_1	tx_ts_clk[1]	
ctl_tx_ptp_st_sync_2	tx_ts_clk[2]	
ctl_tx_ptp_st_sync_3	tx_ts_clk[3]	
ctl_tx_ptp_systemtimer_0[54:0]	tx_ts_clk[0]	System timer input in units of 2^{-8} nanoseconds (unsigned). This field maps to Bits[62:8] of the correction field format in IEEE 1588-2008 (with the bottom 8 bits set to 0).
ctl_tx_ptp_systemtimer_1[54:0]	tx_ts_clk[1]	
ctl_tx_ptp_systemtimer_2[54:0]	tx_ts_clk[2]	
ctl_tx_ptp_systemtimer_3[54:0]	tx_ts_clk[3]	
ctl_tx_resend_pause_0	Asynchronous	Transmit Resend Pause
ctl_tx_resend_pause_1		
ctl_tx_resend_pause_2		
ctl_tx_resend_pause_3		
ctl_tx_send_idle_0	Asynchronous (Parameter) [1]	Transmit Idle code words. If this input is sampled as a 1, the TX path only transmits Idle code words. This input should be set to 1 when the partner device is sending Remote Fault Indication (RFI) code words.
ctl_tx_send_idle_1		
ctl_tx_send_idle_2		
ctl_tx_send_idle_3		
ctl_tx_send_idle_in_0	Asynchronous (Physical Port) [1]	Transmit Send Idle Code Words Input
ctl_tx_send_idle_in_1		
ctl_tx_send_idle_in_2		
ctl_tx_send_idle_in_3		
ctl_tx_send_lfi_0	Asynchronous (Parameter) [1]	Transmit Local Fault Indication (LFI) code word. Takes precedence over RFI.
ctl_tx_send_lfi_1		
ctl_tx_send_lfi_2		
ctl_tx_send_lfi_3		

Port Name	Clock Domain	Description
ctl_tx_send_lfi_in_0	Asynchronous (Physical Port) [1]	Transmit Send Local Fault Indication Input
ctl_tx_send_lfi_in_1		
ctl_tx_send_lfi_in_2		
ctl_tx_send_lfi_in_3		
ctl_tx_send_rfi_0	Asynchronous (Parameter) [1]	Transmit Remote Fault Indication (RFI) code word. If this input is sampled as a 1, the TX path only transmits Remote Fault code words. This input should be set to 1 until the RX path is fully aligned and is ready to accept data from the link partner.
ctl_tx_send_rfi_1		
ctl_tx_send_rfi_2		
ctl_tx_send_rfi_3		
ctl_tx_send_rfi_in_0	Asynchronous (Physical Port) [1]	Transmit Send Remote Fault Indication Input
ctl_tx_send_rfi_in_1		
ctl_tx_send_rfi_in_2		
ctl_tx_send_rfi_in_3		

1. You can either set the parameter or set the value physically at the port. Both have the same effect on the IP. You can change the Physical port dynamically, but parameter can be set only during IP Generation.

MRMAC Status Ports

In addition to the AXI4-Lite registers and counters, the MRMAC IP provides status flag ports to enable ease of integration into user monitoring and interrupt logic. The following table has a list of the output status flags. All status ports are outputs.

Table: MRMAC Status Port Descriptions

Port Name	Clock Domain	Description
stat_rx01_fec_degraded_ser	s_axi_aclk	RX FEC Degraded Symbol Error
stat_rx23_fec_degraded_ser	s_axi_aclk	
stat_tx_local_fault_0	s_axi_aclk	A value of 1 indicates the transmit encoder state machine is in the TX_INIT state.
stat_tx_local_fault_1	s_axi_aclk	
stat_tx_local_fault_2	s_axi_aclk	
stat_tx_local_fault_3	s_axi_aclk	
stat_tx_frame_error_0	s_axi_aclk	Packets with tx_errin set to indicate an End of Packet (EOP) abort.
stat_tx_frame_error_1	s_axi_aclk	
stat_tx_frame_error_2	s_axi_aclk	
stat_tx_frame_error_3	s_axi_aclk	

Port Name	Clock Domain	Description
stat_tx_tsn_preempted_pkt_0	s_axi_aclk	Time-sensitive networking (TSN) preempted packets.
stat_tx_tsn_preempted_pkt_1	s_axi_aclk	
stat_tx_tsn_preempted_pkt_2	s_axi_aclk	
stat_tx_tsn_preempted_pkt_3	s_axi_aclk	
stat_tx_tsn_fragment_0	s_axi_aclk	Time-sensitive networking (TSN) fragmented packets.
stat_tx_tsn_fragment_1	s_axi_aclk	
stat_tx_tsn_fragment_2	s_axi_aclk	
stat_tx_tsn_fragment_3	s_axi_aclk	
stat_tx_pause_valid_0[8:0]	tx_axi_clk_0	Set to 1 when a pause packet is sent. If Bit[8] is set, it means a global pause packet was transmitted.
stat_tx_pause_valid_1[8:0]	tx_axi_clk_0	
stat_tx_pause_valid_2[8:0]	tx_axi_clk_0, tx_axi_clk_2	
stat_tx_pause_valid_3[8:0]	tx_axi_clk_0, tx_axi_clk_2	
stat_tx_axis_unf_0	s_axi_aclk	A value of 1 indicates that the AXI4-Stream interface has experienced an underflow.
stat_tx_axis_unf_1	s_axi_aclk	
stat_tx_axis_unf_2	s_axi_aclk	
stat_tx_axis_unf_3	s_axi_aclk	
stat_tx_axis_err_0	s_axi_aclk	A value of 1 indicates that the AXI4-Stream interface has encountered an error.
stat_tx_axis_err_1	s_axi_aclk	
stat_tx_axis_err_2	s_axi_aclk	
stat_tx_axis_err_3	s_axi_aclk	
stat_tx_flexif_err_0	s_axi_aclk	A value of 1 indicates that the TX Flex I/F has encountered an error.
stat_tx_flexif_err_1	s_axi_aclk	
stat_tx_flexif_err_2	s_axi_aclk	
stat_tx_flexif_err_3	s_axi_aclk	
stat_tx_pcs_bad_code_0[2:0]	s_axi_aclk	A value of 1 indicates that bad PCS code was observed.
stat_tx_pcs_bad_code_1[2:0]	s_axi_aclk	
stat_tx_pcs_bad_code_2[2:0]	s_axi_aclk	
stat_tx_pcs_bad_code_3[2:0]	s_axi_aclk	
stat_tx_flex_fifo_ovf_0	s_axi_aclk	A value of 1 indicates that the Flex I/F FIFO has overflowed.
stat_tx_flex_fifo_ovf_1	s_axi_aclk	
stat_tx_flex_fifo_ovf_2	s_axi_aclk	
stat_tx_flex_fifo_ovf_3	s_axi_aclk	

Port Name	Clock Domain	Description
stat_tx_flex_fifo_udf_0	s_axi_aclk	A value of 1 indicates that the Flex I/F FIFO has underflowed.
stat_tx_flex_fifo_udf_1	s_axi_aclk	
stat_tx_flex_fifo_udf_2	s_axi_aclk	
stat_tx_flex_fifo_udf_3	s_axi_aclk	
stat_tx_bad_fcs_0	s_axi_aclk	Total number of packets greater than 64 bytes that have FCS errors.
stat_tx_bad_fcs_1	s_axi_aclk	
stat_tx_bad_fcs_2	s_axi_aclk	
stat_tx_bad_fcs_3	s_axi_aclk	
stat_tx_ecc_err_0[1:0]	s_axi_aclk	Asserted when any other ECC error is detected. Indicates an ECC error was detected in the memory for that port.
stat_tx_ecc_err_1[1:0]	s_axi_aclk	
stat_tx_ecc_err_2[1:0]	s_axi_aclk	
stat_tx_ecc_err_3[1:0]	s_axi_aclk	
stat_tx_fec_pcs_lane_align_0	s_axi_aclk	Indicates that all the transmit FEC lanes are aligned/deskewed and ready to transmit data.
stat_tx_fec_pcs_lane_align_1	s_axi_aclk	
stat_tx_fec_pcs_lane_align_2	s_axi_aclk	
stat_tx_fec_pcs_lane_align_3	s_axi_aclk	
stat_tx_fec_pcs_block_lock_0[4:0]	s_axi_aclk	Indicates that the PCS has achieved block lock on the corresponding lanes.
stat_tx_fec_pcs_block_lock_1[4:0]	s_axi_aclk	
stat_tx_fec_pcs_block_lock_2[4:0]	s_axi_aclk	
stat_tx_fec_pcs_block_lock_3[4:0]	s_axi_aclk	
stat_tx_fec_pcs_am_lock_0[4:0]	s_axi_aclk	Indicates that all of the PCS lanes have achieved Alignment Marker lock.
stat_tx_fec_pcs_am_lock_1[4:0]	s_axi_aclk	
stat_tx_fec_pcs_am_lock_2[4:0]	s_axi_aclk	
stat_tx_fec_pcs_am_lock_3[4:0]	s_axi_aclk	
stat_rx_block_lock_0[19:0]	s_axi_aclk	Indicates that the PCS has achieved block lock on the corresponding PCS lane as defined by IEEE 802.3. A value of 1 indicates block lock is achieved.
stat_rx_synced_0[19:0]	s_axi_aclk	Word Boundary Synchronized. Indicates whether the PCS is word boundary synchronized (PCS Lane Marker Word has been detected) on the corresponding PCS lane. Corresponds to MDIO register bit 3.52.7:0 and 3.53.11:0 as defined in Clause 82.3.

Port Name	Clock Domain	Description
stat_rx_synced_err_0[19:0]	s_axi_aclk	Word Boundary Synchronization Error. Indicates whether an error occurred during the word boundary synchronization on the corresponding PCS lane.
stat_rx_mf_err_0[19:0]	s_axi_aclk	PCS Lane Marker Word Error. These signals indicate that an incorrectly formed PCS Lane Marker Word was detected in the respective PCS lane. A value of 1 indicates an error occurred.
stat_rx_block_lock_1	s_axi_aclk	Indicates that the PCS has achieved block lock on the corresponding PCS lane as defined by IEEE 802.3. A value of 1 indicates block lock is achieved.
stat_rx_block_lock_2[3:0]	s_axi_aclk	Indicates that the PCS has achieved block lock on the corresponding PCS lane as defined by IEEE 802.3. A value of 1 indicates block lock is achieved.
stat_rx_synced_2[3:0]	s_axi_aclk	Word Boundary Synchronized. Indicates whether the PCS is word boundary synchronized (PCS Lane Marker Word has been detected) on the corresponding PCS lane.
stat_rx_synced_err_2[3:0]	s_axi_aclk	Word Boundary Synchronization Error. Indicates whether an error occurred during the word boundary synchronization on the corresponding PCS lane.
stat_rx_mf_err_2[3:0]	s_axi_aclk	PCS Lane Marker Word Error. These signals indicate that an incorrectly formed PCS Lane Marker Word was detected in the respective PCS lane. A value of 1 indicates an error occurred.
stat_rx_block_lock_3	s_axi_aclk	Indicates that the PCS has achieved block lock on the corresponding PCS lane as defined by IEEE 802.3. A value of 1 indicates block lock is achieved.
stat_rx_flexif_err_0	s_axi_aclk	A value of 1 indicates that the TX Flex I/F has encountered an error.
stat_rx_flexif_err_1	s_axi_aclk	
stat_rx_flexif_err_2	s_axi_aclk	
stat_rx_flexif_err_3	s_axi_aclk	
stat_rx_pcs_bad_code_0	s_axi_aclk	Increment for 64B/66B code violations. This signal indicates that the RX PCS receive state machine is in the RX_E state as specified by the IEEE Std 802.3. This output can be used to generate MDIO register 3.33:7:0 as defined in Clause 82.3.
stat_rx_pcs_bad_code_1	s_axi_aclk	
stat_rx_pcs_bad_code_2	s_axi_aclk	
stat_rx_pcs_bad_code_3	s_axi_aclk	

Port Name	Clock Domain	Description
stat_rx_flex_fifo_ovf_0	s_axi_aclk	A valid of 1 indicates that the Flex I/F experienced an overflow.
stat_rx_flex_fifo_ovf_1	s_axi_aclk	
stat_rx_flex_fifo_ovf_2	s_axi_aclk	
stat_rx_flex_fifo_ovf_3	s_axi_aclk	
stat_rx_flex_fifo_udf_0	s_axi_aclk	A valid of 1 indicates that the Flex I/F experienced an underflow.
stat_rx_flex_fifo_udf_1	s_axi_aclk	
stat_rx_flex_fifo_udf_2	s_axi_aclk	
stat_rx_flex_fifo_udf_3	s_axi_aclk	
stat_rx_status_0	s_axi_aclk	A value of 1 indicates the PCS is aligned and not in hi_ber state. Corresponds to MDIO register bit 3.32.12 as defined in Clause 82.3.
stat_rx_status_1	s_axi_aclk	
stat_rx_status_2	s_axi_aclk	
stat_rx_status_3	s_axi_aclk	
stat_rx_vl_demuxed_0	s_axi_aclk	After word boundary synchronization is achieved on each lane. If a bit of this bus is 1, it indicates that the corresponding PCS lane was properly found and de-MUXed.
stat_rx_vl_demuxed_2	s_axi_aclk	
stat_rx_lane0_vlm_bip7_valid_0	rx_axi_clk_0	Indicates that the bip7 byte value is valid.
stat_rx_lane0_vlm_bip7_valid_2	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_lane0_vlm_bip7_0[7:0]	rx_axi_clk_0	This is the received value of the bip7 byte in the PCS lane0 marker.
stat_rx_lane0_vlm_bip7_2[7:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_aligned_0	s_axi_aclk	When stat_rx_aligned is a value of 1, all of the lanes are aligned/deskewed and the receiver is ready to receive packet data.
stat_rx_aligned_2	s_axi_aclk	
stat_rx_aligned_err_0	s_axi_aclk	When stat_rx_aligned_err is a value of 1, either Lane alignment failed after several attempts, or Lane alignment was lost (stat_rx_aligned was asserted and then it was negated).
stat_rx_aligned_err_2	s_axi_aclk	
stat_rx_misaligned_0	s_axi_aclk	When stat_rx_misaligned is a value of 1, a valid PCS Lane Marker Word was not received on all PCS lanes simultaneously.
stat_rx_misaligned_2	s_axi_aclk	
stat_rx_valid_ctrl_code_0	s_axi_aclk	Indicates that a block with a valid control code was received.
stat_rx_valid_ctrl_code_1	s_axi_aclk	
stat_rx_valid_ctrl_code_2	s_axi_aclk	
stat_rx_valid_ctrl_code_3	s_axi_aclk	

Port Name	Clock Domain	Description
stat_rx_hi_ber_0	s_axi_aclk	When set to 1, the BER is too high as defined by IEEE Std 802.3. Corresponds to MDIO register bit 3.32.1 as defined in Clause 82.3.
stat_rx_hi_ber_1	s_axi_aclk	
stat_rx_hi_ber_2	s_axi_aclk	
stat_rx_hi_ber_3	s_axi_aclk	
stat_rx_bad_code_0	s_axi_aclk	Increment for 64B/66B code violations. This signal indicates that the RX PCS receive state machine is in the RX_E state as specified by the IEEE Std 802.3. This output can be used to generate MDIO register 3.33:7:0 as defined in Clause 82.3.
stat_rx_bad_code_1	s_axi_aclk	
stat_rx_bad_code_2	s_axi_aclk	
stat_rx_bad_code_3	s_axi_aclk	
stat_rx_bad_preamble_0	s_axi_aclk	This signal indicates if the Ethernet packet received was preceded by a valid preamble. A value of 1 indicates that an invalid preamble was received.
stat_rx_bad_preamble_1	s_axi_aclk	
stat_rx_bad_preamble_2	s_axi_aclk	
stat_rx_bad_preamble_3	s_axi_aclk	
stat_rx_bad_sfd_0	s_axi_aclk	Increment bad start of frame delimiter (SFD). This signal indicates if the Ethernet packet received was preceded by a valid SFD. A value of 1 indicates that an invalid SFD was received.
stat_rx_bad_sfd_1	s_axi_aclk	
stat_rx_bad_sfd_2	s_axi_aclk	
stat_rx_bad_sfd_3	s_axi_aclk	
stat_rx_got_signal_os_0	s_axi_aclk	Indicates that a Signal Ordered Set was received.
stat_rx_got_signal_os_1	s_axi_aclk	
stat_rx_got_signal_os_2	s_axi_aclk	
stat_rx_got_signal_os_3	s_axi_aclk	
stat_rx_invalid_start_0	s_axi_aclk	Indicates that a Start code has been detected, but has been invalidated potentially leading to a packet not being recognized. A Start code is invalidated if it immediately follows an Error code, or if it is not preceded by sufficient IPG.
stat_rx_invalid_start_1	s_axi_aclk	
stat_rx_invalid_start_2	s_axi_aclk	
stat_rx_invalid_start_3	s_axi_aclk	
stat_rx_test_pattern_mismatch_0	s_axi_aclk	Total number of test pattern mismatches.
stat_rx_test_pattern_mismatch_1	s_axi_aclk	
stat_rx_test_pattern_mismatch_2	s_axi_aclk	
stat_rx_test_pattern_mismatch_3	s_axi_aclk	
stat_rx_local_fault_0	s_axi_aclk	This output is High when stat_rx_internal_local_fault or stat_rx_received_local_fault is asserted.
stat_rx_local_fault_1	s_axi_aclk	
stat_rx_local_fault_2	s_axi_aclk	
stat_rx_local_fault_3	s_axi_aclk	

Port Name	Clock Domain	Description
stat_rx_internal_local_fault_0	s_axi_aclk	If this bit is sampled as a 1, it indicates a remote fault condition was detected. If this bit is sampled as a 0, a remote fault condition does not exist.
stat_rx_internal_local_fault_1	s_axi_aclk	
stat_rx_internal_local_fault_2	s_axi_aclk	
stat_rx_internal_local_fault_3	s_axi_aclk	
stat_rx_received_local_fault_0	s_axi_aclk	This signal goes High when enough local fault words are received from the link partner to trigger a fault condition as specified by the IEEE fault state machine.
stat_rx_received_local_fault_1	s_axi_aclk	
stat_rx_received_local_fault_2	s_axi_aclk	
stat_rx_received_local_fault_3	s_axi_aclk	
stat_rx_remote_fault_0	s_axi_aclk	If this bit is sampled as a 1, it indicates a remote fault condition was detected. If this bit is sampled as a 0, a remote fault condition does not exist.
stat_rx_remote_fault_1	s_axi_aclk	
stat_rx_remote_fault_2	s_axi_aclk	
stat_rx_remote_fault_3	s_axi_aclk	
stat_rx_truncated_0	s_axi_aclk	Indicates RX frames that were truncated due to their length exceeding the defined maximum length.
stat_rx_truncated_1	s_axi_aclk	
stat_rx_truncated_2	s_axi_aclk	
stat_rx_truncated_3	s_axi_aclk	
stat_rx_pause_valid_0[8:0]	rx_axi_clk_0	RX Pause Valid Each bit of stat_rx_pause_valid_N[8:0] is associated with a pause priority (Bit[8] is for global pause signaling).
stat_rx_pause_valid_1[8:0]	rx_axi_clk_0	
stat_rx_pause_valid_2[8:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_valid_3[8:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta0_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta0_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta0_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta0_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta1_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta1_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta1_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta1_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	

Port Name	Clock Domain	Description
stat_rx_pause_quanta2_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta2_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta2_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta2_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta3_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta3_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta3_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta3_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta4_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta4_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta4_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta4_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta5_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta5_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta5_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta5_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta6_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta6_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta6_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta6_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	

Port Name	Clock Domain	Description
stat_rx_pause_quanta7_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta7_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta7_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta7_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta8_0[15:0]	rx_axi_clk_0	These buses indicate the quanta received for each of the eight priorities in priority-based pause operation and the global pause operation. The value for stat_rx_pause_quanta[8] is used for global pause operation. All other values are used for priority pause operation.
stat_rx_pause_quanta8_1[15:0]	rx_axi_clk_0	
stat_rx_pause_quanta8_2[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_quanta8_3[15:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_req_0[8:0]	rx_axi_clk_0	RX Pause Request Each bit of stat_rx_pause_req_N[8:0] is associated with a pause priority (Bit[8] is for global pause signaling).
stat_rx_pause_req_1[8:0]	rx_axi_clk_0	
stat_rx_pause_req_2[8:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_pause_req_3[8:0]	rx_axi_clk_0, rx_axi_clk_2	
stat_rx_axis_fifo_overflow_0	s_axi_aclk	A value of 1 indicates that the AXI4-Stream interface has experienced an overflow.
stat_rx_axis_fifo_overflow_1	s_axi_aclk	
stat_rx_axis_fifo_overflow_2	s_axi_aclk	
stat_rx_axis_fifo_overflow_3	s_axi_aclk	
stat_rx_axis_err_0	s_axi_aclk	A value of 1 indicates that the AXI4-Stream interface has experienced an error.
stat_rx_axis_err_1	s_axi_aclk	
stat_rx_axis_err_2	s_axi_aclk	
stat_rx_axis_err_3	s_axi_aclk	
stat_rx_ecc_err_0[1:0]	s_axi_aclk	Asserted when any other ECC error is detected. Indicates that an ECC error was detected on the RX of a given port.
stat_rx_ecc_err_1[1:0]	s_axi_aclk	
stat_rx_ecc_err_2[1:0]	s_axi_aclk	
stat_rx_ecc_err_3[1:0]	s_axi_aclk	
stat_rx_bad_fcs_0	s_axi_aclk	Packets received with bad (but not stomped) FCS. A stomped FCS is defined as the bitwise inverse of a good FCS.
stat_rx_bad_fcs_1	s_axi_aclk	
stat_rx_bad_fcs_2	s_axi_aclk	
stat_rx_bad_fcs_3	s_axi_aclk	

Port Name	Clock Domain	Description
stat_rx_tsn_preempted_pkt_0	s_axi_aclk	Packets received with TSN preempted packets.
stat_rx_tsn_preempted_pkt_1	s_axi_aclk	
stat_rx_tsn_preempted_pkt_2	s_axi_aclk	
stat_rx_tsn_preempted_pkt_3	s_axi_aclk	
stat_rx_tsn_fragment_0	s_axi_aclk	Packets received with TSN fragmented packets.
stat_rx_tsn_fragment_1	s_axi_aclk	
stat_rx_tsn_fragment_2	s_axi_aclk	
stat_rx_tsn_fragment_3	s_axi_aclk	
stat_rx_fec_hi_ser_0	s_axi_aclk	Indicates that the number of symbol errors in a 8192-codeword window has exceeded the threshold K (417).
stat_rx_fec_hi_ser_1	s_axi_aclk	
stat_rx_fec_hi_ser_2	s_axi_aclk	
stat_rx_fec_hi_ser_3	s_axi_aclk	
stat_rx_fec_lane_lock_0[3:0]	s_axi_aclk	Indicates that the FEC has achieved lane lock on the indicated lane.
stat_rx_fec_lane_lock_1	s_axi_aclk	
stat_rx_fec_lane_lock_2[3:0]	s_axi_aclk	
stat_rx_fec_lane_lock_3	s_axi_aclk	
stat_rx_fec_corrected_cw_0_0	rx_axi_clk_0	Count of corrected codewords on FEC lane 0.
stat_rx_fec_corrected_cw_0_1	rx_axi_clk_0	Count of corrected codewords on FEC lane 1.
stat_rx_fec_corrected_cw_0_2	rx_axi_clk_0	Count of corrected codewords on FEC lane 2.
stat_rx_fec_corrected_cw_0_3	rx_axi_clk_0	Count of corrected codewords on FEC lane 3.
stat_rx_fec_uncorrected_cw_0_0	rx_axi_clk_0	Count of uncorrected codewords on FEC lane 0.
stat_rx_fec_uncorrected_cw_0_1	rx_axi_clk_0	Count of uncorrected codewords on FEC lane 1.
stat_rx_fec_uncorrected_cw_0_2	rx_axi_clk_0	Count of uncorrected codewords on FEC lane 2.
stat_rx_fec_uncorrected_cw_0_3	rx_axi_clk_0	Count of uncorrected codewords on FEC lane 3.
stat_rx_fec_cw_0_0	rx_axi_clk_0	Count of processed codewords on FEC lane 0.
stat_rx_fec_cw_0_1	rx_axi_clk_0	Count of processed codewords on FEC lane 1.
stat_rx_fec_cw_0_2	rx_axi_clk_0	Count of processed codewords on FEC lane 2.

Port Name	Clock Domain	Description
stat_rx_fec_cw_0_3	rx_axi_clk_0	Count of processed codewords on FEC lane 3.
stat_rx_fec_err_count_0_0[3:0]	rx_axi_clk_0	Count of FEC symbol errors on FEC lane 0.
stat_rx_fec_err_count_0_1[3:0]	rx_axi_clk_0	Count of FEC symbol errors on FEC lane 1.
stat_rx_fec_err_count_0_2[3:0]	rx_axi_clk_0	Count of FEC symbol errors on FEC lane 2.
stat_rx_fec_err_count_0_3[3:0]	rx_axi_clk_0	Count of FEC symbol errors on FEC lane 3.
stat_rx_fec_corrected_cw_1	rx_axi_clk_0	Count of corrected codewords on FEC.
stat_rx_fec_uncorrected_cw_1	rx_axi_clk_0	Count of uncorrected codewords on FEC.
stat_rx_fec_cw_1	rx_axi_clk_0	Count of processed codewords on FEC.
stat_rx_fec_err_count_1[3:0]	rx_axi_clk_0	Count of FEC symbol errors.
stat_rx_fec_corrected_cw_2_0	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 0.
stat_rx_fec_corrected_cw_2_1	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 1.
stat_rx_fec_corrected_cw_2_2	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 2.
stat_rx_fec_corrected_cw_2_3	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 3.
stat_rx_fec_uncorrected_cw_2_0	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 0.
stat_rx_fec_uncorrected_cw_2_1	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 1.
stat_rx_fec_uncorrected_cw_2_2	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 2.
stat_rx_fec_uncorrected_cw_2_3	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC lane 3.
stat_rx_fec_cw_2_0	rx_axi_clk_0, rx_axi_clk_2	Count of processed codewords on FEC lane 0.
stat_rx_fec_cw_2_1	rx_axi_clk_0, rx_axi_clk_2	Count of processed codewords on FEC lane 1.
stat_rx_fec_cw_2_2	rx_axi_clk_0, rx_axi_clk_2	Count of processed codewords on FEC lane 2.
stat_rx_fec_cw_2_3	rx_axi_clk_0, rx_axi_clk_2	Count of processed codewords on FEC lane 3.
stat_rx_fec_err_count_2_0[3:0]	rx_axi_clk_0, rx_axi_clk_2	Count of FEC symbol errors.

Port Name	Clock Domain	Description
stat_rx_fec_err_count_2_1[3:0]	rx_axi_clk_0, rx_axi_clk_2	Count of FEC symbol errors.
stat_rx_fec_corrected_cw_3	rx_axi_clk_0, rx_axi_clk_2	Count of corrected codewords on FEC.
stat_rx_fec_uncorrected_cw_3	rx_axi_clk_0, rx_axi_clk_2	Count of uncorrected codewords on FEC.
stat_rx_fec_cw_3	rx_axi_clk_0, rx_axi_clk_2	Count of processed codewords on FEC.
stat_rx_fec_err_count_3[3:0]	rx_axi_clk_0, rx_axi_clk_2	Count of FEC symbol errors.
stat_rx_fec_aligned_0	s_axi_aclk	Indicates that the FEC is aligned/deskewed and ready to receive packet data.
stat_rx_fec_aligned_1	s_axi_aclk	
stat_rx_fec_aligned_2	s_axi_aclk	
stat_rx_fec_aligned_3	s_axi_aclk	
stat_rx_fec_mapping_0[1:0]	s_axi_aclk	The number corresponding to the detected lane on Lane 0.
stat_rx_fec_mapping_1[1:0]	s_axi_aclk	The number corresponding to the detected lane on Lane 1.
stat_rx_fec_mapping_2[1:0]	s_axi_aclk	The number corresponding to the detected lane on Lane 2.
stat_rx_fec_mapping_3[1:0]	s_axi_aclk	The number corresponding to the detected lane on Lane 3.
stat_rx_fec_delay_0[14:0]	rx_axi_clk_0	The instantaneous delay that has been applied to each of the SerDes lanes in the alignment and deskew block.
stat_rx_framing_err_0[19:0]	s_axi_aclk	Indicates a sync header error detected.
stat_rx_bip_err_0[19:0]	s_axi_aclk	BIP8 error indicator for PCS lane 0. A non-zero value indicates the BIP8 signature was in error.
stat_rx_fec_delay_1[14:0]	rx_axi_clk_0	The instantaneous delay that has been applied to each of the SerDes lanes in the alignment and deskew block.
stat_rx_framing_err_1	s_axi_aclk	Indicates a sync header error detected.
stat_rx_fec_delay_2[14:0]	rx_axi_clk_0, rx_axi_clk_2	The instantaneous delay that has been applied to each of the SerDes lanes in the alignment and deskew block.
stat_rx_framing_err_2[3:0]	s_axi_aclk	Indicates a sync header error detected.

Port Name	Clock Domain	Description
stat_rx_bip_err_2[3:0]	s_axi_aclk	BIP8 error indicator for PCS lane 0. A non-zero value indicates the BIP8 signature was in error.
stat_rx_fec_delay_3[14:0]	rx_axi_clk_0, rx_axi_clk_2	The instantaneous delay that has been applied to each of the SerDes lanes in the alignment and deskew block.
stat_rx_framing_err_3	s_axi_aclk	Indicates a sync header error detected.
stat_rx_cl49_82_convert_err_0	rx_axi_clk_0	Port 0 RX CL49_82 convert error
stat_rx_cl49_82_convert_err_1	rx_axi_clk_0	Port 1 RX CL49_82 convert error
stat_rx_cl49_82_convert_err_2	rx_axi_clk_0, rx_axi_clk_2	Port 2 RX CL49_82 convert error
stat_rx_cl49_82_convert_err_3	rx_axi_clk_0, rx_axi_clk_2	Port 3 RX CL49_82 convert error
stat_tx_cl49_82_convert_err_0	tx_axi_clk_0	Port 0 TX CL49_82 convert error
stat_tx_cl49_82_convert_err_1	tx_axi_clk_0	Port 1 TX CL49_82 convert error
stat_tx_cl49_82_convert_err_2	tx_axi_clk_0, tx_axi_clk_2	Port 2 TX CL49_82 convert error
stat_tx_cl49_82_convert_err_3	tx_axi_clk_0, tx_axi_clk_2	Port 3 TX CL49_82 convert error
stat_rx_flex_mon_fifo_ovf_0	s_axi_aclk	Port 0 RX Flex mon fifo overflow
stat_rx_flex_mon_fifo_ovf_1	s_axi_aclk	Port 1 RX Flex mon fifo overflow
stat_rx_flex_mon_fifo_ovf_2	s_axi_aclk	Port 2 RX Flex mon fifo overflow
stat_rx_flex_mon_fifo_ovf_3	s_axi_aclk	Port 3 RX Flex mon fifo overflow
stat_rx_flex_mon_fifo_udf_0	s_axi_aclk	Port 0 RX Flex mon fifo underrflow
stat_rx_flex_mon_fifo_udf_1	s_axi_aclk	Port 1 RX Flex mon fifo underrflow
stat_rx_flex_mon_fifo_udf_2	s_axi_aclk	Port 2 RX Flex mon fifo underrflow
stat_rx_flex_mon_fifo_udf_3	s_axi_aclk	Port 3 RX Flex mon fifo underrflow

Timestamp Interface

This section describes the ports relating to the operation of the IEEE 1588 Timestamping function. The timestamping interface can be coupled with either the Flex I/F or the AXI4-Stream interface, depending on which interface is active.

When the AXI4-Stream interface is active, the timestamping signals are clocked by the AXI clock. When the Flex I/F is active and the AXI4-Stream interface disabled, the timestamp interface still uses the AXI clock. For this reason, when the

Flex I/F is in use, the `flexif_clk` must drive the `axi_clk` port.

Related Information

IEEE 1588 Timestamping Support

Timestamp Ports

Table: Timestamp Control Signal Descriptions (TX/RX)

Port Name	I/O	Clock Domain	Description
ctl_rx_ptp_systemtimer[54:0], ctl_tx_ptp_systemtimer[54:0]	I	rx/tx_ts_clk	System timer input in units of 2^{-8} nanoseconds (unsigned). This field maps to Bits[62:8] of the correction field format in IEEE 1588-2008 (with the bottom 8 bits set to 0). These inputs are captured on the rx_ts_clk/tx_ts_clk clock domain. It should be stable for several clock period (~10 ns) after the ctl_rx/tx_ptp_st_sync transition.
ctl_rx_ptp_st_sync, ctl_tx_ptp_st_sync	I	rx/tx_ts_clk	Transition indicates point at which ctl_rx/tx_ptp_systemtimer is valid. A DDR phase detector is used on this signal for additional phase compensation. Transitions need to be a minimum of 10 clock cycles apart. The typical delay between each pulse is 64 ns.
ctl_rx_ptp_st_overwrite, ctl_tx_ptp_st_overwrite	I	rx/tx_ts_clk	When asserted, the PTP Timer is overwritten with the ctl_rx/tx_ptp_systemtimer value if the difference is greater than the threshold amount. The overwrite occurs at the next transition of ctl_rx/tx_ptp_st_sync.

Port Name	I/O	Clock Domain	Description
ctl_rx_ptp_st_adjust[31:0], ctl_tx_ptp_st_adjust[31:0]	I	rx/tx_ts_clk	When a bit of ctl_st_adjust_vld (defined below) is asserted, the system timer has the correction in ctl_ptp_st_adjust applied. ✎ **Note:** The correction factor should be in 2's complement form. The correction is applied immediately. The unit of the value on this signal is determined by the adjustment type defined below.
ctl_rx_ptp_st_adjust_type[1:0], ctl_tx_ptp_st_adjust_type[1:0]	I	rx/tx_ts_clk	Specifies the desired adjustment using the value in ctl_ptp_st_adjust: • 2'b00: Adjust system timer; signed, in units of 2^{-8} nanoseconds (the upper 14 bits are ignored). ⚠ CAUTION!No overflow protection is available. • 2'b01: Set the automatic increment value; ctl_ptp_st_adjust value is unsigned, in units of 2^{-8} ns (upper 14 bits ignored). This also clears any existing increment value below 2^{-8} ns (previously set by default or through adjustments). Increments every cycle (Nominal values – non-KP4 : 1.551515 ns; KP4 : 1.50588235 ns) • 2'b10: Adjust automatic increment value; signed, in units of 2^{-40} ns • 2'b11: Unused.

Port Name	I/O	Clock Domain	Description
ctl_rx_ptp_st_adjust_vld, ctl_tx_ptp_st_adjust_vld	I	rx/tx_ts_clk	Transition of this signal triggers adjustment type defined in ctl_rx/tx_ptp_st_adjust_type with a value in ctl_rx/tx_ptp_st_adjust.
stat_rx_ptp_st_sync, stat_tx_ptp_st_sync	O	tx/rx_axi_clk	Reflects ctl_rx/tx_ptp_st_sync after it has been re-sampled to the RX/TX clock domain.
stat_rx_ptp_systemtimer[54:0], stat_tx_ptp_systemtimer[54:0]	O	tx/rx_axi_clk	PTP Timer output in units of 2^{-8} ns. ✎ **Note:** Due to retiming, this value should be captured at least four ts_clk cycles after the stat_rx/tx_ptp_st_sync edge. Also, Bit[54] is dropped and replaced with a capture edge indicator. You can use this information to adjust the system timer value by an additional half rx_serdes_clk/tx_core_clock period.
stat_rx_ptp_st_in_range, stat_tx_ptp_st_in_range	O	tx/rx_axi_clk	Reserved, leave disconnected.

Register Space

The MRMAC IP subsystem registers are accessible in a ZIP file on the AMD website. All registers are accessible through the AXI4-Lite interface.

When 1588 is enabled, an additional AXI4-Lite interface port appears on the MRMAC Wrapper, through which you can configure the Master timer. For more information contact, Support.

Related Information

AXI4-Lite Register Interface

MRMAC Memory Map

The AXI4-Lite interface enables access to the MRMAC configuration and statistics registers. Each port has a set of independent Configuration registers, Status registers, and Statistics counters.

Table: MRMAC Memory Map

Port	Base Address	Region
N/A	0x0000	Revision Registers

Port	Base Address	Region
0	0x0004	Configuration Registers
	0x0740	Status Registers and Statistics Counters
1	0x1004	Configuration Registers
	0x1740	Status Registers and Statistics Counters
2	0x2004	Configuration Registers
	0x2740	Status Registers and Statistics Counters
3	0x3004	Configuration Registers
	0x3740	Status Registers and Statistics Counters

Related Information

Register Space

Configuration Registers

In addition to configuration through the MRMAC IP Wizard, the MRMAC supports a software-based configuration. Some configuration registers (identified as Type RW ATTR) are persistent between resets, meaning that the attribute value of the configuration register does not change as the result of a reset. This allows the logic from the MRMAC to be reset without having to follow the software configuration. When the device is reprogrammed, these MRMAC configuration registers are overwritten with the values generated by the MRMAC IP Wizard. Other configuration registers (identified as Type RW) are set to their default values after reset.

Status Registers

Each port has a set of Status registers to monitor its current state or flag current or historic fault conditions. Two types of Status registers exist: latch-value and real-time. The latch-value status registers latch and hold the active value of a field or bit until it is cleared by writing a 1 to that bit (to clear a latched field, the entire field must be written with 1s). Reading these registers allow historic events to be captured by software. A read of the real-time status registers returns the current values of the associated status flags.

Statistics Monitoring

Each MRMAC port contains an internal statistics engine which monitors packet histogram statistics, and internal error statistics. The internal counters for the statistics are 48 bits wide and saturated when full.

A tick mechanism is used to snapshot the internal counter values and place them in accessible statistics registers. The values remain in the user-accessible registers until a subsequent tick overwrites the values with fresh data. The tick event also resets the internal counters to zero. In this method, the snapshot counter values represent a count of events which have occurred in the interval between tick events, making it easier to window the statistics. A tick event on a port does not affect the statistic engines or user registers of any other port.

A tick event can be triggered by asserting a rising edge on the per-port `pm_tick` input pin for a given port, or by writing a 1 to the tick register of a given port through the AXI4-Lite interface. Tick mode can be set to register instead of pm_tick input pin by writing "1" to MODE register tick_reg_mode_sel_<N> of the respective port.

Transferring data from the internal statistics engine to the user register space takes several clock cycles. Completion of the update process is indicated by the rising edge of the AXI4-Lite interface's `pm_rdy` pin. The port's statistics counters are invalid during the transfer and should not be read.

When a snapshot is completed, the counter values are accessible on the 32-bit AXI4-Lite port where the LSB 32-bit is stored at address `b000` and the MSB 32-bit is stored at address `b100`.

For TX statistics, good packets are defined as packets without FCS or other errors. Bad packets are defined as packets with FCS or any other error.

For RX statistics, good packets are defined as packets without FCS or other errors, including length errors. Bad packets are defined as packets with FCS or any other error. The length field error includes length field error, oversize, and undersize packets.

Related Information

AXI4-Lite Register Interface

Testability Functions

The MRMAC Ethernet example design implements the test pattern generation and checking as defined in Clause 82.2.10 (Test Pattern Generators) and Clause 82.2.17 (Test Pattern Checker). For more information, see the IEEE 802.3 specifications.

Attributes

The majority of MRMAC Subsystem attributes are accessible using the AXI4-Lite interface.

Power-Only Attributes

Several attributes are used by power estimation tools. These power-only attributes are set by the MRMAC Subsystem IP Wizard based on parameters that you choose. These power-only attributes are not controllable through the AXI4-Lite interface. Furthermore, these attributes have no impact on the functional behaviour of the MRMAC Subsystem.

Table: Power-Only Attributes

Attribute Name	Type	Description
ACTIVITY	String	User prediction of how active the block is in their application.
NUM_100G_FEC_ONLY_PORTS	Integer	Number of ports running at 100 Gb/s that include an operational FEC, but no operational MAC nor PCS.
NUM_100G_MAC_PCS_NOFEC_PORTS	Integer	Number of ports running at 100 Gb/s that include an operational MAC and PCS, but no operational FEC.
NUM_100G_MAC_PCS_WITH_FEC_PORTS	Integer	Number of ports running at 100 Gb/s that include an operational MAC and PCS and operational FEC.
NUM_10G_MAC_PCS_NOFEC_PORTS	Integer	Number of ports running at 10 Gb/s that include an operational MAC and PCS, but no operational FEC.

Attribute Name	Type	Description
NUM_10G_MAC_PCS_WITH_FEC_PORTS	Integer	Number of ports running at 10 Gb/s that include an operational MAC and PCS and operational FEC.
NUM_25G_FEC_ONLY_PORTS	Integer	Number of ports running at 25 Gb/s that include an operational FEC, but no operational MAC nor PCS.
NUM_25G_MAC_PCS_NOFEC_PORTS	Integer	Number of ports running at 25 Gb/s that include an operational MAC and PCS, but no operational FEC.
NUM_25G_MAC_PCS_WITH_FEC_PORTS	Integer	Number of ports running at 25 Gb/s that include an operational MAC and PCS and operational FEC.
NUM_40G_MAC_PCS_NOFEC_PORTS	Integer	Number of ports running at 40 Gb/s that include an operational MAC and PCS, but no operational FEC.
NUM_40G_MAC_PCS_WITH_FEC_PORTS	Integer	Number of ports running at 40 Gb/s that include an operational MAC and PCS and operational FEC.
NUM_50G_FEC_ONLY_PORTS	Integer	Number of ports running at 50 Gb/s that include an operational FEC, but no operational MAC nor PCS.
NUM_50G_MAC_PCS_NOFEC_PORTS	Integer	Number of ports running at 50 Gb/s that include an operational MAC and PCS, but no operational FEC.
NUM_50G_MAC_PCS_WITH_FEC_PORTS	Integer	Number of ports running at 50 Gb/s that include an operational MAC and PCS and operational FEC.

Related Information

Register Space

Designing with the Subsystem

General Design Guidelines

Registering Signals

To simplify timing and increase system performance in a programmable device design, keep all inputs and outputs registered between the user application and the subsystem. This means that all inputs and outputs from the user application should come from, or connect to, a flip-flop. While registering signals might not be possible for all paths, it simplifies timing analysis and makes it easier for the AMD tools to place and route the design.

Recognize Timing Critical Signals

The constraints provided with the example design identify the critical signals and timing constraints that should be applied.

Make Only Allowed Modifications

You should not modify the subsystem. Any modifications can have adverse effects on system timing and protocol compliance. Supported user configurations of the subsystem can only be made by selecting the options in the customization IP dialog box when the subsystem is generated.

Clocking

This section describes the clocking for the various supported MRMAC configurations. The MRMAC has a high degree of configurability and the large number of available modes. To achieve a successful implementation, care must be taken to satisfy the clocking requirements described further. You are advised to review the Clocking Use Cases section.

The MRMAC features the ability to dynamically reconfigure ports to different data rates. For example, the MRMAC could be reconfigured during run-time from 4 × 10G mode to 1 × 100G mode. This type of reconfiguration has clocking implications. In 4 × 10G, each port can have independent `tx_core_clk[3:0]` sources. When the MRMAC is reconfigured into 1 × 100GE mode, only `tx_core_clk[0]` is used and the remainder `tx_core_clk[3:1]` is ignored. If reconfiguration is not required, the relevant clocks can be statically connected to the MRMAC.

Related Information

Clocking Use Cases

Clocks

The following table lists the clocks that are present in the MRMAC.

✎ **Note:** All datapath-related clocks are per-port. However, clocks are tied together internally in certain operating modes, and the lower-numbered port clock is used.

Table: Clocks

Clock	Port	Description
AXI4-Lite	s_axi_aclk	AXI4-Lite processor interface clock.
AXI	tx_axi_clk[2:0] rx_axi_clk[2:0]	AXI4-Stream interface clocks. These clocks are used by the AXI4-Stream interface when in independent clock mode. They are also used by certain timestamping signals, and a number of device statistics and flow control signals. The clocks are shared between two ports of the MRMAC. The tx_axi_clk[0] and rx_axi_clk[0] are shared between Ports 0 and 1, The tx_axi_clk[2] and rx_axi_clk[2] are shared between Ports 2 and 3.
core	tx_core_clk[3:0] rx_core_clk[3:0]	Per-port high-speed clocks which drive the MRMAC internals.

Clock	Port	Description
serdes	rx_serdes_clk[3:0]	Per-port high-speed clocks for the GT interface. For RX side: • Narrow mode, rx_serdes_clk[3:0] is used. • Wide mode rx_alt_serdes_clk[3:0] is used. For TX side: • Narrow mode, tx_core_clk[3:0] is used. • Wide mode tx_alt_serdes_clk[3:0] is used.
Ts	tx_ts_clk[3:0] rx_ts_clk[3:0]	Per-port clocks for the timestamp interface.
Flexif	tx_flexif_clk[3:0] rx_flexif_clk[3:0]	Per-port Flex I/F clocks.
alt_serdes	tx_alt_serdes_clk[3:0] rx_alt_serdes_clk[3:0]	Per-port Alternate low-frequency clock for the GT interface For RX side: • Narrow mode, rx_serdes_clk[3:0] is used. • Wide mode rx_alt_serdes_clk[3:0] is used. For TX side: • Narrow mode, tx_core_clk[3:0] is used. • Wide mode tx_alt_serdes_clk[3:0] is used.

System Clocks

The `abp3_clk` is a mandatory clock which must be present and stable in all modes of operation. It is used for internal configuration and synchronization functions in addition to its use as a clock for the processor interface.

Transmit Clocks

The following describes the transmit clocks in the MRMAC.

- **`tx_core_clk[3:0]`**
  This internal high-speed clock drives the bulk of the MRMAC transmit datapath. This per-port clock must be generated by the GT transceiver.
- **`tx_alt_serdes_clk[3:0]`**
  This is an internally-used clock derived from the primary SerDes clocks. These clocks run at exactly half the `tx_core_clk[3:0]` frequencies. This clock is typically generated by the IP Wizard.

- **tx_flexif_clk[3:0]**
  This is used by the flex interface. When the Flex I/F is active, tx_flexif_clk[N] must operate at a frequency ≥ 1/2 of the corresponding tx_core_clk[N] frequency.
- **tx_axi_clk[2][0]**
  AXI4-Stream interface clocks. In addition to clocking the AXI4-Stream interface in independent clocking mode, the tx_axi_clk[2][0] clocks are also used only to clock certain statistics and flow control signals. The tx_axi_clk[2][0] clocks must operate at a frequency high enough to maintain the desired data rate across the AXI4-Stream bus, which is tx_axi_clk[N] must be greater than or equal to 1/2 of the corresponding tx_core_clk[N] frequency.
- **tx_ts_clk[3:0]**
  This clock drives the 1588 timestamping signaling. The MRMAC timestamp timers attempt to synchronize with the external time-of-day source which is running on this clock.

Table: Transmit Clock Data Rate

Selected Data Rate	AXI Clock Minimum Frequency (MHz)	AXI4-Stream Segment Mode	Port AXI4-Stream MODE_REG Field ctl_axis_cfg_<N>
100GE	390.625	Non-Segmented	3'b101
	322.265	Segmented	3'b111
40/50GE	322.265	Non-Segmented	All
25GE	390.625	Non-Segmented	3'b001
25GE	322.265	Non-Segmented	3'b101
10GE	322.265	Non-Segmented	All

Receive Clocks

The following describe the receive clocks in MRMAC:

- **rx_serdes_clk[3:0]**
  This GT SerDes clock is used to drive data across the SerDes interface to the MRMAC.
- **rx_core_clk[3:0]**
  This is the internal high-speed clock which drives the bulk of the MRMAC receive datapath. This per-port clock can be shared with the tx_core_clk[3:0], but depending on the application, it can be derived from the rx_serdes_clk.

  ✎ **Note**: You can opt to use tx_core_clk if rx_core_clk is off or in unstabilized state (in GT RX Reset state), when RX statistics register read is required.
- **rx_alt_serdes_clk[3:0]**
  This is an internally-used clock derived from the primary SerDes clocks. These clocks run at exactly 1/2 the rx_serdes_clk[3:0] frequencies. The logic and hook-ups for this clock are typically generated by the IP Wizard.
- **rx_flexif_clk[3:0]**
  This is used by the flex interface. When the Flex I/F is active and in a multi-lane mode (that is, data rate > 25G), rx_flexif_clk[N] must operate at a frequency ≥ 1/2 of the corresponding rx_core_clk[N] frequency. When the flex interface is in a single lane mode (25G or 10G), it must run at a frequency ≥ 1/2 of the corresponding rx_serdes_clk[N] frequency.
- **rx_axi_clk[2][0]**
  This is a per-port AXI clock. In addition to clocking the RX AXI4-Stream interface in independent clocking mode, the rx_axi_clk[2][0] clocks are also used only to clock certain statistics and flow control signals. The rx_axi_clk[2][0] clocks must operate at a frequency high enough to maintain the desired data rate across the AXI4-Stream bus, which means rx_axi_clk[N] must be greater than or equal to 1/2 of the corresponding rx_core_clk[N] (or rx_serdes_clk[N] in 10GE or 25GE mode) frequency.

  The minimum values for the rx_axi_clk[2][0] clocks for the various modes of operation match the table listed for the tx_axi_clk.

- `rx_ts_clk[3:0]`
  This clock drives the 1588 timestamping signaling. The MRMAC timestamp timers attempt to synchronize with the external time-of-day source which is running on this clock.

Resets

The MRMAC reset structure consists of active-High reset signals for the SerDes, core logic of each port, and a common AXI4-Lite reset.

All resets are asynchronous inputs and are internally synchronized to the correct clock domain for use. When asserted, resets must be held for a minimum of 24 ns or 8 × the cycle time of the associated clock. For example, `rx_core_reset[0]` must be held for 8 × `rx_core_clk[0]` cycles.

The `tx_core_reset` and `rx_core_reset` are the master reset for the pins should be maintained in the reset state until all of the clocks to that port (`rx_core_clk`, `tx_core_clk`, `rx_serdes_clk`, `rx_axi_clk`, `tx_axi_clk`, `rx_flexif_clk`, `tx_flexif_clk`, `tx_ts_clk`, and `rx_ts_clk`) are stable.

In addition to pin-level resets, the MRMAC can be software reset through the AXI4-Lite accessible per-port reset registers.

Related Information

Register Space

Reset Port Description

Reset pins and RESET_REG bits other than `tx_core_reset`, `rx_core_reset`, `tx_serdes_reset`, and `rx_serdes_reset` are for debugging purpose only. It should not be asserted by the user logic.

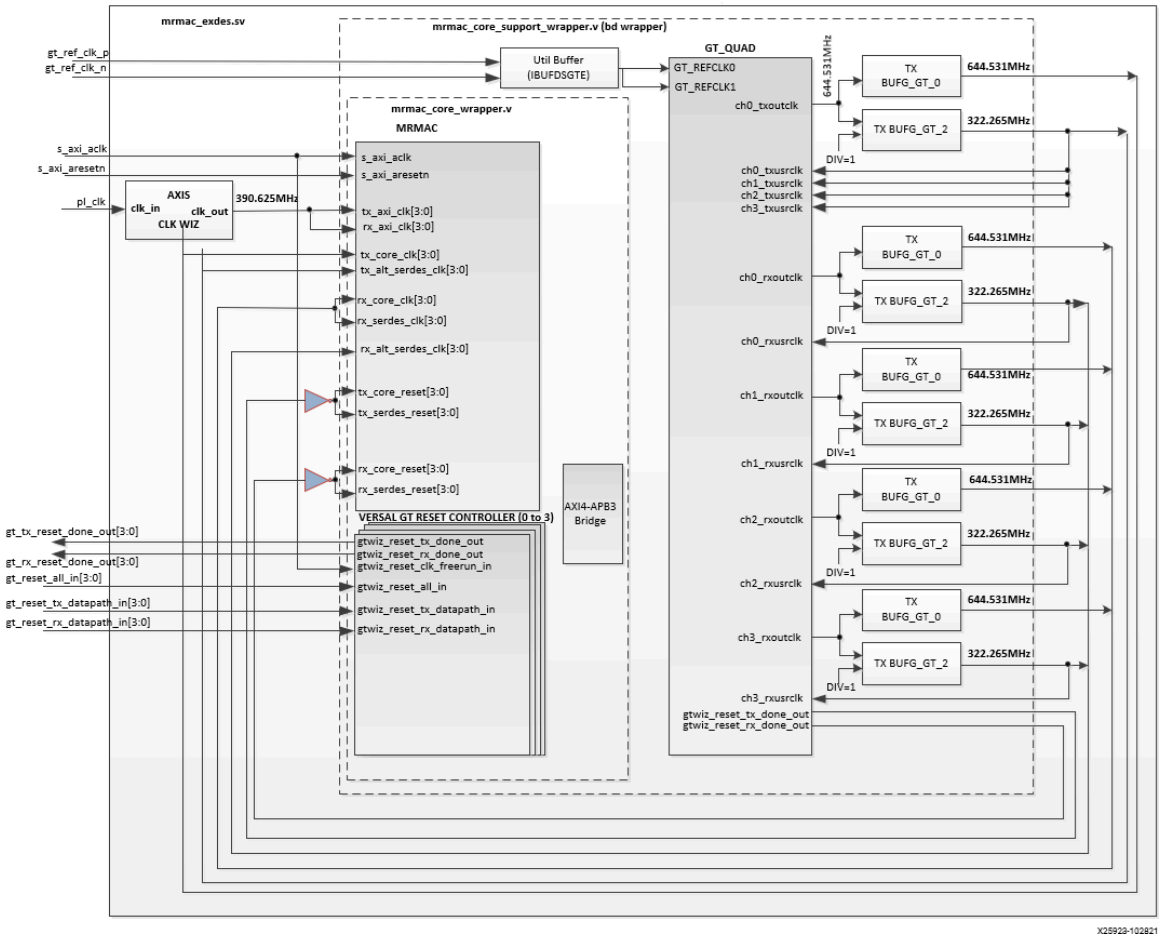
Table: Reset Port Description

MRMAC Reset Pin	Associated RESET_REG Bits (Per Port)	Description
rx_core_reset[3:0] tx_core_reset[3:0]	rx_core_reset tx_core_reset	Per-port TX and RX core reset. Asserting the reset resets the entire RX or TX datapath for that port including: • SerDes interface logic • Status registers • AXI4-Stream user interface • Flex I/F • System Timer interface and logic

MRMAC Reset Pin	Associated RESET_REG Bits (Per Port)	Description
rx_serdes_reset[3:0]	rx_serdes_reset[N:0]	Per-port RX SerDes (GT) interface reset. The RX SerDes can be reset through the register interface on a per-port basis. However, different ports have different allowable configurations and hence there are different reset fields depending on the port. Consequently, each port's RESET_REG has an N-bit rx_serdes_reset field, where N represents the number of PHY lanes present for a Port's configuration. For example, Port 0 can support operation up to and including 100G (requiring four SerDes) and so the reset field is four bits wide. However, if the port were to be configured as 50G, only rx_serdes_reset[1:0] would be in use. Meanwhile, Port 1 only supports lower speed operation and so it is directly tied to a SerDes instance. Therefore, there is only one active reset bit (rx_serdes_reset[0]).
tx_serdes_reset[3:0]	tx_serdes_reset	Per-port TX SerDes (GT) interface reset. A single reset port resets each of the TX GT interface logic. For example, tx_serdes_reset[0] resets all of the GT interface logic for Port 0, regardless of configuration.
apb3_preset		Resets the AXI4-Lite port logic, status, and statistics registers.

The clock and reset connection between MRMAC and GT is shown in the following figure.

Figure: MRMAC and GT Clock and Reset Architecture (Legacy GT Wizard)



GT Reset Control Ports

Table: GT Reset Control Ports

Port Name	I/O	Description
gtwiz_reset_all_in [3:0]	I	Reset input to the GT Reset IP. Resets the PLL and datapath.
gt_reset_tx_datapath_in [3:0]	I	TX reset datapath input to the GT Reset IP.
gt_reset_rx_datapath_in [3:0]	I	RX reset datapath input to the GT Reset IP.
gt_tx_reset_done_out [3:0]	O	TX Reset Done output from the GT Reset IP.
gt_rx_reset_done_out [3:0]	O	RX Reset Done output from the GT Reset IP.

User Interfaces

This section reviews the primary MRMAC user interfaces.

Transmit Operation

In the transmit (outbound) direction, data is presented to the AXI4-Stream interface by the user logic for handling by the MRMAC subsystem. There are two possible AXI configurations set by the port's MODE_REG register:

- Non-segmented
- Segmented

The handshaking and signaling is different for each configuration.

Non-Segmented

In the non-segmented configuration for transmit, there are two operations:

- Normal transmission
- Aborting transmission

Normal Transmission

The timing of a normal frame transfer is shown in the following figure. This example corresponds to a single 25G stream transmitting on Client 0 with an interface 128 bits wide.

When the client wants to transmit a frame, it asserts the `tx_axis_tvalid_0` and places the data and control in `tx_axis_tdata_(0-1)` and `tx_axis_tkeep_(0-1)` in the same clock cycle. When this data is accepted by the core (indicated by `tx_axis_tready_0` being asserted) the client must provide the next cycle of data.

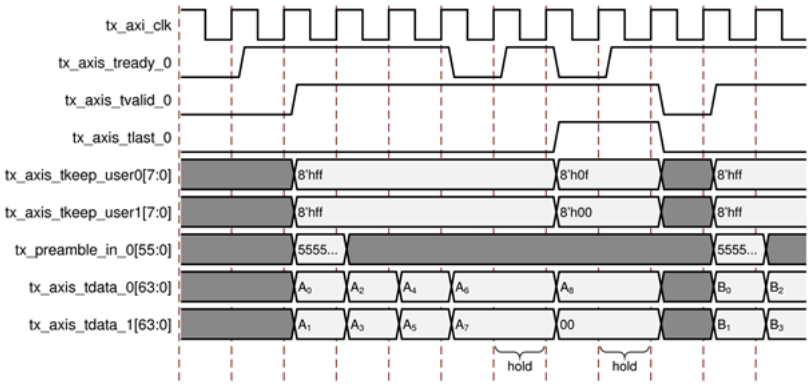
If `tx_axis_tready_0` is not asserted by the core (for example, in the cycles preceding the cycles labeled hold in the figure), the client must hold the current valid data value until it is ready. The transmit MRMAC interface routinely deasserts `tx_axis_tready` as it works to maintain compliant IPG. The user logic must be prepared to stall its egress pipeline as necessary.

The end of packet is indicated to the core by the user logic asserting `tx_axis_tlast` for 1 cycle. The bits of `tx_axis_tkeep_user(0-1)` are set to indicate which bytes in the final data transfer are valid. In this case, only bytes 0, 1, 2, and 3 of `tx_axis_tdata_0` contain valid data. The rest of the data signals do not contain valid data and can be set to any value (0 recommended).

In the example, `tx_axis_tready` happens to deassert as the user logic is presenting the final transfer. Once again the user logic must hold the data on the interface until the transfer is accepted.

✎ **Note:** After `tx_axis_tlast` is deasserted, any data and control is deemed invalid until `tx_axis_tvalid` is asserted next.

Figure: AXI4-Stream Non-Segmented Transmit Normal Frame Transmission



Aborting a Transmission

In cut-through Ethernet architectures, it is sometimes necessary to intentionally abort the transmission of a frame that is currently in flight. A transmit operation can be aborted in one of two ways:

- An explicit error in which a frame transfer is aborted by asserting the appropriate `tx_axis_tkeep_user[8]` (ERR) signal along with the corresponding `tx_axis_tlast` signal.
- An implicit underrun, in which a frame transfer is aborted by deasserting the relevant `tx_axis_tvalid` signal before the frame has completed (that is, before `tx_axis_tlast` has been asserted).

When either of the two scenarios occurs during a frame transmission, the core inserts error codes into the data stream to flag the current frame as an errored frame. It remains the responsibility of the client to re-queue the aborted frame for transmission, if necessary.

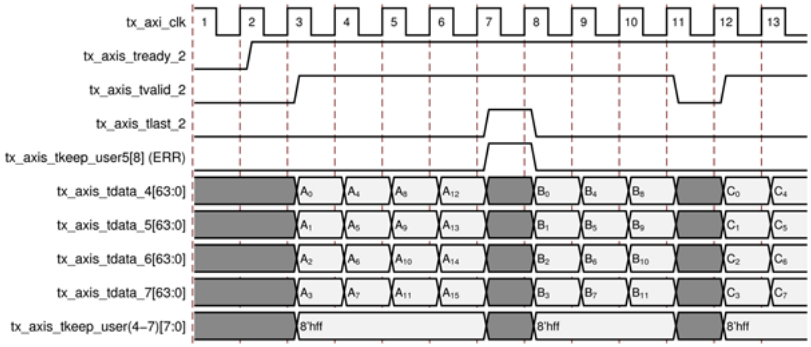
The action taken by the MRMAC when a frame is flagged for abort is configurable. The behavior is configured using the register setting `ctl_tx_corrupt_fcs_on_err` field of the CONFIGURON_TX_REG register. Options include inserting an |E| codeword into the packet, stomping the FCS (all FCS bits flipped), and corrupting the FCS(8 bits of the FCS flipped). The following table lists the codewords you can enter along with their meaning:

Error Code	Description
2'h0	Insert Error Code into Packet
2'h1	Stomp FCS (invert each FCS bit)
2'h2	Corrupt FCS (invert last byte)
2'h3	undefined

In the following 50G (Client 2) example, both styles of frame abort are depicted. On the rising edge of `tx_axi_clk` cycle #8, Frame A is explicitly aborted by asserting ERR. Later, on the rising edge of `tx_axi_clk` cycle #12, Frame B is implicitly aborted by deasserting `tx_axis_tvalid_2` before `tx_axis_tlast_2` is asserted.

✎ **Note:** The next frame can begin immediately. There is no requirement for wait states between terminated frames.

Figure: AXI4-Stream Non-Segmented Transmit Abort Transmission



Related Information

Register Space

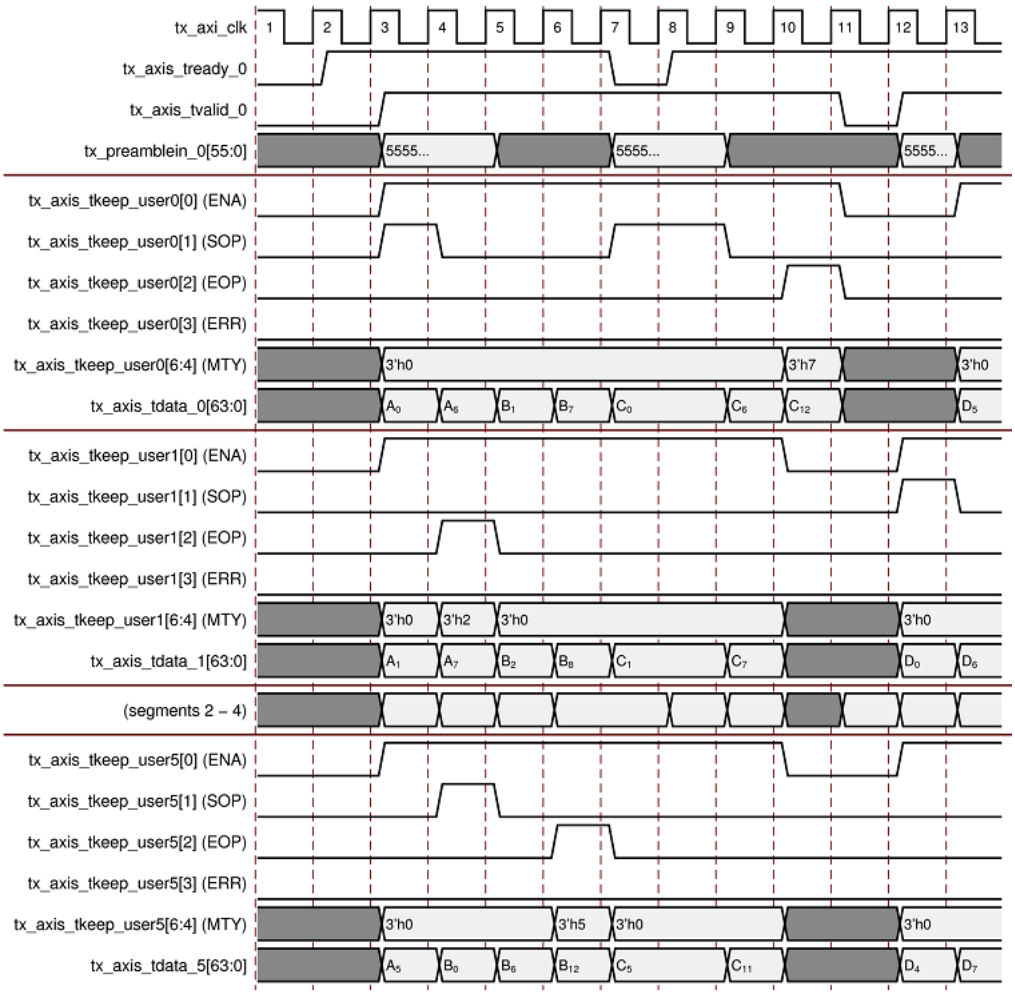
Segmented

In the segmented configuration, the packets span bus segments and the user logic can start or end a packet in any given segment.

Normal Transmission

In the following figure a typical sequence of frame transmits are shown. Segments 2 through 4 are not shown to keep the diagram compact.

Figure: AXI4-Stream Segmented Transmit Normal Frame Transmission



Here are the highlights of this sequence of operations:

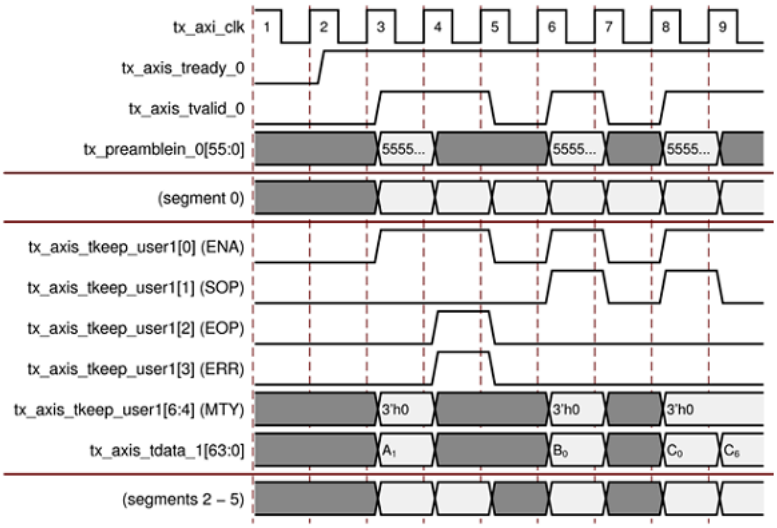
1. In cycle #4, a new frame (Frame A) is started in Segment 0. Frame A is 62 bytes long (it becomes 66B long after FCS is appended). Segment 0 ENA is asserted along with SOP. The first word of data, A_0, appears. Preamble is provided on the tx_preamblein_0 port. The frame continues in Segment 1, 2, 3, 4, and 5.
2. In cycle #5, frame A continues in Segment 0, but ends in Segment 1 when EOP is asserted. The MTY signal toggles to indicate the number of unused bytes in this segment (3'h2 in this case). A new frame, Frame B, begins in Segment 5 (data = B_0). Again, the `tx_preamblein_0` provides the preamble.
3. Frame B continues through cycle #6 across all segments (data B_1 through B_6).
4. In cycle #7, Frame B ends in Segment 5 with data B_{12}. EOP is asserted in this segment.
5. In cycle #8 a new frame, Frame C, is initiated. However, the AXI4-Stream interface has deasserted `tx_axis_tready_0`. The user logic must hold the inputs until `tx_axis_tready_0` is asserted again.
6. In cycle #9 the `tready` signal has been asserted again and so the transfer is accepted. Frame C begins in Segment 0 (SOP asserted) and continues through to Segment 5.
7. Frame C continues in cycle #10, across Segment 0 through 5.
8. In cycle #11, Frame C ends in Segment 0 with the assertion of EOP. The user logic does not have any more data to send so it keeps ENA deasserted in segments 1 through 5.
9. In the subsequent cycle, #12, there is no data at all to send and so the user logic deasserts `tx_axis_tvalid_0` to indicate that the bus is completely idle.
10. Finally, in cycle #13, Segment 1, a new frame, Frame D, begins. The user logic asserts `tx_axis_tvalid_0` again. Frame D continues through to Segment 5.
11. Frame D continues in cycle #14 and beyond.

Aborting a Transmission

As is the case in non-segmented operation, a packet can be aborted on the segmented interface either explicitly (asserting ERR) or implicitly (deasserting `tvalid` without first completing the affected frame).

The following diagram shows both cases. To keep the diagram compact, segments 0 and 2-5 are omitted. Assume they contain valid transactions.

Figure: AXI4-Stream Segmented Transmit Abort Transmission



In the figure, Frame A has started in cycle #4 (in Segment 0, not shown) and continues through to the end of Segment 5. An explicit frame transfer abort occurs in Segment 1 of cycle #5.

Next, a new frame (Frame B) begins in segment 1 of cycle #7, but this frame is aborted implicitly in cycle #8 when `tx_axis_tvalid_0` is deasserted. The diagram ends with Frame C being transferred. It is acceptable to begin a new frame in the segment immediately following an aborted frame. There is no need for any idle bus segments.

Receive Operation

In the receive (inbound) direction data is presented to the user logic by the AXI4-Stream interface. There are two possible MODE_REG register controlled AXI configurations:

- Non-segmented
- Segmented

The handshaking and signaling is different for each configuration.

Non-Segmented

In the non-segmented configuration for receive, there are two operations:

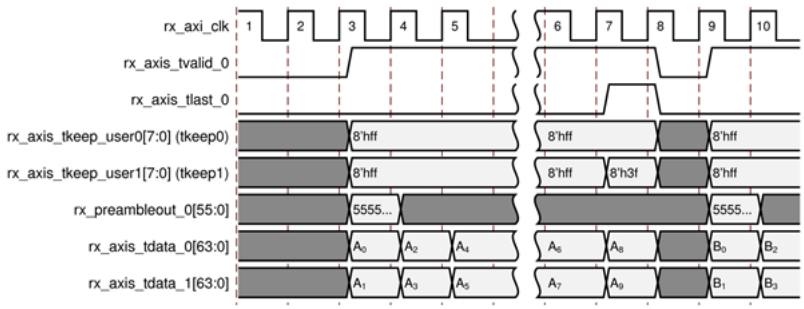
- Normal frame receive
- Frame receive with errors

Normal Frame Receive

The timing of a normal inbound frame transfer is represented in the following figure. The client must be prepared to accept data at any time. There is no buffering within the core to allow for latency in the receive client. When frame

reception begins, data is transferred on consecutive clock cycles to the receive client.

Figure: AXI4-Stream Non-Segmented Normal Frame Receive



During frame reception, `rx_axis_tvalid` is asserted to indicate that valid frame data is being transferred to the client on `rx_axis_tdata`. All bytes are always valid throughout the frame, as indicated by all `rx_axis_tkeep` bits being set to 1, except during the final transfer of the frame when `rx_axis_tlast` is asserted. During this final transfer of data for a frame, `rx_axis_tkeep` bits indicate the final valid bytes of the frame using the mapping from above. The valid bytes of the final transfer always lead out from `rx_axis_tdata[7:0]` (`rx_axis_tkeep[0]`) because Ethernet frame data is continuous and is received least significant byte first.

The `rx_axis_tlast` is asserted and `rx_axis_tuser` is deasserted, along with the final bytes of the transfer, only after all frame checks are completed. This is after the frame check sequence (FCS) field has been received. The core asserts the `rx_axis_tuser` signal to indicate that the frame was successfully received and that the frame should be analyzed by the client. This is also the end of packet signaled by `rx_axis_tlast` asserted for one cycle.

In the figure, the two receive frame transactions on a 128-bit Client 0 interface. Frame A begins in Cycle #4 with the assertion of `rx_axis_tvalid_0`. The frame completes in cycle #8 when `rx_axis_tlast_0` is asserted. The `keep0` and `keep1` signals declare that all eight bytes of `rx_axis_tdata0` are valid, but only six bytes of `rx_axis_tdata1` are good.

Although it is possible for a new frame to begin in the next cycle, that is not the case in this example. However, a new frame, Frame B, does begin in cycle #10.

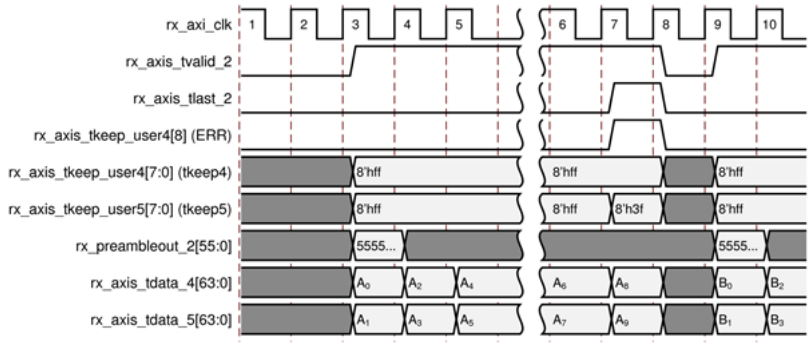
Frame Receive with Errors

The following conditions cause the assertion of `rx_axis_tlast_N[8]` along with `rx_axis_tuser = 1` signifying a bad frame:

- FCS errors is detected
- Packets are shorter than 64 bytes (undersize or fragment frames)
- Frames of length greater than the maximum transmission unit (MTU) Size programmed are received and MTU Size Enable Frames are enabled
- Any control frame that is received is not exactly the minimum frame length unless Control Frame Length Check Disable is set
- Received data stream contains error codes

The case of an unsuccessful frame reception (for example, a run frame or a frame with an incorrect FCS) is shown in the following figure. In this situation, a 128 bit-wide stream active on Client 2 is depicted. The bad frame is received but the signal `rx_axis_tkeep_userN[8]` (ERR) is asserted to the client at the end of the frame (cycle #8). It is the responsibility of the user logic to drop the data already transferred for this frame.

Figure: AXI4-Stream Non-Segmented Receive Frame with Errors



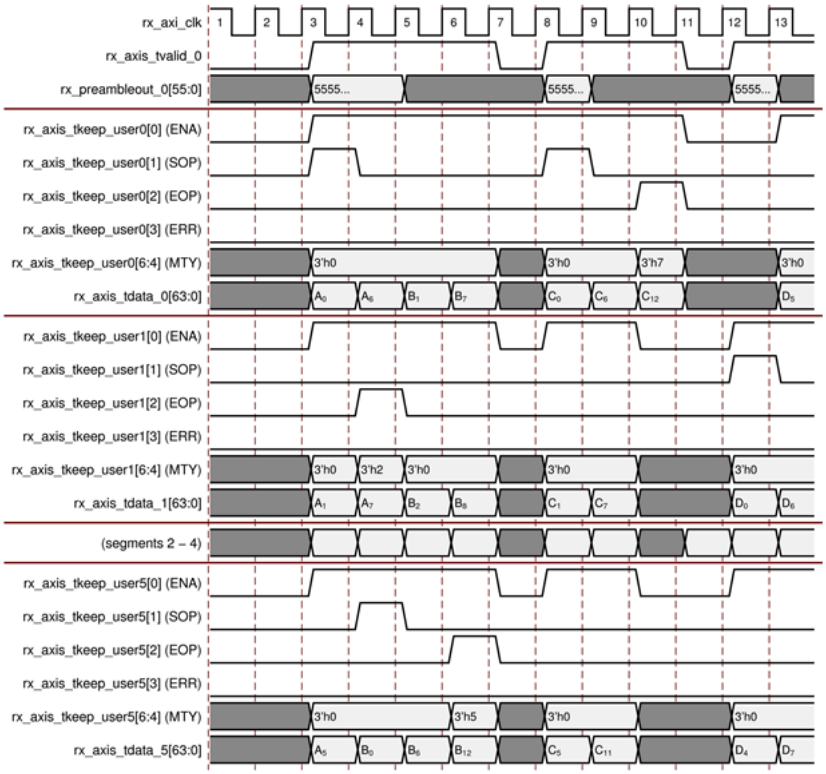
Segmented

In the segmented configuration, the packets span bus segments and the MRMAC can start or end a packet in any given segment.

Normal Frame Receive

In the following figure, a typical sequence of frame transmits are shown. As before, Segments 2 through 4 are omitted to keep the diagram compact. This figure depicts the same sequence of events similar to the previous segmented transmit example, but in the receive direction.

Figure: AXI4-Stream Segmented Normal Frame Receive



As in the earlier example, Frame A starts in Segment 0 of cycle #4 which terminates with EOF in Segment 1 of cycle #5. Frame B begins in Segment 5 of the same cycle, which ends in Segment 5 of cycle #7.

There is no bus activity in cycle #8, therefore the MRMAC deasserts `rx_axis_tvalid_0` for the cycle. A new frame, Frame C, begins in cycle #9 and ends in cycle #11. Cycle #12 is idle (`tvalid` = 0) and the final frame in this example is transferred starting in cycle #13, Segment #1.

Frame Receive with Errors

In the receive direction, a frame is flagged as errored for the same reasons as the non-segmented case. Each segment contains an ERR signal, `rx_axis_tkeep_user_N[3]`. An error is signaled when the MRMAC asserts the ERR signal at the same time as the EOP and ENA signals. In the event of a signaled error, the user logic is responsible for discarding the packet.

Frame Preemption Support

The MRMAC AXI4-Stream provides support for the sending and receiving of Preemption packets (mPackets) as defined in Clause 99 of IEEE Standard for Ethernet (IEEE Std 802.3-2018). Preemption capabilities include outbound preemption header insertion, include outbound preemption header insertion, and mFCS calculation and appending, inbound preemption header extraction, and inbound preempted packet integrity checking.

The user logic is required to handle all other aspects of the preemption protocol, including auto-negotiation of the preemption feature with the link partner and received packet fragment reassembly.

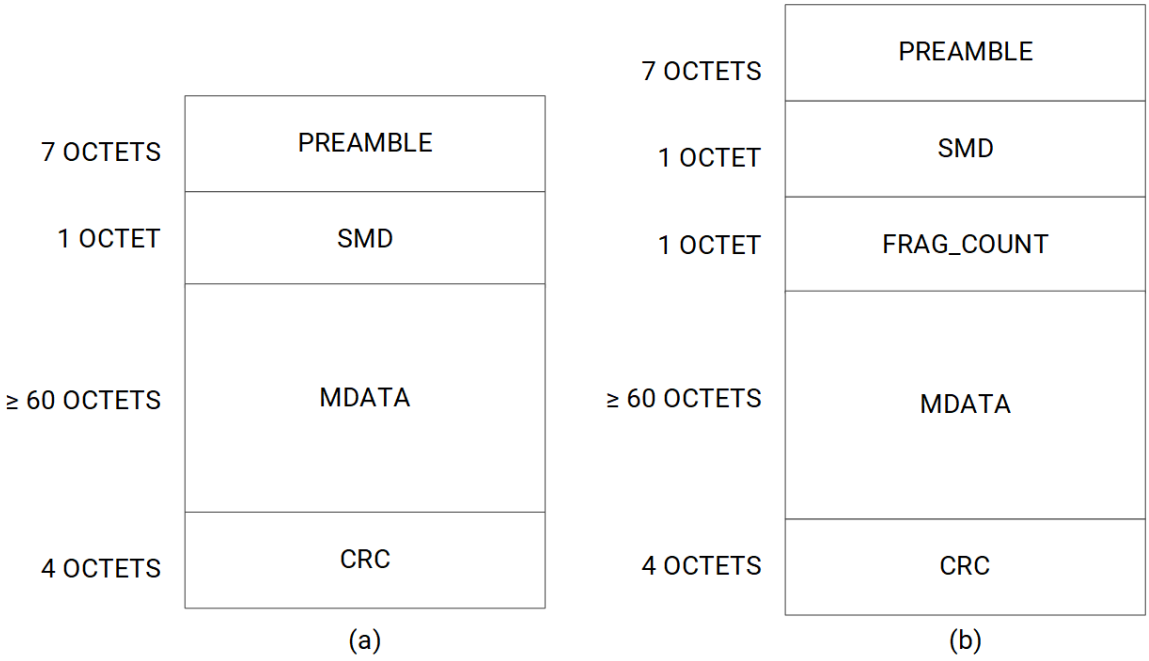
Preemption capability is available on a per-port basis and is enabled by setting the `ctl_preempt_enable_<N>` field of the port's MODE_REG register. Preemption should only be enabled if it is determined that the link partner supports the preemption capability.

✎ **Note:** Preemption is not available when IEEE 1588 1-step timestamping is enabled.

Preemption Frame Structure

The following figure depicts the two mPacket formats. Packet (a) is used for express packets, a complete preemptable packet, or an initial fragment of a packet. Packet (b) is a continuation packet. The SMD field replaces the SFD field of a normal Ethernet frame and is used to indicate which mPacket format is in use. For more information, see IEEE 802.3-2018.

Figure: Preemption Header



Preemption Header Insertion/Extraction

Preemption leverages the MRMAC custom preamble features to insert and extract the necessary Preamble, SMD, and FRAG_COUNT fields. Insertion and extraction of custom preamble is enabled on a per-port basis by setting the `ctl_tx_custom_preamble_enable_<N>` field of the CONFIGURATION_TX_REG register to 1. This feature must be enabled for preemption to work properly.

When custom preamble is enabled, the user logic communicates the desired preamble + SFD/SMD/FRAG_COUNT field for outbound frames on a port-by-port, packet-by-packet basis by means of the signals `tx_preamblein_N[55:0]`. In the receive direction, the signals `rx_preambleout_N[55:0]` are used to communicate the extracted preamble + SFD/SMD/FRAG_COUNT from the received packets.

✎ **Note:** The `tx_preamblein/rx_preambleout` signals are 7B wide. The first byte of preamble is never transmitted in 64B/66B Start codes, so custom preambles would carry 6 bytes of preamble followed by one byte of SMD (preemption header format (a)), or 5 bytes of preamble, followed by one byte of SMD, and one byte of FRAG_COUNT (preemption header format (b)).

Outbound Preemption

In the transmit direction, the MRMAC supports packet preemption by allowing packets currently in flight across the AXI4-Stream interface to be preempted.

When preemption is enabled, the user logic provides appropriate mPacket header fields for all outbound frames using the preamble insertion function. The transmit path does not interpret any of the mPacket header fields but instead places these fields, as provided by `tx_preamblein_<N>`, into the outbound frames.

During packet transfer on TX AXI4-Stream, user logic can indicate that a packet has been preempted by asserting the preempt signal input (a bit in the `tx_axis_tkeep_user_<N>` signal as listed in the port descriptions) along with EOP/tlast. The user logic can then send its express packet.

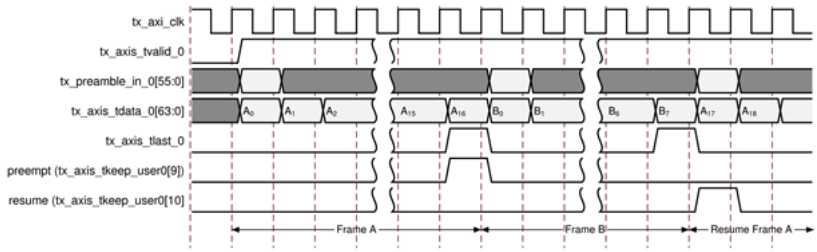
If transmit FCS insert is enabled, the TX logic calculates and inserts an mCRC on the preempted packet. The partially calculated frame CRC is stored away so it can be resumed later.

After the express packet has been transmitted, the user logic can resume the previously preempted frame by asserting the `resume` signal along with SOP/tvalid. The MRMAC resumes calculating the CRC for the interrupted packet. As before, the user logic must provide a suitable mPacket header (with SMD and FRAG_COUNT set appropriately).

Care must be taken by the user logic to ensure that any outbound packet (including fragments) respects the 64B min size for a fragment/packet.

In this example, an in-flight egress frame, Frame A, transmits cycles A_0 through A_{16}. However, it is preempted when the user logic asserts both `tx_axis_tlast_0` and `tx_axis_tkeep_user0[9]` (preempt). The MRMAC calculates an mCRC for Frame A and sets aside the partial FCS calculation for later.

Figure: Transmit Frame Preemption



The priority frame (Frame B) begins in the very next cycle. It is a minimum length frame of 64B. Its final data (B_7) is signaled normally with `tx_axis_tlast_0`. The MRMAC transmits this frame normally.

Finally, Frame A resumes in the next cycle with the user logic asserting the resume signal (`tx_axis_tkeep_user0[10]`). The user logic-provided preamble (`tx_preamble_in_0`) must provide an appropriate preemption header. The partially calculated FCS from earlier is retrieved and packet data transmission resumes with packet data A_{17}.

Related Information

Port Descriptions

Inbound Preemption

The receive logic assists packet preemption by performing integrity checks on received mPackets and extracting the mPacket header fields for presentation on the `rx_preambleout[55:0]` port.

Preempted frames are signaled to the user logic by means of symmetrical preempt and resume `tuser` flags. These signals indicate if a received packet was preempted upstream or if it represents a continuation fragment packet.

For continuation packets, the FRAG_COUNT field, found in the `rx_preambleout` signals, indicates to the user logic that the packet must be reassembled. Reassembly is the responsibility of the user logic.

The AXI4-Stream ERR flag is used by the MRMAC to indicate that a resumed packet has failed the mCRC check. In the event a continuation packet is received with a bad mCRC, it is left to the user logic to take appropriate action.

The RX logic supports two integrity checking modes, which are controlled through bits found in the CONFIGURON_RX_REG register.

Table: Preemption Integrity Checking Modes

ctl_tx_check_sdf	ctl_rx_check_preamble	Result

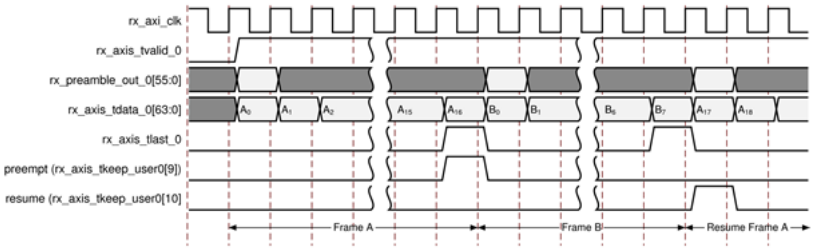
ctl_tx_check_sdf	ctl_rx_check_preamble	Result
1	1	Strict SMD checking is enabled. In this mode, a received packet whose SMD byte does not match valid values from Table 99-1 from the IEEE 802.3-2018 (Clause 99) (see below) results in the AXI4-Stream ERR signal being asserted.
0	0	The RX preemption logic acts in pass-through mode and packets with invalid SMD values are passed through to the user logic without ERR. In this mode, received packets with SMD values not matching Table 99-1 toggles the error statistics (that is, stat_rx_bad_sfd) but not have the ERR flag asserted on the AXI interface. ✎ **Note:** If an SMD value not found in Table 99-1 is received, it indicates to the RX logic that the frame's FCS field must be checked against its expected CRC value, not an mCRC.

Table: SMD Values (Table 99-1)

mPacket Type	Notation	Frame Count	Value
verify packet	SMD-V	-	0x07
respond packet	SMD-R	-	0x19
express packet	SMD-E	-	0xD5
preemptable packet start	SMD-S0	0	0xE6
	SMD-S1	1	0x4C
	SMD-S2	2	0x7F
	SMD-S3	3	0xB3
continuation fragment	SMD-C0	0	0x61
	SMD-C1	1	0x52
	SMD-C2	2	0x9E
	SMD-C3	3	0x2A

In this example, an inbound frame (Frame A) is pre-empted by the arrival of priority packet Frame B. Frame A resumes later with data A_{17}. It is the responsibility of the user logic to parse the extracted `rx_preamble_out_0` to correctly re-assemble Frame A from the two segments.

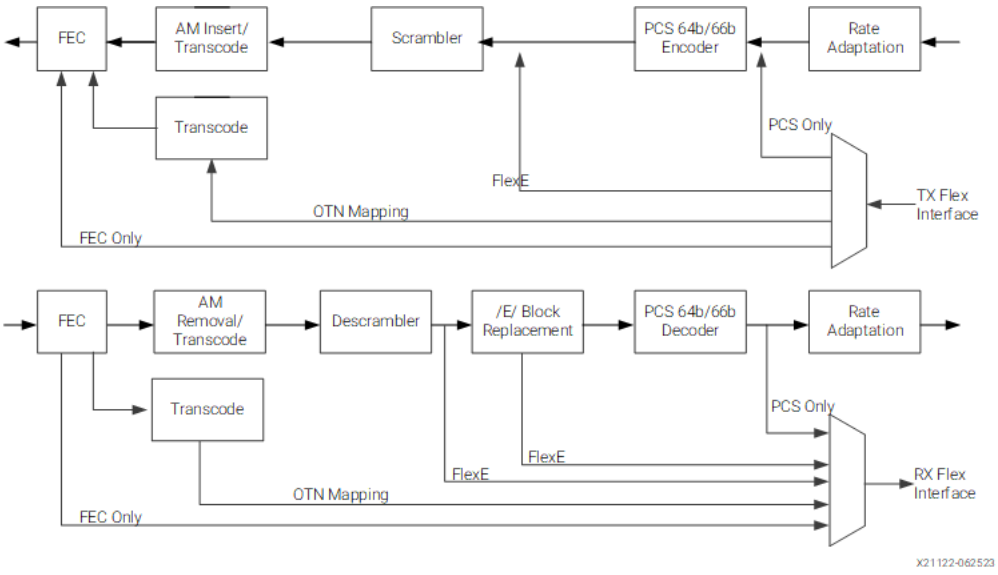
Figure: Receive Frame Preemption



Flexible Interface User Interface

The flexible interface (Flex I/F) is a series of tap points along the PHY/FEC datapath that allows external logic to connect to the internal hardened PCS resources as a PCS client, an OTN PHY client, or a FlexE client. The interface also permits direct access to the on-board FEC resources. The following diagram illustrates RX and TX functions in the path between the PCS Lane alignment logic to the Flex I/F port.

Figure: Flex Interface Tap Points



To support the multiple applications, each flex interface port has logic, controls to enable/disable the descramble/scramble logic, alignment insertion/removal logic, invalid sync-header Error block replacement, and PCS encoder/decoder logic.

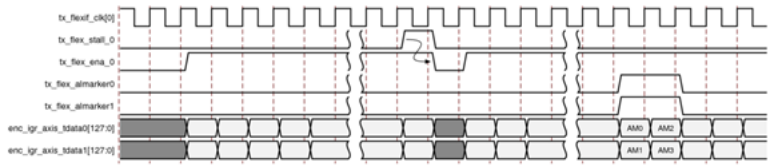
Flexible Interface Examples

Typical Flex I/F TX and RX transactions can be seen in the following diagrams. In this example, data transfer into the Flex I/F port is active on Client 0. The user logic drives data (`tx_flex_data1` and `tx_flex_data2`) into the interface while asserting `tx_flex_ena_0`.

Note the relationship between the per-port `tx_flex_stall_<N>` signal and `tx_flex_ena_<N>`. The stall signal is output to backpressure the transfers into the Flex I/F. The interface requires that you suspend transfers (`tx_flex_enain` == 0) exactly one cycle after the `tx_flex_stall_<N>` is asserted to 1.

✎ **Note:** When the flexif interface is configured for OTN mode (`ctl_tx_flexif_select` = 3'b000), the internal scrambler is disabled, and AMs are passed through but need to be identified. The user logic is responsible for inserting alignment markers. When an alignment marker is inserted, the user logic must assert `tx_flex_alignmarker<N>` to indicate that the data includes an AM. This prevents the scrambler from scrambling the alignment markers. However, other modes do not use `tx_flex_alignmarker<N>`.

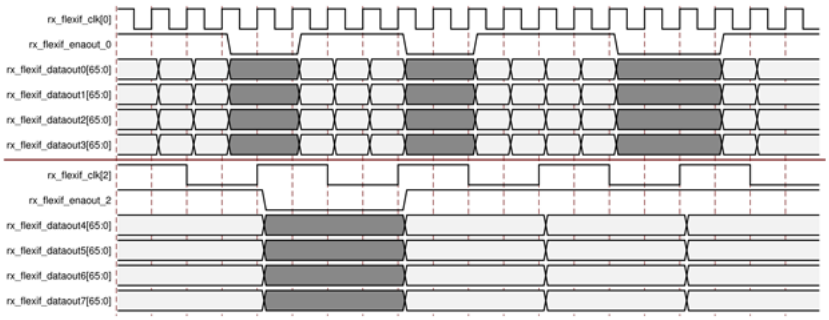
Figure: TX Flex Interface



In the receive direction, the Flex I/F outputs data at the configured port rate. Valid data is indicated by the per-port signal `rx_flex_ena_<N>`. When `rx_flex_ena_<N>` is deasserted, the data is invalid.

✎ **Note:** There is no backpressure from the user logic back to the Flex I/F port. The user logic should be able to keep up with the selected data rate.

Figure: RX Flex Interface



Pause Operation

The MRMAC subsystem provides a comprehensive mechanism for pause packet termination and generation. The TX and RX have independent interfaces for processing pause information as described in this section.

The subsystem is capable of handling 802.3x and priority-based pause operation. The RX path parses pause packets and presents the extracted quanta on the status interface. The TX path can accept pause packet requests from the control interface and inject the requested packets into the data stream. Both global pause packets and priority-based pause packets are handled.

When both `pause_req` and `pause_ack` signals are asserted, there might be a slight variation in the total time due to internal resynchronization of pause interface signals with the core clock and a dependency on the actual bit time on the clock frequency of the serdes interface (which is impacted by the ppm error on the recovered clock). The synchronization error is noticeable for small quantas, while the ppm error is more pronounced for large quantas. For small quantas, you might experience an error of approximately three quanta times due to clock synchronization.

✎ **Note:** 802.3x and global pause are used interchangeably throughout the document.

TX Pause Generation

A pause packet can be requested for transmission using the `ctl_tx_pause_req[8:0]` and `ctl_tx_pause_enable[8:0]` input buses. Bit[8] corresponds to global pause packets and Bits[7:0] correspond to priority pause packets. Each bit of this bus must be held at a steady state for a minimum of 16 cycles before the next transition.

⚠ CAUTION!Requesting both global and priority pause packets at the same time results in unpredictable behavior and must be avoided.

The contents of the pause packet are determined using the following per-port CONFIGURATION_TX_FLOW_CONTROL_* registers which contain the below fields.

Global pause packets (N = 0..3):

```
ctl_tx_da_gpp_N[47:0]
ctl_tx_sa_gpp_N[47:0]
ctl_tx_ethertype_gpp_N[15:0]
ctl_tx_opcode_gpp_N[15:0]
ctl_tx_pause_quanta8_N[15:0]
```

Priority pause packets (N = 0..3):

```
ctl_tx_da_ppp_N[47:0]
ctl_tx_sa_ppp_N[47:0]
ctl_tx_ethertype_ppp_N[15:0]
ctl_tx_opcode_ppp_N[15:0]
ctl_tx_pause_quanta0_N[15:0]
ctl_tx_pause_quanta1_N[15:0]
ctl_tx_pause_quanta2_N[15:0]
ctl_tx_pause_quanta3_N[15:0]
ctl_tx_pause_quanta4_N[15:0]
ctl_tx_pause_quanta5_N[15:0]
ctl_tx_pause_quanta6_N[15:0]
ctl_tx_pause_quanta7_N[15:0]
```

The MRMAC Ethernet subsystem automatically calculates and adds the FCS to the packet. For priority pause packets, the MRMAC Ethernet subsystem also automatically generates the enable vector based on the priorities that are requested.

To request a pause packet, you must set the corresponding bit of the `ctl_tx_pause_req[8:0]` and `ctl_tx_pause_enable[8:0]` bus to a 1 and keep it at 1 for the duration of the pause request (that is, if these inputs are set to 0, all pending pause packets are canceled). The 10G/25G Ethernet subsystem transmits the pause packet immediately after the current packet in flight is completed.

✎ **Important:** Each bit of this bus must be held at a steady state for a minimum of 16 cycles before the next transition.

To re-transmit pause packets, the 10G/25G Ethernet subsystem maintains a total of nine independent timers, one for each priority and one for global pause. These timers are loaded with the value of the corresponding input buses. After a pause packet is transmitted, the corresponding timer is loaded with the similar value of the `ctl_tx_pause_refresh_timer[8:0]` input bus. When a timer times out, another packet for that priority (or global) is transmitted as soon as the current packet in flight is completed. Additionally, you can manually force the timers to 0, and therefore force a retransmission by setting the `ctl_tx_resend_pause` input to 1 for one clock cycle.

To reduce the number of pause packets for priority mode operation, a timer is considered timed out if any of the other timers time out. Additionally, while waiting for the current packet in flight to be completed, any new timer that times out or any new requests are merged into a single pause frame. For example, if two timers are counting down and you send a request for a third priority, the two timers are forced to be timed out. A pause packet for all three priorities is sent as soon as the current in-flight packet (if any) is transmitted. Similarly, if one of the two timers times out without an additional request, both timers are forced to be timed out. Then, a pause packet for both priorities is sent as soon as the current in-flight packet (if any) is transmitted.

You can stop pause packet generation by setting the appropriate bits of `ctl_tx_pause_req[8:0]` or `ctl_tx_pause_enable[8:0]` to 0.

RX Pause Termination

The MRMAC Ethernet subsystem terminates global and priority pause frames and provides a simple handshaking interface to allow user logic to respond to pause packets.

Determining Pause Packets

The following process is used to determine if a given frame is a Pause Frame:

1. Checks are performed to see if a packet is a global or a priority control packet. Packets that pass step 1 are forwarded only if `ctl_rx_forward_control` field of the CONFIGURATION_RX_FLOW_CONTROL_REG1 is set to 1.
2. If step 1 passes, the packet is checked to determine if it is a global pause packet.
3. If step 2 fails, the packet is checked to determine if it is a priority pause packet.

In the following pseudo code, DA is the destination address, SA is the source address, OPCODE is the opcode, and ETYPE is the ethertype/length field that are extracted from the incoming packet.

For step 1, the pseudo code for the checking function is:

```
da_match_gcp = (!ctl_rx_check_mcast_gcp && !ctl_rx_check_ucast_gcp) ||
               ((DA == ctl_rx_pause_da_ucast) && ctl_rx_check_ucast_gcp) ||
               ((DA == 48'h0180c2000001) && ctl_rx_check_mcast_gcp);

sa_match_gcp = !ctl_rx_check_sa_gcp || (SA == ctl_rx_pause_sa);

etype_match_gcp =  !ctl_rx_check_etype_gcp || (ETYPE == ctl_rx_etype_gcp);

opcode_match_gcp = !ctl_rx_check_opcode_gcp ||
                   (OPCODE >= ctl_rx_opcode_min_gcp && OPCODE <= ctl_rx_opcode_max_gcp);

global_control_packet = da_match_gcp && sa_match_gcp && etype_match_gcp &&
                                                opcode_match_gcp && ctl_rx_enable_gcp;

da_match_pcp =  (!ctl_rx_check_mcast_pcp && !ctl_rx_check_ucast_pcp) ||
                ((DA == ctl_rx_pause_da_ucast) && ctl_rx_check_ucast_pcp) ||
                ((DA == ctl_rx_pause_da_mcast) && ctl_rx_check_mcast_pcp);

sa_match_pcp =  !ctl_rx_check_sa_pcp || (SA == ctl_rx_pause_sa);

etype_match_pcp =  !ctl_rx_check_etype_pcp || (ETYPE == ctl_rx_etype_pcp);

opcode_match_pcp = !ctl_rx_check_opcode_pcp ||
                   (OPCODE >= ctl_rx_opcode_min_pcp && OPCODE <= ctl_rx_opcode_max_pcp);

priority_control_packet = da_match_pcp && sa_match_pcp && etype_match_pcp &&
                                                  opcode_match_pcp && ctl_rx_enable_pcp;

control_packet = global_control_packet || priority_control_packet;
```

For step 2, the following pseudo code shows the checking function:

```
da_match_gpp =            (!ctl_rx_check_mcast_gpp && !ctl_rx_check_ucast_gpp) ||
                      ((DA == ctl_rx_pause_da_ucast) && ctl_rx_check_ucast_gpp) ||
                      ((DA == 48'h0180c2000001) && ctl_rx_check_mcast_gpp);
```

```
sa_match_gpp =        !ctl_rx_check_sa_gpp || (SA == ctl_rx_pause_sa);

etype_match_gpp =     !ctl_rx_check_etype_gpp || (ETYPE == ctl_rx_etype_gpp);

opcode_match_gpp =    !ctl_rx_check_opcode_gpp || (OPCODE == ctl_rx_opcode_gpp);

global_pause_packet = da_match_gpp && sa_match_gpp && etype_match_gpp &&
                      opcode_match_gpp && ctl_rx_enable_gpp;
```

For step 3, the following pseudo code shows the checking function:

```
da_match_ppp =                  (!ctl_rx_check_mcast_ppp && !ctl_rx_check_ucast_ppp) ||
                                              ((DA == ctl_rx_pause_da_ucast) && ctl_rx_check_ucast_ppp) ||
                                              ((DA == ctl_rx_pause_da_mcast) && ctl_rx_check_mcast_ppp);

sa_match_ppp =          !ctl_rx_check_sa_ppp || (SA == ctl_rx_pause_sa);

etype_match_ppp =       !ctl_rx_check_etype_ppp || (ETYPE == ctl_rx_etype_ppp);

opcode_match_ppp =      !ctl_rx_check_opcode_ppp || (OPCODE == ctl_rx_opcode_ppp);

priority_pause_packet = da_match_ppp && sa_match_ppp && etype_match_ppp &&
                        opcode_match_ppp && ctl_rx_enable_ppp;
```

Pause Interface

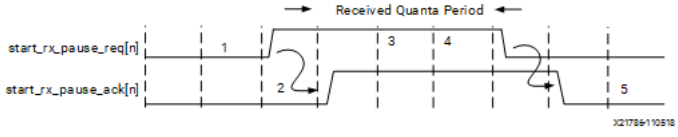
A simple handshaking protocol is used to alert the user logic to the reception of pause packets using the `ctl_rx_pause_enable[8:0]`, `stat_rx_pause_req[8:0]`, and `ctl_rx_pause_ack[8:0]` buses. For these buses, Bit[8] corresponds to global pause packets and Bits[7:0] correspond to priority pause packets.

The following steps occur when a pause packet is received:

1. If the corresponding bit of `ctl_rx_pause_enable[8:0]` is 0, the quanta is ignored and the hard MRMAC stays in step 1. Otherwise, the corresponding bit of the `stat_rx_pause_req[8:0]` bus is set to 1 and the received quanta is loaded into a timer.
   If one of the bits of `ctl_rx_pause_enable[8:0]` is set to 0 (disabled) when the pause processing is in step 2 or later, the subsystem completes the steps as normal until it comes back to step 1.
2. If `ctl_rx_check_ack` input is 1, the subsystem waits for you to set the appropriate bit of the `ctl_rx_pause_ack[8:0]` bus to 1.
3. After you set the proper bit of `ctl_rx_pause_ack[8:0]` to 1, or if `ctl_rx_check_ack` is 0, the subsystem starts counting down the timer.
4. When the timer times out, the subsystem sets the appropriate bit of `stat_rx_pause_req[8:0]` back to 0.
5. If `ctl_rx_check_ack` input is 1, the operation is complete when you set the appropriate bit of `ctl_rx_pause_ack[8:0]` back to 0.
   If you do not set the appropriate bit of `ctl_rx_pause_ack[8:0]` back to 0, the core expects the operation complete after 32 clock cycles.

The following is an example of a pause handshaking event.

Figure: RX Pause Interface



IEEE 1588 Timestamping Support

This section details the packet timestamping function of the MRMAC Ethernet subsystem. This feature provides 1-step and 2-step IEEE 1588v2 functionality.

The MRMAC supports the timestamping of Ethernet frames at both ingress and egress with sub-nanosecond granularity. The option can be used for implementing several IEEE 1588v2 clocks: Ordinary, Transparent, and Boundary. Additionally, 1-step timestamp insertion into outbound precision time protocol (PTP) packets is available. The features can also be used for the generic timestamping of packets at the ingress and egress ports of a system. While the available features can be used for a variety of packet timestamping applications, the rest of this section assumes that you are implementing the IEEE 1588v2 PTP.

IEEE 1588v2 defines a protocol for performing timing synchronization across a network. A 1588 network has a single master clock timing reference, usually selected through a best master clock algorithm. Periodically, this master samples its system timer reference counter and transmits this sampled time value across the network using defined packet formats. This timer should be sampled (a timestamp) when the start of a 1588 timing packet is transmitted. Therefore, to achieve high synchronization accuracy over the network, accurate timestamps are required. If this sampled timer value (the timestamp) is placed into the packet that triggered the timestamp, this is known as one-step operation. Alternatively, the timestamp value can be placed into a follow up packet, this is known as two-step operation.

Other timing slave devices on the network receive these timing reference packets from the network timing master and attempt to synchronize their own local timer references to it. This mechanism relies on these Ethernet ports also taking timestamps (samples of their own local timer) when the 1588 timing packets are received.

Note: Further explanation of the operation of 1588 is outside of this guide. This document describes the 1588 hardware timestamping features for the subsystem.

Related Information

Timestamp Interface

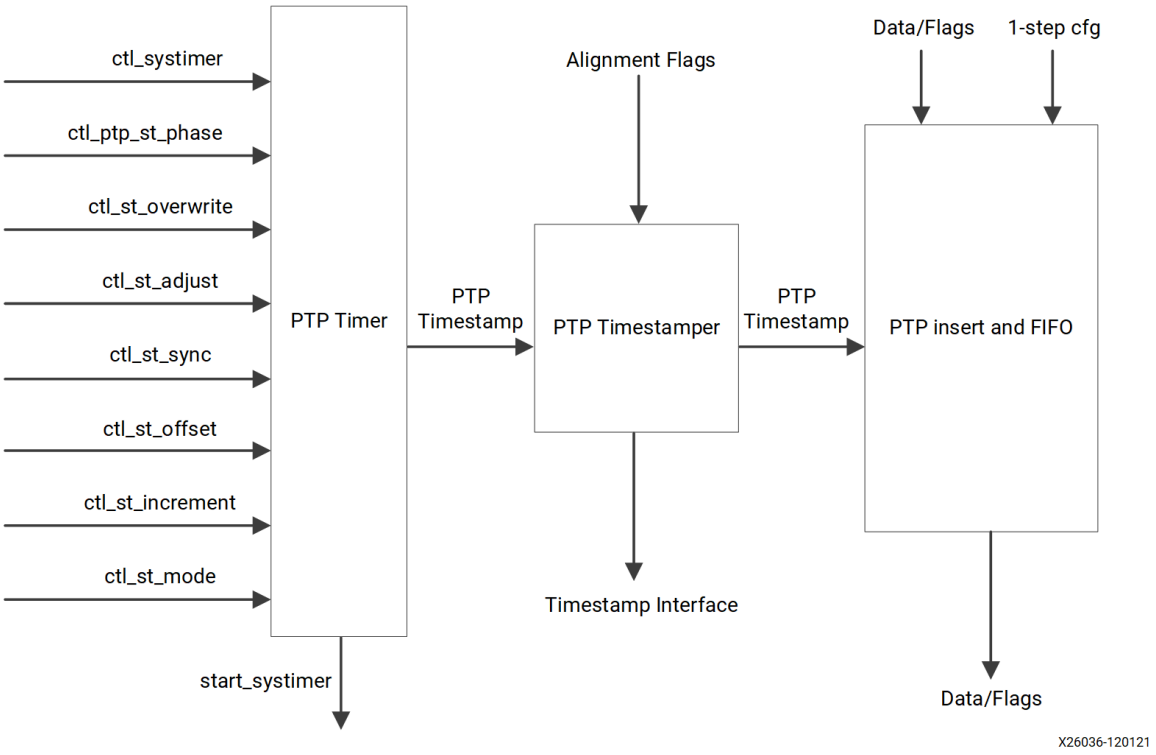
Overview

The MRMAC IEEE 1588 logic consists of several modules:

- PTP Timer, which maintains the system timer (Time-of-Day)
- PTP Timestamper, which timestamps packets
- TX FIFO and Insert block (TX only), which forwards 2-step timestamps to the egress timestamp interface and performs one-step timestamp insertion

The following figure shows the MRMAC 1588/PTP architecture. The diagram is applicable in both transmit and receive directions, although the TX PTP Insert + FIFO is only present in the transmit direction.

Figure: 1588 Block Diagram (MRMAC Internal)



The ctl_ptp_st_sample and ctl_ptp_st_phase are internal signals not exposed to the user. When ctl_ptp_st_sample asserted, it captures the system timer and output it on stat_rx_ptp_systemtimer and stat_tx_ptp_systemtimer. For the user, stat_rx_ptp_systemtimer and stat_tx_ptp_systemtimer values are available when ptp enabled and clocks are provided.

1588 Timestamping support is oriented around an on-board timestamp calculator (PTP Timer), which computes accurate PTP Time-of-Day (TOD) values. The timestamp calculator is adjustable, using a mix of control inputs and internal counters to compensate for differences in clock rates.

The PTP Timestamper calculates a timestamp for inbound and outbound packets using the TOD produced by the PTP Timer. In the transmit direction, timestamps are calculated and made available for all requested packets. Packets are tagged for timestamping by the user logic when presented to the AXI4-Stream interface. The timestamp and accompanying tag can be recovered on the dedicated TX timestamp interface. In the receive direction, all packets are timestamped and the timestamp value is recovered from the AXI4-Stream interface on a dedicated signal.

In the transmit direction, the TX PTP Insert/FIFO logic handles 1588 1-step timestamp insertion into outbound PTP packets. The logic locates and extracts the Correction Field from outbound PTP packets, adjusts it using the computed timestamp, and re-calculates the UDP checksum (if required). The Ethernet FCS value reflects any changes to the Correction Field and the UDP Checksum.

Timestamping

The Timestamper is responsible for tracking the departure and arrival of transmitted and received Ethernet frames. It uses the output of the PTP Timer by applying a timestamp with as much accuracy as possible. The reference point (the point at which the timestamp is sampled) is the first PCS block of a given Ethernet frame.

Egress Timestamping

The operational mode of the egress timestamping function is determined by the settings on the 1588 command port, which can be considered part of the AXI4-Stream interface as the signaling is co-incident with frame signaling. The information contained within the command port indicates one of the following:

- **No Operation**
  The frame is not a PTP frame, and no timestamp action should be taken.
- **1-Step Operation**
  A Correction Field offset value is provided as part of the command port. The frame is timestamped and the captured timestamp is summed with the existing Correction Field contained within the frame. The summed result is overwritten back into the original Correction Field of the frame. Following the frame modification, the frame FCS value is also updated/recalculated. For UDP IPv4 and IPv6 PTP formatted frames, the checksum value in the header of the frame is updated/recalculated upon request.
- **2-Step Operation**
  A tag value (user-sequence ID) is provided as part of the command field. The frame is timestamped and the timestamp made available to the client logic, along with the provided tag value for the frame.

1-Step Insertion

In 1-step insertion mode, egress PTP packets are modified in flight with a calculated timestamp. The PTP Timestamper logic takes in packet data from the TX interface, locates the embedded PTP packet, inserts the timestamp, (optionally) recalculates the UDP checksum, and recalculates the Ethernet FCS field.

✎ **Note:** When `ctl_tx_ptp_1step_enable` = 0 field of the CONFIGURON_1588_REG is set to 0, the timestamp insertion block is bypassed to minimize the latency through the TX MAC. Insertion is not supported for preempted packets (used in TSN).

The 1588 specification defines two supported timestamping modes:

- **Ordinary/Boundary Clock Mode**
  The time-of-day timestamp is inserted into a combination of the Timestamp and the Correction field of the PTP packet.
- **Transparent Clock**
  The Correction field contains a residence time which is calculated by subtracting the ingress timestamp and adding the egress timestamp (the difference is the residence time).

The MRMAC performs packet-level timestamp insertion using a read-modify-write method. When an outbound packet passes through the TX PTP Insert logic, the existing value in the Correction Field is located using the offset provided to the command port `tx_ptp_cf_offset` field. The current timestamp, TS'_{egress}, (provided by the PTP Timer logic) is added to the value discovered in the PTP Correction field. The PTP packet Timestamp field is left untouched.

The following table depicts the handling of the two PTP timestamp fields for each available timestamping mode. The Timestamp Field and Incoming Correction Field columns contain the values that should be seeded into outbound PTP frames by the user logic. The Outgoing Correction Field column is what is placed into the outbound Correction Field by the MRMAC logic.

Table: Correction Field Adjustment Rules

Mode	Timestamp Field	Incoming Correction Field	Outgoing Correction Field
Transparent	TS_{orig}	$CF_{ingress} - TS'_{ingress}$	$CF_{ingress} - TS'_{ingress} + TS'_{egress}$
Ordinary/Boundary	TS_{curr}	$-TS'_{curr}$	$- TS'_{curr} + TS'_{egress}$

From this table, it is apparent that the MRMAC logic does not need to be aware of the selected timestamp mode, as it behaves the same way regardless of the selected mode. The user logic is responsible for setting appropriate initial timestamp values in both the Timestamp and Correction fields of outbound PTP packets.

MRMAC supports different modes to handle wraparound or saturation in the outgoing Correction Field when adding TS'egress would cause an overflow. Setting ctl_tx_ptp_sat_enable=0 causes the result to wrap around (0x7FFFFFFF_FFFFFFFF -> 0x80000000_00000000). Setting ctl_tx_ptp_sat_enable=1 causes the result to saturate to maximum positive (0x7FFFFFFF_FFFFFFFF). Wraparound or saturation is normally used to handle the condition when the incoming Correction Field is a large positive number and TS'egress is large enough to cause an overflow. However, if the TS'egress timestamp has wrapped around while TS'ingress/TS'curr has not (MRMAC assumes TS'egress is later in time than

TS'ingress/TS'curr), this can also cause an overflow condition. MRMAC can detect and adjust for the wraparound by setting ctl_tx_ptp_sat_enable=2. To support this detection, extra information must be added to the incoming Correction Field. Bit 0 should be set to the value of bit 62 of TS'ingress/TS'curr. Bit 1 should be set to 1 for bourdary/ordinary clock mode, or when in transparent mode, set to 1 when ingress correction field bit [63:62] != 2'b01 (where bit 63 is the sign bit and bit 62 is the next MSB) If the result is still an actual overflow condition, the outgoing Correction Field is saturated. In this mode, bits [1:0] are cleared in the outgoing Correction Field.

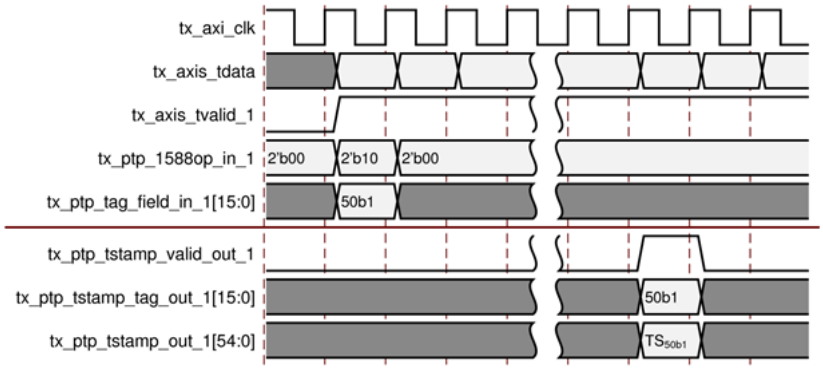
Note: The two PTP fields use different timestamp formats. In this table, TS indicates a timestamp in Timestamp Field format, while TS' means a timestamp in Correction Field format. In most cases, the eight least significant bits of the incoming Correction field are not used in the calculation and are unmodified by the insertion logic. The exception is in the Wrap/Saturate saturation mode.

2-Step Insertion

Egress 2-step timestamping makes use of a timestamping command port which operates in conjunction with the AXI4-Stream data interface. The user logic requests a 2-step timestamp when the first cycle of data for the given frame is presented to the AXI4-Stream bus. Along with this request is a 16-bit timestamp ID, which is carried through to the egress timestamp interface.

After the timestamp is taken, the timestamp value (TS) is made available on the timestamp interface, along with the corresponding timestamp ID. The timestamp is a 55-bit value in the following figure.

Figure: Egress 2-Step Timestamp



Related Information

Time Representation

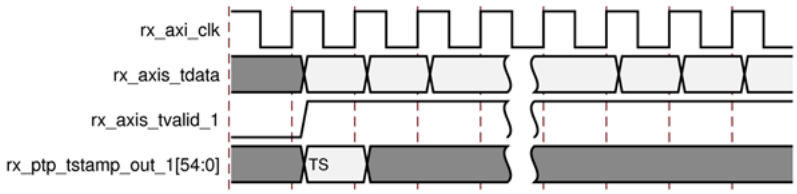
Ingress Timestamping

The ingress logic does not parse the ingress packets to search for 1588 (PTP) frames. Instead, it takes a timestamp for every received frame and outputs this value to the user logic. The feature is always enabled, but the timestamp output can be ignored if you do not require this function. The timestamp can be extracted from the `rx_ptp_tstamp_out_<N>` signal, where N corresponds to the bus segment.

Timestamps are filtered after the PCS decoder to retain only those timestamps corresponding to a start of packet (SOP). These timestamps output on the AXI4-Stream interface. The 55-bit timestamp is valid when the first byte of the frame is present on the system interface.

Note: `rx_ptp_tstamp_out` should be ignored for short packets.

Figure: Timestamp Ingress Interface



Flex Interface Timestamping

The 1588 timestamping is available when the MRMAC is in the flexible interface mode. Only 2-step timestamping is available and 1-step timestamping (insertion) is not supported. To enable the feature and associated ports, set the control bit `ctl_pcs_ts_en` to 1.

Note: In both RX and TX Flex I/F timestamping, Clause 49/82 conversion causes inaccurate timestamps. If conversion is required with 1588, it should be done external to the MRMAC. Also, timestamping is not supported in FEC only or FC32 modes.

The RX Flex I/F timestamps use the same output port as the normal ingress timestamps (`rx_ptp_tstamp_out_<N>`). Enabling Flex I/F timestamps switches these ports over to the `flexif` clock. As decoding is not performed on the packet, the timestamp corresponds to the first word on the datapath and not the first word of the frame. For high accuracy, user logic can add block offsets to the timestamp value if the start of packet is not on the first word of the datapath.

In the egress direction, the Flex I/F timestamping feature uses a different set of signals to request a timestamp and provide a tag (`tx_ptp_flex_*_<N>`). However, the usual egress timestamp interface is used to produce the results.

Note: The egress timestamp output uses the AXI4-Stream clock regardless of the chosen timestamp mode.

An additional signal is present on the Flex I/F in the egress direction: `tx_ptp_flex_1588loc_in_<N>[1:0]`. In the egress direction, the user logic is required to explicitly identify the lane containing the start of the packet (that is, the lane in which data following SFD is present) by asserting Bit[0] of the per-lane (`<N>`) signal. If the Flex I/F is in Clause 74 (KR-FEC) mode, Bit[1] of the relevant signal should be set to 1 if the start is offset by 32 bits (if the Flex I/F is in Clause 74 KR-FEC) mode. In this method, the TX timestamp accurately reflects the position of the start of packet.

Related Information

Flex Interface Modes

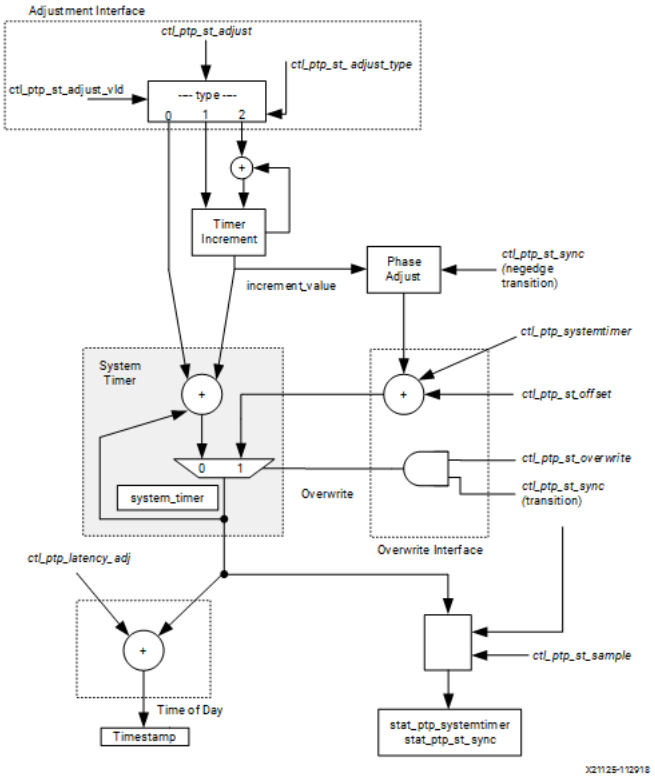
Timer Operation

The PTP timer is essentially a free running timer with several fine-tuning capabilities. The timer acts as a local copy of the external master Time-of-Day (TOD) timer and can update and adjust at any time. Various hooks and adjustment mechanisms are provided to keep the timer in sync with the master TOD timer.

Timer Block Diagram

The architecture of the PTP timer is shown in the following figure.

Figure: System Timer and Adjustment Inputs Diagram (MRMAC Internal)



Note: The ctl_ptp_st_sync represents ctl_rx_ptp_st_sync and ctl_tx_ptp_st_sync.

The heart of the timer is an accumulator with a programmable periodic increment. You can specify the initial timer value and the increment value. These values can be further adjusted, if necessary, using various optional correction methods (including phase and frequency adjustments) to achieve improved accuracy between the system timer and the master TOD timer.

The main elements of the system timer include:

- **system_timer**
  MRMAC timer, source of generated timestamps.
- **Timer Increment**
  The amount by which `system_timer` is incremented each clock period. The amount of increment can be set or adjusted as needed to synchronize the `system_timer` with the external master clock. The established increment can also be temporarily bypassed to provide a specific timer adjustment.
- **Overwrite**
  A one-shot overwrite of the `system_timer` value, triggered by a transition of `ctl_ptp_st_sync` while `ctl_tx/rx_ptp_st_overwrite` input signal is set.
- **Timestamp**
  The generated timestamp to the system, micro-adjusted to account for any desired latency by the value `ctl_ptp_latency_adj`.

Time Representation

The IEEE 1588 uses two types of time field formats:

- **Time-of-Day (ToD) Format**
  IEEE 1588-2008 format consisting of an unsigned 48-bit second field and a 32-bit nanosecond field. This format is compatible with version 1 of the standard.

- **Correction Field Format**
  Introduced in version 2 to handle Transparent clocks and better than 1 ns precision. It is a signed 64-bit integer in units of 2^{-16} ns. For example, 2.5 ns is represented as `64'h0000_0000_0002_8000`.

The `system_timer` is implemented as an internal, unsigned linear counter which counts time in units of 2^{-40} ns. Each clock cycle `system_timer` is incremented by the quantity `increment_value`, which represents the period of the system timer clock in the same units. For example, for a nominal 644.0625 MHz clock, the increment value would be `42'h18D_3018_D302` (1,705,908,949,762 $\times$ 2^{-40} ns). This increment value is configured by the MRMAC at start-up. If KR4-FEC is used, a different initial value is configured: `42'h181_8181_8182`.

The MRMAC exposes 55 bits of the `system_timer` through the various timestamping interfaces (that is, TX and RX packet timestamps). This is equivalent to a timer which counts in units of 2^{-8} ns (that is, ~3.9 ps). User logic can map these 55 bits directly into Bits[62:8] of a Correction Field format timestamp. The bottom eight correction field bits should be set to `8'h0` as the architecture does not support this degree of precision.

Note: This same mapping is used in 1-step timestamp insertion.

The least-significant 32 bits of the MRMAC (that is, $< 2^{-8}$ ns portion of the counter) is not available through the timestamping interfaces. This allows the `system_timer` to be adjusted at sub-nanosecond levels to track an external master clock, as described in the next section.

Timer Adjustment Mechanisms

To provide time-of-day timestamps on the local clock (either RX recovered clock or TX clock), the PTP/System Timer running on the MRMAC local clock needs to be synchronized to the master time-of-day timer running on System Timer (`rx_ts_clk_N`, `ts_ts_clk_N`). The MRMAC system timer can be synchronized with a master timer in a number of methods. Each port has independent internal free-running `system_timer` per RX and TX. For the below description, the tx/rx variable should be substituted with the desired TX and RX direction target and the N variable should be substituted by the target port number.

Overwrite Interface

The straightforward available adjustment method is to perform a direct update of the `system_timer` using the Overwrite interface. In this procedure, the user logic provides its current timer value at regular intervals. The per-port inputs `ctl_tx/rx_ptp_systemtimer_N` are used to provide the master clock value. A transition on `ctl_tx/rx_ptp_st_sync_N` triggers the overwrite. Both of these signals are latched on the master time-of-day clock, `ts_tx/rx_clk_N`.

The overwrite mechanism is very effective. An overwrite interval of 64 ns, using the default settings for the `timer_increment`, allows the MRMAC to maintain the target accuracy even with the clock at $\pm$200 PPM.

During the overwrite process, a fixed offset can be applied to the `ctl_tx/rx_ptp_systemtimer_N` value using the `ctl_tx/rx_st_offset_N` input. This allows the system to account for any system latency (for example, latency due to re-sampling delays). It also allows you to move the reference plane, the point in the datapath that the timestamp represents.

Note: By default, the offset should be set to 4.75x the clock period.

When the master time-of-day clock domain `ts_tx/rx_clk_N` and the `ctl_tx/rx_ptp_st_sync_N` signals are asynchronous to the local clock, it is possible that the new system timer value is retimed and sampled in the wrong clock cycle, leading to unwanted inaccuracy. To counter this, an embedded DDR phase detector automatically adjusts the provided `system_timer` value to account for whether the new timer value was sampled on a rising or falling clock edge. This is the phase adjust function depicted in the diagram.

If additional accuracy is required, the user logic can overwrite the system timer at start-up and then continually monitor the `stat_tx/rx_ptp_systemtimer_N` making frequency and phase adjustments to the system timer as necessary to precisely match the master timer. Using this method, accuracy can be improved significantly.

Adjustment Interface

After a value has been established in the `system_timer`, it begins to increment by a fixed amount on a cycle-to-cycle basis. This fixed amount is labeled as `timer_increment` on the block diagram.

The amount of the increment can be adjusted in one of three methods. The desired method is selected using the signal `ctl_tx/rx_ptp_st_adjust_type_N` input. The update value is provided by `ctl_tx/rx_ptp_st_adjust_N` and the operation is triggered by toggling the value of `ctl_tx/rx_ptp_st_adjust_vld_N`.

Updates can be performed at any time, although changes should only be performed once every 10 clocks (system clocks) to avoid issues with retiming.

- **Method 1: Phase Adjustment (type = 2'b00)**
  The `system_timer` can be shifted in time by a specific amount (that is, a phase shift) using the system timer phase adjustment method. The provided adjustment is defined to be in units of 2^{-8} ns and is directly added to `system_timer` in addition to the usual timer increment. It is a one-time adjustment and you must repeat the operation again to add another amount. The maximum shift is $(218 - 1) \times 2^{-8}$ ns, or approximately 1 µs.
- **Method 2: Coarse Frequency Set (type = 2'b01)**
  In this mode, you can define a new increment value that is to be added to `system_timer` on a clock-by-clock basis. The provided increment value is interpreted as an unsigned integer in units of 2^{-8} ns. To make more fine-grained adjustments ($< 2^{-8}$ ns) to `increment_value`, the next method must be used.
- **Method 3: Fine Frequency Adjust (type = 2'b10)**
  This method allows you to make sub-nanosecond adjustments to the interval value. The provided increment value is interpreted as a signed integer in units of 2^{-40} ns. It is added to the existing timer increment. This is a signed value, so subtraction is possible if a negative adjustment is provided.

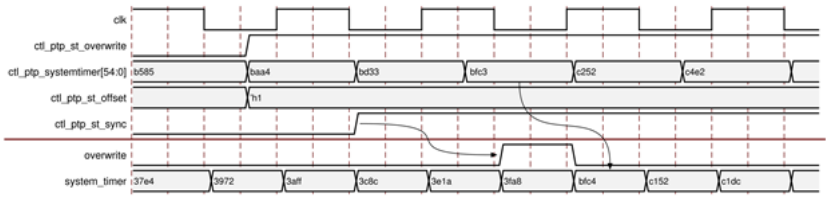
Examples

The following shows timing diagram examples for the timer adjustments.

System Timer Overwrite

To overwrite the system timer, assert `ctl_tx/rx_ptp_st_overwrite_N` and drive the desired 54-bit value onto the input signal `ctl_tx/rx_ptp_systemtimer_N`. The overwrite is triggered by toggling the signal `ctl_tx/rx_ptp_st_sync_N`.

Figure: Timer Overwrite



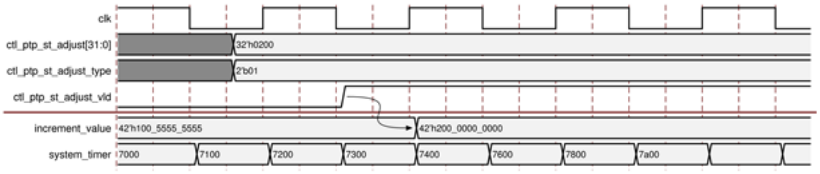
Note: The `ctl_tx/rx_ptp_systemtimer_N` and `ctl_tx/rx_ptp_st_sync_N` signals come from a clock domain that is asynchronous to the timer's clock domain, hence some small delays for retiming.

As mentioned earlier, if the `ctl_tx/rx_ptp_st_sync_N` transition is sampled to the falling edge of the timer's clock, a DDR phase compensator automatically adds half a clock period (that is, `timer_increment` / 2) to the system timer to help improve timer resolution.

Timer Frequency Set Coarse Adjustment

In this example, the `system_timer` frequency is set by providing a coarse adjustment to the `timer_increment`.

Figure: System Timer – Set Increment Value

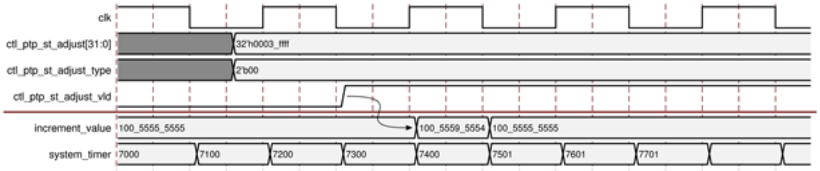


Note: The 32-bit `ctl_ptp_st_adjust` value is interpreted as being in units of 2^{-8} ns, which means it is mapped into the coarse portion of the `increment_value`. The sub-nanosecond portion of the `increment_value` is set to `32'd0`. To adjust the sub-nanosecond portion of the increment value, use the Timer Frequency Adjust method.

Timer Frequency Adjust Fine Adjustment

Here is an example of fine-tuning the `system_timer` frequency by adjusting the sub-nanosecond portion of the increment value (Method 3 on the Adjustment interface). The adjustment is specified to be in units of 2^{-40} ns. The provided signed adjustment value is added to the existing `increment_value` and used moving forward.

Figure: Fine Frequency Adjust

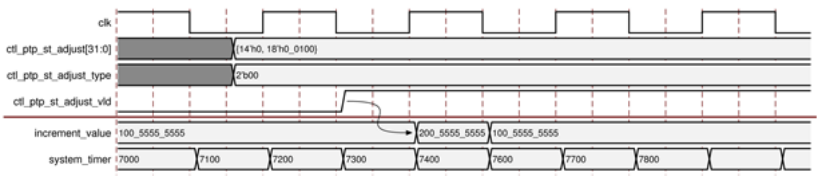


In this example, a signed adjustment value of `32'hBA9D_1F60` is provided. Because this adjustment value is negative, it results in a subtraction of `42'h4562_E0A0`, reducing the increment value from `42'h18D_8AC5_C140` to `42'h18D_4562_E0A0`.

Phase Shift

In this diagram, a phase shift to the timer value is shown (Method 1 using the Adjustment interface).

Figure: Phase Adjustment



Note: The `increment_value` only changes for one `system_timer` iteration (it is a one-shot adjustment) then returns to its previous value. The maximum allowable adjustment is `18'h3_FFFF` in units of 2^{-8} ns.

Other Adjustment Mechanisms and Considerations

This section describes the additional adjustment mechanisms to use for greater timestamp accuracy.

RSFEC Codeword Offset

When RSFEC is enabled, you can choose to adjust timestamps due to the presence of RSFEC codewords. To facilitate this when RSFEC is enabled, the timestamp interfaces (both ingress and egress) provide an RSFEC codeword offset from the start of packet. You can use this information to apply offsets to the timestamp if desired.

RSFEC Compensation ROM

Additional timestamp compensation can be obtained when RSFEC is active by enabling the RSFEC Compensation ROM. The user logic might want to enable this feature if it requires that transcoder compression and parity insertion effects be included in the produced timestamp. This compensation is optional and is enabled using the `ctl_tx_ptp_rsfec_comp_en` field of the CONFIGURATION_1588_REG register. Enabling the compensation ROM is similar to moving the timestamp timing plane to the point where data is RSFEC encoded. Typically this would not be required.

The compensation ROM contains a table of pre-calculated adjustment factors which adjust the timestamp to compensate for the effects of transcoder compression and RSFEC parity bits insertion.

When enabled, the compensation is performed automatically in the egress direction, which is particularly useful when 1-step timestamp insertion is active. In the ingress (receive) direction, you must perform your own RSFEC compensation as the ROM is only available in the egress (transmit) direction. To accomplish this, the user logic can generate its own compensation look-up table. The input to the table is the codeword offset (`D_ptp_rsfec_offset_out_N`) produced by the receive timestamp interface. The output from the table is described by the following pseudo code:

RSFEC KR4:

```
for (cw_offset=0;cw_offset<80;i++) begin
 adj_val[cw_offset] = round((256*(1-257/264)*cw_offset*64/25)); # (Nominal KR4 block period)
end
```

RSFEC KP4:

```
for (cw_offset=0;cw_offset<80;i++) begin
 adj_val[cw_offset] = round((256*(1-257/272)*cw_offset*64/25)); # (Nominal - KP4 block period)
end
```

The tables generated by the pseudo code reflect a nominal clock with a standard KR4/KP4 block period with a 25G PCS transmission period (in units of 2^{-8} ns). The code should be adjusted for the appropriate data rate as necessary.

1588 Control and Status Registers

These registers are used to configure and fine-tune the Timestamping functionality.

Table: 1588 Control and Status Registers

CONFIGURATION_TX_1588 CONFIGURATION_RX_1588 Register Fields	Type	Description

CONFIGURATION_TX_1588 CONFIGURATION_RX_1588 Register Fields	Type	Description
ctl_ptp_st_offset[15:0]	RW	This is used to apply a fixed offset to ctl_ptp_st_systemtimer on an overwrite. The units of this bus are 2^{-8} ns. Currently no default, but could be set to the equivalent of four clock cycles to match the latency due to metastability and pipeline flops.
sample_ptp_systimer[54:0]	RO	The current value of the system timer is captured on read (not using the ctl_ptp_sync).
sample_ptp_increment[41:0]	RO	The current value of the system timer increment value (in units of 2^{-40} ns). This is for monitoring only. Adjustments are made using the PTP adjust interface.
ctl_tx_ptp_1step_enable	RW	This signal, when asserted enables 1-step operation.

ctl_tx_ptp_sat_enable[1:0]	RW	Saturation mode of the 1-step insertion logic.
		• 2'b00: Wraparound (0x7FFFFFFF_FFFFFFFF → 0x80000000_00000000)
		• 2'b01: Saturate to max positive
		• 2'b10: Use Wrap/Saturate indications in incoming Correction Field Bits[1:0]. Note, in this mode, the eight LSB bits of the correction field are cleared.
		  ○ CF Bit[0]: Bit[62] of Ingress RX Timer Value (correction field format)
		  ○ CF Bit[1]: Sign of incoming correction/software generated field

ctl_tx_ptp_latency_adjust[19:0]	RW	This bus can be used to apply a fixed offset to the TX timestamp. The units of this bus are 2^{-8} ns in 2's complement form.
ctl_rx_ptp_latency_adjust[19:0]	RW	This bus can be used to apply a fixed offset to the RX timestamp. The units of this bus are 2^{-8} ns in 2s complement form.

Related Information

Register Space

Design Flow Steps

This section describes customizing and generating the subsystem, constraining the subsystem, and the simulation, synthesis, and implementation steps that are specific to this IP subsystem. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator (UG994)
- Vivado Design Suite User Guide: Designing with IP (UG896)
- Vivado Design Suite User Guide: Getting Started (UG910)
- Vivado Design Suite User Guide: Logic Simulation (UG900)

GT Quad Integration with AMD IP Cores

AMD GT-based IP cores such as Aurora, PCIe® , 40G/50G Ethernet, MRMAC, and DCMAC provide Block Automation in IP integrator to enable you to connect multiple AMD parent IP cores to GT Quad IP cores seamlessly. IP Block Automation instantiates GT Quad IP cores and creates datapath and clock connections (USRCLK and GT `REFCLK`).

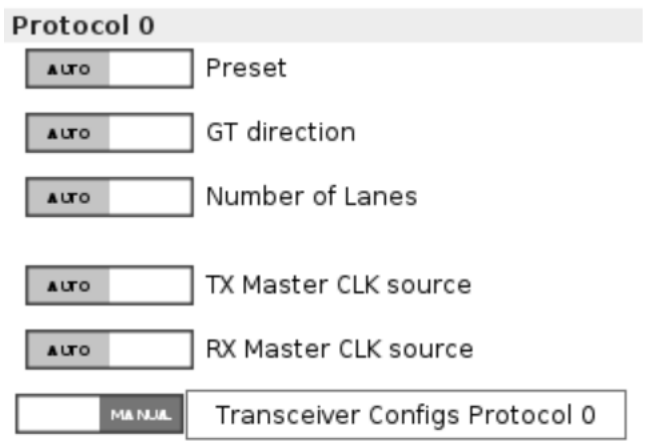
Perform the following steps to connect the MRMAC instance using Block Automation:

1. Add the MRMAC instance using the Add IP option in the IP integrator canvas.
2. Configure the MRMAC instance.
3. Click Run_Block_Automation. In the Block Automation screen, select one of the options Auto, Start_with_New_Quad, or Customized_Connections.
4. Perform steps 2 and 3 to add more MRMAC instances based on your system requirements.

✎ **Note:** For Customized_Connections options, a GT Quad needs to be manually added to the BD design first to allow you to select lanes.

GT Quad IP core parameters are propagated from any connected IP cores when the design is validated. Therefore all GT Quad parameters are marked Auto in the Transceiver Wizard configuration screen. However, you can change the Auto option to Manual for the transceiver configuration, as shown in the following figure, to fine tune parameters such as insertion loss, drive strength, equalization, and other advanced settings. After toggling to Manual mode, any changes to the parent IP configuration followed by validation, no longer propagate GT Quad parameters from the parent IP core to the GT Quad IP core. Manual changes should only be performed after all essential parent IP core parameters are propagated to the GT Quad IP core.

Figure: Auto to Manual Options Switch in Transceiver Wizard



Customizing and Generating the Subsystem

This section includes information about using AMD tools to customize and generate the subsystem in the AMD Vivado™ Design Suite.

If you are customizing and generating the subsystem in the Vivado IP integrator, see the Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator (UG994) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP subsystem using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the Vivado Design Suite User Guide: Designing with IP (UG896) and the Vivado Design Suite User Guide: Getting Started (UG910).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

Select 100G Ethernet IP from the IP catalog, a window displays showing the different available configurations. These are organized in various tabs for better readability and configuration purposes. The details related to these tabs are shown in the following figures.

MRMAC Configuration Tab

The MRMAC tab configures the MRMAC subsystem mode.

Figure: MRMAC Configuration Tab

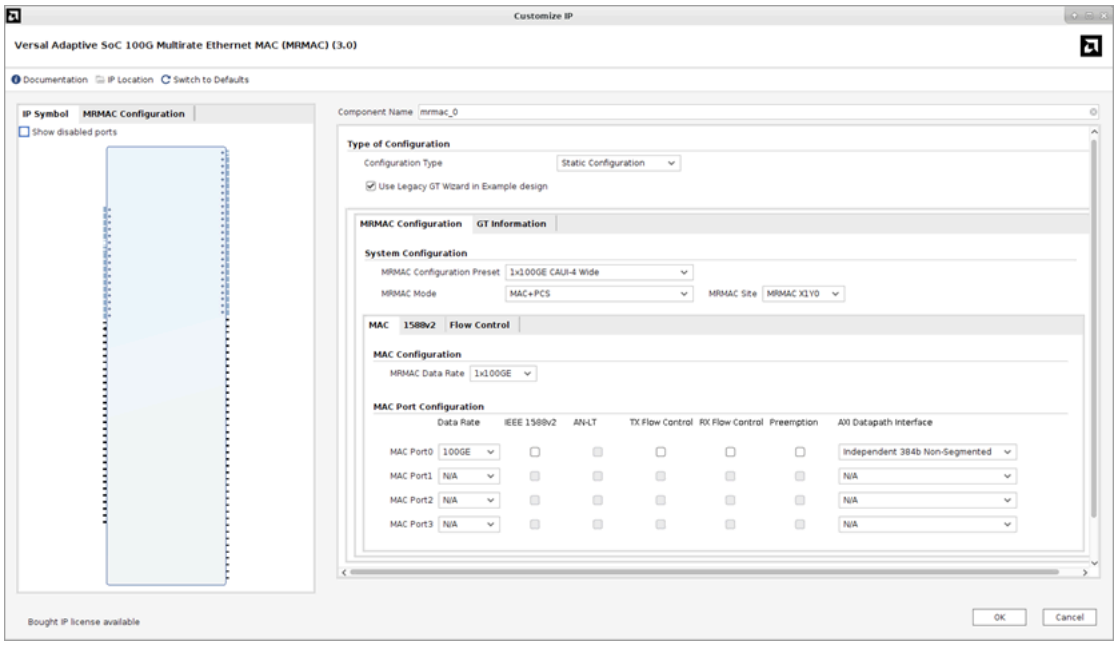
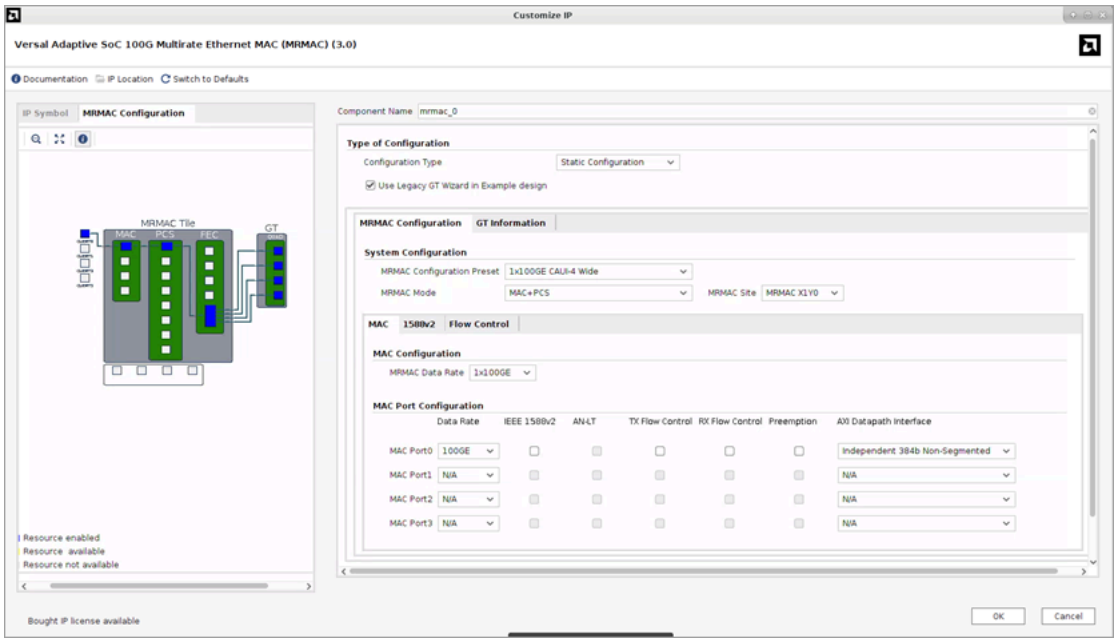


Figure: MRMAC Configuration Tab (Resource View)

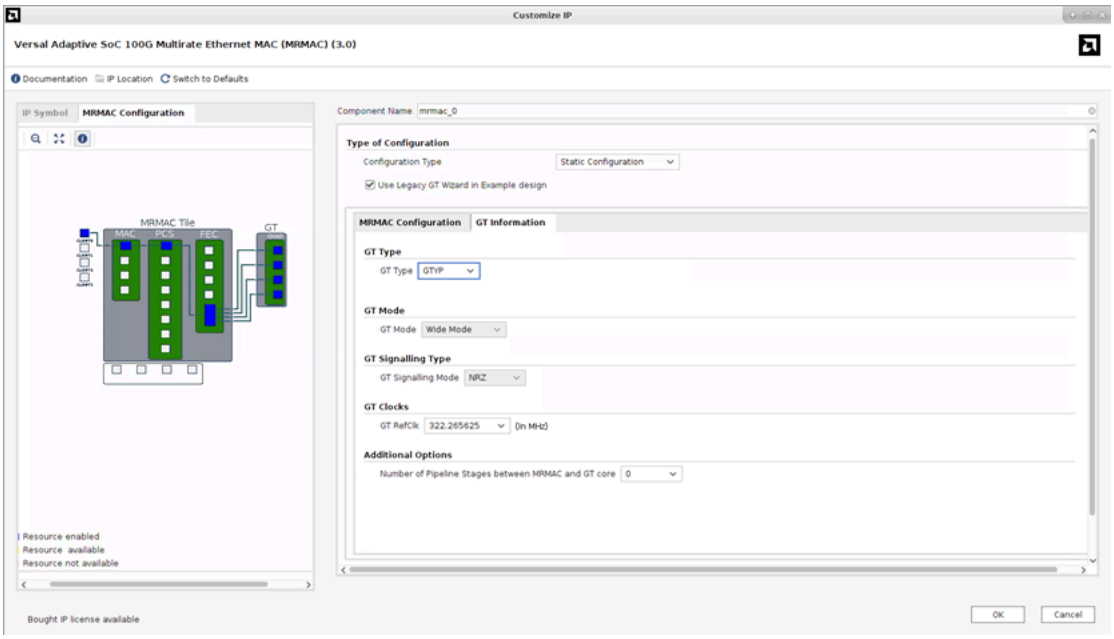


- Type of Configuration
  Configures the MRMAC configuration type – Static or Dynamic.
- System Configuration
  Enables the MRMAC Configuration Preset, MRMAC Mode, and MRMAC Site.
- MAC
  Configures MRMAC Data Rate and MRMAC-AXI Datapath Interface.
- MAC Port Configuration
  Sets the MAC ports for Data Rate, IEEE 1588v2, TX Flow Control, and RX Flow Control.
- **Use Legacy GT Wizard In Example Design**
  When not selected MRMAC example design uses the GT Wizard Subsystem (gtwiz_versal IP) in RTL mode.

GT Information Tab

The GT information tab shows the GT configuration information that is needed based on the MRMAC mode configured as shown.

Figure: GT Information Tab



- GT Type
  Enables the GT type for your configuration.
- GT Mode
  Specifies whether the selected preset is in Wide mode or in Narrow mode. For the Start from Scratch preset, you can select your required mode.
- GT Signalling Type
  Specifies whether the selected preset signaling type is NRZ or PAM4
- Additional Options
  Additional access to choose the number of pipeline stages between MRMAC and GT, to ease timing.

Output Generation

For details, see the Vivado Design Suite User Guide: Designing with IP (UG896).

Constraining the Subsystem

Required Constraints

When MRMAC is configured in dynamic mode, timing arcs are enabled for all possible combinations. Except for narrow and wide distinction, as switching between these two Serdes widths is not allowed. When in dynamic configuration, Vivado attempts to close timing for all potential modes/rates of the MRMAC. To limit the cases considered by the timing engine, set_false_path commands can be added to design XDCs to exclude unneeded cases.

The MRMAC example design.XDC file for dynamic configuration contains a sample set of constraints to disable the unneeded cases in commented form. You can explore it by tweaking it as per the requirement. A sample for 100G RX is shown here.

```
### RX
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG_G
T_U/O1}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
```

```
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG_G
T_U/O2}]]
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O1}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]]
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG_G
T_U/O2}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]]
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_2/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O1}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_2/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]]
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG_G
T_U/O2}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_2/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]]
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_3/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O1}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_3/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]]
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG_G
T_U/O2}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_3/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]]
set_false_path -from [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_2/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]] -to [get_clocks -of_objects [get_pins -hier -filter { name =~
*/*_exdes_support_wrapper/*_exdes_support_i/*_gt_wrapper/mbufg_gt_1_3/U0/USE_MBUFG_GT_SYNC.GEN_MBUFG_GT[0].MBUFG
_GT_U/O2}]]
```

Device, Package, and Speed Grade Selections

This section is not applicable for this IP subsystem.

Clock Frequencies

This section is not applicable for this IP subsystem.

Clock Management

This section is not applicable for this IP subsystem.

Clock Placement

This section is not applicable for this IP subsystem.

Banking

This section is not applicable for this IP subsystem.

Transceiver Placement

This section is not applicable for this IP subsystem.

I/O Standard and Placement

This section is not applicable for this IP subsystem.

Simulation

For comprehensive information about AMD Vivado™ simulation components, as well as information about using supported third-party tools, see the Vivado Design Suite User Guide: Logic Simulation (UG900).

For information regarding simulating the example design, see the related information.

Related Information

Simulating the Example Design

Synthesis and Implementation

For details about synthesis and implementation, see the Vivado Design Suite User Guide: Designing with IP (UG896).

For information regarding synthesizing and implementing the example design, see the related information.

Related Information

Synthesizing and Implementing the Example Design

Example Design

Overview

This chapter explains the AMD Versal™ Devices MRMAC example design and various test scenarios implemented within the example design.

For the MRMAC example design, the GT subcore is always in the example design. This example design demonstrates switching speeds between 1x100GE, 1x50GE, 1x40GE, 4x25GE, and 4x10GE.

Note: Example design does not support mixed or custom mode configurations (configurations having disabled MAC ports). They support port enablement and core generation.

Example Design Hierarchy

The following figure shows the MRMAC example design with the Legacy GT Wizard. In the block design, the MRMAC and GT Quad Base IP are connected along with the MBUFG_GTs. For more information on the GT Quad Base IP (Legacy GT Wizard), see the Versal Adaptive SoC Transceivers Wizard LogiCORE IP Product Guide (PG331).

As shown in the following figure, the MRMAC example design simulation uses a packet generator and monitor to generate and check Ethernet traffic. A test bench controls and monitors the packet generator and also configures the MRMAC IP through an AXI4-Lite interface. When configuring the MRMAC core with the Legacy GT Wizard, the IP integrator is the only supported flow using Block Automation to generate and configure Versal Adaptive SoC Transceiver cores. When configuring the MRMAC core with the GT Wizard subsystem, the AMD Vivado™ Integrated Design Environment flow (RTL flow) is the only supported flow.

For Implementation, as shown in the subsequent figure, CIPS triggers the generator and monitor and configures the MRMAC IP through the AXI4-Lite interface. The IP folder provides a sample C- Code. Both simulation and validation of the example design starts with GT rate configuration followed by reset. Then the core is configured through the AXI4-Lite

interface. This is followed by a loop-back test using the generator and monitor blocks in example design. Finally, it reads the MRMAC statistics to compare the results of RX with TX.

✎ **Note:** For ease of meeting timing in generator and monitors in slow speed devices, the AXIS clock is set to 370 MHz.

Figure: MRMAC Example Design (Legacy GT Wizard) (Simulation)

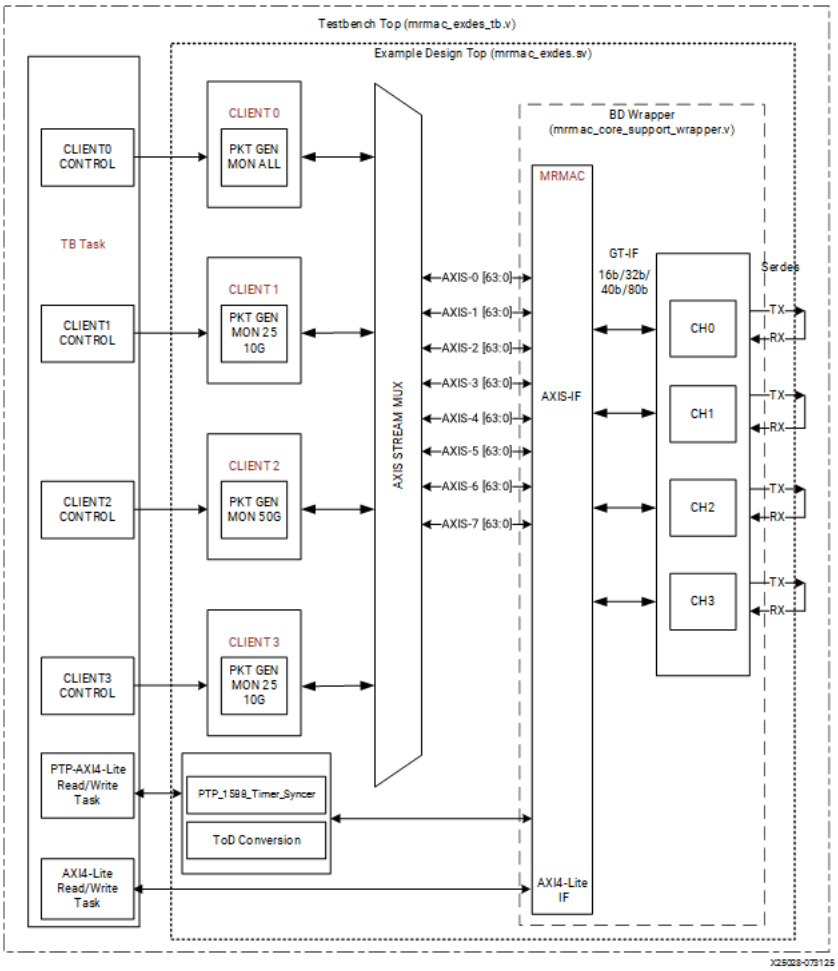
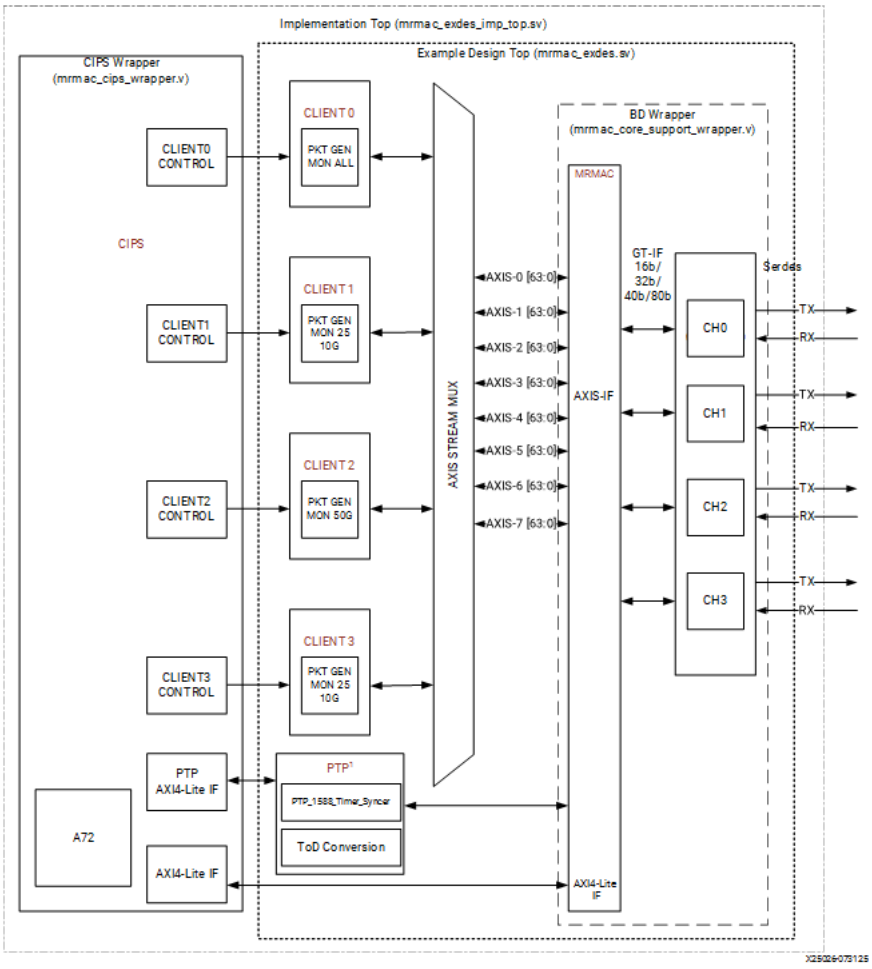


Figure: MRMAC Example Design (Legacy GT Wizard) (Implementation)

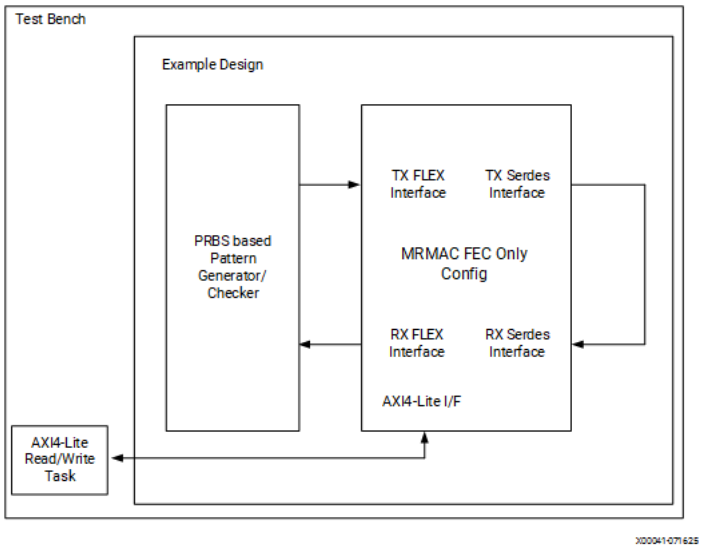


✎ Note: The MRMAC Tile section in device is currently done by MRMAC GUI (default set to "X0Y3"). A sample constraint for MRMAC location "X0Y3" is applied to core.xdc (mrmac_0.xdc) file. A suitable GT also suggested in exdes.xdc (mrmac_0_example_top.xdc) file to lock appropriate GT for the selected MRMAC. If you change these locations, you need to take care of the clocking region and routing feasibly.

MRMAC FEC Only Configuration Example Design

The following figure shows the MRMAC FEC Only example design simulation. In the block diagram, the MRMAC FEC Only example design uses a pseudo-random binary sequence (PRBS) based packet generator and monitor to generate and check Ethernet traffic. A test bench controls and monitors the packet generator and also configures the MRMAC IP through an AXI4-Lite interface.

Figure: MRMAC FEC Only Example Design (Simulation)



✎ Note: The MRMAC FEC Only configuration example design only supports simulation

MRMAC PCS Only Configuration Example Design

In the MRMAC PCS Only example design, the MRMAC and GT Quad Base IP are connected along with the MBUFG_GTs. For more information on the GT Quad Base IP, refer to the Versal Adaptive SoC Transceivers Wizard LogiCORE IP Product Guide (PG331).

The MRMAC PCS Only example design simulation uses a block RAM based packet generator and monitor to generate and check Ethernet traffic. A test bench controls and monitors the packet generator and also configures the MRMAC IP through an AXI4-Lite interface.

✎ Note: The MRMAC PCS Only configuration example design only supports simulation.

Transceiver Selection Rules

✎ Important: The design must meet the following rules when connecting the Versal MRMAC Hard IP core to the transceivers:

1. GTs have to be contiguous.
2. For MRMAC operating in wide SerDes mode (CTL_SERDES_WIDTH_[0..3]<2> == 1), the MRMAC and connected GT Site (GTY_QUAD/ GTYE5_QUAD/ GTME5_QUAD) instance should be placed within the same Clock Region.
3. For MRMAC operating in narrow SerDes mode (CTL_SERDES_WIDTH_[0..3]<2> == 0), the MRMAC and connected GT Site (GTY_QUAD/ GTYE5_QUAD/ GTME5_QUAD) instance should be placed either within the same Clock Region, or one horizontal Clock Regions above or below the MRMAC instance.
4. MRMAC 100GE, 40GE mode should use four contiguous GTs (GTY/GTYP/GTM), and 50GE should use two contiguous GTs (GTY/GTYP/GTM) from the same GTYE5_QUAD/ GTME5_QUAD.

✎ Note: Apart from the previously mentioned rules, it is not recommended to place MRMAC and GT Site (GTY_QUAD/GTYE5_QUAD/GTME5_QUAD) in horizontally opposite locations. However, in extreme cases, when you have to intentionally violate one or more than one of the said rules, timing violation might occur, and it is the user's

responsibility to resolve it. Although not guaranteed, you can explore options like adding pipeline registers between MRMAC and GT in such cases.

User Interface

GT Serdes pins and clock interface pins.

Table: User I/O Ports

Name	Size	I/O	Description
gt_ref_clk_p	1	I	Differential input clk to GT
gt_ref_clk_n	1	I	Differential input clk to GT
gt_rxp_in	4	I	Differential serdes input to GT
gt_rxn_in	4	I	Differential serdes input to GT
gt_txp_out	4	O	Differential serdes output from GT
gt_txn_out	4	O	Differential serdes output from GT

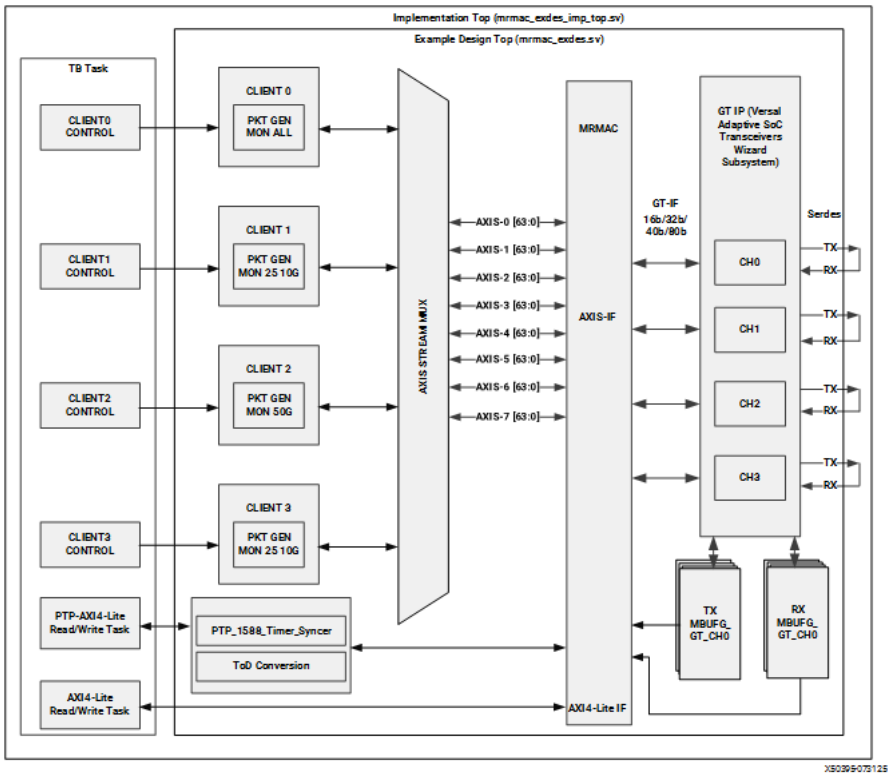
Example Design with GT Wizard Subsystem (gtwiz_versal IP)

The following figures show the MRMAC example design with GT Wizard Subsystem (gtwiz_versal IP). In the block diagram, the MRMAC and gtwiz_versal IPs are connected along with the MBUFG_GTs. MRMAC, gtwiz_versal, and MBUFG_GTs are integrated through RTL instantiation in the example design.

For more information on the gtwiz_versal IP, see the Versal Adaptive SoC Transceiver Subsystem Product Guide (PG442).

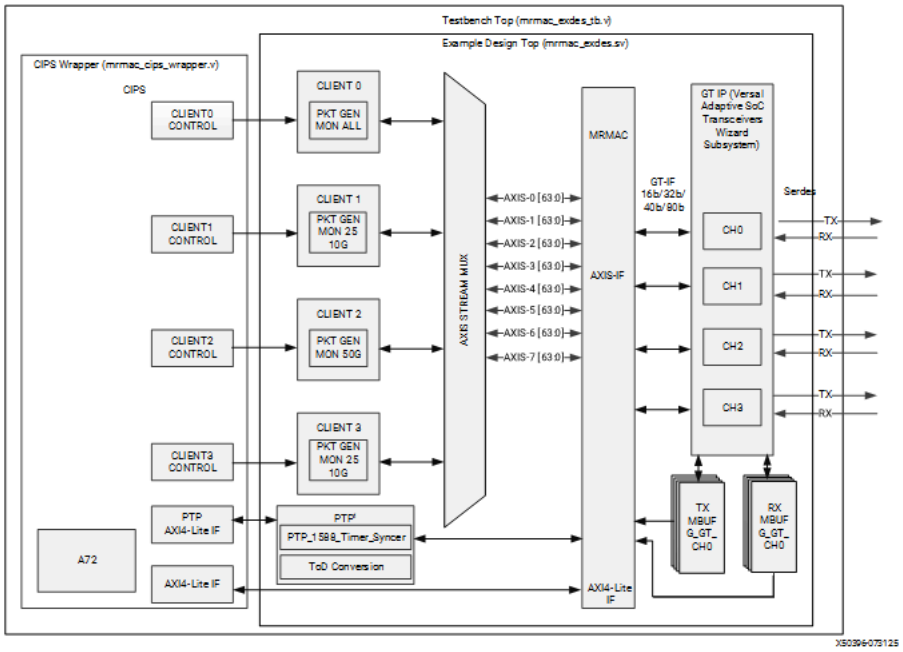
As shown in the following figure, the MRMAC example design simulation uses a packet generator and monitor to generate and check Ethernet traffic. A test bench controls and monitors the packet generator and also configures the MRMAC IP through an AXI4-Lite interface.

Figure: MRMAC Example Design with GT Wizard Subsystem (gtwiz_versal IP) (Simulation)



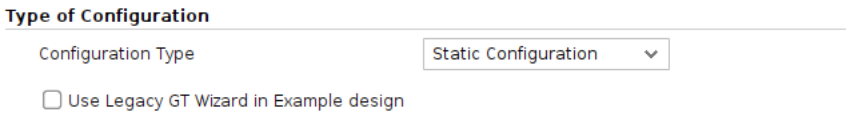
For Implementation, as shown in the subsequent figure, CIPS triggers the generator and monitor and configures the MRMAC IP through the AXI4-Lite interface. The IP folder provides a sample C- Code. Both simulation and validation of the example design starts with GT rate configuration followed by reset. Then the core is configured through the AXI4-Lite interface. This is followed by a loop-back test using the generator and monitor blocks in example design. Finally, it reads the MRMAC statistics to compare the results of RX with TX.

Figure: MRMAC Example Design with GT Wizard Subsystem (gtwiz_versal IP) (Implementation)



When generating the IP, disable the Use Legacy GT Wizard in Example Design option in the MRMAC IP configuration tab to enable the GT Wizard Subsystem (gtwiz_versal IP) in the example design.

Figure: MRMAC Configuration Tab with Use Legacy GT Wizard in Example Design Option



Simulating the Example Design

The example design provides a quick way to simulate and observe the behavior of the MRMAC IP subsystem example design projects generated using the AMD Vivado™ Design Suite. For step by step guide to perform the example design simulation, see Example Design Creation, Simulation, and Device Image Generation.

Supported Simulators

The currently supported simulators are:

- Vivado simulator (default)
- Mentor Graphics® Questa® Advanced Simulator/ModelSim (integrated in the Vivado IDE)
- Synopsys VCS® and VCS MX
- Cadence Xcelium Parallel Simulator
- Aldec Riviera-PRO

The simulator uses the example design test bench and test cases provided along with the example design.

Running a Simulation

For any project (MRMAC IP subsystem) generated out of the box, the simulations can be run in the following ways:

1. In the Sources Window, right-click the example project file (.xci), and select Open IP Example Design. The Vivado IP integrator block design-based example project is created.
2. In the Flow Navigator (left-hand pane), under Simulation, right-click Run Simulation and select Run Behavioral Simulation.

   ✎ **Note:** The post-synthesis and post-implementation simulation options are not supported for the MRMAC IP subsystem.

   After the Run Behavioral Simulation Option is running, you can observe the compilation and elaboration phase through the activity in the Tcl Console, and in the Simulation tab of the Log Window.
3. In Tcl Console, type the run all command and press Enter. This runs the complete simulation as per the test case provided in example design test bench.
   After the simulation is complete, the result can be viewed in the Tcl Console.

Changing the Simulator

To change the simulators:

1. In the Flow Navigator, under Simulation, select Simulation Settings.
2. In the Project Settings for Simulation dialog box, change the Target Simulator to QuestaSim/ModelSim.
3. When prompted, click Yes to change and then run the simulator.

✎ **Note:** The Language option should be set to `Mixed`.

Simulation Speed Up

Simulation can take a long time to complete due to the time required to complete alignment. A `` `define SIM_SPEED_UP `` is available to improve simulation time by reducing the PCS lane alignment marker (AM) spacing to speed up the time the IP takes to achieve alignment. Setting `` `define SIM_SPEED_UP `` changes the following parameters:

- `CTL_TX_VL_LENGTH_MINUS1_100GE_0` and `CTL_RX_VL_LENGTH_MINUS1_100GE_0` from `16'h3FFF` to `16'h01FF`
- `CTL_TX_VL_LENGTH_MINUS1_25GE_0` and `CTL_RX_VL_LENGTH_MINUS1_25GE_0` from `16'h4FFF` to `16'h04FF`
- `CTL_TX_VL_LENGTH_MINUS1_25GE_1` and `CTL_RX_VL_LENGTH_MINUS1_25GE_1` from `16'h4FFF` to `16'h04FF`
- `CTL_TX_VL_LENGTH_MINUS1_25GE_2` and `CTL_RX_VL_LENGTH_MINUS1_25GE_2` from `16'h4FFF` to `16'h04FF`
- `CTL_TX_VL_LENGTH_MINUS1_25GE_3` and `CTL_RX_VL_LENGTH_MINUS1_25GE_3` from `16'h4FFF` to `16'h04FF`

The `SIM_SPEED_UP` option can be used for simulation when in serial loopback or if the AM spacing can be reduced at both endpoints. This option is compatible with the example design simulation, which uses serial loopback.

✎ **Note:** Altering the value of `CTL_TX_VL_LENGTH_MINUS1_100GE_0` and `CTL_RX_VL_LENGTH_MINUS1_100GE_0` from the default value of `0x3FFF` or `CTL_TX_VL_LENGTH_MINUS1_25GE_*` and `CTL_RX_VL_LENGTH_MINUS1_25GE_*` from the default values of `16'h4FFF` violates the IEEE 802.3 spec.

Decreasing the AM spacing results in less than MRMAC bandwidth being available on the link.

This change can be made only in simulation. For a design to work in hardware, the default values must be used.

Full rate simulation without the `SIM_SPEED_UP` option should still be run.

`SIM_SPEED_UP` is available only when running RTL simulations. The option is not available for post-synthesis or post-implementation simulations.

VCS

Use the vlogan option: `+define+SIM_SPEED_UP`.

QuestaSim

Use the vlog option: `+define+SIM_SPEED_UP`.

Vivado Simulator

Use the xvlog option: `-d SIM_SPEED_UP`.

Xcelium

Use the xmvlog option: `+define+SIM_SPEED_UP` .

Riviera-PRO

Use the vlog option: `+define+SIM_SPEED_UP`.

Synthesizing and Implementing the Example Design

To run synthesis and implementation on the example design in the Vivado Design Suite, perform the following steps:

1. Go to the XCI file, right-click, and select Open IP Example Design. A new Vivado tool window opens with the project name `example_project` within the project directory.
2. In the Flow Navigator, click Run Synthesis and Run Implementation.

★ **Tip:** Click Run Implementation first to run both synthesis and implementation. Click Generate Bitstream to run synthesis, implementation, and then bitstream.

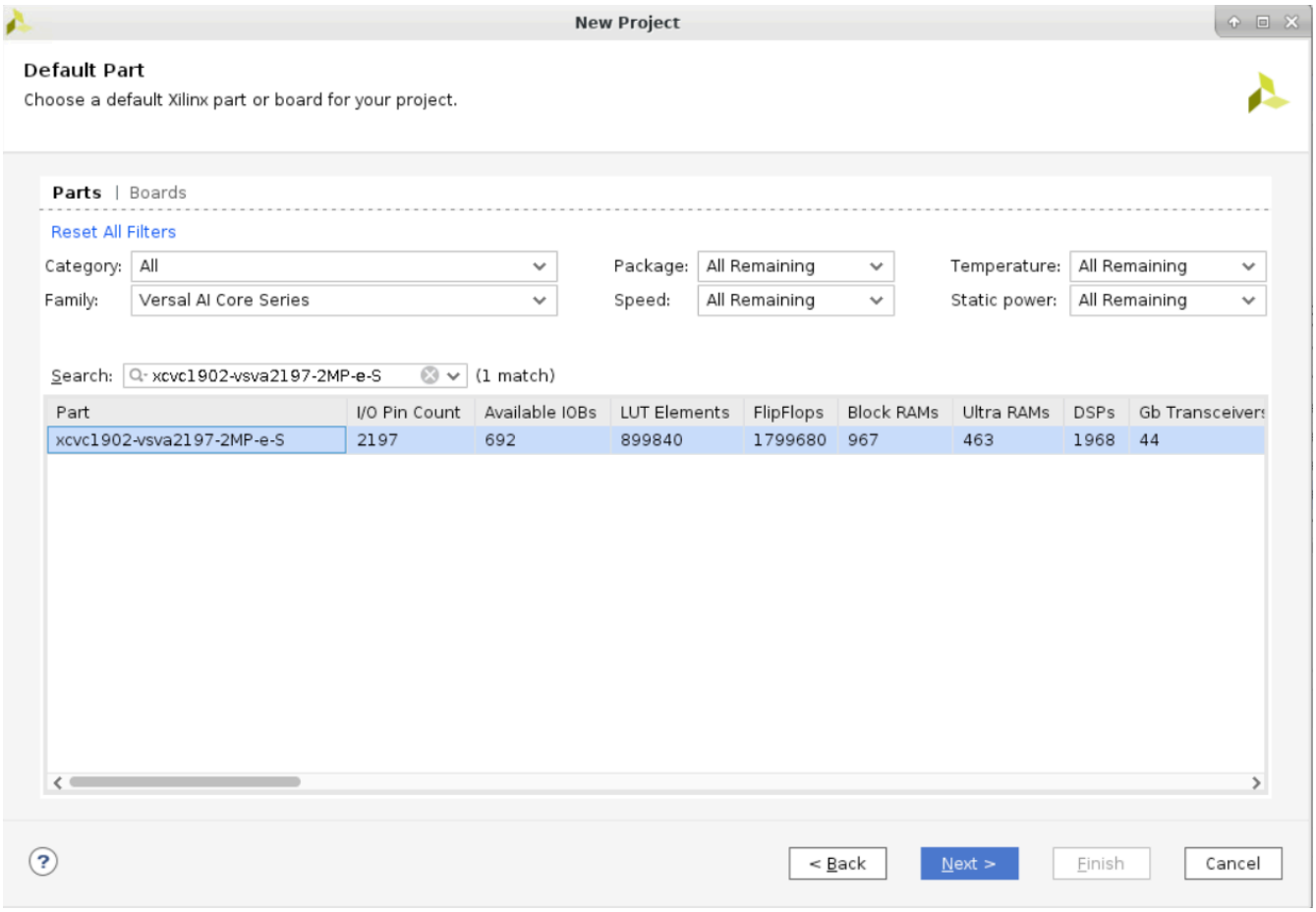
✎ **Note:** For step by step guide to perform example design synthesis, implementation, and validation of the VCK190 board, see Example Design Creation, Simulation, and Device Image Generation.

MRMAC Example Design Simulation and Validation Steps

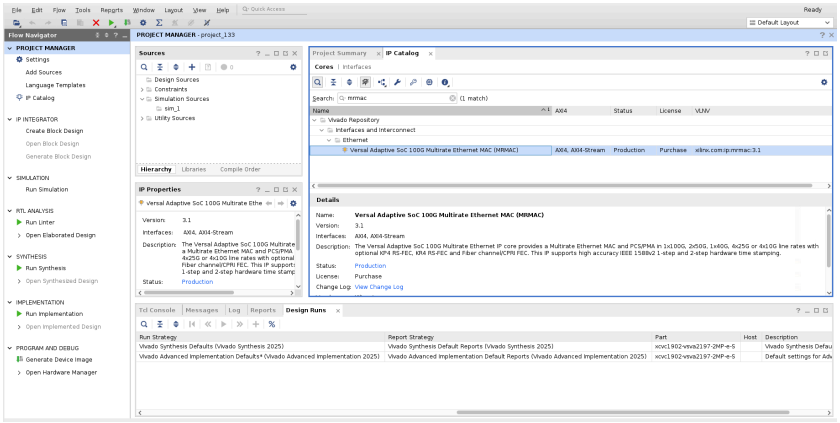
Example Design Creation, Simulation, and Device Image Generation

1. Open the AMD Vivado™ Design Suite in GUI mode.
2. Select File > Project > New.

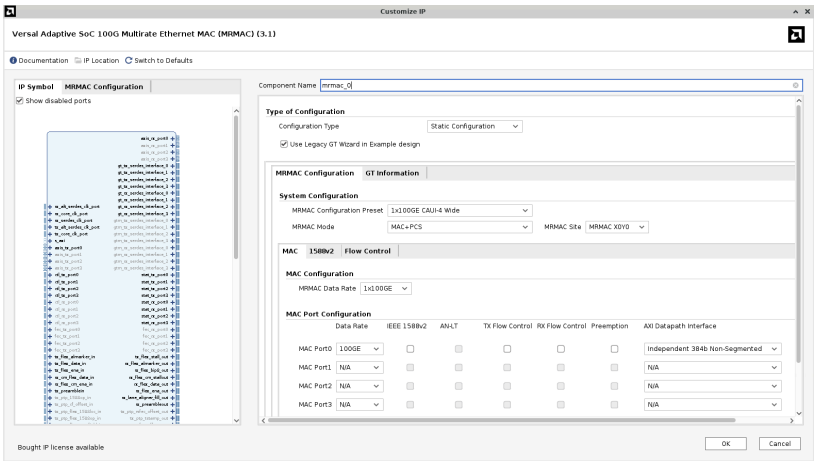
3. Create a new project with any AMD Versal™ device xcvc1902-vsva2197-2MP-e-S (for VCK190 Board) or xcvm1802-vsva2197-2MP-e-S (for VMK180 Board).



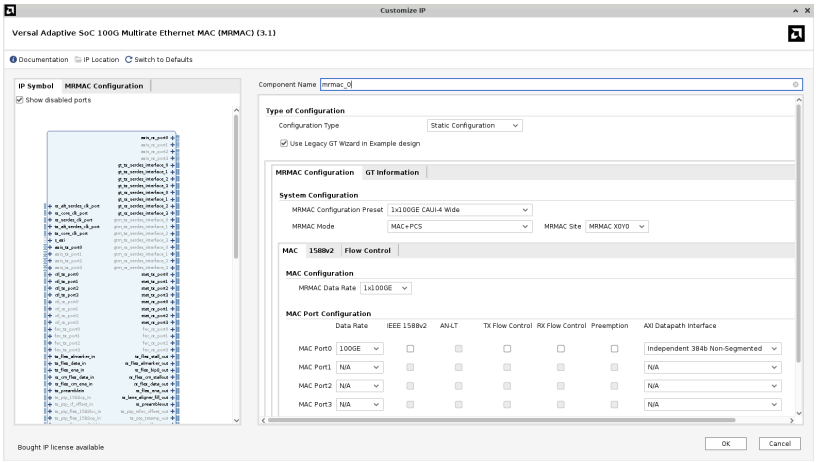
4. Search for and select the MRMAC IP from the IP catalog.



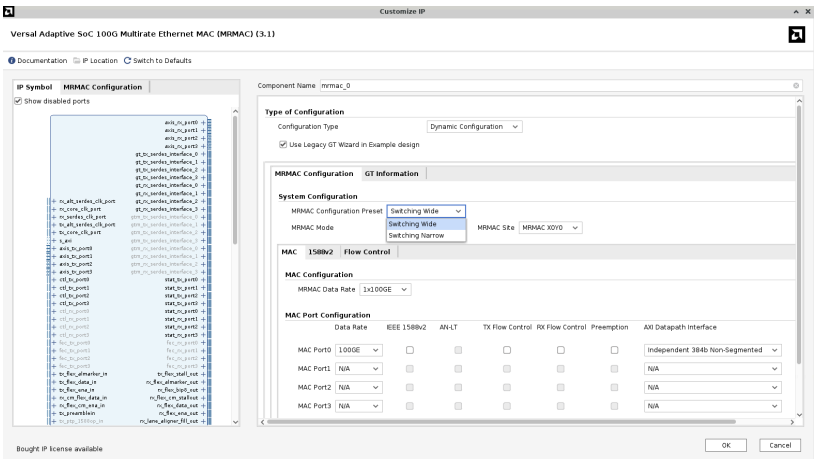
5. Customize the IP.
   a. For VCK190 bank 200, for the MRMAC site, select MRMAC X0Y0.



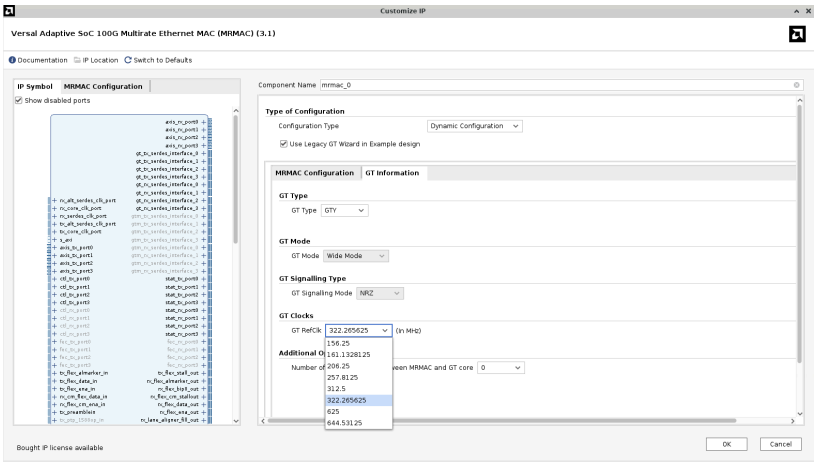
   b. Select the required MRMAC Configuration Type. All the switching presets are present under the Dynamic configuration type and the static presets under the Static Configuration type. For example, if Switching Wide preset is required, select the Dynamic Configuration type.

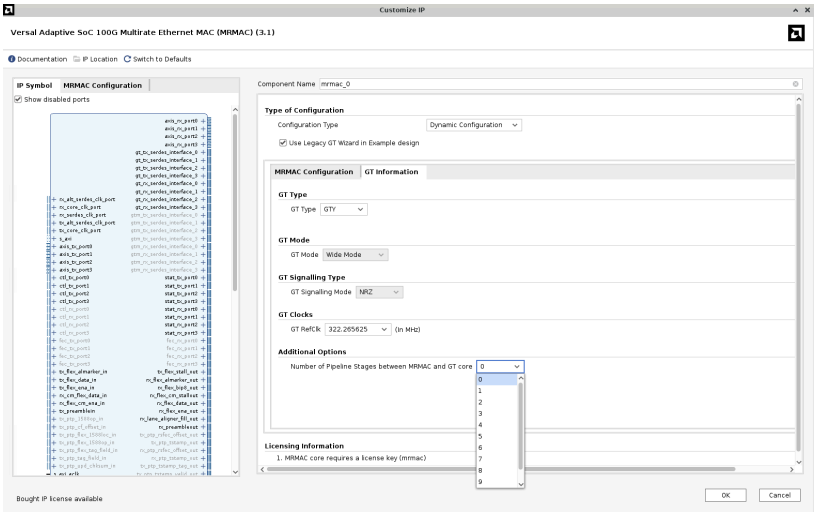


   c. Select the required GT Wizard. For the legacy GT Wizard, enable Use Legacy GT Wizard in Example Design and disable it for the New GT wizard selection. This option will be deprecated in a future release.
   d. Select Switching Wide from the MRMAC Configuration Preset .

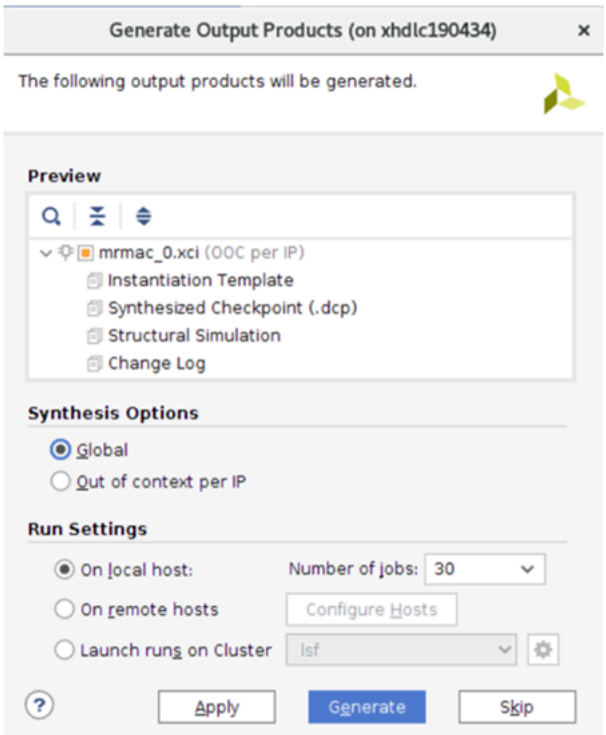


e. Click the GT information tab, select GT refCLK, for example 322.26525, and number of pipeline stages, for example 0. Click OK to finish the IP customization.

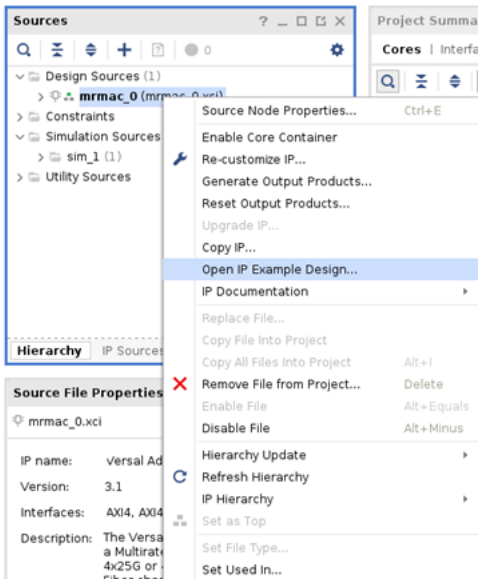




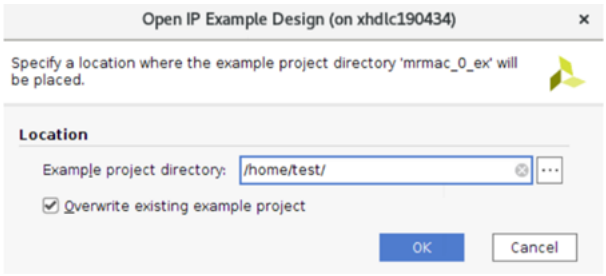
6. After customizing, the Generate Output Products window appears. Click Generate.



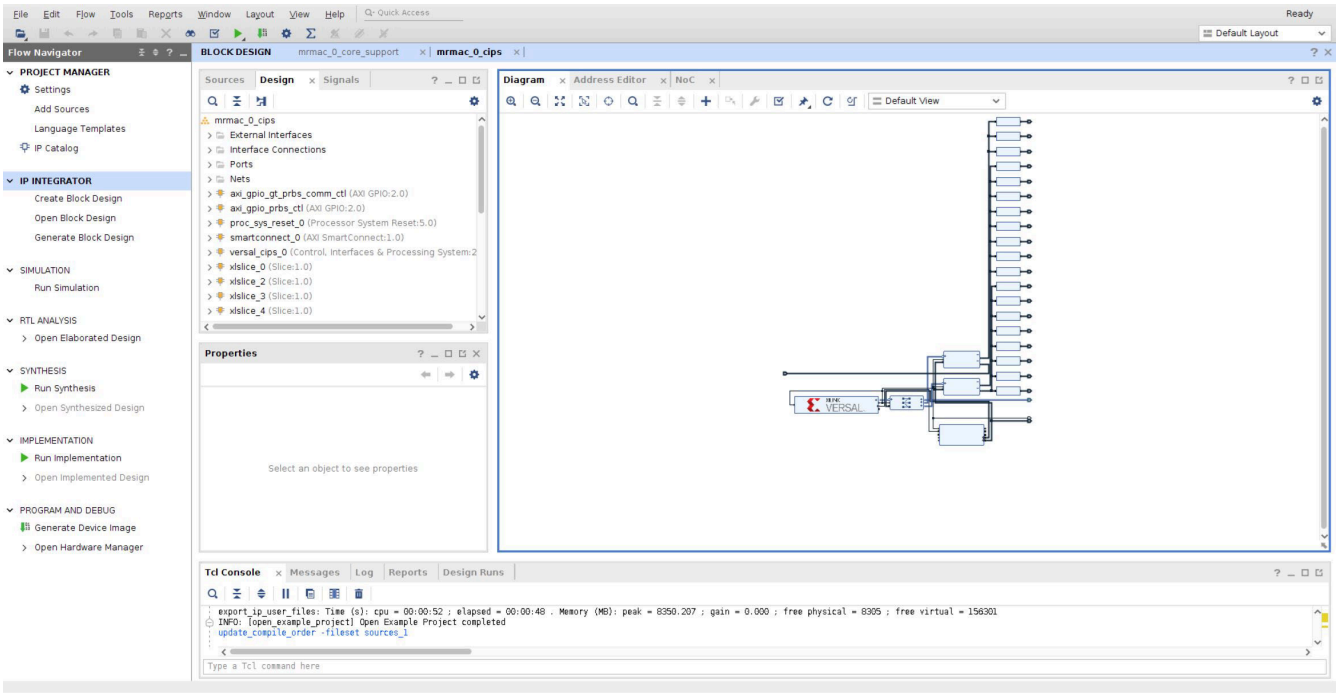
7. After generating the IP, right-click mrmac_0 from the Design Sources and select Open IP Example Design.



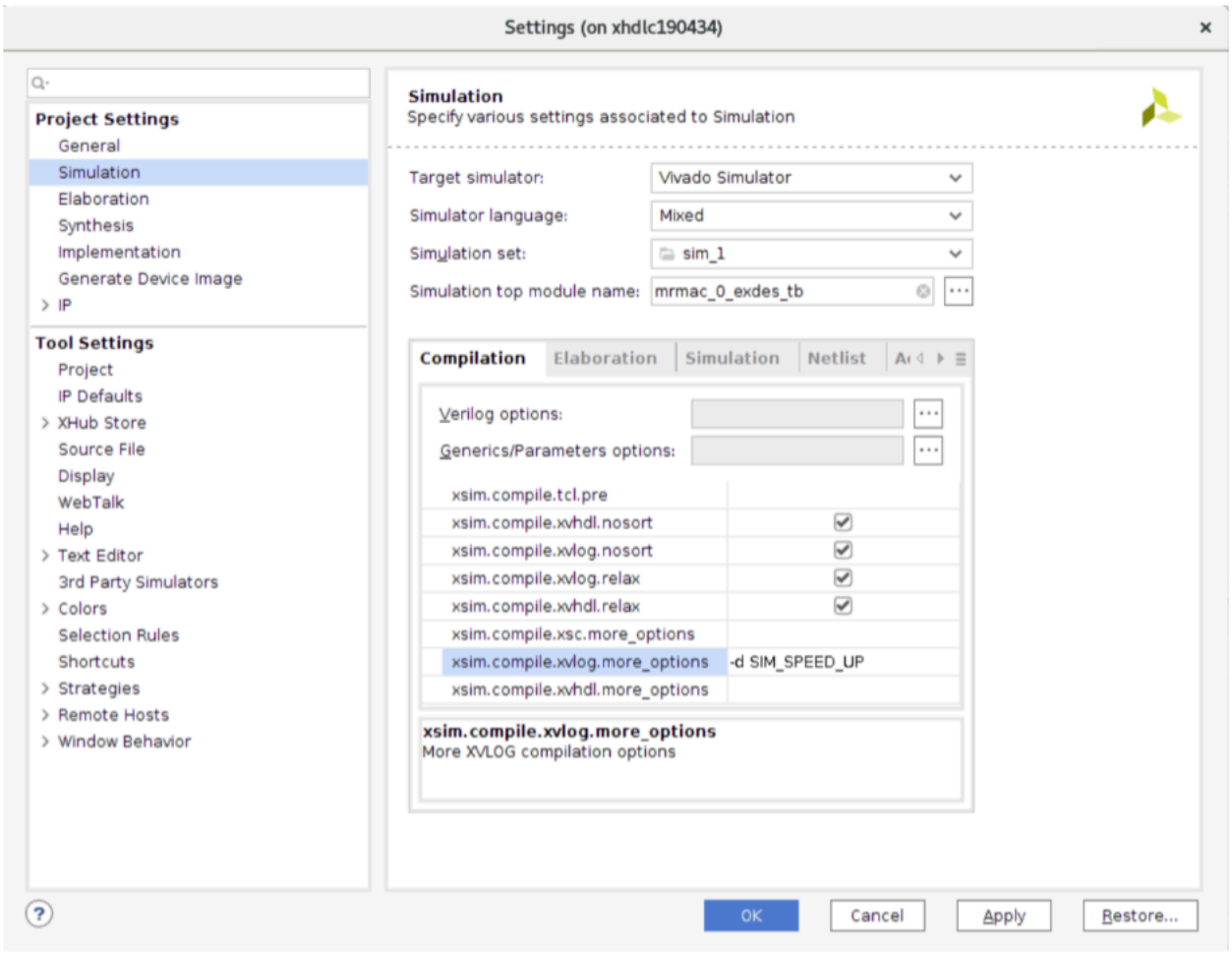
8. Specify the path where you want to save the example design, and click OK.



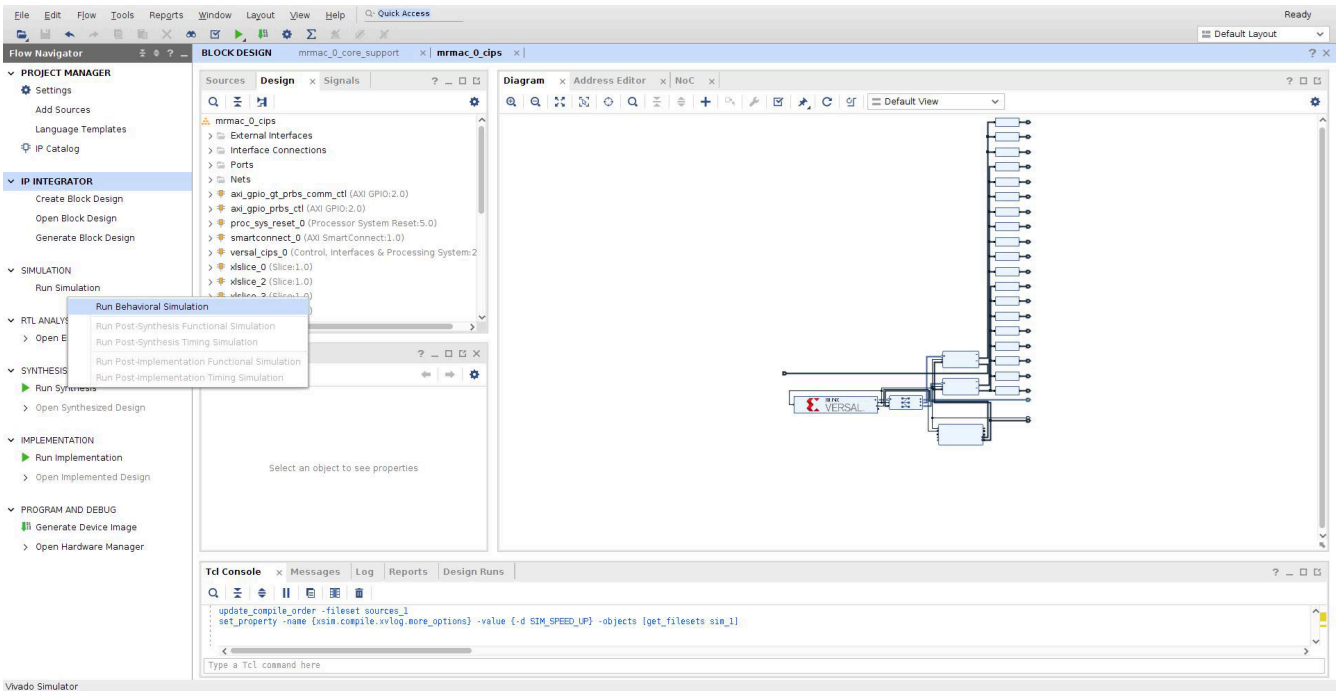
The Vivado project opens with example design ‘mrmac_0’.



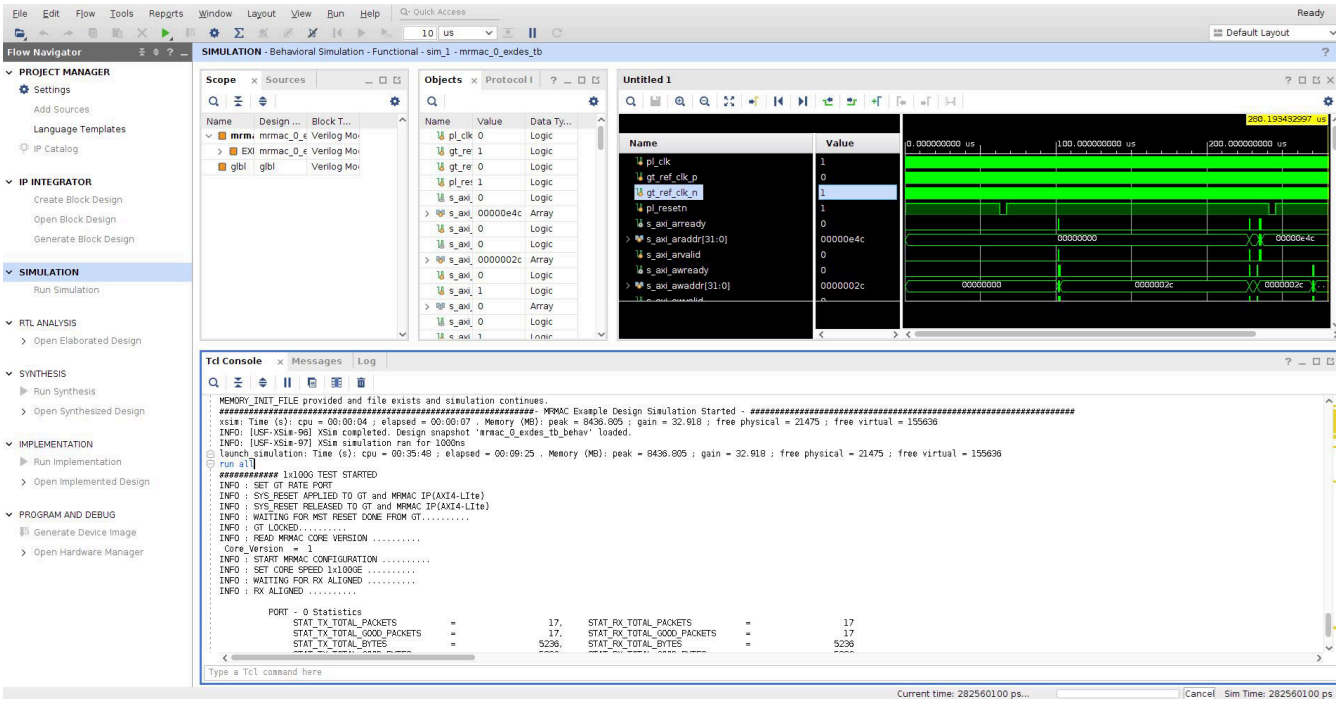
9. Right-click Simulation and select Simulation settings.
   Select the Target simulator as required. The Vivado simulator is shown in the following figure.



10. Once the simulation setup is done, click Run Behavior Simulation. Observe the simulation results and log on to the Tcl console.



The example design simulation starts with a core speed of 100GE then does data sanity by sending and receiving few packets in loopback mode. Upon completion, it prints the MRMAC statistics and also the test status. Then, it switches to 50GE, 40GE, 4x25GE, 4x10GE, and repeats the same test.

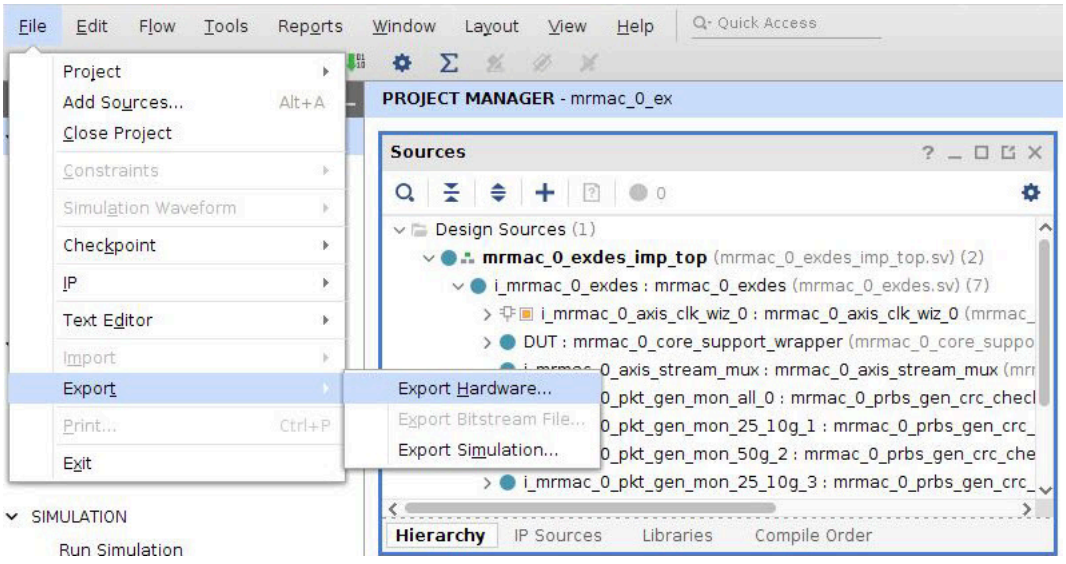


Once the simulation is successful, you can generate the bitstream for board validation.

11. Write the constraints in the XDC file with respect to the selected device/board.
    For VCK190/VMK180, uncomment the following in the example design .xdc file:

```
###### Below is the constraints for VCK190 Board(xcvc1902-vsva2197-2MP-e-S-es1) example design MRMAC_X0Y0-
GTY Bank 200 .  Uncomment below to use.
##### GTY Bank 200
##set_property PACKAGE_PIN AF2 [get_ports {gt_rxp_in[0]}]
##set_property PACKAGE_PIN AF1 [get_ports {gt_rxn_in[0]}]
##set_property PACKAGE_PIN AF7 [get_ports {gt_txp_out[0]}]
##set_property PACKAGE_PIN AF6 [get_ports {gt_txn_out[0]}]
##set_property PACKAGE_PIN AE4 [get_ports {gt_rxp_in[1]}]
##set_property PACKAGE_PIN AE3 [get_ports {gt_rxn_in[1]}]
##set_property PACKAGE_PIN AE9 [get_ports {gt_txp_out[1]}]
##set_property PACKAGE_PIN AE8 [get_ports {gt_txn_out[1]}]
##set_property PACKAGE_PIN AD2 [get_ports {gt_rxp_in[2]}]
##set_property PACKAGE_PIN AD1 [get_ports {gt_rxn_in[2]}]
##set_property PACKAGE_PIN AD7 [get_ports {gt_txp_out[2]}]
##set_property PACKAGE_PIN AD6 [get_ports {gt_txn_out[2]}]
##set_property PACKAGE_PIN AC4 [get_ports {gt_rxp_in[3]}]
##set_property PACKAGE_PIN AC3 [get_ports {gt_rxn_in[3]}]
##set_property PACKAGE_PIN AC9 [get_ports {gt_txp_out[3]}]
##set_property PACKAGE_PIN AC8 [get_ports {gt_txn_out[3]}]
### GTREFCLK 0 Configured as Output (Recovered Clock) which connects to 8A34001 CLK1_IN
### GTREFCLK 1 ( Driven by 8A34001 Q1 )
##set_property PACKAGE_PIN AD11 [get_ports {gt_ref_clk_p}]
##set_property PACKAGE_PIN AD10 [get_ports {gt_ref_clk_n}]
```

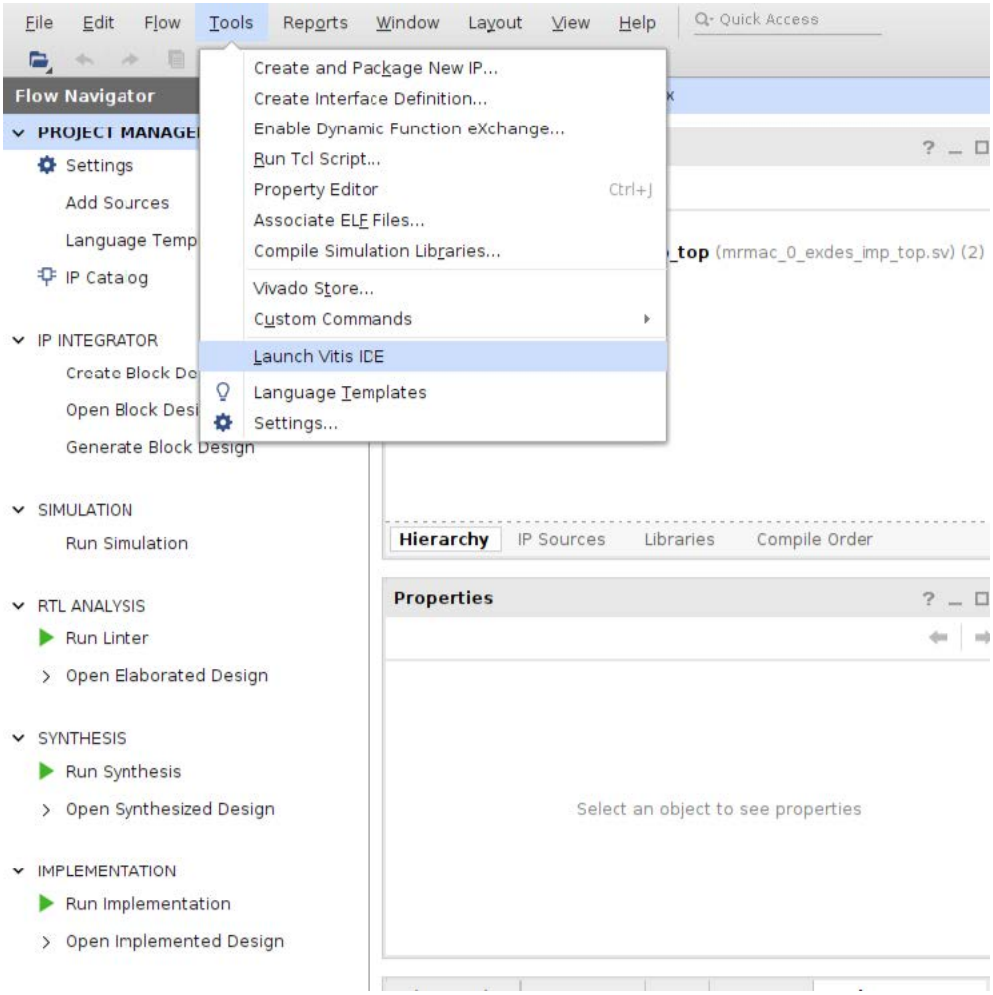
12. Click Generate Device Image. It starts from synthesis and implementation, then the image (.pdi) is generated.
13. After generating the image, export the hardware for application creation by navigating to File > Export > Export Hardware.



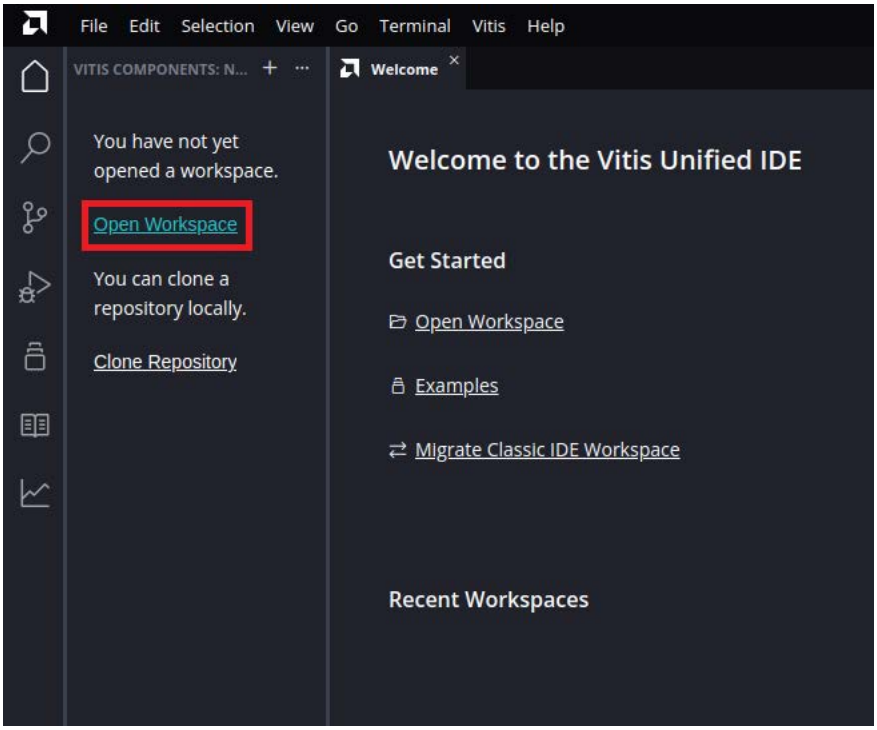
    The Export window opens.
14. Select Fixed and include device image.
15. Generate the .xsa file by navigating to Fixed > Next > Include Device Image > Next > file_name > Finish.

Test ELF Creation

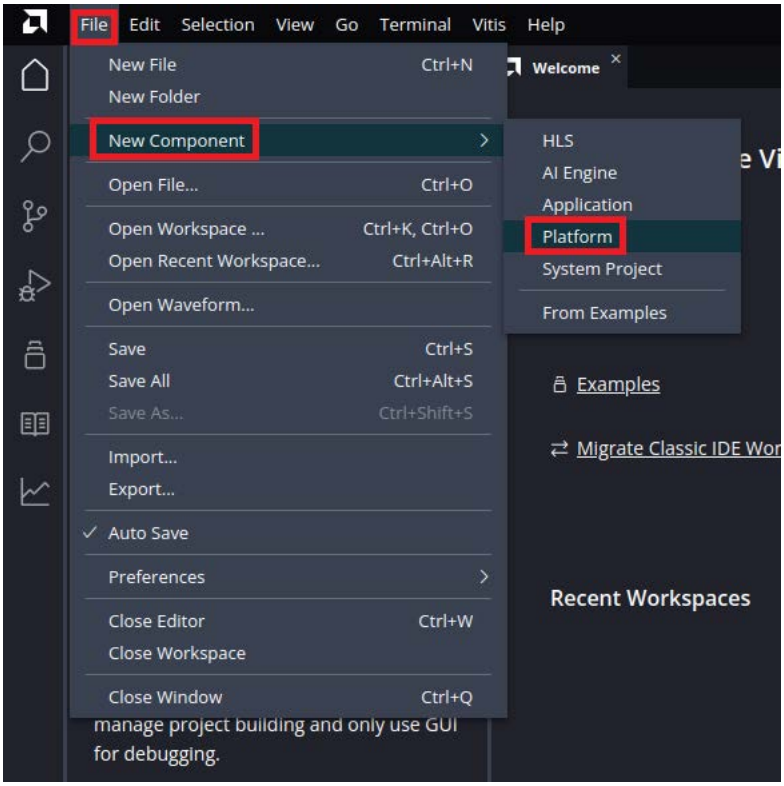
1. Select Tools > Launch Vitis IDE to create an application using the AMD Vitis™ Unified IDE.



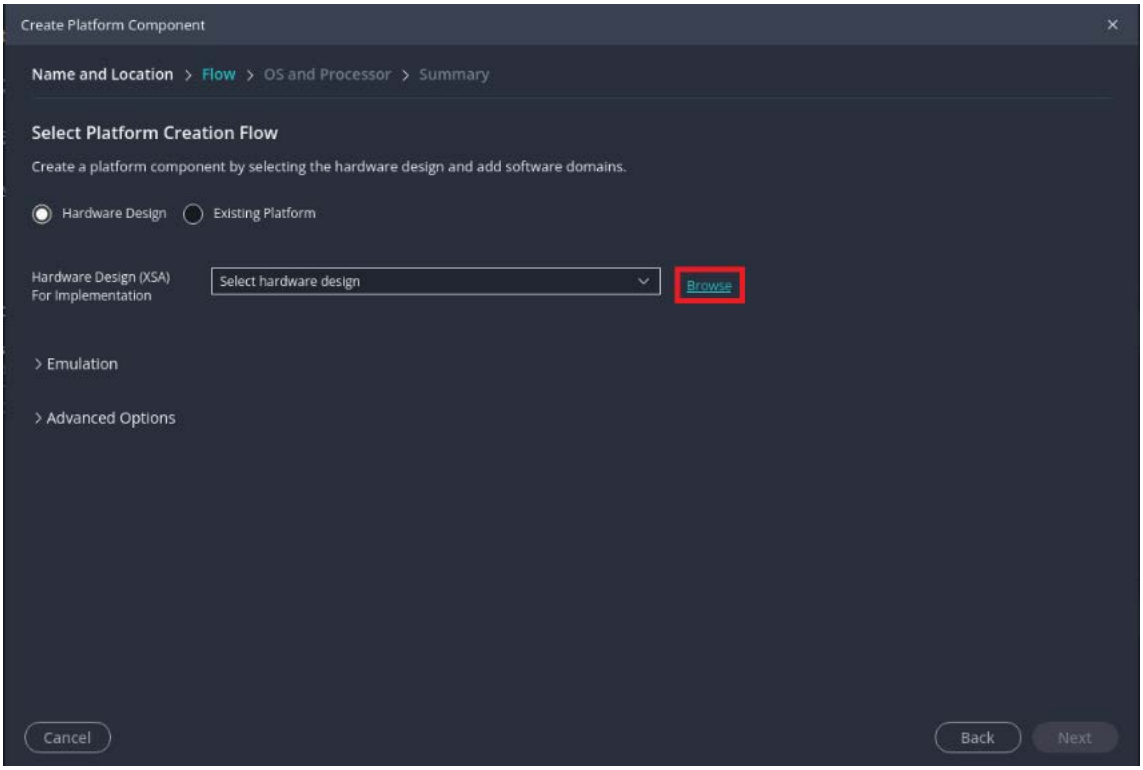
2. Select Open Workspace. Provide a workspace path to create the application. After the set up, the Vitis window is launched.



3. Select File > New Component > Platform to create a platform for the application.

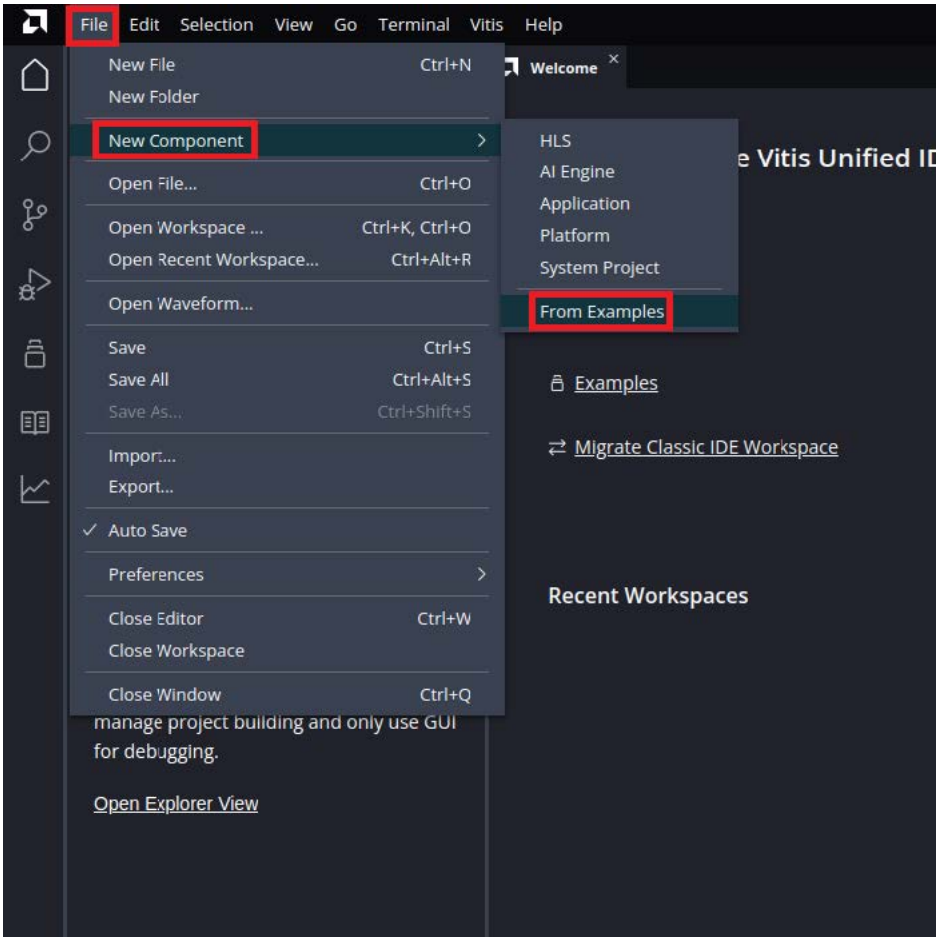


4. Enter the platform component name (for example, platform) and click Next.
5. Browse the exported hardware file (.xsa) path.

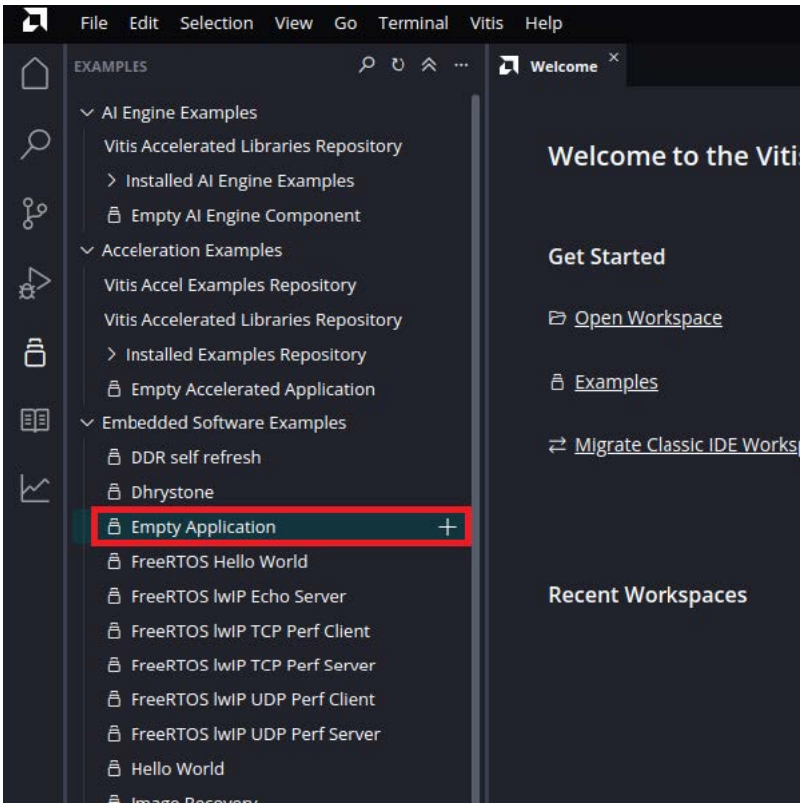


6. Click Next > Next > Finish. The platform for the application is created.

7. Click File > New Component > From Examples.

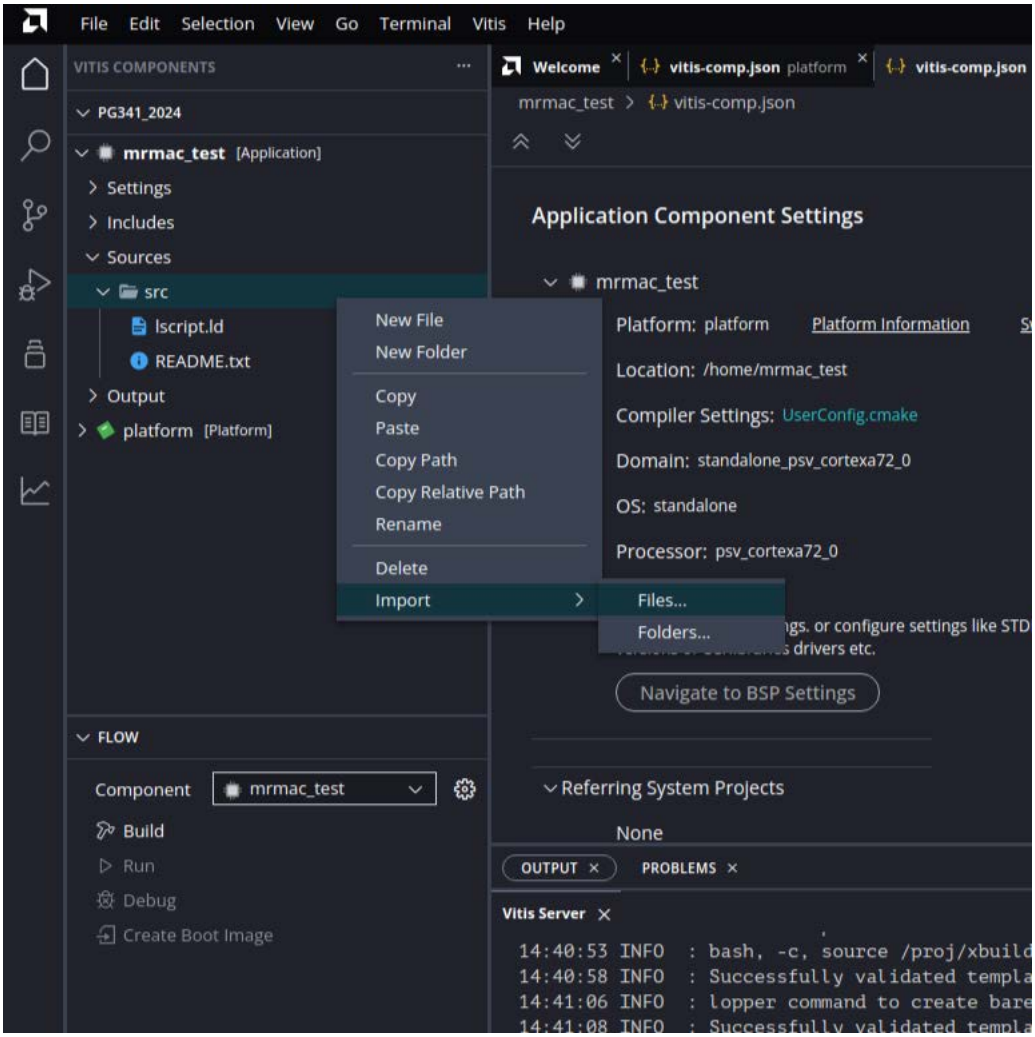


8. Click Empty Application.



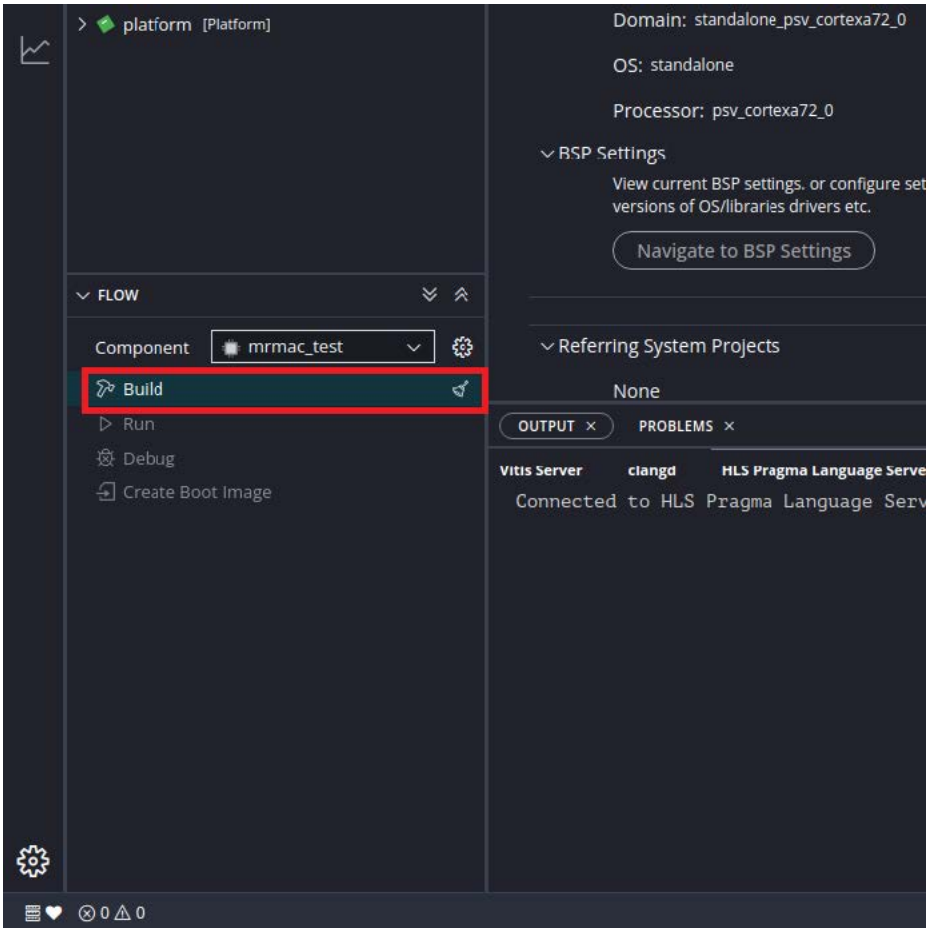
9. Enter the application component name (for example, `mrmac_test`) and click Next.
10. Select the platform created earlier and click Next.
11. Select the standalone_psv_cortexa72_0, and click Next.

12. Add a `C` file to the application. Right-click Import and select Files...

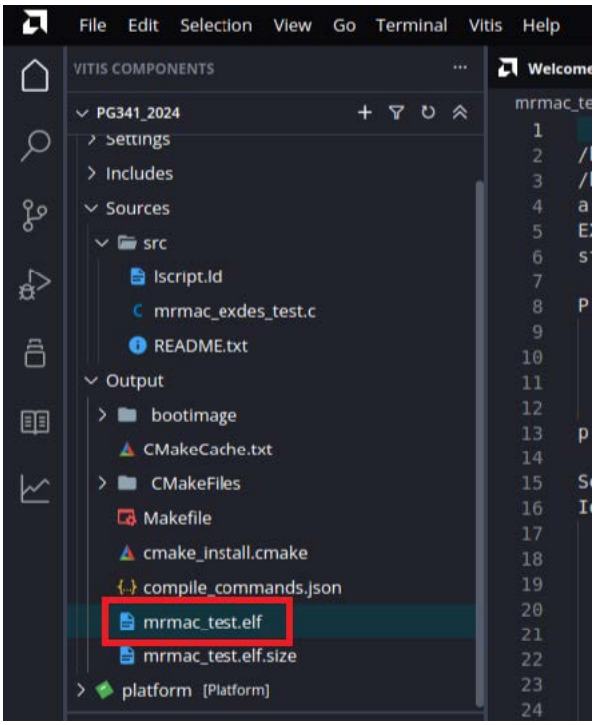


13. Specify the File path:
<file_path>../mrmac_0_ex/mrmac_0_ex.gen/sources_1/bd/mrmac_0_exdes_support/ip/mrmac_0_exdes_support_mrmac_0_co
and select mrmac_exdes_test.c

14. Click Build to create the project.

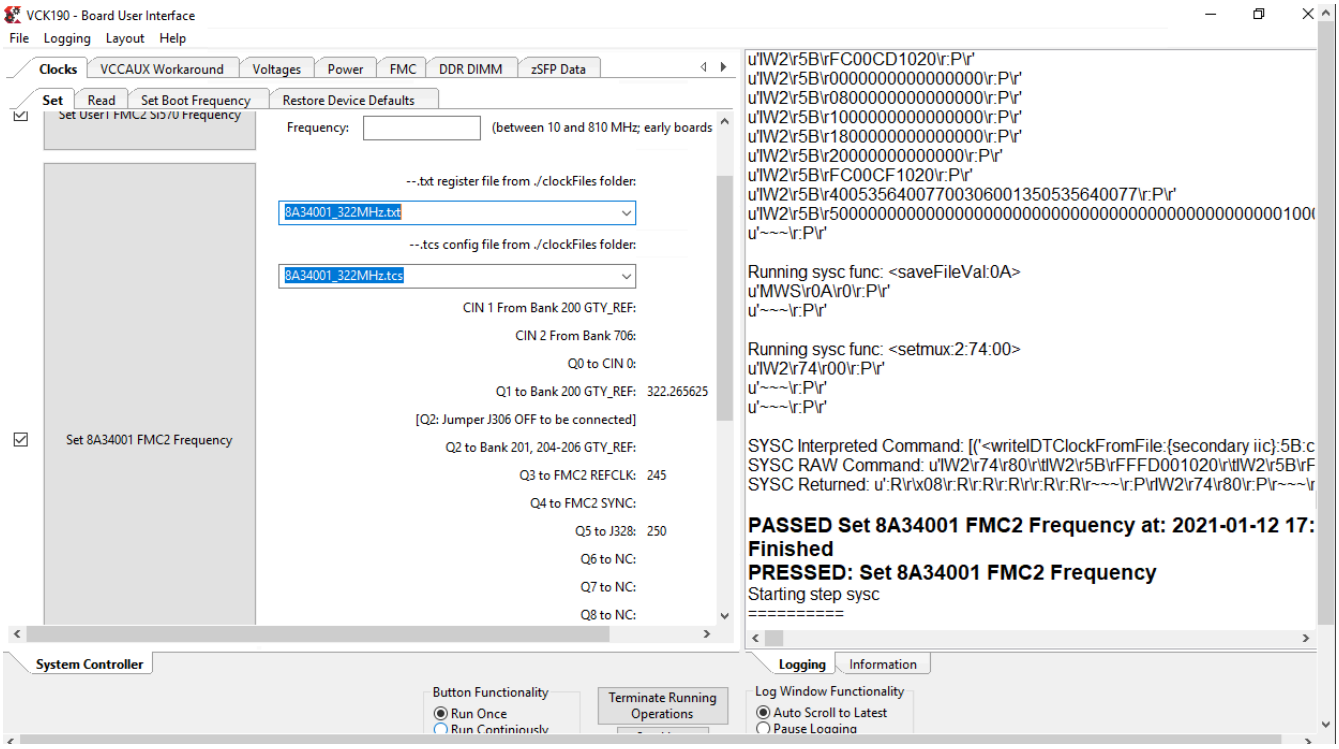


15. An .elf file is generated in the Output folder.



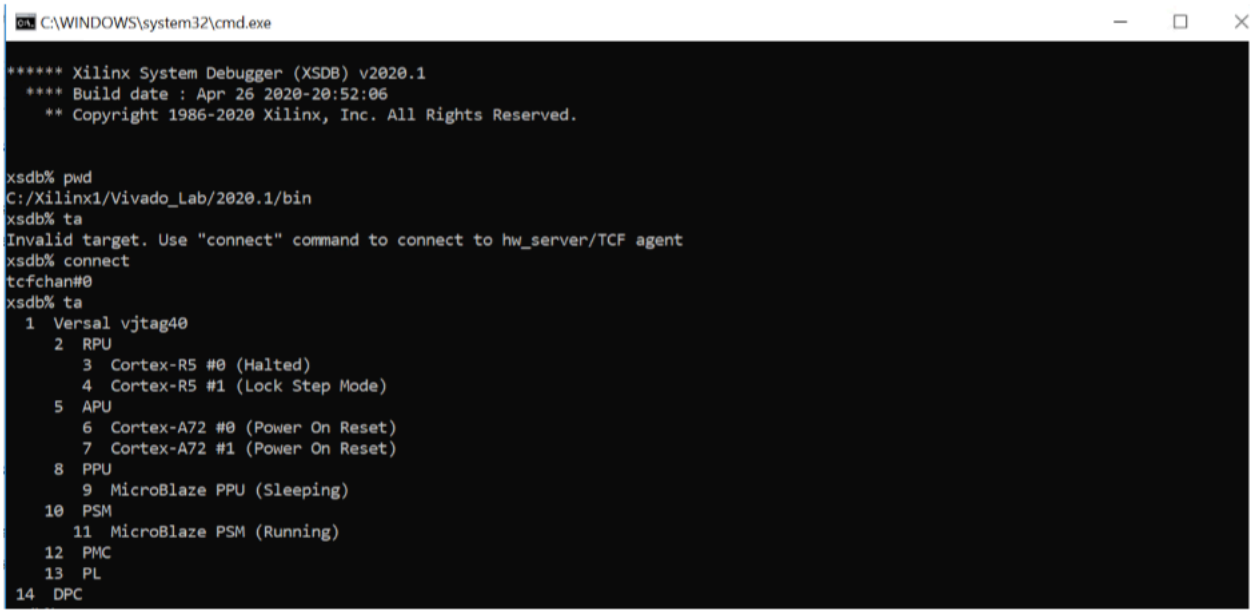
Validate the Design on the VCK190/VMK180

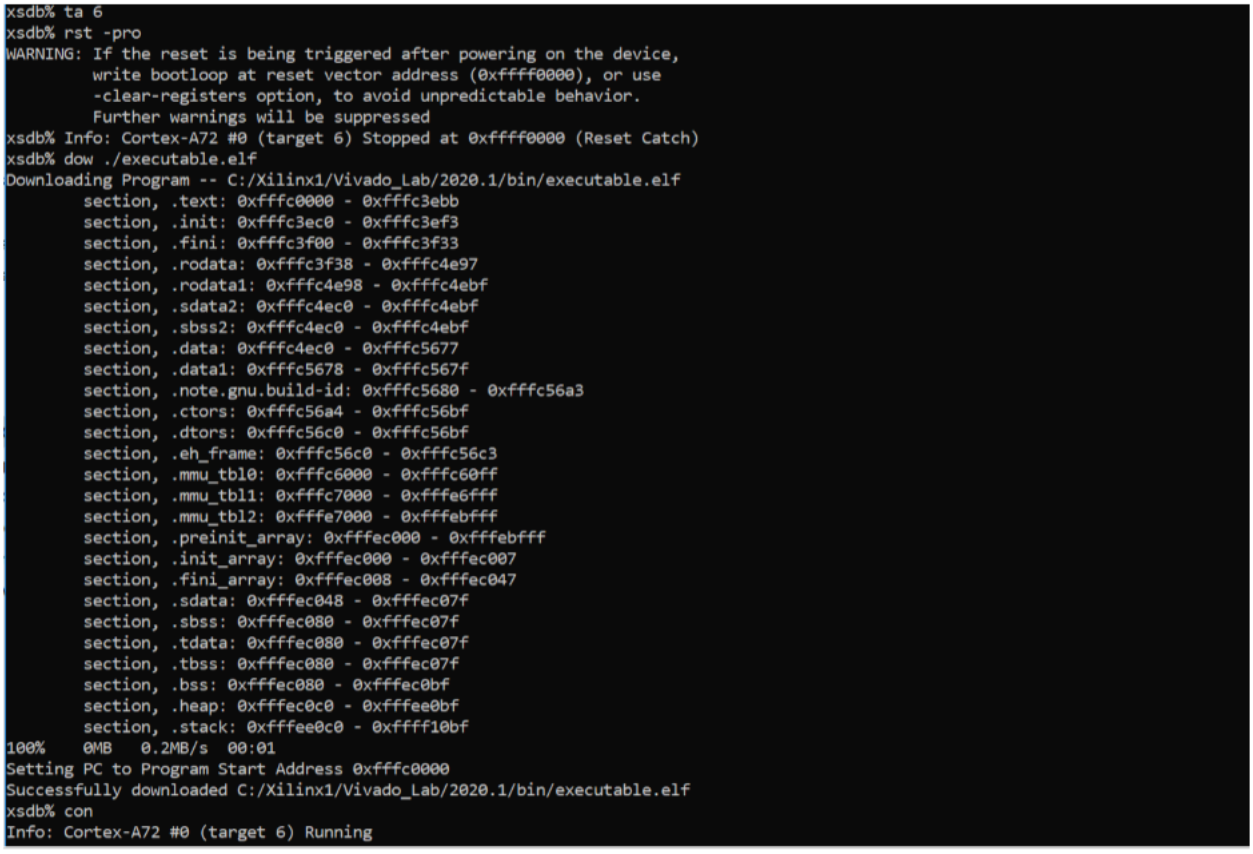
1. Once the bitstream (.pdi) and the application file (.elf) are ready, power ON the Versal board.
   Ensure that all power UART and loopback cable connections are properly connected. Refer to the VCK190 Evaluation Board User Guide (UG1366) and VMK180 Evaluation Board User Guide (UG1411) for additional information.
2. Before dumping the MRMAC image, configure the device with the required reference frequency, as shown in the following figure.



3. Launch the Vivado hardware manager. Select the Versal device and program the device.
   a. Launch the Vivado Lab Edition, and click Open Hardware Manager.
   b. Select Open Target > Open New Target > Next > Next > Finish.
   c. Right click vjtag40_1, and select Program Device as shown in the following figure.
   d. Browse to the path where the bitstream (.pdi) is located, and then select Program.

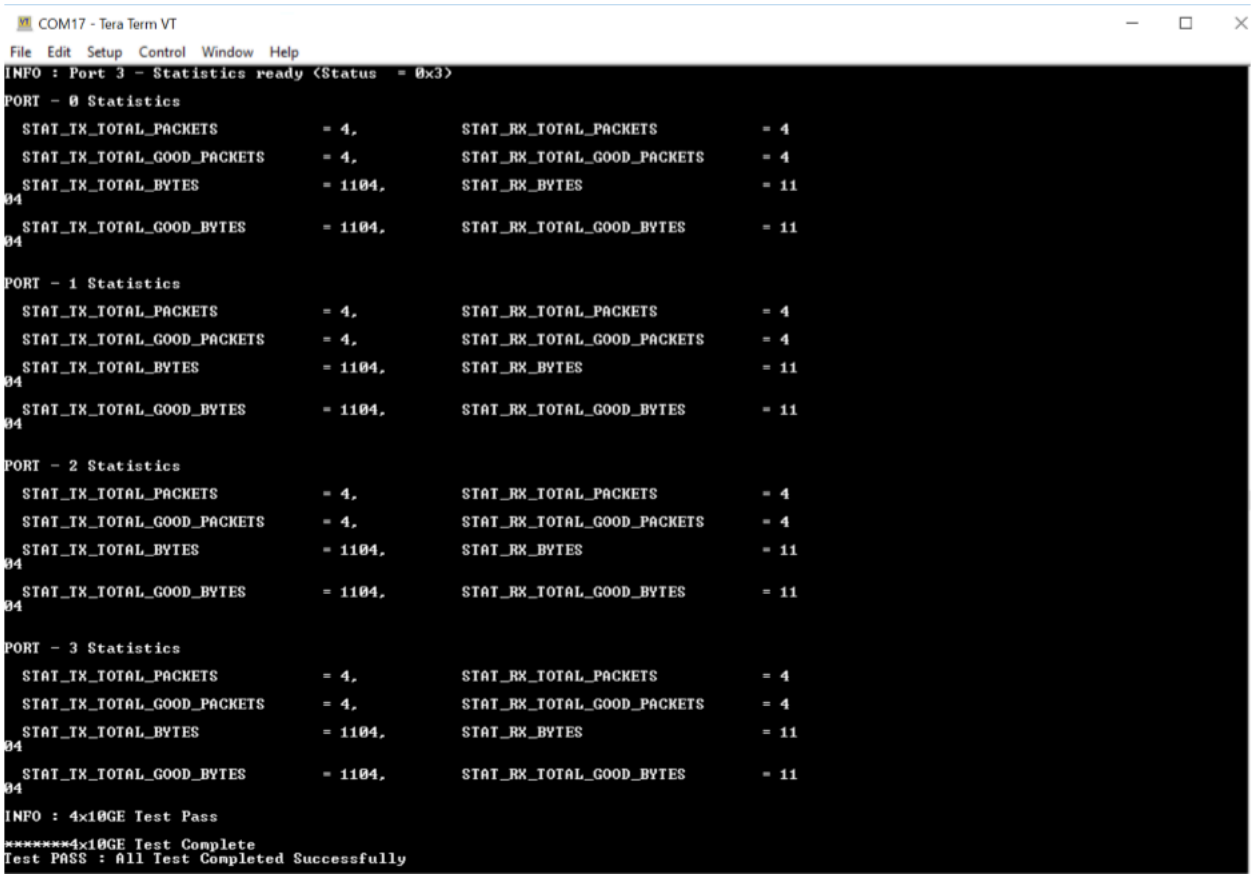
4. Open the xsdb console in C:\Xilinx\Vivado_Lab\2021.1\bin. Follow the procedure as shown in the following snapshot and observe the results in Tera Term.





✎ Note: Set the .elf file path before downloading .elf.

See the results in Tera Term.



PTP 1588 Timer Syncer IP

The PTP 1588 Timer Syncer IP provides reference time to all the Ethernet ports in the example design. It implements an IEEE1588 real time clock timer and can accurately recreate the 1588 timer in a local port's clock domain.

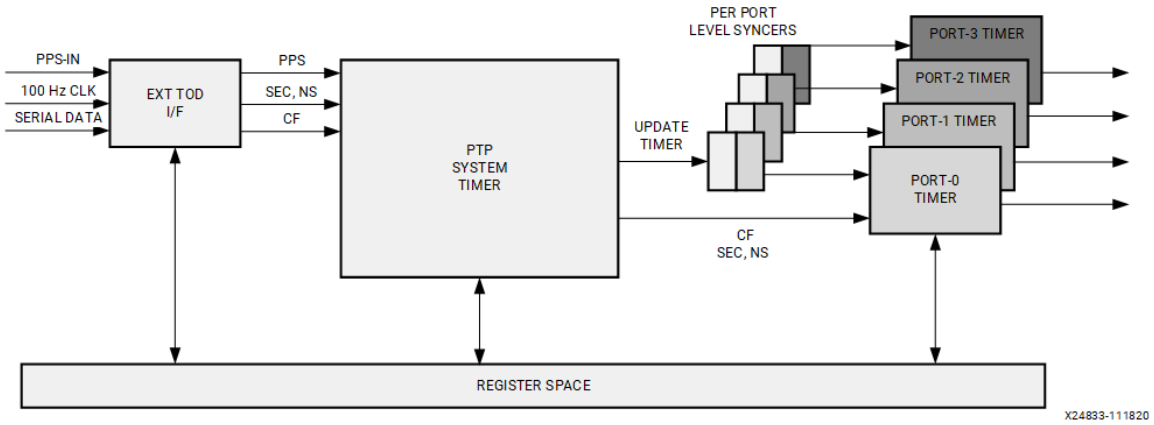
Features

- Can be configured as Master System Timer, Master System and Port Timer.
- Supports the ToD (Sec - NanoSec) and Continuous/Correction Time (CF) formats.
- Implements External ToD bus interface logic for synchronization to high precision clock sources.
- Crosses System ToD to port timer's clock domain.
- AXI4-Lite I/F for configuration and monitoring.

Core Overview

Figure-1 identifies the major functional blocks of the Timer Syncer IP. In this figure only four instances of the port timer are shown for simplicity. The number of port instances are user selectable at the timer of generating core. The core supports up-to 16 instances of port timers.

Figure: Timer Syncer IP functional block diagram



The Timer Syncer IP contains all of the functions and interfaces needed to implement a variety of ToD topologies and applications. Further, the IP can be controlled with either SW or HW devices.

The System Timer maintains time on the free-running system time clock (ts_clk) and provides mechanism to synchronize this timer value to the various Port timers, each of which might be clocked on their separate clocks (phy_clk).

To System Timer IP provides timer values in two formats: Timestamp format (or ToD format) is composed of 80 bits containing fields of unsigned positive seconds and nanoseconds as {seconds[47:0], nanoseconds[31:0] }. Correction Field (CF) format which is a signed 64b value in nanoseconds multiplied by 2^{+16}.

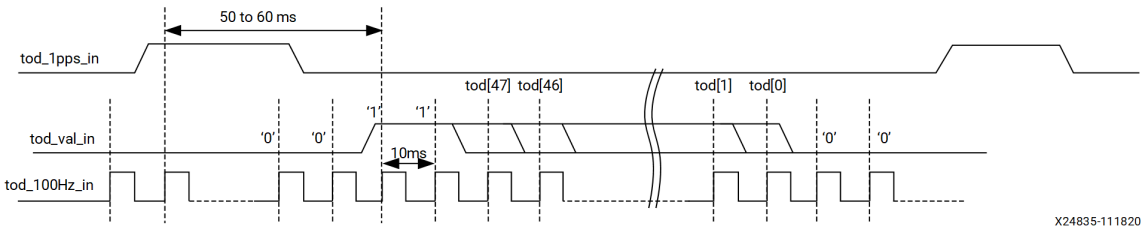
The description of the functional blocks and the interfaces are provided in the subsequent sections.

External ToD/IF

This module is optional and is present only when the IP is generated with “Enable Ext ToD Bus I/F” option selected. Its function is to load the reference seconds value onto the PTP System Timer and synchronize the System Timer to an external 1PPS source. It is intended to be interfaced with a high precision clock source/timing device.

The inputs to this module are: 1PPS pulse, serial 1-bit data bus which carries the reference ToD seconds time which is clocked by a serial clock input. The serial clock is expected to be less than or equal to the free-running clock (ts_clk) of the Timer Sync block. The process of loading the reference 48-bit second is shown in Figure-3.

Figure: ToD Value (Seconds) Interface



The External ToD I/F block stores the serially input ToD seconds value in a holding register. At the next received positive edge of the 1PPS signal, the holding register’s value is incremented by +1 second, and the result be presented to the PTP System Timer sub-block. The positive edge of the 1PPS signal is retimed within the External ToD I/F block to the ts_clk clock and output output for use by the PTP System Timer block.

PTP System Timer Sub Block

This is the Master or the main ToD timer and is clocked by a free running clock (ts_clk). The system timer maintains time in the ToD (48-bit seconds and 30-bit nano-sec) and Continuous time/Correction Field (63-bits CF) formats.

Registers are provided to initialize the timers' seconds and nanoseconds counter values or to read back a snap-shot of the values. A set of offset registers are provided for seconds and nanoseconds values which are added to the ToD timer value prior to being output to the port timers.

After initialization, the PTP System Timer's internal ToD counters can be optionally synchronized to external devices by either the External ToD interface block's output 1PPS signal, or by the software control via register operations.

A 1PPS output indicates when the sys timer's ToD ns field rolls over from 999_999_999 ns to 1 sec. Also, the values of system timer can be read from snapshot registers. The process of transferring the system timer's ToD counters to the port timers is configurable and can be triggered by a 1PPS pulse of the external bus, or by a write to the TOD_SW_LOAD register. When a transfer is triggered, the PTP System Timer outputs a load-pulse (synchronous to the ts_clk domain) and places the value of its internal timer on the output bus.

PTP Port Timer Sub Block

These sub-blocks contain the per port TX and RX timers which are typically clocked by their respective TX and RX PHY clocks (tx_phy_clk and rx_phy_clk). The timer synchronizer IP allows for a maximum of sixteen port timers to be enabled at time of generation.

These port timers maintain time in both the ToD (48-bit seconds and 30-bits nano-sec) and Continuous time/Correction Field (63-bits CF) formats, and provide those outputs synchronized to the tx_phy_clk and rx_phy_clk for use by AMD Ethernet IP.

It is expected that each port's TX and RX clock domains might be asynchronous with the Master timer's ts_clk clock. The port timers contain synchronization logic to domain cross the PTP System Timer's load-pulse and timer update values.

As the PTP System Timer is initialized, or synchronized to the high precision reference clock, it in turn pushes its updated timer value to all the port times which are connected to it. After initialization, the Timer Syncer IP can be configured such that the Port Timers are continuously kept in-sync with the PTP System Timer's master ToD value, or the Port Timers can be independently controlled.

Port Specification

Timer Syncer IP Interface Ports

Table: Timer Sync IP Top Level Ports

Signal	Direction	Clock Domain	Description
Clock Resets			
ts_clk	I	N/A	Free running clock which clocks the System Timer's counters
ts_rst	I	ts_clk	System timer reset -active-High
tod_intr	O	ts_clk	Interrupt asserted on 1-PPS event
External ToD Bus Interface			
tod_1pps_in	I	N/A	External Bus 1-PPS input

Signal	Direction	Clock Domain	Description
tod_sec_clk_in	I	N/A	External Bus clock, used for tod_sec_in
tod_sec_in	I	tod_sec_clk_in	External Bus seconds serial data
tod_1pps_out	O	ts_clk	1-PPS output. Asserted when the System Timer's nano-second field rolls-over.
tod_sec_clk_out	I	N/A	Echo of External Bus clock
tod_sec_out	I	tod_sec_clk_in	Serial output of internal ToD seconds
Per-Port Port Timer Interface _m denotes the port number (0 ≤ m ≤ 15)			
tx_phy_clk_m	I		Port TX PHY Clock
rx_phy_clk_m	I		Port RX PHY Clock
tx_phy_rst_m	I	tx_phy_clk_m	Port TX Reset
rx_phy_rst_m	I	rx_phy_clk_m	Port RX Reset
tx_tod_sec_m[47:0]	O	tx_phy_clk_m	Port TX Timer seconds field
tx_tod_ns_m[31:0]	O	tx_phy_clk_m	Port TX Timer nano-seconds field
tx_tod_corr_m[63:0]	O	tx_phy_clk_m	Port TX Timer CF field
rx_tod_sec_m[47:0]	O	rx_phy_clk_m	Port RX Timer seconds field
rx_tod_ns_m[31:0]	O	rx_phy_clk_m	Port RX Timer nano-seconds field
rx_tod_corr_m[63:0]	O	rx_phy_clk_m	Port RX Timer CF field
AXI4-Lite Interface			
s_axi_aclk	I		AXI4-Lite I/F clock
s_axi_aresetn	I	s_axi_aclk	AXI4-Lite I/F reset
s_axi_awaddr[31:0]	I	s_axi_aclk	Write address
s_axi_awvalid	I	s_axi_aclk	Write Address Valid
s_axi_awready	O	s_axi_aclk	Write Address Ready
s_axi_wdata[31:0]	I	s_axi_aclk	Write Data
s_axi_wstrb[3:0]	I	s_axi_aclk	Write Data Byte Valid. Tie to 4'hF
s_axi_wvalid	I	s_axi_aclk	Write Valid
s_axi_wready	O	s_axi_aclk	Write Ready
s_axi_bresp	O	s_axi_aclk	Write Response

Signal	Direction	Clock Domain	Description
s_axi_bvalid	O	s_axi_aclk	Write Response Valid
s_axi_bready	I	s_axi_aclk	Write Response Ready
s_axi_araddr[31:0]	I	s_axi_aclk	Read Address
s_axi_arvalid	I	s_axi_aclk	Read Address Valid
s_axi_arready	O	s_axi_aclk	Read Address Ready
s_axi_rdata[31:0]	O	s_axi_aclk	Read Data
s_axi_rresp	O	s_axi_aclk	Read Response
s_axi_rvalid	O	s_axi_aclk	Read Data Valid
s_axi_rready	I	s_axi_aclk	Read Ready

Internal Sub-Block Ports

The following tables contain select ports present on the IP sub-blocks to assist in debugging.

Table: Internal Sub-Block Ports

Signal	Direction	Clock Domain	Description
External ToD Bus Ports			
ext_tod_bus_sec	O	ts_clk	Seconds value sent to PTP System Timer
ext_tod_bus_ns	O	ts_clk	Nanoseconds value sent to PTP System Timer
ext_tod_bus_1pps	O	ts_clk	1PPS sent to PTP System Timer
PTP System Timer Ports			
sys_tod_sec[48:0]	O	ts_clk	System Timer ToD seconds field.
sys_tod_ns[31:0]	O	ts_clk	System Timer ToD nano-seconds field.
sys_tod_corr[63:0]	O	ts_clk	System Timer ToD CF format.
update_timer	O	ts_clk	Sync update pulse to the Port Timer Blocks. Asserted when the Master timer is synchronized either by the Ext ToD I/F or via register updates
sys_timer_1pps_O	O	ts_clk	1-PPS output to External ToD Bus Block. This port is asserted when the System Timer's nano-second field rolls-over.

Signal	Direction	Clock Domain	Description
Port Timer Ports			
update_port_timer	I	ts_clk	Sync update pulse driven by the System Timer
sys_tod_sec_in[48:0]	I	ts_clk	System Timer seconds field. Present only when the Timer Format is ToD or Both
sys_tod_ns_in[31:0]	I	ts_clk	System Timer nano-seconds field. Present only when the Timer Format is ToD or Both
sys_tod_corr_in[63:0]	I	ts_clk	System Timer CF field. Present only when the Timer Format is CF or Both

Register Spaces

All the control and status registers are memory mapped. After power-up or reset, you can reconfigure the core parameters from their defaults at any time. The following table lists the access type details. Table 2 provides register map details.

Table: Register Access Type Definitions

Access Type	Description
RO	Read Only Readable register; write has no effect
RW	Read-Write Readable and writable register
RW1T	Read Write '1' to trigger Write '1' to trigger. Write '0' is ignored. Read returns '0'.
RW1C	Read, write '1's to Clear Writing a '1' clears the corresponding bit position in the register to '0'; Writing a '0' leaves the corresponding bit unchanged. For example, it can be used to acknowledge interrupt status.
WO	Write Only Writable register, the value is not readable (returns '0')

Table: Register Map

Offset	Register Name	Access	Description

Offset	Register Name	Access	Description
0x0000	TOD_CONFIG	RW	Main Configuration [0] – Enable System Timer: • A '0' to this bitfield disables the system timer IP. [1] – Enable External ToD Bus: • Write '1' to enable the ToD Bus signals. The Overwrite Mode field further defines how signaling is used. This is present only when the core is generated with External ToD Bus I/F support. [3:2] – Overwrite Mode: • 0x0 System Timer counter is not overwritten at 1-PPS event from External ToD Bus. • 0x1 System Timer counter is overwritten with the External ToD Bus seconds input at 1-PPS event from External ToD Bus. • 0x2 System Timer counter is overwritten with value stored in the SW TOD_SW_SEC_0/1 register at 1-PPS event from the External ToD Bus. • 0x3 Reserved The above modes are only present when the core is generated with Ext ToD Bus interface support. [4] – Enable Sys_timer auto-refresh. Write a '1' to enable the System Timer block to automatically refresh the port-timers

Offset	Register Name	Access	Description
			with the latest ToD and CTIME (if enabled) values [5] – Enable timer snapshot on external 1PPS [15:6] – Reserved [16:31] – Enable Port TX and RX Timers • [16] = Enable Port-0 TX and RX • [17] = Enable Port-1 TX and RX • ……. • [31] = Enable Port-15 TX and RX The upper limit for port number depends on the number of ports enabled at the time of generating core. [31:20] – Reserved
0x0004	TOD_SNAPSHOT	RW1T	[0] – Snapshot all timers Writing 1'b1 snapshots all Counters (System, External ToD Bus, and all enabled ports) [31:1] – Reserved
0x0008	TOD_INTR_ENABLE	RW	Interrupt enable register [0] – 1-PPS interrupt (Master RTC sec field) [15:1] – Reserved [16] – 1-PPS interrupt (External 1pps input) [31:1] – Reserved
0x000C	TOD_INTR_STATUS	RW1C	Interrupt clear register [0] – 1-PPS interrupt (Master RTC sec field) [15:1] – Reserved [16] – 1-PPS interrupt (External 1pps input) [31:1] – Reserved

Offset	Register Name	Access	Description
0x0010	TOD_SW_SEC_0	RW	[31:0] – Overwrite Master Timer's Second field bits [31:0]
0x0014	TOD_SW_SEC_1	RW	[15:0] – Overwrite Master Timer's Second field bits [47:32] [31:16] – Reserved
0x0018	TOD_SW_NS	RW	[29:0] – Overwrite Master Timer's Second field bits [31:30] – Reserved
0x001C	TOD_SW_LOAD	RW1T	[0] – Write '1' initiates a load of the System Timer's ToD values from the TOD_SW_SEC_0/1, TOD_SW_NS, and TOD_SW_CTIME_0/1 registers. [1] – Write '1' initiates a load of the System Timer's ToD Offset value from the TOD_SEC_SYS_OFFSET_0/1, and TOD_NS_SYS_OFFSET_0 registers. Note: Offset is added by logic prior to the System Timer's output to the Port Timers. As such, the offset is not reflected by System Timer ToD read backs. [31:0] – Reserved
0x0020	TOD_SW_CTIME_0	RW	[31:0] – Overwrite Master Timer's CF field bits [31:0]
0x0024	TOD_SW_CTIME_1	RW	[30:0] – Overwrite Master Timer's Second field bits [63:32] [31] – Reserved

Offset	Register Name	Access	Description
0x0028	TOD_SEC_SYS_OFFSET_0	RW	{TOD_SEC_SYS_OFFSET_1[15:0], TOD_SEC_SYS_OFFSET_0[31:0]} – Represents System timer 48b seconds field signed offset value. TOD_NS_SYS_OFFSET_0[29:0] – Represents System timer 30b nano second field signed offset value. Signed bit interpreted as follows for TOD_SEC_SYS_OFFSET: • If [47] = 1'b1 then subtract from the system timer • If [47] = 1'b0 then add to the system timer Need to apply trigger bit [1] at register 0x001C
0x002C	TOD_SEC_SYS_OFFSET_1	RW	
0x0030	TOD_NS_SYS_OFFSET_0	RW	
0x0034	TOD_SYS_PERIOD_0	RW	System Timer TS clock period expressed in 2-48 ns For example, 3.2 ns is represented as: • TOD_SYS_PERIOD_0[31:0] = 0x3333_3333 • TOD_SYS_PERIOD_1[23:0] = 0x0003_3333 • TOD_SYS_PERIOD_1[31:24] reserved A write to the TOD_SYS_PERIOD_1 register 'commits' the updated period value to System timer.
0x0038	TOD_SYS_PERIOD_1	RW	
0x0100	TOD_SYS_SEC_0	RO	[31:0] – Snapshot of System Timer's Second Field [31:0]
0x0104	TOD_SYS_SEC_1	RO	[15:0] – Snapshot of System Timer's Second Field [47:32] [31:16] – Reserved

Offset	Register Name	Access	Description
0x0108	TOD_SYS_NS	RO	[29:0] – Snapshot of System Timer's Nanosecond Field [29:0] [31:30] – Reserved
0x010C	TOD_SYS_OFFSET	RW	[31:0] – Signed offset to be applied to seconds value loaded from the External ToD bus, and to be added to 0 nanoseconds field when a 1PPS event occurs on the External ToD bus. The signed value is expressed in 2^{-16} ns. This is present only when the core is generated with Ext ToD Bus support
0x0110	TOD_SYS_CTIME_0	RO	[31:0] – Snapshot of System Timer's CF Field [31:0]
0x0114	TOD_SYS_CTIME_1	RO	[30:0] – Snapshot of System Timer's CF Field [63:32] [31] – Reserved
0x0120	TODBUS_SEC_0	RO	Current value of the Ext ToD Bus's Second Field [31:0]
0x0124	TODBUS_SEC_1	RO	[15:0] – Current value of the Ext ToD Bus's Second Field [47:32] [31:16] – Reserved
0x012C	TODBUS_SYS_DIFF	RO	[31:0] – Once a second comparison of External ToD bus captured value and System Timer's internal ToD value in signed unit of 2^{-16} ns.
Port Timer Registers: This register set is replicated for every Port Timer present (0 to 15) at the offsets listed for Ports 1 to 15.			
Port 0 Registers: 0x0200 to 0x027F			
0x0200	TX0_CTIME_0	RO	[31:0] – Snapshot of Port0 TX Timer's CF Field [31:0]
0x0204	TX0_CTIME_1	RO	[30:0] – Snapshot of Port0 TX Timer's CF Field [63:32] [31] – Reserved

Offset	Register Name	Access	Description
0x0208	TX0_PERIOD_0	RW	Port0 TX clock period expressed in 2^{-48} ns For example, 3.2 ns is represented as: • TX0_PERIOD_0[31:0] = 0x3333_3333 • TX0_PERIOD_1[23:0] = 0x0003_3333 • TX0_PERIOD_1[31:24] reserved A write to the TX0_PERIOD_1 register will 'commit' the updated period value to Port timer.
0x020C	TX0_PERIOD_1	RW	
0x0210	TX0_SYS_OFFSET	RW	[31:0] – Signed offset applied to Port0 TX Timer's ToD output expressed in 2^{-16} ns.
0x0214	TX0_NS_SNAP	RO	[29:0] – Snapshot of Port0 TX Timer's Nanosecond Field [29:0] [31:30] – Reserved
0x0218	TX0_SEC_0_SNAP	RO	[31:0] – Snapshot of Port0 TX Timer's Second Field [31:0]
0x021C	TX0_SEC_1_SNAP	RO	[15:0] – Snapshot of Port0 TX Timer's Second Field [47:32] [31:16] – Reserved
0x0220	RX0_CTIME_0	RO	[31:0] – Snapshot of Port0 RX Timer's CF Field [31:0]
0x0224	RX0_CTIME_1	RO	[30:0] – Snapshot of Port0 RX Timer's CF Field [63:32] [31] – Reserved

Offset	Register Name	Access	Description
0x0228	RX0_PERIOD_0	RW	Port0 RX clock period expressed in 2^{-48} ns For example, 3.2 ns is represented as: • RX0_PERIOD_0[31:0] = 0x3333_3333 • RX0_PERIOD_1[23:0] = 0x0003_3333 • RX0_PERIOD_1[31:24] reserved A write to RX0_PERIOD_1 register will 'commit' the updated period value to Port timer.
0x022C	RX0_PERIOD_1	RW	
0x0230	RX0_SYS_OFFSET	RW	[31:0] – Signed offset applied to Port0 RX Timer's ToD output expressed in 2^{-16} ns.
0x0234	RX0_NS_SNAP	RO	[29:0] – Snapshot of Port0 RX Timer's Nanosecond Field [29:0] [31:30] – Reserved
0x0238	RX0_SEC_0_SNAP	RO	[31:0] – Snapshot of Port0 RX Timer's Second Field [31:0]
0x023C	RX0_SEC_1_SNAP	RO	[15:0] – Snapshot of Port0 RX Timer's Second Field [47:32] [31:16] – Reserved
0x0240	CORE_TX0_PERIOD_ 0	RO	Port0 TX clock period configured in core, expressed in 2^{-48} ns For example, 3.2 ns is represented as: • CORE_TX0_PERIOD_ 0 = 0x3333_3333 • CORE_TX0_PERIOD_ 1 = 0x0003_3333 • CORE_TX0_PERIOD_ 1[31:24] reserved
0x0244	CORE_TX0_PERIOD_ 1	RO	

Offset	Register Name	Access	Description
0x0248	CORE_RX0_PERIOD_ 0	RO	Port0 RX clock period configured in core, expressed in 2^{-48} ns For example, 3.2 ns is represented as: • CORE_RX0_PERIOD_ 0 = 0x3333_3333 • CORE_RX0_PERIOD_ 1 = 0x0003_3333 • CORE_RX0_PERIOD_ 1[31:24] reserved
0x024C	CORE_RX0_PERIOD_ 1	RO	
0x0250	PORT0_SEC_ OFFSET_0	RW	PORT0_SEC_SYS_OFFSET_1[15:0], PORT0_SEC_SYS_OFFSET_0[31:0]} – Represents the System timer 48b seconds field signed offset value. PORT0_NS_SYS_OFFSET_0[29:0] – Represents the System timer 30b nanosecond field signed offset value. Signed bit interpreted as follows for PORT0_SEC_SYS_OFFSET: • If [47] = 1'b1 then subtract from system timer • If [47] = 1'b0 then add to system timer These registers are used to apply a large port-specific offset. A write to PORT0_NS_SYS_OFFSET_0 is required to trigger the application of the port's offset.
0x0254	PORT0_SEC_ OFFSET_1	RW	
0x0258	PORT0_NS_OFFSET_0	RW	
Port 1 Registers: 0x0280 to 0x02FF (Port # x 0x0280)			
Port 2 Registers: 0x0300 to 0x037F			
Port 3 Registers: 0x0380 to 0x03FF			
Port 4 Registers: 0x0400 to 0x047F			
Port 5 Registers: 0x0480 to 0x04FF			
Port 6 Registers: 0x0500 to 0x057F			

Offset	Register Name	Access	Description
Port 7 Registers: 0x0580 to 0x05FF			
Port 8 Registers: 0x0600 to 0x067F			
Port 9 Registers: 0x0680 to 0x06FF			
Port 10 Registers: 0x0700 to 0x077F			
Port 11 Registers: 0x0780 to 0x07FF			
Port 12 Registers: 0x0800 to 0x087F			
Port 13 Registers: 0x0880 to 0x08FF			
Port 14 Registers: 0x0900 to 0x097F			
Port 15 Registers: 0x0980 to 0x09FF			

S/W Driver Core Initialization

Its recommended that the S/W drivers follow these steps sequentially to properly initialize the core.

1. Initialize other elements of the design such that the clocks (including ts_clk, and any tx/rx_phy_clk* clocks) are present and stable. Release the resets for those clock domains (ie. ts_rst, tx/rx_phy_rst*).
2. Configure the TOD_CONFIG register:
    - In Timer or Timer Syncer mode set bit [0] to enable the System Timer.
    - If an external ToD bus (1PPS synchronization and optionally serial seconds input) is to be used, then set bit [1] to '1'.
    - Set the mode bit field [3:2] to the setting matching the intended synchronization method used in your system. For example, if your system uses an external device connected to the External ToD bus (1PPS input and second's value serial input) the mode field 0x1.
    - Finally enable (set to '1') the appropriate bits of the Port Timer enable bitfield [19:4] to enable the port TX and RX Timers.
3. If the System Timer's initial value is to be set by the SW through register programming, an alternative to the external ToD bus interface, then write appropriate initial values to SW loading registers: TOD_SW_SEC_0/1, TOD_SW_NS, TOD_SW_CTIME_0/1. Also, configure any static offset to be loaded by writing the TOD_SEC_SYS_OFFSET_0/1 and TOD_NS_SYS_OFFSET_0 registers.
4. If the SW registers were updated in step 2, then a write of '1' to the appropriate bits of TOD_SW_LOAD[1:0] triggers the System Timer to load the SW values.
5. Program the Port TX/RX Timer. This includes the period values and any required static offset to the appropriate values by writing to the Port timer's registers: TX<M>_PERIOD_0/1, RX<M>_PERIOD_0/1, and TX<M>/RX<M>_SYS_OFFSET.

Timer Syncer IP

When the 1588 checkbox is enabled in MRMAC GUI, the Timer Syncer IP automatically generates and appropriately connects in the example design. For reference, you can generate the Example design of the MRMAC IP core.

PTP Timestamp Interface

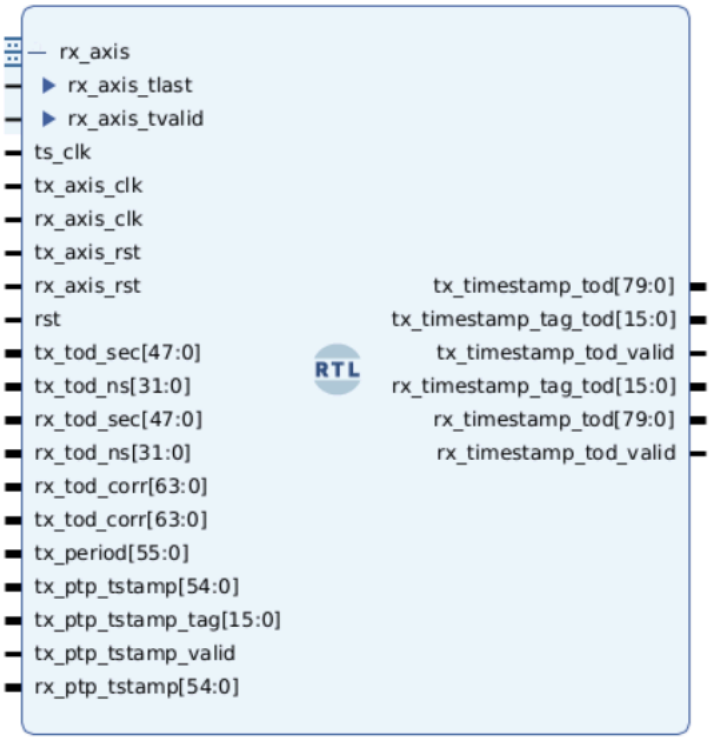
The PTP Timestamp Interface is an encrypted RTL module that demonstrates the timestamp conversion from CF format to ToD format. Following are the port details.

Table: PTP Timestamp Interface Ports

Signal	Direction	Description
ts_clk	I	Timestamp clock. Default clock period of 4.0 ns
rst	I	Reset sync with ts_clk
tx_axis_clk	I	TX AXIS streaming Clock
rx_axis_clk	I	TX AXIS streaming Clock
tx_axis_rst	I	Reset sync with tx_axis_clk
rx_axis_rst	I	Reset sync with rx_axis_clk
tx_tod_sec[47:0]	I	Port TX Timer seconds field
tx_tod_ns[31:0]	I	Port TX Timer nanoseconds field
rx_tod_sec[47:0]	I	Port RX Timer seconds field
rx_tod_ns[31:0]	I	Port RX Timer nanoseconds field
tx_tod_corr[63:0]	I	Port TX Timer CF format
rx_tod_corr[63:0]	I	Port RX Timer CF format
tx_period[55:0]	I	TX period value. Unused and reserved for future enhancement. Tie to 56'd0.
rx_axis_tvalid	I	RX AXIS control signals from MRMAC of the respective port.
rx_axis_tlast	I	RX AXIS control signals from MRMAC of the respective port.
tx_ptp_tstamp[54:0]	I	TX Timestamp output of MRMAC. To be connected to respective port of MRMAC signal "tx_ptp_tstamp_out_<N>." N is the port Number 0/1/2/3.
tx_ptp_tstamp_tag[15:0]	I	TX Timestamp Tag output of MRMAC. To be connected to respective port of MRMAC signal "tx_ptp_tstamp_tag_out_<N>." N is the port Number 0/1/2/3.
tx_ptp_tstamp_valid	I	TX Timestamp Valid output of MRMAC. To be connected to respective port of MRMAC signal "tx_ptp_tstamp_valid_out_<N>." N is the port Number 0/1/2/3.
rx_ptp_tstamp[54:0]	I	RX Timestamp output of MRMAC. To be connected to respective port of MRMAC signal "rx_ptp_tstamp_out_<N>." N is the port Number 0/1/2/3.
tx_timestamp_tod[79:0]	O	User interface For TX Timestamp in ToD format
tx_timestamp_tag_tod[15:0]	O	User interface For TX Timestamp TAG .

Signal	Direction	Description
tx_timestamp_tod_valid	O	User interface For TX Timestamp valid.
rx_timestamp_tod[79:0]	O	User interface For RX Timestamp in ToD format
rx_timestamp_tag_tod[15:0]	O	User interface For RX Timestamp TAG. This is generated using internal counter. Can be used for debug purpose only.
rx_timestamp_tod_valid	O	User interface For RX Timestamp valid.

Figure: PTP Timestamp Interface



PTP SYSTIMER BUS Interface

The PTP SYSTIMER BUS Interface is an encrypted RTL module used to demonstrate the system timer bus control. It demonstrates the generation of the `st_sync` pulse every 64 ns, converts the ToD timer value to CF format, and provides it to system timer interface of the core. The following table lists the port details.

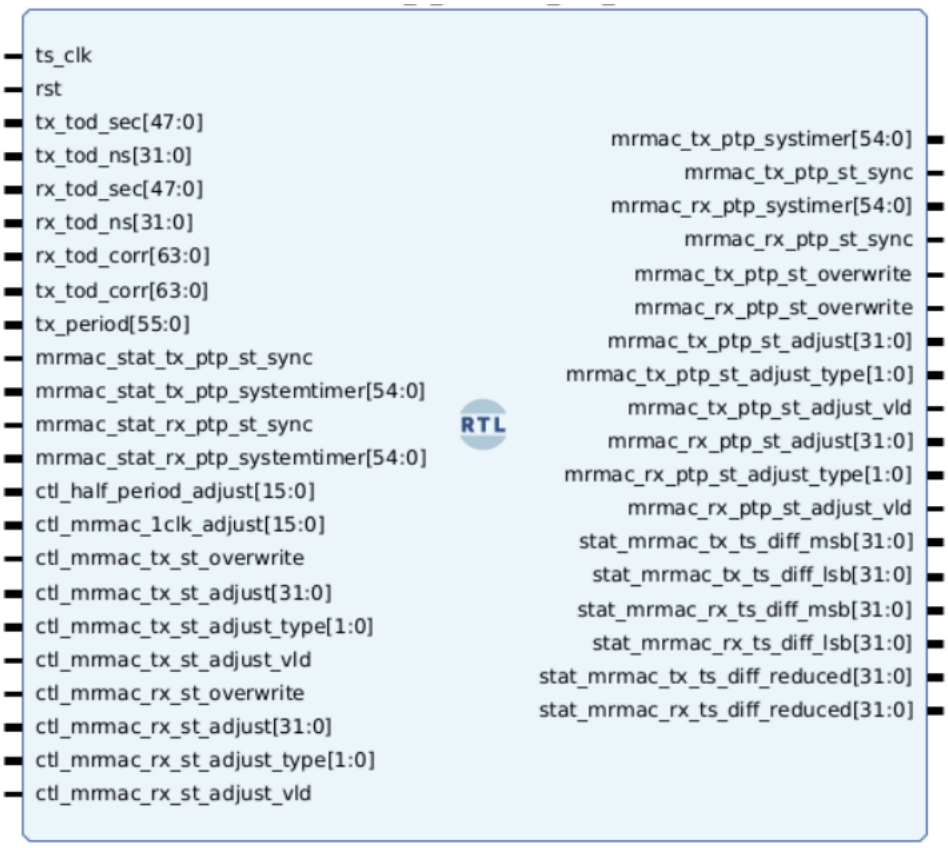
Table: PTP SYSTIMER BUS Interface

Signal	Direction	Description
ts_clk	I	Time-stamp clock. Default clock period of 4.0 ns
rst	I	Reset sync with ts_clk
tx_tod_sec[47:0]	I	Port TX Timer seconds field

Signal	Direction	Description
tx_tod_ns[31:0]	I	Port TX Timer nanoseconds field
rx_tod_sec[47:0]	I	Port RX Timer seconds field
rx_tod_ns[31:0]	I	Port RX Timer nanoseconds field
tx_tod_corr[63:0]	I	Port TX Timer CF format
rx_tod_corr[63:0]	I	Port RX Timer CF format
mrmac_tx_ptp_systimer[54:0]	O	MRMAC TX system timer. This needs to be connected to MRMAC IP port ctl_tx_ptp_systemtimer[54:0]
mrmac_tx_ptp_st_sync	O	MRMAC TX st_sync pulse. This needs to be connected to MRMAC IP port ctl_tx_ptp_st_sync
mrmac_tx_ptp_st_overwrite	O	MRMAC TX st_overwrite. This needs to be connected to MRMAC IP port ctl_tx_ptp_st_overwrite
mrmac_rx_ptp_systimer[54:0]	O	MRMAC RX system timer. This needs to be connected to MRMAC IP port ctl_rx_ptp_systemtimer[54:0]
mrmac_rx_ptp_st_sync	O	MRMAC RX st_sync pulse. This needs to be connected to MRMAC IP port ctl_rx_ptp_st_sync
mrmac_rx_ptp_st_overwrite	O	MRMAC RX st_overwrite. This needs to be connected to MRMAC IP port ctl_rx_ptp_st_overwrite
mrmac_tx_ptp_st_adjust[31:0]	O	MRMAC TX st_adjust value. This needs to be connected to MRMAC IP port ctl_tx_ptp_st_adjust[31:0]
mrmac_tx_ptp_st_adjust_type[1:0]	O	MRMAC TX st_adjust type. This needs to be connected to MRMAC IP port ctl_tx_ptp_st_adjust_type[1:0]
mrmac_tx_ptp_st_adjust_vld	O	MRMAC TX st_adjust valid. This needs to be connected to MRMAC IP port ctl_tx_ptp_st_adjust_vld
mrmac_rx_ptp_st_adjust[31:0]	O	MRMAC RX st_adjust value. This needs to be connected to MRMAC IP port ctl_rx_ptp_st_adjust[31:0]
mrmac_rx_ptp_st_adjust_type[1:0]	O	MRMAC RX st_adjust type. This needs to be connected to MRMAC IP port ctl_rx_ptp_st_adjust_type[1:0]
mrmac_rx_ptp_st_adjust_vld	O	MRMAC RX st_adjust valid. This needs to be connected to MRMAC IP port ctl_rx_ptp_st_adjust_vld
ctl_mrmac_rx_st_adjust_vld	I	Tie to 1'b0
ctl_mrmac_rx_st_adjust_type[1:0]	I	Tie to 2'b00
ctl_mrmac_rx_st_adjust[31:0]	I	Tie to 32'd0

Signal	Direction	Description
ctl_mrmac_rx_st_overwrite	I	Tie to 1'b1
ctl_mrmac_tx_st_adjust_vld	I	Tie to 1'b0
ctl_mrmac_tx_st_adjust_type[1:0]	I	Tie to 2'b00
ctl_mrmac_tx_st_adjust[31:0]	I	Tie to 32'd0
ctl_mrmac_tx_st_overwrite	I	Tie to 1'b1
tx_period[55:0]	I	TX period value. Unused and reserved for future enhancement. Tie to 56'd0.
ctl_half_period_adjust[15:0]	I	Half period Adjust value. Unused and reserved for future enhancement. Tie to 16'd0.
ctl_mrmac_1clk_adjust[15:0]	I	1Clock Adjust value. Unused and reserved for future enhancement. Tie to 16'd0.
mrmac_stat_tx_ptp_st_sync	I	Unused and reserved for future enhancement. Tie to 1'b0.
mrmac_stat_tx_ptp_systemtimer[54:0]	I	Unused and reserved for future enhancement. Tie to 55'd0.
mrmac_stat_rx_ptp_st_sync	I	Unused and reserved for future enhancement. Tie to 1'b0.
mrmac_stat_rx_ptp_systemtimer[54:0]	I	Unused and reserved for future enhancement. Tie to 55'd0.
stat_mrmac_tx_ts_diff_msb[31:0]	O	Unused and reserved for future enhancement.
stat_mrmac_tx_ts_diff_lsb[31:0]	O	Unused and reserved for future enhancement.
stat_mrmac_rx_ts_diff_msb[31:0]	O	Unused and reserved for future enhancement.
stat_mrmac_rx_ts_diff_lsb[31:0]	O	Unused and reserved for future enhancement.
stat_mrmac_tx_ts_diff_reduced[31:0]	O	Unused and reserved for future enhancement.
stat_mrmac_rx_ts_diff_reduced[31:0]	O	Unused and reserved for future enhancement.

Figure: SYSTIMER BUS Interface



MRMAC Auto-Negotiation and Link Training

MRMAC uses an Auto-Negotiation and Link Training (ANLT) block inside the GT Wizard. Currently, the following MRMAC modes support ANLT with GTM Only:

Table: Supported MRMAC Modes

MRMAC Mode	Description
1x100GE	100CAUI-4, 100GAUI-1 Only
2x50GE	50GAUI-1 Only
1x40GE	
4x25GE	Although it is four ports, Master Lane is lane 0. Therefore, AN only performs on Lane 0 (Port-0).

For additional details, refer to the Versal Auto Negotiation and Link Training Interface User Guide (UG1790) (registration required) and AR000038153.

Clocking Use Cases

This appendix illustrates the clocking strategy for various modes of MRMAC operation.

Emphasis is placed on showing how one mode can dynamically evolve into another. For example, the MRMAC might start off configured as 4 × 10G clients but then dynamically re-configure into a 1 × 50G + 10G + 25G configuration.

Table: Clocking Use Cases Index

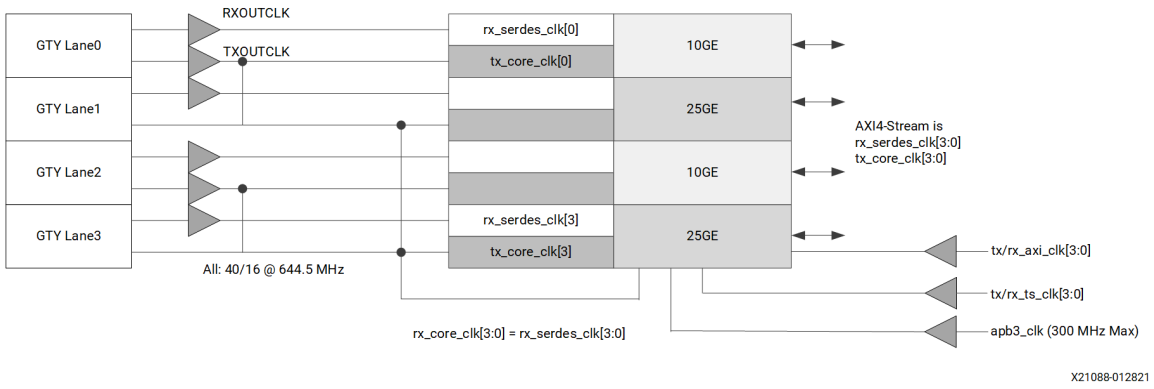
Example	Configuration
Standard Ethernet MAC + PCS (Low Latency Mode) End Point	
1a	4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s) Low Latency AXI Operating Mode
1b	1 × 50GE (1 × 53.12 Gb/s), 2 × 10/25GE (2 × 10.31/25.78 Gb/s) Low Latency Operating Mode (50GE IEEE 50GBASE-R)
1c	1 × 50GE (2 × 26.56 Gb/s), 2 × 10/25GE (2 × 10.31/25.78 Gb/s) Low Latency Operating Mode (50GE IEEE 50GBASE-R External Gearbox)
1d	1 × 100GE (4 × 25.78/26.56 Gb/s) Operating Mode
Standard Ethernet MAC + PCS (Low Frequency AXI4-Stream) End Point	
2a	4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s) Low Speed AXI Operating Mode
2b	1 × 50GE (1 × 53.12 Gb/s), 2 × 10/25GE (2 × 10.31/25.78 Gb/s) Low Speed AXI Operating Mode (50GE IEEE 50GBASE-R)
2c	1 × 50GE (2 × 26.56 Gb/s), 2 × 10/25GE (2 × 10.31/25.78 Gb/s) Low Speed AXI Operating Mode (50GE IEEE 50GBASE-R External Gearbox)
2d	1 × 100GE (4 × 25.78/26.56 Gb/s) Low Speed AXI Operating Mode
2e	4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s) Low Speed AXI opt2 Operating Mode
2f	1 × 50GE (1 × 53.12 Gb/s), 2 × 10/25GE (2 × 10.31/25.78 Gb/s) Low Speed AXI Operating Mode (50GE IEEE 50GBASE-R)
2g	4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s) Independent Lanes, Low Speed AXI opt2 Operating Mode
2h	4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s) Independent Clock Lanes, Low Speed AXI Operating Mode
2i	4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s) Low Speed PCS Data (-1LP), Low Speed AXI Operating Mode
2j	1 × 100GE (4 × 28.21 Gb/s) Overclock, Low Speed AXI Operating Mode
2k	1 × 100GE (2 × 56.42 Gb/s) Overclock, Low Speed AXI Operating Mode
PCS Only and FEC Only End Point Use Cases	
3a	4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s) PCS Only, Low Speed AXI Operating Mode
3b	4 × 28.21GE FEC Only, Operating Mode

Standard Ethernet MAC + PCS (Low Latency Mode)

Example 1a

- 4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s)
- Low Latency AXI Operating Mode

Figure: Example Diagram 1a

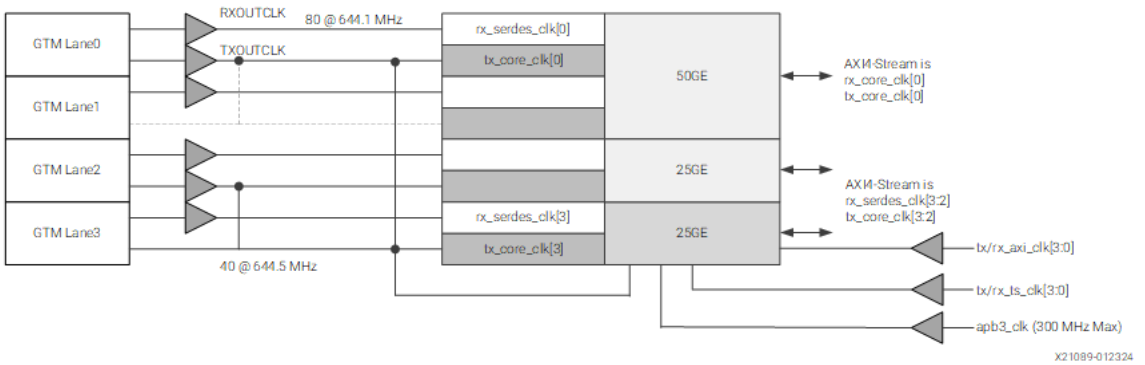


Example 1b

- 1 × 50GE (1 × 53.12 Gb/s)
- 2 × 25GE (2 × 25.78 Gb/s)
- 50GE IEEE 802.3cd
- Low Latency AXI Operating Mode
- SerDes interface is in the narrow mode

In the example diagram 1b, the unused lane can be dark but still connected for switchable use cases.

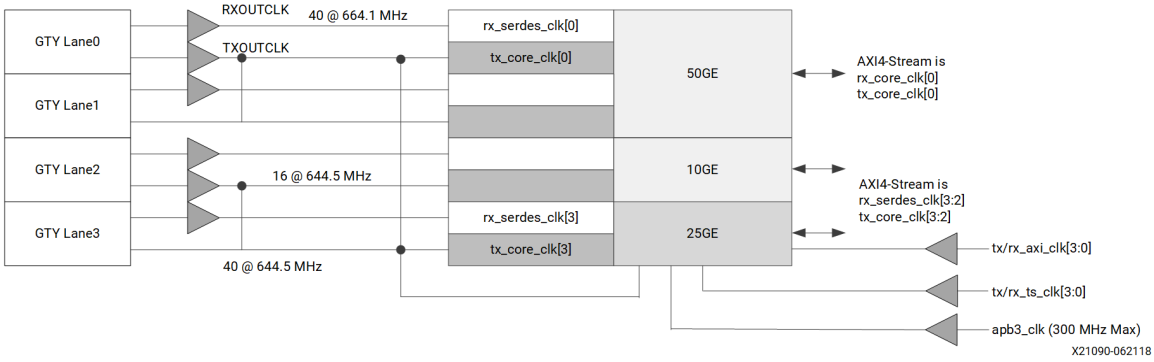
Figure: Example Diagram 1b



Example 1c

- 1 × 50GE (2 × 26.56 Gb/s)
- 2 × 10/25GE 2 × 10.31/25.78 Gb/s)
- 50GE IEEE 802.3cd External Gearbox
- Low Latency AXI Operating Mode

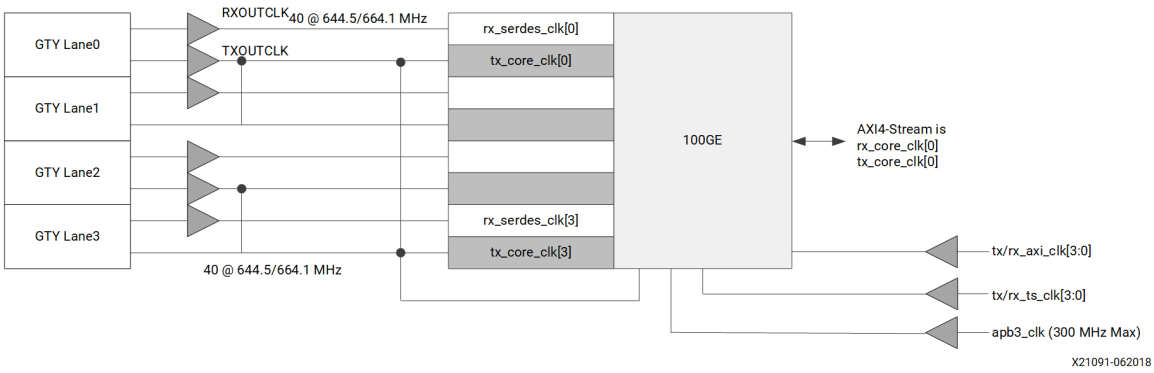
Figure: Example Diagram 1c



Example 1d

- 1 × 100GE (4 × 25.78/26.56 Gb/s)
- Low Latency AXI Operating Mode

Figure: Example Diagram 1d

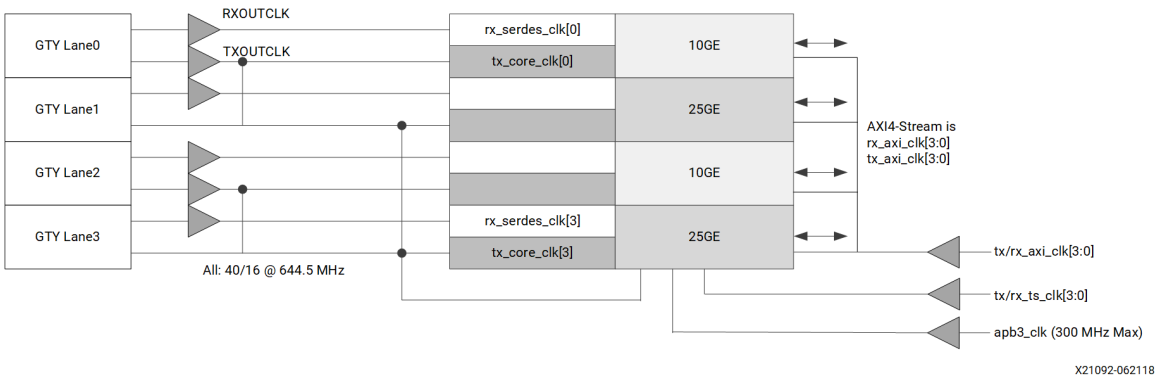


Standard Ethernet MAC + PCS (Low Frequency AXI4-Stream)

Example 2a

- 4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s)
- Low Latency AXI Operating Mode

Figure: Example Diagram 2a

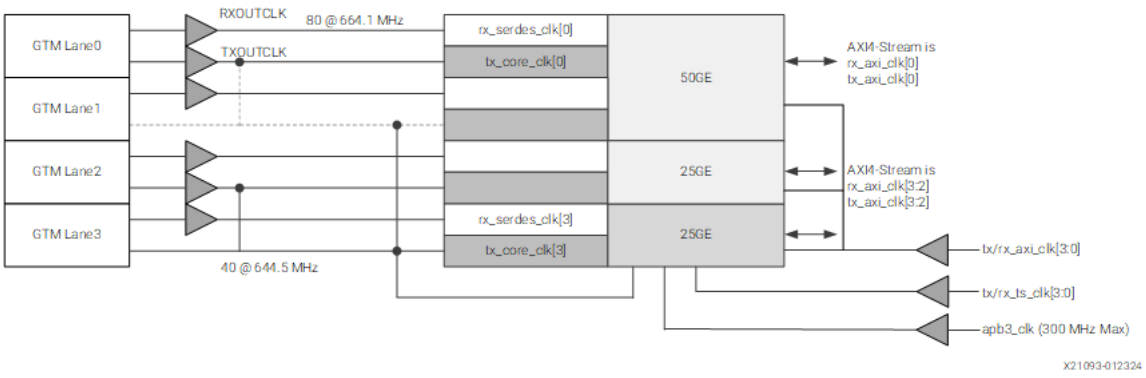


Example 2b

- 1 × 50GE (1 × 53.12 Gb/s)
- 2 × 25GE (2 × 25.78 Gb/s)
- 50GE IEEE 50GBASE-R
- Low Frequency AXI Operating Mode

In the example diagram 2b, the unused lane can be dark but still connected for switchable use cases.

Figure: Example Diagram 2b



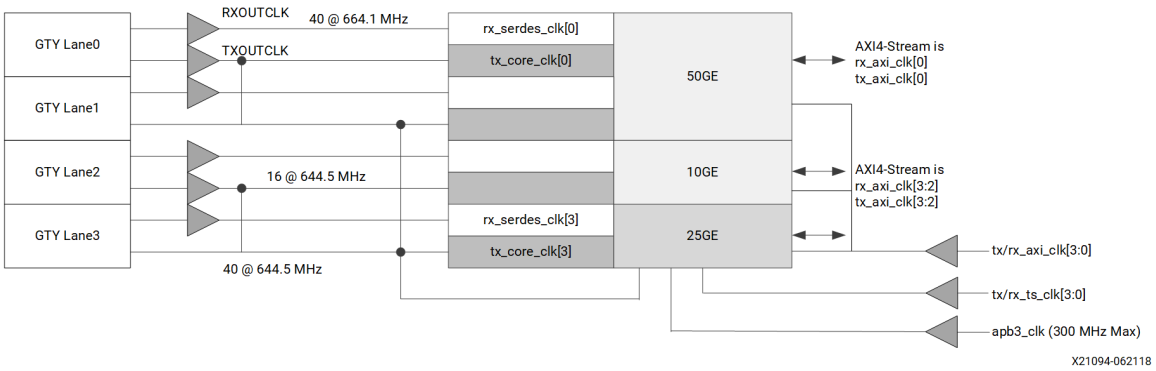
Example 2c

- 1 × 50GE (2 × 26.56 Gb/s)
- 2 × 10/25GE 2 × 10.31/25.78 Gb/s)
- 50GE IEEE 50GBASE-R External Gearbox
- Low Frequency AXI Operating Mode

The following shows the clocks used in the example diagram 2c use case:

- 4 × RX SerDes clock
- 2 × TX SerDes clock (and `tx/rx_core_clk[3:0]`)
- 1 × System timer clock
- 3 × AXI4-Stream clock (independent AXI)
- 1 × AXI4-Stream clock (shared AXI) 1 × APB3 clock
- 11 clocks (independent AXI)
- 21 clocks for DD MRMAC (share APB3)
- 9 clocks (shared AXI)
- 17 clocks for DD MRMAC

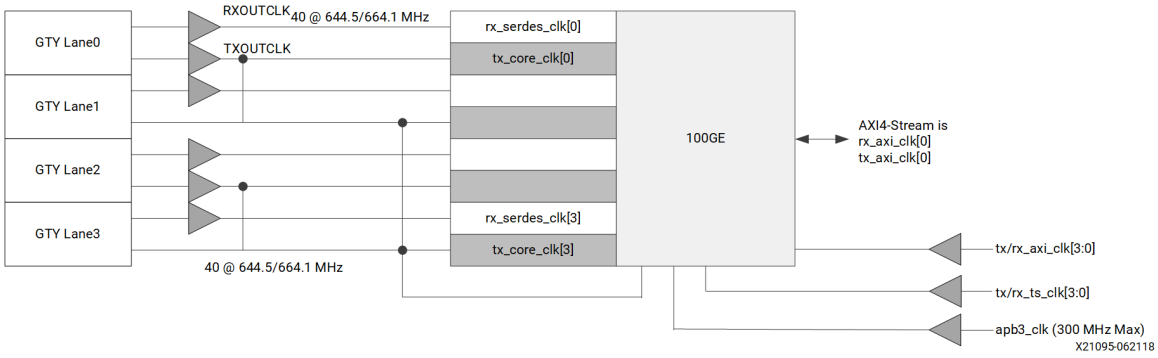
Figure: Example Diagram 2c



Example 2d

- 1 × 100GE (4 × 25.78/26.56 Gb/s)
- Low Frequency AXI Operating Mode

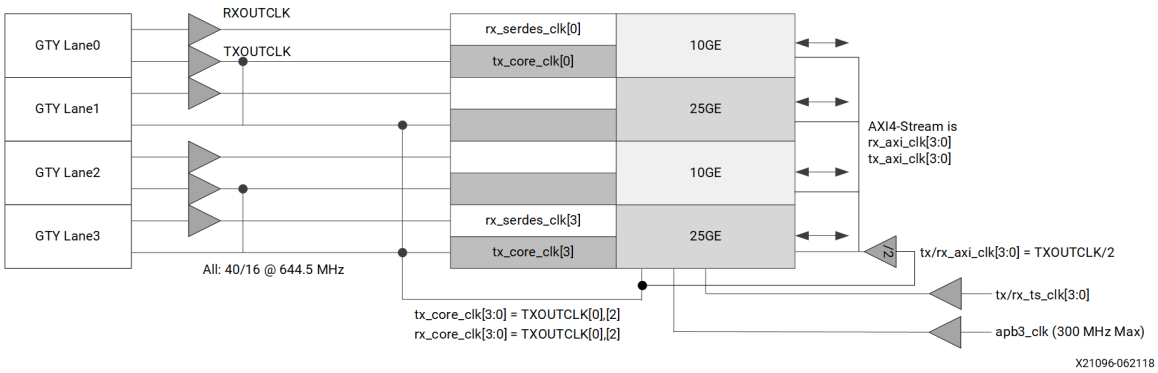
Figure: Example Diagram 2d



Example 2e

- 4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s)
- Low Frequency AXI Operating Mode
- Drive `tx/rx_axi_clk[3:0]` with TXOUTCLK/2

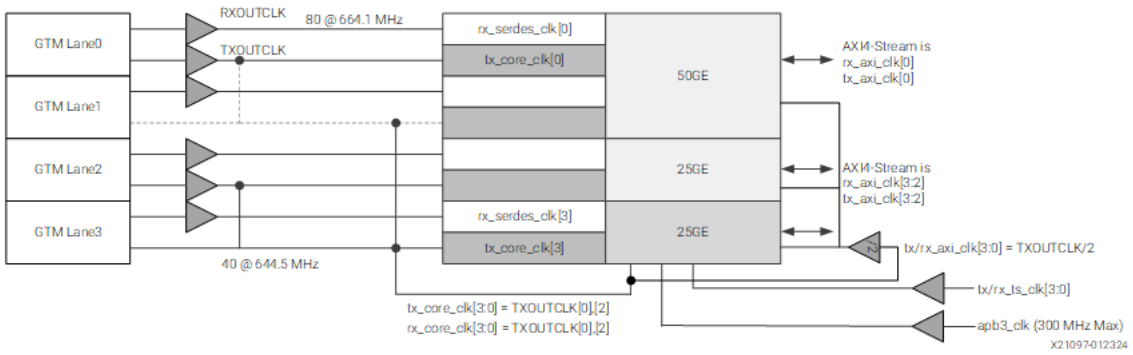
Figure: Example Diagram 2e



Example 2f

- 1 × 50GE (1 × 53.12 Gb/s)
- 2 × 25GE (2 × 25.78 Gb/s)
- Low Frequency AXI Operating Mode
- 50GE IEEE 50GBASE-R

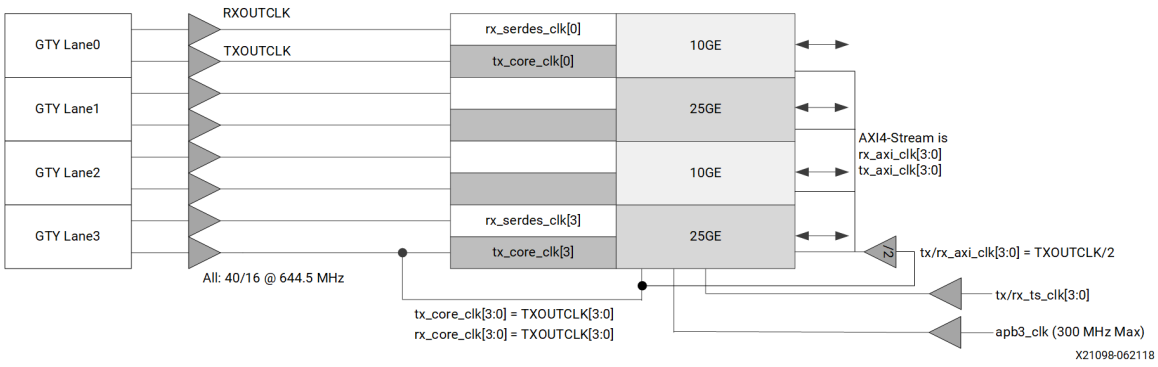
Figure: Example Diagram 2f



Example 2g

- 4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s)
- Independent Clock Lanes
- Low Frequency AXI Operating Mode
- Drive `tx/rx_axi_clk[3:0]` with TXOUTCLK/2

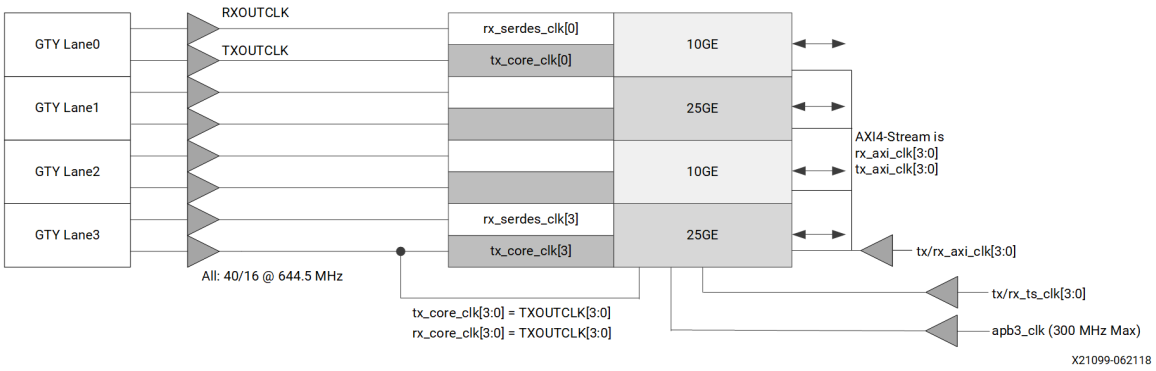
Figure: Example Diagram 2g



Example 2h

- 4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s)
- Independent Clock Lanes
- Low Frequency AXI Operating Mode

Figure: Example Diagram 2h

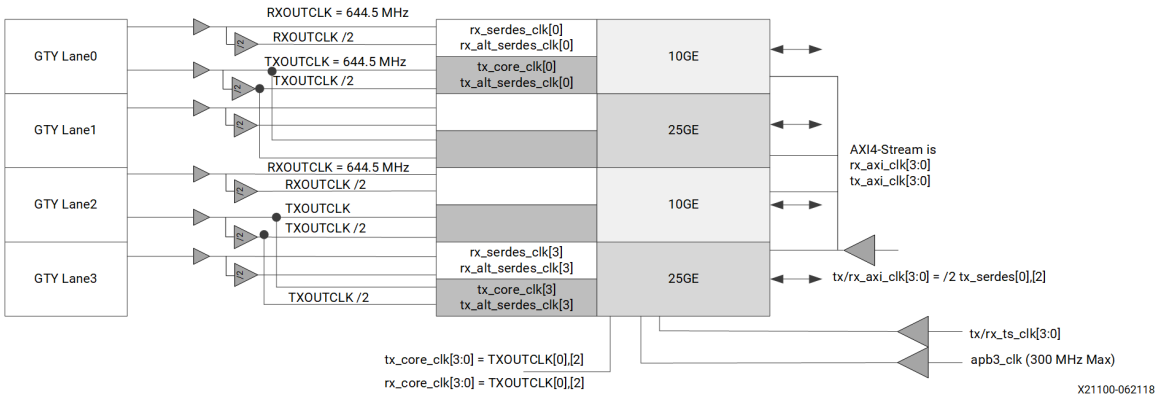


Example 2i

- 4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s)
- Low Speed PCS Data (-1LP)
- Low Frequency AXI Operating Mode

In the example diagram 2i, 25GE Data out 80 @ 322.26 MHz and 10GE Data out 32 @ 323.26 MHz. TX/RXOUTCLK is ×2 data rate and local /2 bufg in RCLK region generates the clock to capture the SerDes Data.

Figure: Example Diagram 2i

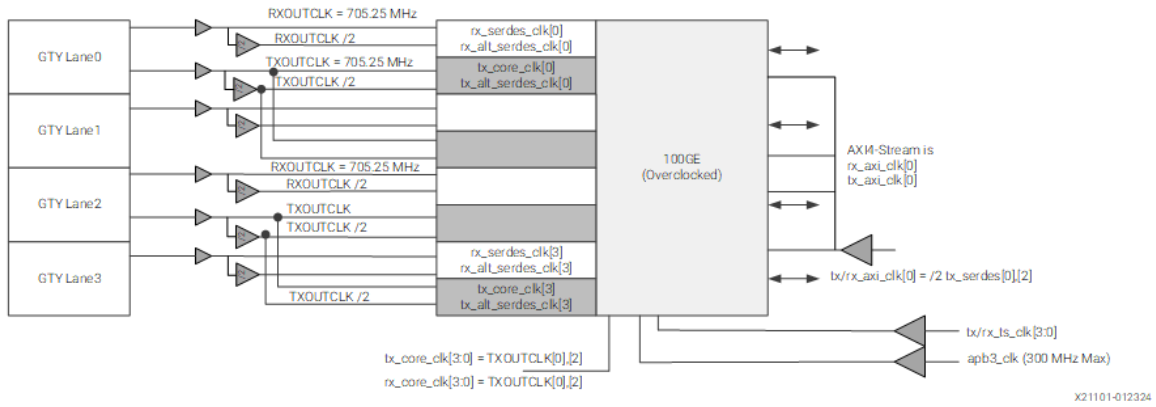


Example 2j

- 1 × 100GE (4 × 28.21 Gb/s) Overclock
- Low Frequency AXI Operating Mode

Each lane has 28.21 Gb/s Data out 80 @ 352.62 MHz. TX/RXOUTCLK is ×2 data rate and local /2 bufg in RCLK region generates the clock to capture the SerDes Data.

Figure: Example Diagram 2j

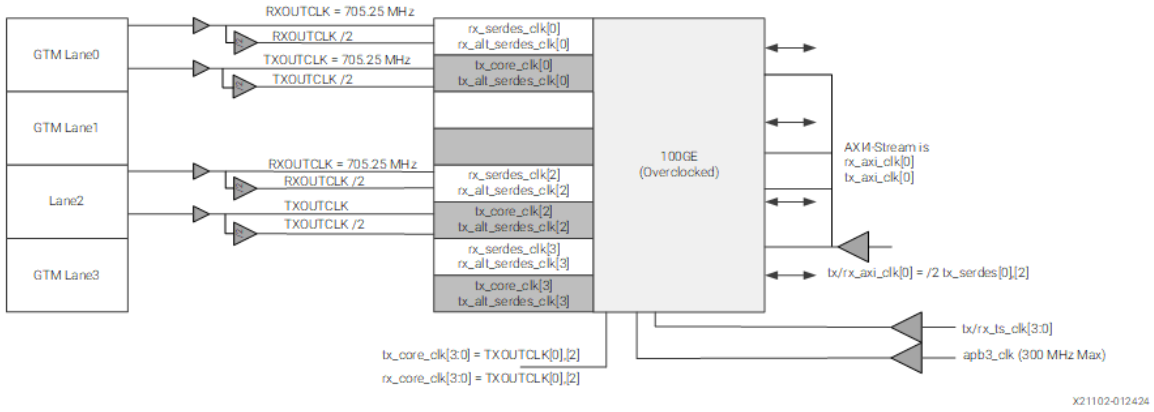


Example 2k

- 1 × 100GE (2 × 56.42 Gb/s) Overclock
- Low Frequency AXI Operating Mode

Each lane has 56.42 Gb/s Data out 160 @ 352.62 MHz. TX/RXOUTCLK is ×2 data rate and local /2 bufg in RCLK region generates the clock to capture the SerDes Data.

Figure: Example Diagram 2k

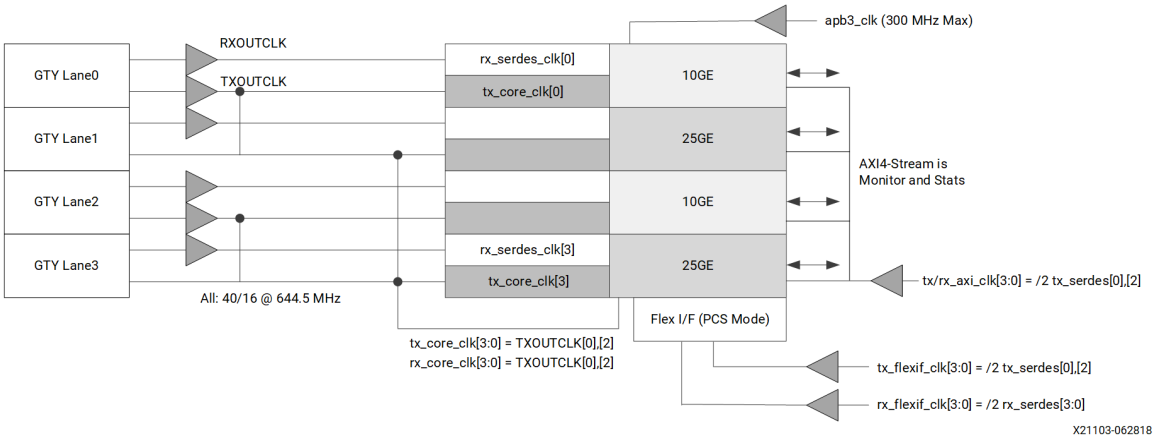


PCS Only and FEC Only

Example 3a

- 4 × 10/25GE (4 × 10.3125/4 × 25.78 Gb/s)
- PCS Only
- Low Speed AXI Operating Mode

Figure: Example Diagram 3a

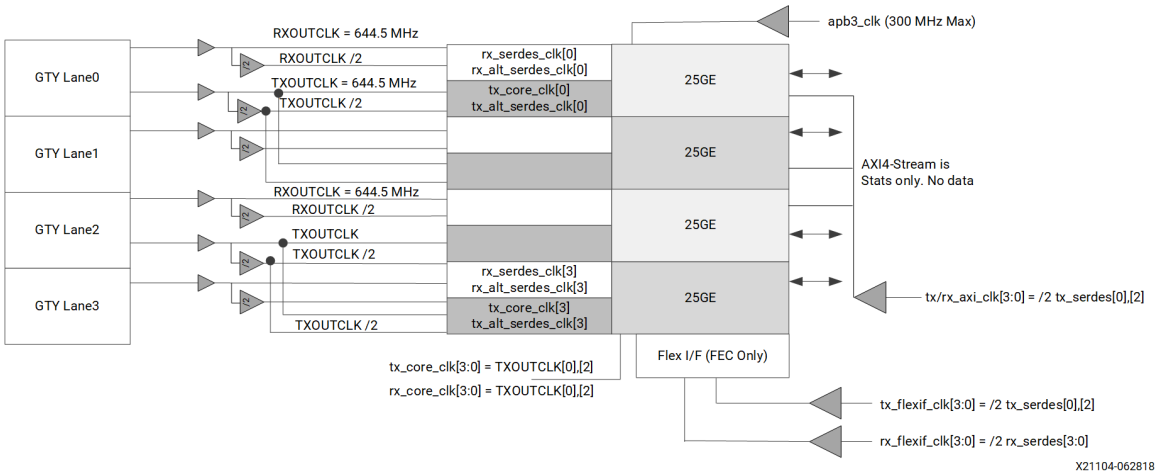


Example 3b

- 4 × 28.21GE
- FEC Only Operating Mode

In the example diagram 3b, the 28.05 Gb/s Data out 80 @ 350.62 MHz. TX/RXOUTCLK is ×2 data rate and local /2 bufg in RCLK region generates the clock to capture the SerDes Data.

Figure: Example Diagram 3b



Debugging

This appendix includes details about resources available on the Support website and debugging tools.

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)

✎ **Note:** IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the subsystem, the Support web page contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The Community Forums are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the subsystem. This guide, along with documentation related to all products that aid in the design process, can be found on the AMD Adaptive Support web page or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the Downloads page. For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this subsystem can be located by using the Search Support box on the main AMD Adaptive Support web page. To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the MRMAC

AR 75817

Technical Support

AMD Adaptive Computing provides technical support on the Community Forums for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the Community Forums.

Debug Tools

There are many tools available to address MRMAC design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the Vivado Design Suite User Guide: Programming and Debugging (UG908).

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to https://docs.amd.com.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select Help > Documentation and Tutorials.
- On Windows, click the Start button and select Xilinx Design Tools > DocNav.
- At the Linux command prompt, enter `docnav`.

✎ **Note:** For more information on DocNav, refer to the Documentation Navigator User Guide (UG968).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the Design Hubs View tab.
- Go to the Design Hubs web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see Support.

References

These documents provide supplemental material useful with this guide:

1. Versal Adaptive SoC Transceiver Subsystem Product Guide (PG442)
2. Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973)
3. IEEE Standard for Ethernet IEEE 802.3-2022
4. IEEE Standard 1588-2019, IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems
5. 25G and 50G Ethernet Consortium Schedule 3 version 1.6 (August 18, 2015)
6. ITU-T G.709.5 (03/2024) - Flexible OTN short-reach interfaces
7. Arm® AMBA® 4 AXI4-Stream Protocol v1.0 Specification (ARM IHI 0051A)
8. Arm AMBA 3 APB v1.0 Specification (ARM IHI 0024B)
9. AXI to APB Bridge LogiCORE IP Product Guide (PG073)
10. Vivado Design Suite: AXI Reference Guide (UG1037)
11. Vivado Design Suite Tutorial: Logic Simulation (UG937)
12. Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator (UG994)
13. Vivado Design Suite User Guide: Designing with IP (UG896)
14. Vivado Design Suite User Guide: Getting Started (UG910)
15. Vivado Design Suite User Guide: Logic Simulation (UG900)
16. Vivado Design Suite User Guide: Programming and Debugging (UG908)
17. Versal Adaptive SoC Transceivers Wizard LogiCORE IP Product Guide (PG331)
18. VCK190 Evaluation Board User Guide (UG1366)
19. VMK180 Evaluation Board User Guide (UG1411)
20. Versal Auto Negotiation and Link Training Interface User Guide (UG1790) (registration required)

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
09/02/2025 Version 3.1	
Standards	Updated IEEE 802.3-2022 standard version.

Section	Revision Summary
Performance and Resource Utilization	Added topic.
Port FEC Mode	Updated table.
MRMAC Configuration Tab	Updated section.
MRMAC FEC Only Configuration Example Design	Added section.
MRMAC PCS Only Configuration Example Design	Added section
Example Design with GT Wizard Subsystem (gtwiz_versal IP)	Added section.
Example Design Creation, Simulation, and Device Image Generation	Updated section.
MRMAC Auto-Negotiation and Link Training	Added section
Clocking Use Cases	Updated table.
Standard Ethernet MAC + PCS (Low Frequency AXI4-Stream)	Updated examples.
References	Updated IEEE standards versions.
11/13/2024 Version 3.0	
MRMAC Configuration Tab	Updated the section.
GT Information Tab	Updated the section.
05/30/2024 Version 2.3	
• MRMAC Control Ports • Flexible Interface Examples • Customizing and Generating the Subsystem • Flex I/F Active Bus Ports by Configuration • Test ELF Creation	• Modified MRMAC Control Port Descriptions table. • Added a note and modified TX Flex Interface timing diagram. • Modified the screenshots. • Modified the table notes. • Updates the flow and images.
01/29/2024 Version 2.2	
• MRMAC Configuration Tab • GT Information Tab • Constraining the Subsystem	Updated for v2.2.
07/11/2023 Version 2.1	
• Flex Interface Modes of Operation • MRMAC Configuration Tab • GT Information Tab • Example Design Creation, Simulation, and Device Image Generation	Updated for v2.1.
Entire document	General updates.
01/30/2023 Version 2.0	
General updates	Updated the Register Reference Files URL.

Section	Revision Summary
01/18/2023 Version 2.0	
Simulation Speed Up	Added Xcelium, Riviera-PRO.
Entire document	General updates.
07/13/2022 Version 1.6	
General updates	Updated the register map ZIP file link
07/07/2022 Version 1.6	
General updates	Updated with minor technical changes.
01/10/2022 Version 1.5	
PTP 1588 Timer Syncer IP	Added PTP TIMESTAMP and SYSTIMER BUS information
Register Spaces	Updated Register Map
11/15/2021 Version 1.4	
Product Specification	• PTP Updates • Port Table updates for 1588
Example Design	Updated Example Design
Design Flow Steps	Updated for v1.4
02/05/2021 Version 1.3	
GT Quad Integration with AMD IP Cores	Added new section under Design Flow Steps
02/03/2021 Version 1.3	
Initial release.	N/A

THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright